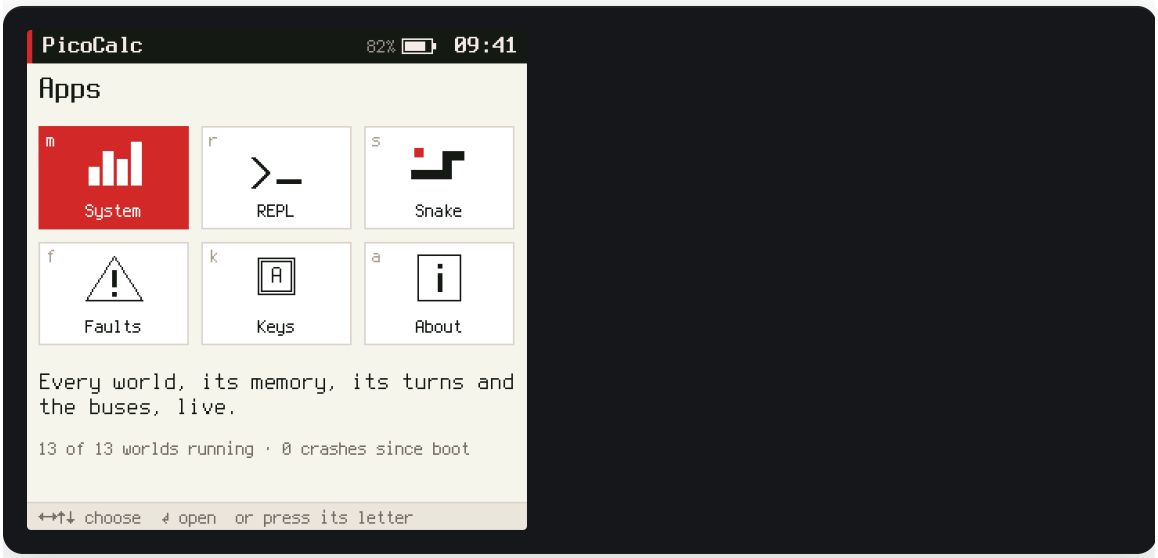
A COMPANION TO SMALL WORLDS

From L4 to Small Worlds

Microkernel Lessons for a JavaScript Operating System, and the Workbench and Agent That Test Them

With PicoCalc OS as the worked example: a JavaScript operating system for an ESP32-P4 hand-held, its C nucleus, drivers and apps, simulated, measured and broken on purpose.



The home screen, read from the simulated ST7365P panel's memory.

Preface

PicoCalc OS runs every driver, service and app of a pocket computer as a *world*: a separate JavaScript runtime with its own heap and its own time budget, scheduled by a small C nucleus that delivers messages, hands out hardware as handles and restarts whatever fails. *Small Worlds* [Small Worlds 2026] teaches operating systems with that system as its worked example. This book is its companion, and it asks a narrower question: what does thirty years of research on the L4 family of microkernels have to teach a system like this one, and what happens when the system acts on it?

The L4 family is a good teacher because its builders wrote down what they learnt. Jochen Liedtke showed in the early 1990s that a microkernel could be fast if it was designed around a few primitives and built with care for the machine. Twenty years later Kevin Elphinstone and Gernot Heiser went back through those decisions and reported which ones had survived [Elphinstone and Heiser 2013]. Along the way seL4 became the first general-purpose kernel with a machine-checked proof that its C code implements its specification [Klein et al. 2009], and its builders then moved to statically described systems, in which a short file says which components exist and exactly how each may reach the others. Each of those steps answers a question that PicoCalc OS also has to answer.

What this book does

Part I reads the research: the path from Brinch Hansen's nucleus to L4, seL4's object model and its proofs, and the static systems built on seL4 (the Microkit and LionsOS). It reads them as an engineer reads a design document, looking for the mechanism, the reason for it and the evidence behind it, and it compares each mechanism with what PicoCalc OS does.

Part II is the *first pass*: the changes PicoCalc OS made after that reading. The manifest became the whole description of who may talk to whom, which channels each world may post and hear, and how much of the kernel's tables each may use. The ask graph must have no cycles. A turn that never ends is now a crash. Each chapter shows the defect with a world written to exploit it, the L4 idea behind the fix, the nucleus code that implements it, and the same world failing afterwards. The last chapter of the part is about evidence: what PicoCalc OS can know about itself without a proof, and how the first pass was shown to change nothing it should not.

Part III is the workbench: the same nucleus and the same QuickJS compiled to WebAssembly and running in a page, with the device, the code, the kernel's tables and its trace beside each other, a coding agent that works through a single tool, and an eval suite that boots a fresh system for every check. The workbench also carries this book in its Book view, where a code excerpt's label leads to the lines it quotes, and the agent reads and cites the same text. The part then designs the second pass, what to borrow next, with the evals that would show each step works, and ends with the second pass as it was built: endpoint capabilities, a passive sandbox, one driver per device and supervision in a world, each with its cost and its evidence.

How to read it

Read Part I for the research and Part II for the system; either works first. A reader who wants to change PicoCalc OS today can start with Chapter 5 and Part III. Two appendices document the manifest and the workbench's API.

Code excerpts labelled with a file name are taken from the PicoCalc OS sources and checked against them when the book is built. Chapters 1 to 12 describe the system as it stood after the first pass; their excerpts of files the second pass changed quote the copy kept in `history/first-pass/`, and their labels say so. Chapter 13 describes the system as it is now, and `SECOND-PASS.md` in the repository records the same pass. Numbers in milliseconds are virtual milliseconds of the host build, calibrated against the ESP32-P4, unless the text says otherwise. PicoCalc OS runs on a simulated board; its port to the device is planned, and this book does not describe it as done. Every reference to another system's design is to its primary source, linked in the bibliography.

Contents

PART I. THE L4 LINEAGE

1 **Why read L4 now** 8

Where PicoCalc OS stands beside L4, seL4 and the Microkit: a microkernel's structure without its hardware, the defects the comparison found, which L4 lessons can transfer to JavaScript worlds in one address space, and how this book tests each claim.

2 **From the nucleus to L4** 30

The microkernel idea from Brinch Hansen's RC 4000 nucleus through Mach to Liedtke's L3 and L4: why the first microkernels were slow, what minimality means, what twenty years of L4 kept and dropped, and what a PicoCalc message costs beside an L4 IPC.

3 **seL4: capabilities, proofs and time** 56

seL4's design decisions read as mechanisms with reasons: capabilities to kernel objects, kernel memory from untyped, endpoints, the derivation tree and revocation, scheduling contexts, time protection, and what the proofs establish, assume and cover today.

4 **Static systems: Microkit and LionsOS** 79

Why embedded systems are described statically: CAmkES, capDL and the seL4 Microkit, a Microkit system description read element by element, LionsOS and the sDDF driver framework, what is verified about the Microkit, and the Microkit beside PicoCalc OS's manifest.

PART II. THE FIRST PASS

5 **Authority from the manifest** 98

Three attacks by a world with no authority, L4's path from global thread identifiers through clans and chiefs to endpoint capabilities, and the first pass's answer: the manifest states who may send, ask, publish and hear, checked at boot and at every message.

6 **Asks that cannot wait for each other** 121

The ask deadlock that a closed mailbox creates, why L4 dropped IPC timeouts and how the Microkit keeps calls acyclic, and the first pass's four mechanisms: cycles refused at boot, downward asks noted, timed-out asks withdrawn, and turn deadlines.

7 **No shared pools** 142

Four kernel pools any world could drain, seL4's answer of no kernel heap beside Genode's and Linux's, and the first pass's per-world quotas: written in the manifest, charged to the world that caused each entry, checked at boot, and what static partitioning costs.

8 **Evidence without a proof** 168

How strong the evidence for the first pass is without a proof: a ladder of instruments from examples to machine-checked proofs, seL4's proof in one page, deterministic simulation as PicoCalc's main instrument, the first pass re-verified with real output, and what the instruments miss.

PART III. THE WORKBENCH AND THE AGENT

9 **The system in a page** 188

How the real nucleus and QuickJS run in a browser page as a WASI reactor, how closely that build matches the native one and how the match was measured, and how the workbench is built around it, with a live system, fresh systems, hot reload, views and a deliberately plain design.

10 **An agent with one tool** 212

The workbench's coding agent, built on Claude through the artifact platform's sample capability, works through a single eval tool that runs JavaScript against a curated scope; this chapter explains why one tool, how a turn runs, what the model is told and shown, and how modes, snapshots and revert keep a person in control.

11 **Evals on fresh systems** 237

How the workbench's nineteen evals boot a fresh PicoCalc OS for every check, why determinism makes a before-and-after comparison exact, how results become a baseline with regressions marked, how much the suite catches when bugs are seeded into the system, and how the same machinery could evaluate the agent.

12 **The second pass** 264

Five more borrowings from the L4 family (endpoint capabilities, asks charged to the asker, one driver per device, supervision in a world, a hardware fence for the nucleus) and the workbench features that would test them, each with its source, a design sketch, what it fixes, what it costs and its evals, and an order in which to do them.

13 **The second pass, built** 282

How the designs of Chapter 12 became code: typed endpoint handles numbered per world, one driver for the STM32, restart policy in a supervisor world, a passive sandbox that runs on its askers' priority and budget, and the purge of a dead instance's events; with what each cost, the regression an eval found, and the evidence that nothing else changed.

APPENDICES

A **The manifest** 314

The complete schema of js/system.json after the second pass (groups, defaults, the supervisor, endpoints and worlds), its defaults and limits, every check the boot makes, every message the manifest's rules produce at boot and at run time, and the manifest PicoCalc OS ships with.

B **The workbench's API** 342

The agent's eval tool, its time limit and Read-only mode, every name in its scope, apply's report, the eval API with metrics, pins, generated and recorded evals, the trace diff, wb links, the views, the JavaScript API of the WebAssembly build, the keyboard shortcuts and the headless tests, as the code defines them.

· **References** 368

CHAPTER 1 PART I · THE L4 LINEAGE

Why read L4 now

PicoCalc OS has the structure of a microkernel system without the hardware a microkernel relies on. Its drivers, services and apps are separate components that share no memory and communicate only by messages through a small C nucleus, as in a microkernel. They all run in one address space at one privilege level, as in a unikernel. What keeps them apart is the QuickJS runtime each one runs in. *Small Worlds* [Small Worlds 2026] teaches operating systems with this system and, in its Chapter 8, places it among kernel structures in general. This chapter makes a narrower and sharper comparison, with the L4 family of microkernels and with its most developed member, seL4, together with the Microkit, the framework for building static systems on it. The comparison is worth making for two reasons. The L4 builders spent three decades finding out which microkernel mechanisms are worth their cost and wrote down what they learnt [Elphinstone and Heiser 2013; Heiser and Elphinstone 2016]. And the comparison predicts, one by one, the defects that *Small Worlds* lists as known issues. By the end of the chapter the reader should be able to say precisely where PicoCalc OS resembles an L4 system and where it does not, which of L4's lessons survive being carried across a boundary enforced by a language rather than an MMU, and how the rest of this book carries them.

1.1 PicoCalc OS in one page

A **world** is the unit of everything in PicoCalc OS. Each world is a JavaScript program evaluated in its own QuickJS runtime, with its own heap, a heap limit (`mem_kb` in the manifest) and a time budget for each piece of work it does (`budget_ms`). The shipped system has thir-

teen worlds. Three are drivers (keyboard, power and display), which run at priority 3. The compositor and flaky, a service that fails on purpose, are services at priority 2. Seven are apps at priority 1: the launcher, the system monitor, the REPL, Snake, Faults, Keys and About. The sandbox, a service that evaluates what is typed at the REPL, is placed at priority 0 by the manifest so that nothing waits behind it. *Small Worlds* builds the system up from these pieces in its Chapters 1 and 3.

The **nucleus** is the C program underneath. It runs one world at a time on a single **lane**: it picks the highest-priority world that has work, runs one **segment** of it (the handling of one message, or one continuation after an awaited result), counts the interpreter's steps, and charges the segment to a **virtual clock** at $\tau = 665$ ns per step plus $c_0 = 40$ µs per segment. Every effect a segment asks for (a message, a bus transfer, a timer) is deferred until the segment ends, and discarded if the segment crashes. A **turn** is everything a world does for one message; it spans several segments when the handler awaits. Messages are copied as JSON. `send` delivers one message, `ask` delivers one and returns a promise of the reply, and `publish` delivers a copy to every world that hears a channel. Hardware is reached only through **handles**, references held in a kernel table, with rights `r`, `w` and `g`, that a world may narrow, pass in a message, release or revoke (*Small Worlds*, Chapter 9). When a world crashes, the nucleus discards its runtime, its handles and its pending effects, and restarts it after 100, 200, 400 and then 800 ms; the fifth crash within ten seconds stops it (*Small Worlds*, Chapter 14).

The host build runs the nucleus against a simulated board, the same C code that would run on the ESP32-P4, with a simulated ST7365P panel controller and a simulated STM32 keyboard controller. Its `stats` command prints the worlds as the nucleus sees them. Here it is after two virtual seconds of an idle system:

```
$ printf 'wait 2000\nstats\nquit\n' | ./build/picocalc-host -
t = 2000.0 ms, lane busy 22.4 ms (1.1%), 140 turns, 6 published, 12 delivered, 0 crashes
  world       kind     state        turns    cpu ms   mem KB  limit  rst  last error
  keyboard    driver   idle            96     10.73    189.2    320    0
  power       driver   idle             1      0.30    186.8    320    0
  display     driver   idle             4      2.90    191.9    320    0
  compositor  service  idle            13      4.86    268.6    768    0
  launcher    app      idle             1      0.28    200.8    384    0
  sysmon      app      idle             4      0.37    206.6    512    0
  repl        app      idle             0      0.06    192.3    384    0
  sandbox     service  idle             0      0.06    193.1    512    0
  snake       app      idle            19      2.42    201.0    320    0
  faults      app      idle             0      0.07    191.5    384    0
  flaky       service  idle             0      0.06    183.4    256    0
  keys        app      idle             1      0.14    188.5    320    0
  about       app      idle             1      0.12    185.1    320    0
  bus i2c0  busy   583.80 ms, 97 transfers, 390 bytes, 0 errors
  bus spi2  busy    64.02 ms, 38 transfers, 318842 bytes, 0 errors
```

The keyboard driver has run 96 turns, nearly all of them its 20 ms poll of the STM32 over I2C. Snake has run 19, one for each 110 ms tick of its game clock (a tick does nothing while Snake is not in front). Most other worlds have run once, to start. The lane was busy for 22.4 ms of the 2,000, which is the first number to keep in mind when the costs of messages come up in Chapter 2.

The **manifest**, `js/system.json`, is the static description of the whole system. In *Small Worlds* it listed each world's file, its **claims** (the devices and system rights it may open handles for), its budget, its heap limit and, for the sandbox, its priority. After the first pass it also says which channels each world may publish and hear, which worlds it may send to and ask, and how much of the kernel's tables it may use:

▸ history/first-pass/js/system.json

```
{
  "groups": {
    "apps": ["launcher", "sysmon", "repl", "snake", "faults", "keys", "about"]
  },
  "defaults": { "handles": 4, "ask_slots": 8, "mbox": 64, "mbox_kb": 128, "bufs": 1, "turn_ms": 8000 },
  "worlds": [
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
        "budget_ms": 20,  "mem_kb": 320,
      "provides": ["keys"] },
…
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
        "budget_ms": 80,  "mem_kb": 768,
      "needs": ["frames", "keys", "power", "worlds.status"], "sends": ["display", "power", "apps"],
          "asks": ["display"],
      "handles": 6, "bufs": 2, "mbox": 128 },
…
    { "name": "repl",       "file": "apps/repl.js",
        "mem_kb": 384,
      "asks": ["sandbox"] },
    { "name": "sandbox",    "file": "system/sandbox.js",
        "budget_ms": 200, "mem_kb": 512, "priority": 0 },
…
```

Before the first pass the same two worlds were described by one line each, and nothing in the file spoke about communication:

```
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
        "budget_ms": 20,  "mem_kb": 320 },
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
        "budget_ms": 80,  "mem_kb": 768 },
```

Figure 1.1 draws the communication the new manifest declares. Every edge in it is one the shipped code uses, and the nucleus refuses any send or ask that is not an edge.

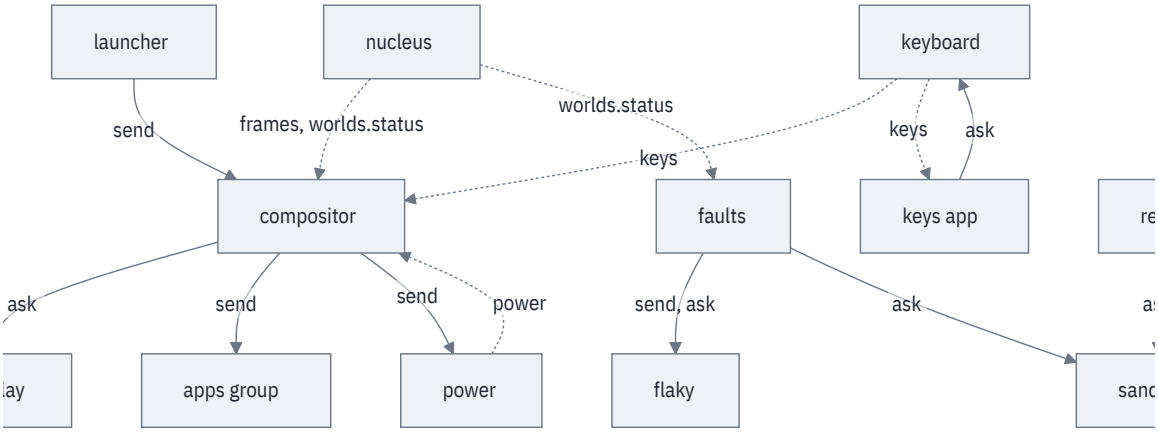


Figure 1.1. Who may talk to whom in the shipped manifest. Solid edges are sends and asks a world's entry lists; dotted edges are channels, from their one provider to the worlds that hear them. The compositor sends to the group of seven apps, whichever is in front.

Three facts about the system matter more than any other in what follows. Its isolation is language-based: the worlds and the nucleus share one address space, and nothing in hardware stops a memory-safety bug in QuickJS from reaching the nucleus (*Small Worlds*, Chapters 8 and 10). Its trusted code is the nucleus, 2,771 lines of C in seven source files and one header (font data aside), measured with `wc -l` after the first pass, against 2,374 lines in the same files before it, plus the 80,864 lines of QuickJS that every world's confinement depends on. And it runs today on the host build and in a browser, against simulated peripherals; the port to the ESP32-P4 is planned, so every time given in this book is a virtual time on the host build unless the text says otherwise.

This section has fixed the vocabulary: worlds, the nucleus and its lane, segments and turns, messages, handles, supervision and the manifest. The next section sets that vocabulary beside seL4's.

1.2 Beside seL4 and the Microkit

The comparison needs one concrete L4 system, and the natural choice is seL4 with the Microkit. seL4 is the member of the family that, in its builders' words, "pushed the L4 model furthest" [Heiser and Elphinstone 2016], and it was the first general-purpose kernel with a machine-checked proof that its C implementation refines its specification [Klein et al. 2009]. The Microkit is the layer seL4's builders recommend for simple static systems; it

is "a thin abstraction layer over seL4 that hides much of seL4's complexities" and requires an architecture in which "the set of modules and their communications is defined at system configuration time" [Heiser 2020]. That makes it the L4 system closest in shape to PicoCalc OS, whose manifest also fixes the set of components at boot. Chapter 3 and Chapter 4 describe both in detail; this section needs only the facts in Table 1.1.

A Microkit system is a set of **protection domains** (PDs), each an seL4 address space with its code, entered through `init`, `notified`, and optionally `protected` and `fault` entry points. PDs are connected by **channels**, each of which "connects exactly two PDs", and over which they exchange notifications and **protected procedure calls** (PPCs); they may also share memory regions. All of this is declared in an XML **system description file** that the `microkit` tool reads [Microkit manual 2026]. Underneath, seL4 provides capabilities to kernel objects, endpoints and notifications for communication, and untyped memory from which every kernel object is made; its builders "completely remove the kernel heap" [Heiser and Elphinstone 2016].

Table 1.1. PicoCalc OS beside seL4 with the Microkit, concern by concern. The middle column is the system *Small Worlds* describes; the right column is the system after the first pass (FIRST-PASS.md).

Concern	seL4 with the Microkit	PicoCalc OS in *Small Worlds*	PicoCalc OS after the first pass
Unit of isolation	a protection domain, built on an seL4 address space ("thin wrappers abstracting hardware page tables")	a world: a QuickJS runtime with its own heap	unchanged
What enforces it	the MMU, programmed by the kernel	the language runtime; one address space, one privilege level	unchanged
Naming a peer	a channel declared in the system description, connecting exactly two PDs; capabilities underneath	a world's name; any world may send to or ask any other	names checked against the entry's `sends` and `asks` (worlds or groups)
Channels	notifications on a declared channel	publish on any channel the world's own file says it provides	`provides` and `needs` come from the manifest; one provider per channel
Calls	a PPC only to a PD of strictly higher priority, which "prevents deadlocks"	ask anyone; two worlds that ask each other wait until a timeout	the `asks` graph must have no cycle (checked at boot)
Kernel memory	no kernel heap; kernel objects are made by retyping untyped memory the user holds	shared tables (128 handles, 1,024 asks, 16 framebuffers) and unbounded mailboxes	per-world quotas that must fit the tables at boot; mailbox quotas
Processor time	a priority, a budget and a period per PD (seL4's MCS scheduling contexts)	fixed priority by kind; a budget per segment	plus `turn_ms`, a deadline on a whole turn
Failure	a PD's fault goes to its parent PD's `fault` entry point, or to the system's monitor	supervision inside the nucleus: restart with backoff, stop after five crashes in ten seconds	unchanged
Description of the system	the system description file; the tool generates a capDL specification from it	the manifest: worlds, claims, budgets, heap limits	the manifest also fixes the graph and the quotas; any inconsistency stops the boot

Concern	seL4 with the Microkit	PicoCalc OS in *Small Worlds*	PicoCalc OS after the first pass
Trusted code	a kernel "of the order of ten thousand lines of source code"	2,374 lines of nucleus plus 80,864 of QuickJS	2,771 lines of nucleus plus the same engine
Evidence of correctness	machine-checked proofs of functional correctness and, for a correctly configured system, of integrity and confidentiality (storage channels, not timing channels); a WCET analysis for Arm v6	deterministic simulation, pixel captures, benchmarks	plus an attack suite run before and after

The seL4 column is drawn from the whitepaper [Heiser 2020], the Microkit manual [Microkit manual 2026] and the lessons paper [Heiser and Elphinstone 2016]. The PicoCalc columns are drawn from the code; each entry in the right column is a function in `nucleus/kernel.c` or `nucleus/vm.c` that later chapters quote.

Read row by row, the table separates into two kinds of difference, and the separation is the main result of this chapter. The first two rows and the last two are about how isolation is *enforced* and how strongly its correctness is *known*. There PicoCalc OS differs from seL4 by construction. It has no MMU boundary between worlds because it chose a language runtime instead, and it has no proof, because nobody has specified the nucleus formally and its isolation rests on 80,864 lines of QuickJS, about eight times the size of seL4's kernel. No change to the nucleus closes those rows, and Chapter 8 asks what can be known without them.

The middle rows are about what the system *describes*: who may name whom, who may post on a channel, which calls may wait for which, how much of the kernel's state each component may consume, and for how long a piece of work may run. In *Small Worlds*' system each of these was decided at run time by the worlds themselves, or not decided at all. In seL4 and the Microkit each is decided ahead of time, by a description no component writes, and checked by the kernel or by the tool that builds the system. None of these decisions depends on whether isolation comes from page tables or from a garbage-collected heap. That is why they can be ported, and the right-hand column records the first port. The failure row belongs with them, although the first pass left it alone: when to restart a component and when to give up is a policy, and where that policy lives is a design decision that Chapter 2 and Chapter 12 take up.

Two of the middle rows can be checked against the shipped system. The Microkit's call rule, applied to PicoCalc OS's priorities, would accept three of the five asks in the manifest: the compositor (priority 2) asks the display driver (3), Keys (1) asks the keyboard driver (3), and Faults (1) asks flaky (2). It would refuse the other two, the REPL's and Faults' asks to the sandbox at priority 0, which the host build reports at boot as `note:` lines. The graph has no cycle, so it cannot deadlock, and the first pass checks that property directly instead of the priority rule that implies it in the Microkit, because the REPL's design puts untrusted evaluation below every other world and needs the downward edge. The kernel-memory row has its numbers in the host's `snapshot` output: the thirteen worlds' handle quotas add up to 54 of the table's 128 entries, their ask slots to 104 of 1,024, and the one world with the `dma` claim, the compositor, may hold 2 of the 16 framebuffers. The unused capacity is what it costs to make exhaustion impossible, a cost Chapter 7 weighs.

1.3 What the comparison found

Small Worlds closes with an appendix of known issues, each reproduced on the host build. Read against Table 1.1, its items about authority, hang detection and uncounted memory turn out to sit in the rows where PicoCalc OS left a decision to the worlds or made none. Each has an L4 answer, and each became an attack in `bench/rogue.py` or a check in `bench/firstpass.py`, written by the authors of the first pass to demonstrate the defect on the old nucleus and its absence on the new one.

The clearest is the unauthenticated message. In the old nucleus, `op_send` looked up the receiver by name and went straight on to check the handles in the payload (*Small Worlds*, Chapter 12 quotes it). Any world could send any message type to any world it could name. The first pass put one call between the lookup and the rest:

▸ history/first-pass/nucleus/vm.c

```
static bool may_talk(JSContext *ctx, world_t *W, world_t *T, const char *type, bool ask) {
  uint64_t set = ask ? W->may_ask : W->may_send;
  if (T != W && !(set >> T->id & 1)) {
    fail(ctx, "%s may not %s %s: its manifest entry does not list it under \"%s\"", W->name, ask ?
        "ask" : "send to", T->name, ask ? "asks" : "sends");
    return false;
  }
  if (world_needs(T, type)) {
    fail(ctx, "\"%s\" is a channel %s hears; only its provider can post it, by publish", type, T-
        >name);
    return false;
  }
  return true;
}

static JSValue op_send(JSContext *ctx, world_t *W, int argc, JSValueConst *argv) {
  char *to = sarg(ctx, argv, argc, 0), *type = sarg(ctx, argv, argc, 1), *json = sarg(ctx, argv, argc,
      2);
  int hs[MAX_MSG_HANDLES], nh = argc > 3 ? int_list(ctx, argv[3], hs, MAX_MSG_HANDLES) : 0;
  world_t *T = to ? world_by_name(to) : NULL;
  JSValue r = JS_UNDEFINED;
  if (!T) r = fail(ctx, "there is no world called \"%s\"", to ? to : "");
  else if (!may_talk(ctx, W, T, type ? type : "", false)) r = JS_EXCEPTION;
…
```

The world is still named by a string, and `world_by_name` still finds it; what changed is that the name is now checked against a bit set, `may_send`, that `kernel_boot` computed from the manifest before any world ran. The second test in `may_talk` closes a subtler hole. A world entitled to send to the compositor could still send it a message whose type was `frames`, the channel on which the nucleus delivers every app's screen, and the compositor's handler could not tell a forged frame from a real one. A type that names a channel the receiver hears may now arrive only by `publish`, from the channel's one provider. This is the L4 lesson in its smallest form. L4 kernels once addressed messages to global thread identifiers, and later kernels replaced them with endpoints, because the global identifiers gave "poor information hiding" and "introduced covert channels" [Heiser and Elphinstone 2016]. PicoCalc OS has not yet gone as far as endpoints (Chapter 12 takes that step), but it has stopped treating a name as permission.

The attack suite shows the difference directly. The same rogue worlds run against the nucleus from before the pass and against the current one:

▸ bench/rogue-before.txt

```
forge_frame_allowed: the same from a world allowed to send to the compositor
  ATTACK SUCCEEDED: [  1502.390] compositor <- frames from rogue_frame2
      {"app":"launcher","title":"Home","frame":{"tree":{"t":"text","s":"forged by rogue","kids":
      []},"hints":""}}

flush_display: ask the display driver to flush the whole screen
  ATTACK SUCCEEDED: [  1543.872] rogue_flush: OUTCOME flush_display answered
      {"px":102400,"ms":41.399000000000115}
```

▸ bench/rogue-after.txt

```
forge_frame_allowed: the same from a world allowed to send to the compositor
  REFUSED: [  1502.340] rogue_frame2: OUTCOME forge_frame refused: "frames" is a channel compositor
      hears; only its provider can post it, by publish

flush_display: ask the display driver to flush the whole screen
  REFUSED: [  1502.336] rogue_flush: OUTCOME flush_display refused: rogue_flush may not ask display:
      its manifest entry does not list it under "asks"
```

The second attack is Hardy's confused deputy [Hardy 1988]. The display driver holds the SPI handle and the compositor's framebuffer view, and before the pass it flushed all 102,400 pixels, reporting 41.4 ms for the flush, for any world that asked, because it had no way to know whose authority the request carried. The fix makes the deputy unreachable, since only the compositor may now ask the display driver anything.

Table 1.2 lists all six defects the suite and the checks exercise, with the L4 idea behind each fix and the chapter that tells the story.

Table 1.2. The known issues of *Small Worlds* Appendix D, with the deadlock of its Chapter 3, the attack or check that demonstrates each (bench/rogue-before.txt, bench/rogue-after.txt, bench/firstpass-after.txt), the L4 idea behind the fix, and the chapter that describes it.

Known issue	Before the first pass	L4 idea	After	Chapter
Self-declared channels	a world with no claims declared `provides("keys")` and its forged key reached the compositor	every channel is declared in a system description that no component writes [Microkit manual 2026]	`provides` and `needs` from the manifest; a world whose file declares more is stopped at start	Chapter 5
Unauthenticated messages	a forged `frames` message replaced the launcher's screen; the display driver flushed the screen for a stranger	endpoints instead of global thread identifiers [Heiser and Elphinstone 2016]; the confused deputy [Hardy 1988]	`sends` and `asks`; the channel-type rule	Chapter 5
Handle-table exhaustion	15 framebuffers, then 108 narrowings, filled the shared tables; after a crash the compositor ended up stopped	no kernel heap: kernel objects come from memory the user holds [Elphinstone and Heiser 2013]	per-world `handles` and `bufs` quotas that must fit the tables	Chapter 7
Uncounted mailbox memory	one world queued 3,000 messages in another's mailbox in one turn	the same; and the RC 4000 nucleus limited the messages a process could have outstanding [Hansen 1970]	`mbox` and `mbox_kb` quotas; a post to a full subscriber is dropped and counted	Chapter 7
Ask cycles	an ask between two worlds that ask each other failed only at its 2,000 ms timeout	a protected call only to strictly higher priority, which prevents deadlocks [Microkit manual 2026]	the boot refuses a cycle in the `asks` graph and prints it	Chapter 6
No hang detection between segments	a turn waiting on a promise that never settled was never noticed	processor time as an object with a budget, the MCS scheduling context [Heiser 2020]	`turn_ms`: a turn unfinished 8,000 ms after it began is a crash	Chapter 6

Two rows need a caution. The last one borrows a principle more than a mechanism: seL4's scheduling contexts bound how much processor time a thread may consume in each period, while `turn_ms` bounds how long a turn may remain unfinished, which is closer to a watchdog. And L4 abandoned IPC timeouts because there is "no theory, or even good heuristics, for choosing timeout values" [Elphinstone and Heiser 2013], yet PicoCalc OS keeps both an ask timeout and now a turn deadline. Chapter 6 argues why a system whose turns can await promises needs them anyway, and why the 8,000 ms default must exceed the REPL's 5,000 ms ask timeout.

The comparison also points at decisions the first pass left with the worlds. The known issues file names two: authority is per world, not per message type, so the launcher may send the compositor `open` for any app; and a framebuffer's pixels still do not count against its owner's heap limit. A third was found while this chapter was being checked against the code. A world's priority comes from the manifest only if its entry has a `priority` key, and only the sandbox's does. Every other world gets its priority from the kind its own file declares, in `boot_world`:

▸ history/first-pass/nucleus/kernel.c

```
if (W->prio < 0) W->prio = !strcmp(W->kind, "driver") ? 3 : !strcmp(W->kind, "service") ? 2 : 1;
```

A file that begins `driver(self)` therefore runs at driver priority whatever the manifest intended. A two-world manifest in a scratch directory shows it: an entry with no `priority` key, whose file declares itself a driver, is reported by the `snapshot` command as `"kind":"driver"` and `"priority":3`. In a Microkit system description a PD's priority is an attribute of the PD in the description [Microkit manual 2026], so no component chooses its own. Exercise 4 measures what a self-promoted world can do to the keyboard's latency.

The six rows of Table 1.2, and the three items after it, establish the claim of the previous section in detail. Every authority and resource defect PicoCalc OS was known to have sat in a row where seL4 and the Microkit decide ahead of time and PicoCalc OS decided at run time.

1.4 What transfers and what does not

A reader of the L4 literature meets many mechanisms, and only some of them make sense in a system whose components are JavaScript runtimes in one address space. The test that sorts them follows from the previous two sections. A mechanism transfers when its purpose is to decide authority or account for a resource, because those decisions are the same whatever enforces isolation. A mechanism does not transfer when its purpose is to make hardware-enforced isolation cheap or to prove it correct, because PicoCalc OS has no hardware-enforced isolation to make cheap and no specification to prove against. Table 1.3 applies the test.

Table 1.3. L4 ideas sorted by whether they transfer to a system isolated by a language runtime, with where this book takes each up.

L4 idea	What it is for in L4	Transfers?	In this book
Isolation by address spaces	a driver bug cannot write another component's memory	no	the language does it; fencing the nucleus with RISC-V PMP is a sketch (Chapter 12)
Fast synchronous IPC	make the cost of crossing an address-space boundary small	the techniques no; the discipline of measuring the path yes	Chapter 2
A machine-checked proof	know that the kernel implements its specification	no	Chapter 8
Capabilities instead of global names	authority travels only with a reference that grants it	partly	names checked against the manifest (Chapter 5); endpoint capabilities (Chapter 12)
A static description of the system	every component and connection fixed and checked before anything runs	yes	the manifest (Chapter 5, Appendix A)
No kernel heap	no user can exhaust kernel memory that others need	yes, as quotas	Chapter 7
Calls only to higher priority	the call graph cannot deadlock	yes, as an acyclic `asks` graph	Chapter 6
Budgets as objects	processor time is granted and bounded like any other resource	partly	`budget_ms` and `turn_ms`; charging an ask's work to the asker (Chapter 12)

The first three rows deserve a sentence each, because they are where a careless port would go wrong. The ESP32-P4 has region-based physical memory protection, but ESP-IDF gives tasks no per-task page tables, and a few PMP regions cannot describe thirteen heaps scattered through PSRAM (*Small Worlds*, Chapter 8). Isolation by address spaces is therefore unavailable in the form L4 uses it. L4's IPC techniques (messages in registers, a direct switch to the receiver, a short kernel path) attack the cost of a hardware context switch and a TLB refill; a PicoCalc message costs about 56 µs one way on the virtual clock, almost all of it the cost of entering a JavaScript runtime and running the prelude's dispatch code, and none of

those techniques touches that cost. What transfers is Liedtke's habit of measuring the path a message takes and designing for it, which Chapter 2 applies. And the proofs depend on a formal specification of the kernel and a C semantics for its code; PicoCalc OS's isolation depends on QuickJS, for which neither exists.

The remaining rows are the ones the first pass took. Each became a manifest key, a boot-time check in `kernel_boot`, and a run-time check in `vm.c` or `kernel.c`. None changed the API a world is written against, so the thirteen shipped worlds needed no edits. The first pass's verification confirms this: the 22 captures of the native 32-bit and 64-bit builds are pixel-identical to the builds before it, and every virtual-time result of `bench/run.py` is unchanged [First pass 2026]. A world that never tried to exceed its authority cannot tell that the checks are there.

Two rows are marked partly, for the same reason: the first pass took the static half of the idea and left the dynamic half. A capability both names an object and carries the right to use it, and it can be passed, narrowed and revoked while the system runs (*Small Worlds*, Chapter 9). A `sends` list names whom a world may reach, but it is fixed at boot; no world can give another the right to talk to it, take that right back, or limit it to one message type. In seL4's MCS model processor time is itself a capability, a scheduling context with a budget and a period [Heiser 2020]. PicoCalc OS's budgets belong to the world that runs, so when the REPL asks the sandbox, the evaluation is charged to the sandbox and runs at the sandbox's priority. *Small Worlds* (Chapter 12) traces one such ask that took 1.6 ms from send to resumption, against 119 µs on an idle lane, almost all of it spent waiting behind higher-priority worlds. Both dynamic halves need new objects that exist at run time, which the first pass deliberately avoided, and Chapter 12 designs them.

The test of this section, authority and accounting transfer while enforcement and proof do not, is the one the rest of the book uses to decide what to borrow next.

1.5 The method of this book

Each borrowing in this book follows the same four steps, shown in Figure 1.2. The steps are simple; the discipline is in never skipping one.

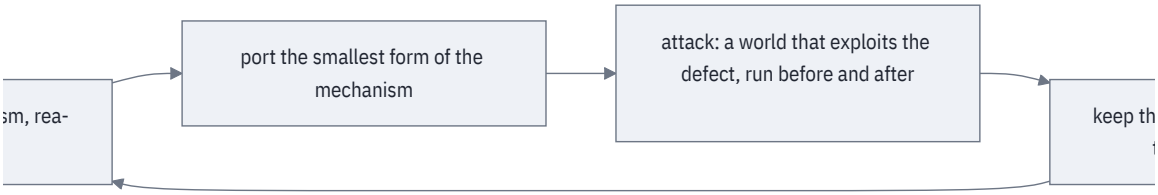


Figure 1.2. The four steps every borrowing in this book follows. The evidence kept at the last step is what the next reading is checked against.

Read the paper. The reading looks for three things in a source: the mechanism, the reason the authors give for it, and the evidence they offer that it works. The reason matters most, because it decides whether the mechanism transfers by the test of Section 1.4. The Microkit's rule that a PPC may only go to strictly higher priority comes with its reason in the manual, that the restriction "prevents deadlocks" [Microkit manual 2026]. That reason holds for any system in which a caller waits for its callee, which PicoCalc OS's asks do, so the rule transfers; the priority mechanism that enforces it in the Microkit does not, and Chapter 6 explains why PicoCalc OS checks the graph instead.

Port the smallest form. A port changes the nucleus as little as the mechanism allows and changes nothing a well-behaved world can observe. The first pass added 397 lines to the nucleus and no operation to the API a world is written against. Its new state lives in fields of `world_t`, `msg_t` and `dmabuf_t` in `nucleus/nucleus.h`, and its checks run once at boot or inside operations that already existed: a send, an ask, the start of a world, the moment a turn first awaits.

Attack it. For every defect, a world is written that exploits it, using only the authority the defect gives it. The attack is run against the nucleus from before the change, where it must succeed, and against the nucleus after it, where it must fail with a message a person can act on. `bench/rogue.py` holds six such attacks, one of them in two forms; `python3 bench/rogue.py` runs them against the current build and `--bin` against another. The first is short enough to show whole. It is an app with no claims that declares itself the provider of `keys` and, a moment after starting, publishes a down-arrow:

▸ bench/rogue/rogue_key.js

```
app(self).provides("keys").onStart(async () => {
  await sleep(1500);
  try {
    publish("keys", { code: 0xb6, key: "down", ch: "", state: "down", caps: false });
    log("OUTCOME forge_key published a down-arrow on keys");
  } catch (e) { log("OUTCOME forge_key refused: " + e.message); }
});
```

Against the old nucleus the compositor handled the forged key as it would a real one. Against the current one the world never reaches its `sleep`: `read_meta` in `nucleus/vm.c` finds that the file declares a channel its manifest entry does not give it, and the world is stopped at start with that reason, which is the first verdict in `bench/rogue-after.txt`. The rows of Table 1.2 were all reproduced this way. An attack that cannot be made to succeed on the old code shows that the defect was misunderstood, so the run before the change is as much a part of the evidence as the run after it.

Keep the evidence. The outputs of each run are kept in the repository beside the code they test (`bench/rogue-before.txt`, `bench/rogue-after.txt`, `bench/firstpass-after.txt`), together with the checks that the rest of the system did not change: captures compared pixel for pixel across builds, virtual-time benchmarks compared number for number. Because the host build is deterministic, the same script gives the same pixels and the same times on every run, so a difference is a change in behaviour and never noise. The workbench of Part III turns this step into a tool, with evals that boot a fresh system for each check (Chapter 11).

The method is a weaker form of what the seL4 team did. They specified the kernel, proved the code against the specification, and kept the proofs as the evidence [Klein et al. 2009]. PicoCalc OS states its invariants in prose and code, attacks them, and keeps the outputs. Chapter 8 measures how much weaker that is.

1.6 The plan of the book

Part I reads the research. Chapter 2 follows the microkernel idea from Brinch Hansen's RC 4000 nucleus through Mach to Liedtke's L3 and L4, the family that grew from them and the concepts twenty years of deployment kept and abandoned, and reads PicoCalc OS's nucleus

with Liedtke's minimality principle. Chapter 3 describes seL4's object model, its kernel memory from untyped, its IPC and capability derivation, its proofs and what they assume, and its mixed-criticality and time-protection work. Chapter 4 describes the static systems built on seL4, the Microkit and LionsOS, and ends with a table that matches each Microkit concept with its PicoCalc counterpart.

Part II is the first pass. Chapter 5 makes the manifest the source of authority over channels and messages, and Chapter 6 makes the ask graph acyclic, takes timed-out asks out of mailboxes and adds turn deadlines. Chapter 7 replaces the shared tables with per-world quotas. Chapter 8 asks what PicoCalc OS can know about itself without a proof, and how the first pass was shown to change nothing it should not. Each chapter in this part shows the defect with an attack, the L4 idea, the nucleus code and the attack refused.

Part III is the workbench. Chapter 9 compiles the same nucleus and QuickJS to WebAssembly and runs them in a page beside views of the kernel's tables. Chapter 10 describes a coding agent that works through a single tool, and Chapter 11 the eval suite that boots a fresh system for every check. Chapter 12 closes the book with the second pass: endpoint capabilities, asks charged to the asker, one driver per device, supervision moved out of the nucleus, and fencing the nucleus with PMP, each with a design and the evals that would test it. Appendix A documents the manifest and every message the nucleus prints about it, and Appendix B the workbench's API.

The parts depend on each other in one direction. Each chapter of Part II takes its L4 idea from Part I: Chapter 5 from seL4's endpoints (Chapter 3) and the Microkit's channels (Chapter 4), Chapter 6 from the Microkit's call rule and from L4's history with IPC timeouts (Chapter 2), Chapter 7 from seL4's untyped memory, and Chapter 8 from seL4's proofs and what they assume. Part III depends on Part II for its subject. The workbench's Graph view draws the communication graph that Chapter 5 puts in the manifest, and the eval suite of Chapter 11 includes the first pass's security checks, each run on a freshly booted system. Chapter 12 then returns to Part I for the ideas the first pass left aside, so the book ends where it began, with a reading of the research, now against a system that has absorbed part of it.

KEY POINTS

- PicoCalc OS has a microkernel's decomposition into components that communicate only by messages, a unikernel's single address space, and isolation enforced by one QuickJS runtime per world.
- seL4 with the Microkit is the closest L4 system in shape: protection domains, channels between exactly two of them, protected calls only upward in priority, and a system description fixed before the system runs.
- The differences between the two systems are of two kinds: how isolation is enforced and proved, which PicoCalc OS cannot change, and what the system describes ahead of time, which it can.
- Every authority and resource defect *Small Worlds* recorded (self-declared channels, unauthenticated messages, handle-table exhaustion, uncounted mailboxes, ask cycles, no hang detection) sat where the Microkit decides ahead of time and PicoCalc OS decided at run time.
- `may_talk` in `nucleus/vm.c` is the first pass's smallest borrowing: a name is still looked up, but it is checked against a bit set computed from the manifest, and channel types may arrive only by publish.
- An L4 idea transfers when it decides authority or accounts for a resource, and does not when it makes hardware isolation cheap or proves it correct.
- Some decisions remain with the worlds after the first pass, among them the per-type authority of a send and a world's priority, which its file's declared kind sets unless the manifest gives one.
- Every borrowing follows four steps: read the paper for mechanism, reason and evidence; port the smallest form; attack the old and new code; keep the outputs as evidence.

The comparison with seL4 and the Microkit gives PicoCalc OS a map of its own weaknesses, and the test of Section 1.4 says which parts of the L4 literature can repair them. The next chapter goes back to the start of that literature, to see how microkernels arrived at the designs this chapter compared against, and what their builders learnt about cost and minimality on the way.

EXERCISES

1. BASIC From `js/system.json`, list for each world the worlds it may send to, the worlds it may ask, and the channels it hears, expanding the group `apps`. Which asks go to a world of lower priority? Check your answer against the `note:` lines the host build prints at boot with `-v`.

2. BASIC For each row of Table 1.1, say whether the difference between the seL4 column and the PicoCalc columns is one of enforcement and proof or one of description, and justify each answer in one sentence.

3. ANALYSIS The shipped manifest's handle quotas add up to 54 of the kernel's 128 entries and its ask slots to 104 of 1,024; only the compositor holds the `dma` claim, with 2 of the 16 framebuffers. How many more worlds with the default quotas can be added before the boot fails, and which limit stops it first? Check by adding worlds to a copy of the manifest and booting with `-m`.

4. CODE In a scratch copy of `js/`, add a world whose file begins `driver(self)` and which runs a busy loop of about 40 ms every 50 ms, with no `priority` in its manifest entry. Measure the keyboard's key-to-pixels latency with the `latency` command with and without the world, then add `"priority": 1` to its entry and measure again. Explain the three results with `pick` in `nucleus/kernel.c`.

5. CODE In a scratch copy of the repository, make `may_talk` in `nucleus/vm.c` return `true` at once, rebuild, and run `python3 bench/rogue.py --bin` on the new binary. Predict before running which of the seven scenarios succeed, and explain why the others are still refused.

6. ANALYSIS A Microkit channel connects exactly two PDs, while PicoCalc OS's `keys` channel has one provider and two worlds that hear it. Describe how the same flow of key events would be expressed in a Microkit system description: how many channels, which notifications, and what would carry the key data. What does PicoCalc OS's channel give up by comparison?

7. DISCUSSION Suppose PicoCalc OS ran each world in its own hardware address space. Which rows of Table 1.2 would that have fixed, and which would remain? What does the answer say about the relation between isolation and authority?

8. PROJECT The known issues file says a world allowed to send to another may send it any type that is not a channel; the launcher may send the compositor `open` for any app. Write a seventh scenario for `bench/rogue.py` that demonstrates this with a world that holds only `"sends": ["compositor"]`, and decide from the compositor's code what evidence in the kernel log should count as success. Then sketch the manifest syntax that would refuse it.

9. DISCUSSION seL4 names a communication partner with an endpoint capability; PicoCalc OS names it with a string checked against a bit set computed at boot. List three things an endpoint capability can express that the `sends` list cannot, and for each, say whether the shipped system needs it today.

10. BASIC The first pass claims that a well-behaved world cannot tell the checks are there. Name two observations that would refute the claim, and the command or script in the repository that makes each observation.

FURTHER READING

[Elphinstone and Heiser 2013] The SOSP retrospective on twenty years of L4: what the family kept, what it replaced and what it abandoned, with the reasons. The source of most of the lessons this book borrows.

[Heiser and Elphinstone 2016] The longer journal version of the same retrospective, with the family tree, the deployments and a fuller account of each design decision.

[Heiser 2020] The seL4 Foundation's introduction to seL4: capabilities, the proofs and their assumptions, mixed criticality, the Microkit and the deployments, in one document.

[Microkit manual 2026] The Microkit manual: protection domains, channels, protected calls and the system description file. Short, and the closest published design to PicoCalc OS's manifest.

[Hardy 1988] Three pages on the confused deputy, the failure the display driver's `flush` handler reproduced.

[Small Worlds 2026] Chapters 8, 9, 12 and 14 and Appendix D describe the system this chapter compares, its handles, its messages, its supervision and its known issues.

[First pass 2026] The first pass in one file: the manifest schema, every new message, the code changes and the verification.

CHAPTER 2 PART I · THE L4 LINEAGE

From the nucleus to L4

A microkernel keeps in the kernel only what cannot be done anywhere else, and moves the rest into components that communicate by messages. The idea is older than the word, and its history is largely a history of cost. Each generation of microkernels was judged by what a message cost, and the designs that survived were shaped by that judgement. This chapter follows the idea from Per Brinch Hansen's nucleus for the RC 4000, through Mach and the measurements that made first-generation microkernels look inherently slow, to Jochen Liedtke's L3 and L4, the family of kernels that grew from them, and what that family kept and abandoned after twenty years of deployment. It ends with two readings of PicoCalc OS: its nucleus examined with Liedtke's minimality principle, and its messages costed beside L4's. By the end the reader should be able to explain why the first microkernels were slow, what Liedtke changed and why, which of L4's original decisions survived, which parts of PicoCalc OS's nucleus are policy by Liedtke's test, and how far a PicoCalc message is from an L4 IPC in cost, and for what reasons.

2.1 The RC 4000 nucleus

Brinch Hansen's 1970 paper on the RC 4000 multiprogramming system opens with a complaint about operating systems built around one way of running programs (batch processing, priority scheduling, real-time scheduling or conversational access), which could not be changed without replacing them. His conclusion was that the main problem was not to define the functions of one operating system but to supply a nucleus "that can be extended with new operating systems in an orderly manner" [Hansen 1970]. The nucleus he and his

colleagues built for Regnecentralen's RC 4000 provided processes, a hierarchy in which a parent could create, start, stop and remove its children, and communication between processes. Scheduling strategy and resource allocation were left to ordinary programs, which could be replaced while the system ran; the paper notes that "the difference between operating systems and production programs is one of jurisdiction only".

Communication was by messages, and the paper explains why. Semaphores were sufficient "from a logical point of view", but did not meet the system's "requirements of safety and efficiency" in an environment where some processes "may turn out to be black sheep and break the rules of the game". The nucleus therefore administered "a common pool of message buffers and a message queue for each process", with four primitives: send message, wait message, send answer and wait answer. A process that sent a message was told the identity of the buffer that carried it, and the answer came back in the same buffer, so that "the selection of a buffer" was "the creation of an identification of a conversation". The nucleus stored the sender and the receiver in each buffer and checked them whenever a process tried to answer or wait for an answer in it, so that no process could interfere with a conversation between two others.

Three further rules read today as answers to questions PicoCalc OS also had to answer. A message to a process that did not exist received "a dummy answer generated by the system nucleus". When a process was removed, the nucleus found the unanswered messages sent to it, and "these are returned as dummy answers to the senders". And the pool of buffers was shared, so one process could exhaust it by sending messages to "ignorant processes, which do not respond with answers", blocking all further communication; therefore "a limit is set to the number of messages a process can send simultaneously". With the answer travelling in the buffer the request arrived in, this "placed the entire risk of a conversation on the process that opens it". Table 2.1 sets the RC 4000's rules beside PicoCalc OS's.

Table 2.1. The RC 4000 nucleus's message rules [Hansen 1970] and their counterparts in PicoCalc OS's nucleus.

RC 4000 nucleus	PicoCalc OS
send message, then wait answer	`ask`, which returns a promise of the reply
wait message	the lane runs the receiver's handler with the next message in its mailbox
send answer, in the received buffer	the handler's return value, which settles the asker's promise through its entry in the ask table
a buffer identifies a conversation	an ask's number identifies it; the entry records the asker and its epoch
dummy answer for a nonexistent receiver	`there is no world called "X"`, thrown at the call
dummy answers for messages to a removed process	`fail_mailbox` rejects every waiting ask with `X is stopped`
a limit on the messages a process can have outstanding	the `ask_slots` quota, added by the first pass

The last row is the one PicoCalc OS lacked until the first pass. Its 1,024-entry ask table was shared by every world, and nothing stopped one world from filling it. The first pass gave each world a quota of waiting asks, checked in `kernel_ask` before the message is made:

▸ history/first-pass/nucleus/kernel.c

```
int kernel_ask(world_t *from, world_t *to, const char *type, const char *json, const int *hs, int nh,
    int promise, vtime_t timeout) {
  if (to->status == W_STOPPED) {
    char t[160]; snprintf(t, sizeof t, "%s is stopped", to->name);
…
  if (from->used_asks >= from->q_asks) {
    char t[160]; snprintf(t, sizeof t, "%s already has %d asks waiting for replies, its quota", from-
        >name, from->used_asks);
    char *j = json_string(t);
    defer_settle(from, promise, 0, j, 0);
    free(j);
    return 0;
  }
…
  ask_t *a = &K.asks[id % MAX_ASKS];
  *a = (ask_t){ .id = id, .from = from->id, .from_epoch = from->epoch, .to = to->id, .promise =
      promise, .live = true, .deadline = timeout };
  from->used_asks++;
  m->ask = id;
  for (int k = 0; k < nh; k++) m->hs[m->nh++] = hs[k];
  defer_deliver(to, m);
  deferred_t *d = calloc(1, sizeof(*d));
  d->run = run_ask_timer; d->op = id; d->delay = timeout;
  defer(d);
  return id;
}
```

A world over its quota gets a rejected promise and nothing else happens, so the risk of an ask stays with the asker, as it did with the process that opened an RC 4000 conversation. The sum of all quotas must fit the table, a check the RC 4000 did not need because its limits were set by each parent from its own resources. Chapter 7 tells the rest of that story.

The nucleus was small, and its conversations were expensive for its machine. Its primitives occupied 2,400 words of the RC 4000's 24-bit store, and the whole nucleus with its process descriptions and buffers 4,800 words. A complete conversation, the four primitives together, took 2 ms on a computer whose typical instruction took 4 µs, about 500 instruction times. Brinch Hansen's analysis divided those 2 ms among validity checking (25%), process activation (45%) and message buffering (30%), and concluded that the distribution was too even for "additional, ad hoc machine instructions" to help: "The only realistic solution is to make the hardware faster." Two decades later Liedtke would reach the opposite conclusion about the microkernels of his time.

The RC 4000 nucleus established the pattern the rest of this chapter follows: messages as the only communication, the nucleus as the keeper of buffers and of the identity of each conversation, policy in replaceable processes, and a bound, per process, on the nucleus's shared buffers.

2.2 Mach and the price of IPC

Mach, developed at Carnegie Mellon in the 1980s, made the nucleus idea the basis of a UNIX-compatible system. Its kernel provided tasks, threads, ports and messages, and moved the management of paging into user-level external pagers [Accetta et al. 1986]; *Small Worlds,* Chapter 12, describes its ports and the semantics of its messages [Small Worlds 2026]. Mach became the reference against which microkernels were measured, and the measurements were unfavourable.

Liedtke summarised them in 1996. Each request to a service in a microkernel system is a remote procedure call, two IPCs. "Steady progress yielded up to twofold improvement in speed", but by 1991 the gains had dwindled, and "Mach 3 stabilized at about 115 µs per IPC on a 486-DX50". A conventional Unix system call, roughly 20 µs on the same processor, had "about 10 times less overhead than the Mach RPC", and "it seemed that 100 µs was the inherent cost of an IPC" [Liedtke 1996]. Mach itself was large, with "approximately 140 system calls" and "more than 300 Kbytes of code". Its builders, and those of Chorus, answered the cost by moving "the most critical servers and drivers" back into the kernel, as Mach's kernel-loaded tasks and Chorus's supervisor actors.

Two papers drew conclusions from the same evidence. Bershad argued in 1992 that IPC performance was becoming increasingly irrelevant to the performance of microkernel-based systems as a whole [Bershad 1992]. Chen and Bershad then traced thirteen workloads on DEC's Ultrix and on Mach 3.0 with CMU's UNIX server, and measured each system's memory cycles per instruction (MCPI), the average number of cycles an instruction spends waiting for the memory system. They found that the microkernel-based system executed "substantially more non-idle system instructions for an equivalent workload", that the average instruction under Mach had a higher MCPI than under Ultrix, and that optimisations "focusing exclusively on IPC", without regard to other overheads such as MCPI, "will have a limited impact on overall system performance" [Chen and Bershad 1993, p. 125].

Liedtke's own measurements located part of the overhead precisely. On a 486 at 50 MHz, Mach's `mach_thread_self` call, which does little more than enter and leave the kernel, took 18 µs, "whereas the bare machine time for user—kernel—user is 2 µs"; using Mach's system-call path alone would have quadrupled the cost of L3's 8-byte IPC [Liedtke 1993]. The overhead was in the kernel's path, before any message was moved.

By the early 1990s the evidence therefore admitted two readings. Either the microkernel idea carried an inherent cost, in messages and in memory-system behaviour, which a system should accept or avoid by moving services back into the kernel, or the cost belonged to how the first microkernels had been built. Liedtke set out to show the second.

2.3 Liedtke's argument

Liedtke's L3 was a complete operating system, with persistence built in, developed at GMD from the mid-1980s for i386 machines [Heiser and Elphinstone 2016]. His 1993 paper describes the rebuilding of its process and communication sections for speed; it is a paper about L3, the predecessor of L4. Its abstract states the problem with one number and the result with another: microkernels typically required "100 µs for a short message transfer on a modern processor" at 50 MHz, and "we achieve 5 µs; a twentyfold improvement". Against Mach on the same machine the gains ranged "from a factor of 22 (8-byte messages) to 3 (4-Kbyte messages)" [Liedtke 1993].

The paper's method is stated as a list of principles for the redesign. "Ipc performance is the Master": anything that might make IPC faster had to be discussed, and "in case of doubt, decisions in favour of ipc have to be taken". "All design decisions require a performance discussion." "The design has to cover all levels from architecture down to coding." And "the design has to aim at a concrete performance goal", because a goal is what exposes weak techniques. The abstract adds that "there is no single trick to obtaining this high performance"; instead "a synergetic approach in design and implementation on all levels is needed". The result was a kernel designed around its message path, measured at every level from the choice of abstractions to the order of instructions.

Two years later Liedtke turned from how to build a microkernel to what to put in it. "On µ-kernel construction" states the principle that every later L4 kernel cites:

> **DEFINITION: MINIMALITY PRINCIPLE**
>
> A concept is admitted to the kernel only if moving it outside, and so permitting competing implementations, "would prevent the implementation of the system's required functionality" [Liedtke 1995].

The paper is explicit that the "determining criterion used is functionality, not performance". Applied to a system with protection and page-based virtual memory, the principle leaves three things in the kernel: address spaces, constructed recursively outside the kernel with three operations (grant, map and flush) so that memory managers and pagers can be ordinary programs; threads with IPC; and unique identifiers. The same paper answers the performance objections with numbers. Its table of one-byte remote procedure calls puts L3 at 10 µs for the round trip on a 486 at 50 MHz, 250 cycles per one-way IPC, and Mach on the same processor at 230 µs, 5,750 cycles; the other systems in the table with full IPC semantics (QNX, Mach on two other processors, SRC RPC, Amoeba and Spin) range from 1,254 to 6,916 cycles one way. Its reanalysis of Chen and Bershad's data argues that the additional cache misses came from "Mach itself, including trap handling, IPC and memory management", and concludes that "properly constructed µ-kernels will automatically avoid the memory system degradation measured for Mach". The abstract puts the whole claim in one sentence: the inefficiency and inflexibility of current microkernels "is not inherited from the basic idea" but comes "mostly from overloading the kernel and/or from improper implementation" [Liedtke 1995].

The 1996 article in *Communications of the ACM* presented the argument to a general readership [Liedtke 1996]. Table 2.2 collects its comparison of the first and second generations.

Table 2.2. Mach 3 and L4 compared by Liedtke [Liedtke 1996]; the cycle counts are from his table of one-byte remote procedure calls [Liedtke 1995], whose entry on the L4 side is L3. All IPC times are on an Intel 486 at 50 MHz.

	Mach 3	L4
System calls	about 140	7
Kernel code	more than 300 KB	12 KB
One IPC, 8 bytes	115 µs	5 µs
One IPC, 512 bytes	172 µs	18 µs
One-way IPC in a one-byte RPC	5,750 cycles	250 cycles

"With 2 x 5 µs," Liedtke wrote, the basic L4 RPC was "twice as fast as a conventional Unix system call". The same article locates the first generation's lack of flexibility in one sentence: "Conceptually, the weakness is the policy inside the microkernel."

Liedtke's argument has two halves that are easy to run together. The minimality principle decides *what* belongs in the kernel, and its test is function. The performance discipline decides *how* to build what remains, and its test is measurement down to the instruction. PicoCalc OS can apply the first half directly (Section 2.7). The second applies only in a general form, because the cost of a PicoCalc message lies somewhere Liedtke's techniques do not reach (Section 2.8).

2.4 L4Linux and the performance of a whole system

A fast IPC is a microbenchmark result, and Chen and Bershad's objection was about whole systems. Härtig and his colleagues at Dresden and IBM answered it in 1997. They ported Linux to run as a user-level server on L4, as L4Linux, and compared it with native Linux and with MkLinux, Linux on a Mach-derived kernel, in two versions: one whose Linux server runs in user mode, and one co-located in the kernel's address space [Härtig et al. 1997]. The abstract gives the result. On the AIM benchmarks, L4Linux's maximum throughput was "only 5% lower than that of native Linux", and the corresponding penalty was five times higher for MkLinux co-located in the kernel and seven times higher for MkLinux at user level. Recompiling the Linux server, L4Linux was 6–7% slower than native Linux and 10–20% faster than both versions of MkLinux.

The microbenchmarks show where the difference came from. Table 2.3 gives the paper's measurement of `getpid`, the shortest Linux system call, invoked in a tight loop on a 133 MHz Pentium. Under L4Linux a system call becomes an IPC to the Linux server and its reply.

Table 2.3. The cost of getpid on a 133 MHz Pentium [Härtig et al. 1997]. The trampoline variant enters the Linux server through a trampoline instead of the shared emulation library.

System	Time	Cycles
Linux	1.68 µs	223
L4Linux	3.95 µs	526
L4Linux, trampoline	5.66 µs	753
MkLinux, server in the kernel	15.41 µs	2,050
MkLinux, server in user mode	110.60 µs	14,710

L4Linux needed about 300 cycles more than native Linux for the call, and MkLinux needed 3.9 times (in-kernel) or 29 times (user mode) as many cycles as L4Linux. The paper states the limit of its own result too: "L4Linux still needs 2.4 times as many cycles as native Linux." A fast kernel narrowed the gap to a few per cent on whole workloads, but it did not close it on the shortest path.

One remark in the paper's discussion of method bears on everything this book reports. Explaining why its microbenchmarks show very small standard deviations, the authors note that "a deterministic system is being measured that does not behave stochastically", that "for fast operations, most measurement errors are systematic", and that they therefore report several different microbenchmarks, whose disagreement gives "an impression of the absolute error". PicoCalc OS's host build is deterministic by construction: the virtual clock gives the same times for the same script on every run. A standard deviation over repeated runs of one PicoCalc script is zero and says nothing about accuracy. The systematic error lies in the clock's calibration, one parameter fitted to one measurement on the ESP32-P4 (*Small Worlds*, Chapter 18), and Chapter 8 returns to it.

L4Linux settled the whole-system question for its time: a conventional operating system could run as a server on a fast microkernel within a few per cent of native speed, and the speed of the kernel's IPC decided how close it came.

2.5 The L4 family

After 1995 L4 was less a single kernel than an interface that several groups implemented, and Heiser and Elphinstone's account of the family is the source for this section [Heiser and Elphinstone 2016]. Liedtke first used the name L4 for the "V2" interface, circulated from 1995, and the original kernel was written in assembler for the i486. At TU Dresden a new kernel for the same interface, Fiasco, became the first L4 written almost entirely in a higher-level language, C++; it later became Fiasco.OC, with object capabilities. Liedtke moved to Karlsruhe, where Hazelnut was the first L4 kernel ported to a second architecture, and where, after his death in 2001 [KIT n.d.], his students implemented a revised interface, V4, in the open-source L4Ka::Pistachio. At UNSW and NICTA, Pistachio was forked as L4-embedded for resource-constrained systems; Qualcomm adopted it as a protected-mode real-time operating system for the firmware of its wireless modem processors, and OK Labs renamed it OKL4. NOVA, also from Dresden, was designed from scratch for hardware-supported virtualisation on x86 [Steinberg and Kauer 2010], and seL4 was designed from scratch for formal verification [Klein et al. 2009]. Figure 2.1 draws these relations.

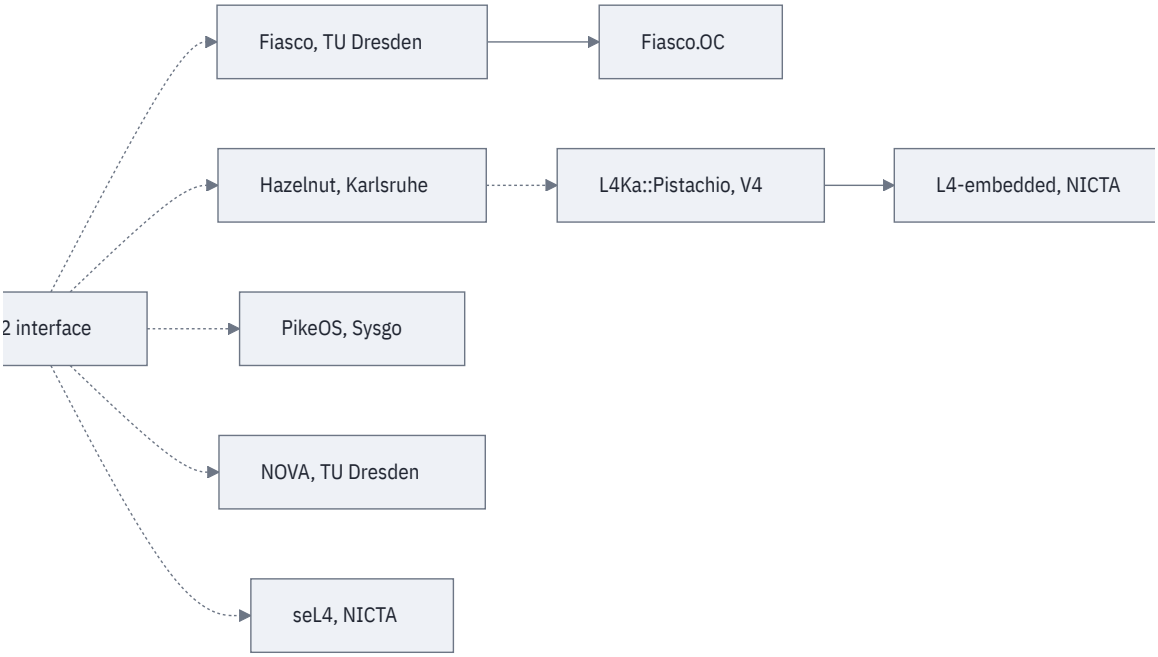


Figure 2.1. The L4 kernels this chapter mentions, after Heiser and Elphinstone's account [Heiser and Elphinstone 2016]. A solid arrow marks code that was forked; a dotted arrow marks a new implementation of an earlier design or interface.

Deployment followed. Elphinstone and Heiser report that OKL4's deployment through Qualcomm reached the billions [Elphinstone and Heiser 2013], and the seL4 white paper says the OKL4 microkernel "shipped on billions of Qualcomm cellular modem chips" [Heiser 2020]. The 2016 account adds that an L4-based kernel runs on the security processor of recent Apple iOS devices, that PikeOS, a commercial reimplementation of the V2 interface by Sysgo, is certified for avionics and deployed in aircraft and trains, and estimates Fiasco's commercial shipments at up to 100,000 [Heiser and Elphinstone 2016].

Through all of this the cost of the central operation stayed small. Table 2.4 gives the one-way IPC cost of members of the family over twenty years, from the table Elphinstone and Heiser compiled.

Table 2.4. One-way IPC cost of L4 kernels [Elphinstone and Heiser 2013, Table 1]. Rows that differ between this table and its 2016 version are left out.

Kernel	Year	Processor	MHz	Cycles	Time
Original	1993	i486	50	250	5.00 µs
Original	1997	Pentium	160	121	0.75 µs
Hazelnut	2002	Pentium 4	1,400	2,000	1.38 µs
Pistachio	2005	Itanium 2	1,500	36	0.02 µs
NOVA	2010	Core i7 (Bloomfield), 32-bit	2,660	288	0.11 µs
seL4	2013	Core i7 4770 (Haswell), 32-bit	3,400	301	0.09 µs
seL4	2013	ARM11	532	188	0.35 µs
seL4	2013	Cortex A9	1,000	316	0.32 µs

Apart from the Pentium 4, every row lies between 36 and 316 cycles, so twenty years of new processors and new kernels changed the cost of an IPC, measured in the processor's own cycles, by less than a factor of ten. The family's history is a sequence of reimplementations of a small interface that changed (V2, V4, then capabilities) while the message path stayed short.

The family could exist because the interface was small enough to implement again. An interface of seven system calls can be rebuilt in C++ for new processors, forked for an embedded product, or designed afresh so that it can be verified, and each of those happened. PicoCalc OS has a small interface of the same kind: every route from a world's code to the rest of the system passes through two host functions, `__k` and `__ka` (*Small Worlds*, Chapter 10). *Small Worlds* (Chapter 17) builds a second small system that follows PicoCalc OS's rules in a few hundred lines of JavaScript, minios, whose worlds are Web Workers, and Chapter 9 builds the unchanged nucleus for WebAssembly. Neither is a competing implementation in Liedtke's sense, but both rely on the property that made the L4 family possible, a list of what a component may ask of the kernel short enough to implement again.

2.6 What twenty years taught

In 2013 Elphinstone and Heiser went back through L4's original design decisions and sorted them into those kept, those replaced and those abandoned [Elphinstone and Heiser 2013]; the journal version of 2016 extends the account [Heiser and Elphinstone 2016]. Their conclusion is that minimality, generality and fast IPC remain the main drivers of the design. Synchronous IPC remains the main mechanism for client–server interaction in seL4, although OKL4 replaced it with a fully asynchronous model. User-level device drivers remain what they call "a core feature", and the kernel remains mostly non-preemptible, with preemption points placed in long operations such as object deletion.

Much of the original design did not survive, and Table 2.5 lists the parts that matter for this book, with the reason given for each and the corresponding part of PicoCalc OS.

Table 2.5. Decisions of the original L4 that later kernels replaced or abandoned, with the reasons Elphinstone and Heiser give [Elphinstone and Heiser 2013; Heiser and Elphinstone 2016], and PicoCalc OS's counterpart.

Original decision	What became of it	Reason given	PicoCalc OS
IPC timeouts	abandoned in seL4 and OKL4; a flag chooses between polling and blocking	of little use against denial of service; no theory for choosing values, so in practice only zero and infinity were used	asks keep timeouts, 2,000 ms by default (Chapter 6)
Long IPC (large messages copied by the kernel)	abandoned	much kernel complexity and many corner cases; rarely used, since shared buffers were preferred for bulk data	payloads copied as JSON, at most 64 KiB; framebuffers shared through handles
Global thread identifiers as destinations	replaced by endpoints	poor information hiding, and the global identifiers introduced covert channels	world names, checked against the manifest since the first pass (Chapter 5)
Clans and chiefs (redirection of messages that leave a clan)	abandoned for capabilities	"the strict thread hierarchy was unwieldy in practice"; for mandatory access control it "quickly deteriorated into a chief per process"	none; the manifest's `sends` and `asks`
Recursive page mappings	replaced in seL4 by the right to map, held in a frame capability	the mapping database took 25% to 50% of kernel memory in real systems	narrowed handles, whose derivation tree lives in the 128-entry handle table
Lazy scheduling	replaced by Benno scheduling	its worst case was bounded only by the number of threads, which defeated worst-case execution-time analysis	`pick` scans every world, at most 32 (`MAX_WORLDS`)
A kernel heap	removed in seL4: every kernel object is made from untyped memory the user holds	all kernel memory becomes subject to capabilities, so its use is authorised like any other resource	fixed tables, now with per-world quotas (Chapter 7)

Read together, the abandoned decisions fall into three groups; the grouping is this book's reading of the table. Some made the kernel hold state on users' behalf without a bound it could enforce: the wakeup lists that timeouts required, the mapping database, the nested page faults of long IPC. Some made the kernel spend time without a bound that could be

analysed, as lazy scheduling did. And some made names carry authority that their holders could not control: global thread identifiers, which any thread could use and which revealed the structure of a server, and clans and chiefs, which tried to restrict what names could reach by redirecting messages through a hierarchy. What replaced them is the seL4 design that Chapter 3 describes, in which communication is named by capabilities to endpoints and notifications, and every kernel object is made by retyping untyped memory, which the authors call "the only way to create objects in seL4" [Heiser and Elphinstone 2016].

The right-hand column shows PicoCalc OS on both sides of the line. It kept timeouts, which L4 abandoned, and Chapter 6 argues why a system whose turns await promises needs them, and adds a turn deadline, a watchdog of the kind Elphinstone and Heiser recommend in their place [Elphinstone and Heiser 2013]. It names its peers with global names, the decision L4 replaced, and the first pass checks those names against the manifest without making them capabilities. Before the first pass its handles reproduced the failure of the recursive mapping database closely: narrowing makes a new entry in a fixed kernel table, and a world given a single read-only view could narrow it until the table was full (*Small Worlds*, Chapters 8 and 9). Its scheduler's scan is bounded by the 32-world limit, which is the property Benno scheduling was introduced to give. Its sends are asynchronous, as all of OKL4's messages are, so a sender never waits. Its asks behave differently, because a turn stays open until the reply comes, and Chapter 6 shows that this brings back the deadlock L4's timeouts were meant to break.

The lesson of the twenty years, in the form this book uses it, is that a kernel decision survived when it bounded the kernel's state and time and tied authority to something a component holds, and failed when it did not.

2.7 Minimality applied to the nucleus

Liedtke's principle can be applied to PicoCalc OS's nucleus as a reading. For each part of `nucleus/`, ask whether moving it into a world, so that competing implementations could exist, would prevent the system from providing its required functionality. This section does that and proposes nothing; Chapter 12 proposes the changes.

Most of the nucleus passes the test as mechanism. The lane runs one world at a time and applies a segment's effects all or nothing (`pick`, `run_segment` and `apply_deferred` in `nucleus/kernel.c`), and no world could provide that for the others, because it is what makes a world's crash harmless to them. Messages, mailboxes and the ask table, handles

and their checks, budgets and the virtual clock are mechanism for the same reason: each is a check, or the custody of a resource, that no world can be trusted to perform for the others. The manifest checks in `kernel_boot` are a borderline case. They enforce a description, which is mechanism, but nothing requires them to run on the device; the Microkit performs the equivalent checks in the tool that builds the system image (Chapter 4). Table 2.6 lists the parts that fail the test or come close.

Table 2.6. Parts of PicoCalc OS's nucleus read with Liedtke's minimality principle. Line counts are from the current sources.

Part	Where	Size	Reading
The supervision schedule	`crash_at`, `nucleus/kernel.c`	38 lines	policy: a window, an intensity and a backoff that a supervisor world could hold
Drawing: fills, text, the damage diff	`nucleus/gfx.c`, `nucleus/font_data.h`	179 lines, and 2,080 lines of font tables	policy, kept in C for speed
The I2C transaction	`run_i2c`, `nucleus/hw.c`	27 lines	serialising a shared bus is mechanism; the write, wait and read shape is the keyboard controller's protocol
Priority from a world's kind	`boot_world`, `nucleus/kernel.c`	1 line	policy, and set by the world's own file (Chapter 1)
Frames	`run_segment`, `nucleus/kernel.c`	13 lines	close: the unforgeable sender name is mechanism; publishing the frame is a service the apps could perform
The manifest checks	`kernel_boot` and its helpers	225 lines	mechanism that could run when the system is built

The supervision schedule is the clearest case of policy in the nucleus. When a world crashes, `crash_at` decides between a restart and a stop:

▸ history/first-pass/nucleus/kernel.c

```
  // crashes in the last ten seconds decide between a restart and a stop
  vtime_t now = kernel_now();
  int keep = 0;
  for (int i = 0; i < W->n_crash_times; i++) if (now - W->crash_times[i] < MS(10000)) W-
      >crash_times[keep++] = W->crash_times[i];
  W->n_crash_times = keep;
  if (W->n_crash_times < 8) W->crash_times[W->n_crash_times++] = now;
  int recent = W->n_crash_times;
…
  teardown(W, t);
  if (recent >= 5 || W->config_error) {
    W->status = W_STOPPED;
…
  W->status = W_RESTARTING;
  vtime_t delay = MS(100) << (recent - 1);
  if (delay > MS(2000)) delay = MS(2000);
  event_t *e = new_event(E_RESTART, now + delay);
```

Every number in it is a choice: the ten-second window, the stop at the fifth crash, the 100 ms base and its doubling, the 2,000 ms cap. Other systems put the same choices outside the kernel. Erlang/OTP gives each supervisor process its own restart intensity and period [Erlang/OTP n.d.]; MINIX 3 recovers drivers from a user-level reincarnation server [Herder et al. 2006]; and the Microkit delivers a protection domain's fault to its parent protection domain, which decides what to do [Microkit manual 2026]. `teardown`, which discards a dead world's runtime, handles, settles and frame and advances its epoch, is mechanism and would stay. A supervisor world would need two mechanisms, and PicoCalc OS already has both in some form: `worlds.status` tells its subscribers of every crash, and `sys.restart`, open to a world with the `sys.admin` claim, calls `kernel_restart_world`. Moving the schedule out would not prevent any required function, so by Liedtke's test it does not belong in the nucleus.

The drawing code fails the test just as plainly, and stays in the nucleus for speed, the reason Mach and Chorus moved servers back into their kernels. The compositor could fill rectangles and draw glyphs in JavaScript, at 665 ns per interpreted step on the virtual clock, and a full-screen fill touches 102,400 pixels; *Small Worlds* (Chapter 8, Exercise 8) asks for the resulting slowdown. Liedtke's criterion is function, so by his test the code is policy that the nucleus keeps for performance, which is the trade the minimality principle was written to resist. The difference from Mach is that here the trade is made knowingly and in one place.

The I2C transaction mixes the two. Serialising a shared bus is mechanism: the keyboard and power drivers both hold handles for the STM32 at address 0x1F, and neither can be trusted to wait for the other. The shape of the transaction is the device's:

▸ history/first-pass/nucleus/hw.c

```
static void run_i2c(deferred_t *d, vtime_t at) {
  handle_t *h = find(d->handle);
  vtime_t start = bus_start(BUS_I2C0, at);
  // a write phase (address + bytes, 9 clocks each), an optional gap with the bus held, then a read
      phase
  double bits = (d->nbytes ? (1 + d->nbytes) * 9 + 2 : 0) + (d->nread ? (1 + d->nread) * 9 + 2 : 0);
  vtime_t end = start + wire_us(bits, h ? h->hz : 10000) + d->delay;
```

The write, the gap with the bus held and the read exist because the STM32 answers a read with whatever its last register write prepared, so a read that follows another driver's write gets the wrong answer. *Small Worlds* (Chapter 13) tells how two drivers' reads interleaved in exactly that way at the first boot of the full system in the simulator, and how this transaction in the nucleus's API fixed it for every driver. A single driver that owns the controller and serves the other as a client would make the transaction an internal matter of that driver, which is the arrangement Chapter 4 describes for LionsOS's drivers and the third step of Chapter 12.

Frames come close to passing. After every segment in which a world's screen changed, `run_segment` calls the world's `screen()` function and publishes the result on the `frames` channel, wrapped with the world's name. The nucleus does not interpret the tree it publishes; the prelude builds it and the compositor draws it. What the nucleus contributes that no world can is the name, because `frames` is the nucleus's own channel and the first pass refuses a sent message of that type (Chapter 1). The publication itself could move: each app could send its frame, and the compositor could check the sender the prelude passes to every handler as `ctx.from`. That would move a check from the nucleus into a receiver, the arrangement *Small Worlds* (Chapter 9) contrasts with complete mediation. By the letter of Liedtke's test the publication is a service the apps could perform; the identity it carries is mechanism, and it is already part of every message.

The reading finds the nucleus's policy small and concentrated: thirty-eight lines of supervision, one line of priority, one transaction shape, a convenience around frames, and the drawing code, which is policy kept for speed. The rest is mechanism by Liedtke's own test.

The test concerns function, and the next section turns to speed.

2.8 What a PicoCalc message costs

Liedtke judged a microkernel first by its IPC, and PicoCalc OS's messages can be judged the same way, with care about what is compared. The measurement uses two worlds written for this chapter and not part of the repository, a caller that asks a callee 100 times in a row and reports the mean round trip, and a callee that answers each ask with the number 1:

```
// Ask round trips between two worlds of the same priority, as in chapter 2.
service(self)
  .onStart(async () => {
    await sleep(20);
    const n = 100, t0 = now();
    for (let i = 0; i < n; i++) await ask("callee", "ping", null);
    log("RESULT ask round trip " + ((now() - t0) / n * 1000).toFixed(1) + " us");
  });
```

```
service(self).on("ping", () => 1);
```

With a two-world manifest that lets the caller ask the callee, the host build reports:

```
$ printf 'wait 200\nquit\n' | ./picocalc-host -v -j js -m js/l4.json -
calibration: the 100k loop is 200002 steps, so a step is 665 virtual ns; fib(20) is 43783 steps = 29.1
    virtual ms (31 ms on the device)
[     0.000] booting 2 worlds from js/l4.json
[    31.527] caller: RESULT ask round trip 113.0 us
```

The trace of the same run shows where the 113 µs go. Each ask is one message and two segments, the callee's turn and the caller's continuation:

▸ 14book/traces/02-ask.jsonl


```
{"k":"seg","w":"caller","t":20174,"dt":57,"steps":24,"native":2,"kind":"settle","type":"start"}
{"k":"msg","t":20231,"from":"caller","to":"callee","type":"ping","ask":true,"bytes":4}
{"k":"seg","w":"callee","t":20231,"dt":55,"steps":24,"native":0,"kind":"msg","type":"ping"}
{"k":"seg","w":"caller","t":20286,"dt":58,"steps":27,"native":1,"kind":"settle","type":"start"}
{"k":"msg","t":20344,"from":"caller","to":"callee","type":"ping","ask":true,"bytes":4}
```


The caller's first ask is made in a segment that starts at 20,174 µs, and the message is delivered when that segment ends, at 20,231 µs, because `kernel_ask` only defers the delivery (the `defer_deliver` call in Section 2.1). The callee runs at once, for 55 µs: c_0 = 40 µs for entering and leaving its runtime, plus 24 interpreter steps of the prelude's dispatch at 665 ns each, about 16 µs, rounded down. Its return value settles the caller's promise at 20,286 µs, and the caller's continuation runs for 58 µs (40 µs, 27 steps and 1 µs of native work) and makes the next ask, delivered at 20,344 µs. Figure 2.2 draws one round trip.

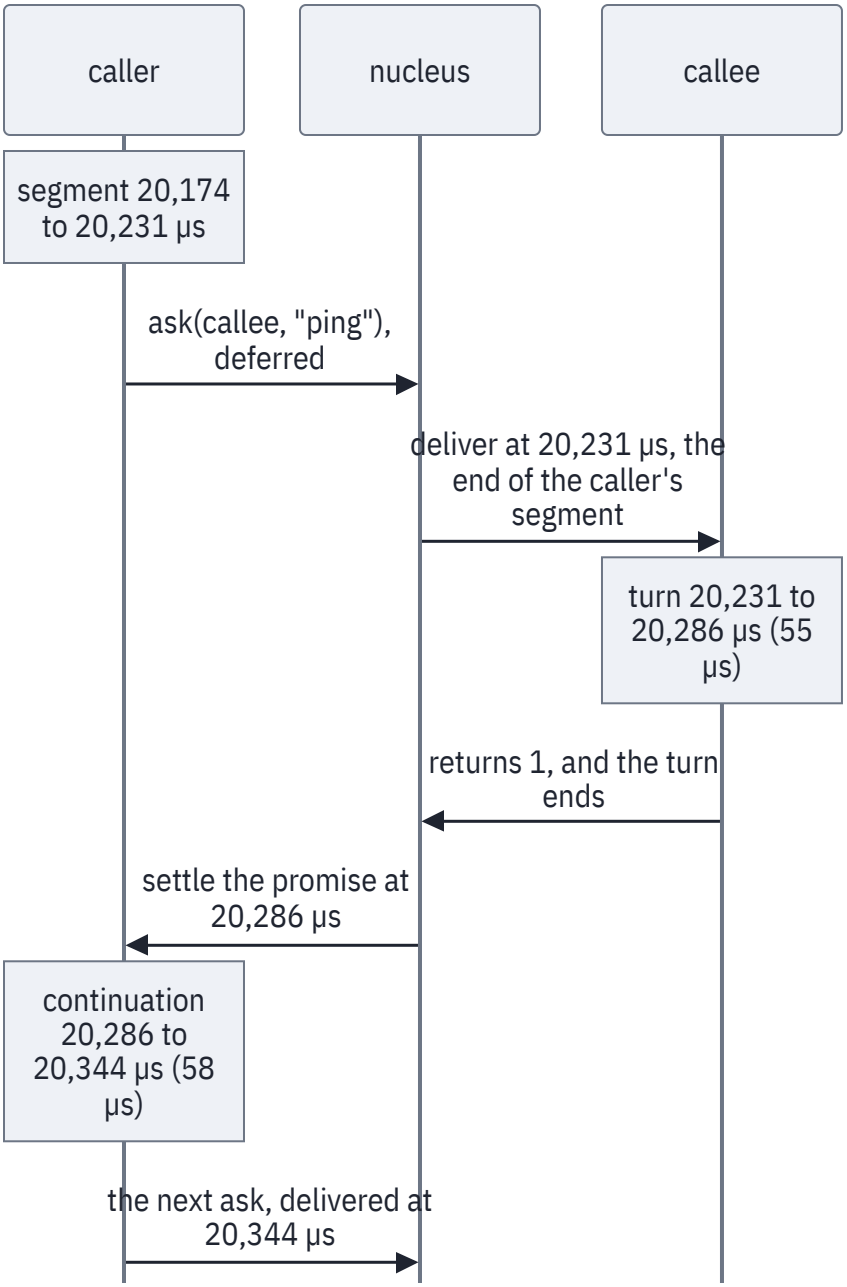


Figure 2.2. One ask on the virtual clock, with the times of the trace above. Nothing moves between the worlds until a segment ends; each side's cost is one segment.

Small Worlds (Chapter 12) measured the same operation with the benchmark suite's payload and found 119 µs, and 64 µs per message for a stream of one-way sends. Table 2.7 sets one direction of a PicoCalc ask beside the systems of this chapter.

Table 2.7. One direction of a message in five systems. The PicoCalc OS row is a virtual time on the host build; its cycle count assumes a 400 MHz core and the virtual clock's calibration.

System	Machine	One way	Round trip	In the machine's terms	Source
RC 4000 nucleus	RC 4000, about 4 µs per instruction	1.0 ms	2.0 ms	about 250 instruction times one way	[Hansen 1970]
Mach 3	i486, 50 MHz	115 µs	230 µs	5,750 cycles	[Liedtke 1995; Liedtke 1996]
L3	i486, 50 MHz	5 µs	10 µs	250 cycles	[Liedtke 1993; Liedtke 1995]
seL4	Cortex A9, 1 GHz	0.32 µs		316 cycles	[Elphinstone and Heiser 2013]
PicoCalc OS	ESP32-P4, as modelled	55 to 58 µs	113 µs	about 22,000 cycles	this section

If the virtual clock's calibration holds, one direction of a PicoCalc ask costs about 22,000 cycles of a 400 MHz core: about four times what Mach 3 spent on a 486, and about seventy times what seL4 spends on a Cortex A9. By instruction times it is also slower than Brinch Hansen's nucleus. Three cautions apply before any conclusion is drawn. First, 40 of the 55 µs are c_0, the virtual clock's charge for entering and leaving a world's runtime, and c_0 is an estimate that has not been measured on the device; only the 24 steps of dispatch rest on the calibration of τ, which was fitted to a measurement on the ESP32-P4 (*Small Worlds*, Chapter 18). Second, an L4 IPC switches between address spaces that hardware protects, while a PicoCalc message enters another JavaScript runtime in the same address space and parses JSON there; the two operations buy different isolation at different prices. Third, the machines differ, and a cycle on one processor is not a cycle on another.

What survives the cautions is the structure of the cost. Almost none of it is the moving of bytes: a payload is charged 25 ns per byte when it is encoded and again when it is decoded (*Small Worlds*, Chapter 12), so 100 bytes add about 5 µs to a round trip of 113 µs. Almost all

of it is the fixed cost of a segment. Liedtke's first principle, that IPC performance is the master, becomes for PicoCalc OS the rule that segments are the master: a protocol's cost is the number of segments it needs, and a design is judged by counting them. The techniques L4 used to shorten its path (messages in registers, a direct switch to the receiver, a kernel entry tuned instruction by instruction) attack the cost of a hardware context switch, which PicoCalc OS does not pay, and none of them reaches c_0. If c_0 measured on the device turned out large, the place to work would be the cost of entering a QuickJS runtime and the length of the prelude's dispatch, which is Liedtke's method applied to a different path.

Liedtke's last principle, that the design "has to aim at a concrete performance goal", transfers whole. PicoCalc OS's goal is the time from a key press to the first changed pixels, whose median over 100 presses is 30.9 ms on the virtual clock, with a maximum of 40.7 ms. *Small Worlds* (Chapter 1) shows that the keyboard's 20 ms poll and its two I2C reads at 10 kHz account for 83% of that time and the processor for less than 3 ms, so against the goal a message costs about 0.2% of the median, and in the two seconds of the idle system shown in Chapter 1 the lane was busy for 1.1% of the time. The first pass was held to a stricter goal, that it change no virtual time at all, and `bench/run.py` confirmed that it changed none [First pass 2026]. Its checks run inside host calls the virtual clock already charges, and they add no segment, so on the measure this section uses they cost nothing.

KEY POINTS

- Brinch Hansen's RC 4000 nucleus (1970) already had the microkernel's essentials: messages as the only communication, buffers and conversation identities kept by the nucleus, policy in replaceable processes, and a per-process limit on outstanding messages that PicoCalc OS gained only with its `ask_slots` quota.
- First-generation microkernels were slow: Mach 3 spent about 115 µs per IPC on a 486 at 50 MHz, Mach and Chorus moved critical servers back into their kernels, and Chen and Bershad found higher system instruction counts and memory cycles per instruction under Mach than under Ultrix.
- Liedtke showed that the cost lay in how those kernels were built: L3 achieved 5 µs on the same processor, 22 times faster than Mach for 8-byte messages, by designing every level of the kernel around IPC.

- The minimality principle admits a concept to the kernel only if moving it out would prevent required functionality; its criterion is function, and it left address spaces, threads with IPC and unique identifiers.
- L4Linux ran within 5% of native Linux's throughput on the AIM benchmarks, where MkLinux's penalty was five to seven times larger, and so showed that a fast kernel made a whole system competitive.
- Over twenty years the L4 family kept minimality, fast synchronous IPC and user-level drivers, abandoned IPC timeouts, long IPC, clans and chiefs and the kernel heap, and replaced global thread identifiers with endpoints; the abandoned decisions left kernel state or time unbounded or let names carry authority.
- Read with Liedtke's test, PicoCalc OS's nucleus is mostly mechanism; its policy is the supervision schedule in `crash_at`, the priority taken from a world's kind, the I2C transaction's shape and the drawing code, which is kept in C for speed.
- A PicoCalc ask costs 113 µs on the virtual clock, two segments of about 56 µs, most of each the estimated runtime entry cost c_0; the transferable lesson is to count segments and aim at a stated goal, and L4's techniques for a short IPC path do not apply.

The microkernel idea survived its first generation because Liedtke separated the question of what belongs in a kernel from the question of how fast the kernel's path can be made, and answered both with evidence. PicoCalc OS inherits the first answer almost unchanged and the second only as a method. The next chapter, Chapter 3, follows the family to the kernel that pushed the L4 model furthest, where the minimal set of mechanisms became a set of capability-protected objects with no kernel heap, and where the evidence became a proof.

EXERCISES

1. `BASIC` Map each of the RC 4000's four message primitives onto PicoCalc OS's world API and the nucleus function that implements it. Which of the RC 4000's rules did PicoCalc OS lack before the first pass, and which function in `nucleus/kernel.c` implements it now?

2. ANALYSIS Liedtke's 1993 paper measures 5 µs for an 8-byte L3 IPC, and gives L3's time for longer messages as about $7 + 0.02n$ µs for n bytes up to 2 KB and $10 + 0.04n$ µs beyond, and Mach's as about $120 + 0.08n$ µs. Use these to reproduce the paper's "factor of 22 (8-byte messages) to 3 (4-Kbyte messages)" as closely as the formulas allow, and explain why the factor falls with message size.

3. ANALYSIS Using the cost model of Section 2.8 (40 µs per segment, 665 ns per step, 25 ns per payload byte at each end), predict the round trip of an ask with a 1,000-byte payload. *Small Worlds* (Chapter 12) measured 171 µs. Where does your prediction differ, and why?

4. CODE Repeat the measurement of Section 2.8 in a scratch directory with payloads of 0, 100 and 1,000 bytes, then give the callee `"priority": 0` and add a third world at priority 1 that keeps the lane busy with 5 ms segments. Report the round trips, and explain the last result with `pick` in `nucleus/kernel.c`.

5. CODE Count the segments in one key press on the launcher, from the keyboard driver's poll to the display driver's last flush, using a trace from `trace FILE` (*Small Worlds*, Chapter 12, reads one). Multiply by the cost of a segment from Section 2.8 and compare with the 30.9 ms median. How large would c_0 have to be for segments to account for half of the median?

6. ANALYSIS Apply Liedtke's minimality test to two parts of the nucleus this chapter did not discuss: `kernel_publish`'s policy of dropping a post for a full subscriber and logging the drops at powers of two, and the 600 ms budget for evaluating a world's file at boot. For each, say whether it is mechanism or policy and where it could live instead.

7. DISCUSSION Elphinstone and Heiser abandoned IPC timeouts because there is no theory for choosing their values. PicoCalc OS keeps a 2,000 ms default for asks, the REPL uses 5,000 ms, and `turn_ms` defaults to 8,000 ms. Is PicoCalc OS's situation different enough to justify keeping them? Compare your answer with Chapter 6.

8. ANALYSIS The recursive mapping database of the original L4 took 25% to 50% of kernel memory in real systems. Estimate the fraction of PicoCalc OS's 128-entry handle table that the shipped system uses after two virtual seconds (the `snapshot` command reports each world's `used.handles`), and the fraction the quotas allow. What does the difference say about static partitioning?

9. PROJECT Design a supervisor world that takes over the schedule in `crash_at`: the manifest syntax that names it and the worlds it supervises, the messages it receives, the nucleus operations it calls, and what happens when the supervisor itself crashes. Compare your design with Erlang/OTP's supervisors and MINIX 3's reincarnation server, and list the evals that would show it behaves like the current nucleus. (Chapter 12 gives one design; write yours before reading it.)

10. DISCUSSION Brinch Hansen concluded that the only realistic way to make his nucleus faster was faster hardware; Liedtke concluded that the microkernels of his time were slow because of their design. Which conclusion fits PicoCalc OS's 113 µs ask better, and what measurement on the ESP32-P4 would decide it?

FURTHER READING

[Hansen 1970] Five pages that contain the microkernel idea, message buffering with conversation identities, and the per-process limit on outstanding messages. Read the section on process communication first.

[Accetta et al. 1986] The Mach paper: tasks, threads, ports, messages and external pagers, the design the second generation reacted against.

[Chen and Bershad 1993] The careful measurement of Mach against Ultrix that framed the performance debate. Read it with Liedtke's reanalysis in the 1995 paper.

[Liedtke 1993] How L3's IPC was made twenty times faster, with the design principles stated as a list. A model of designing a kernel around one measured path.

[Liedtke 1995] The minimality principle, the recursive construction of address spaces, and the argument that first-generation microkernels were slow by construction.

[Liedtke 1996] The same argument for a general audience, with the comparison of Mach and L4 that Table 2.2 reproduces.

[Härtig et al. 1997] L4Linux: the whole-system evidence that a fast microkernel can carry a conventional operating system, and a short, careful discussion of measuring a deterministic system.

[Elphinstone and Heiser 2013] Twenty years of L4 sorted into kept, replaced and abandoned, with reasons. The source of Table 2.5.

[Heiser and Elphinstone 2016] The extended account, including the family tree and the deployments.

CHAPTER 3 PART I · THE L4 LINEAGE

seL4: capabilities, proofs and time

seL4 is the member of the L4 family whose C implementation has a machine-checked proof that it does what its specification says [Klein et al. 2009]. The proof was possible because the kernel was designed for it, and the design decisions that made it possible are the ones this chapter is about. Every resource is a kernel object reached through a capability. The kernel has no heap, so every byte it uses on behalf of a thread comes from memory that thread's authority covers. Capabilities are recorded in a derivation tree, so that authority given out can be taken back. Messages are addressed to endpoint objects. And in the mixed-criticality version, processor time is an object too.

This chapter reads each decision as a mechanism with a reason, sets it beside what PicoCalc OS has in its place (the handles and quotas of *Small Worlds* [Small Worlds 2026], measured on the host build), and then states what the proofs establish, what they assume and where the verification stands at the end of 2026. The reader should finish able to explain why each part of seL4's object model exists, what its theorems say and do not say, and which of its properties a JavaScript system on a microcontroller can have.

3.1 The object model

The seL4 kernel provides a small number of object types and makes every operation an operation on one of them. In the white paper's words, "a system call in seL4 is a capability invocation", with arguments that say what to do to the object the capability references [Heiser 2020, §4.1]. The kernel checks that the capability authorises the operation and refuses it otherwise. The only system call that names no capability is `seL4_Yield`, which

gives up the rest of the caller's time [seL4 manual 2026, §2.2]. Sending a message, creating a thread, mapping a page and acknowledging an interrupt are therefore the same act at the system-call boundary, the invocation of a capability the caller holds, with a message saying what to do.

The white paper lists ten object types, all referenced by capabilities [Heiser 2020, §4.1]. Table 3.1 gives them with the structure in PicoCalc OS that plays a similar part, where there is one.

Table 3.1. seL4's ten object types and their nearest counterparts in PicoCalc OS. Most of PicoCalc's counterparts are not capabilities; only hardware is reached through handles.

seL4 object	What it represents	Nearest counterpart in PicoCalc OS
Untyped	free physical memory, which can be retyped into any other object	none; the nucleus allocates from the C heap, bounded by per-world quotas in the manifest
CNode	a table of capability slots; CNodes form a thread's capability space	the `holders` bits of the entries in the 128-entry handle table
Thread control block	a thread of execution	a world (`world_t`), with its own QuickJS runtime
Scheduling context	a budget and a period on one core (MCS kernels only)	`budget_ms`, a limit per segment, with no period
Endpoint	a rendezvous point for synchronous messages	none; messages are addressed to a world's name, checked against `sends` and `asks`
Reply object	the return path of one call (MCS kernels only)	an entry in the ask table, with the asker's epoch
Notification	a word of binary semaphores	a channel post, which carries JSON and is queued for each subscriber
Address space	page tables enforced by the MMU	a world's heap, enforced by the language
Frame	physical memory that can be mapped, device registers included	a framebuffer handle (`H_DMA`); bus handles (`H_I2C`, `H_SPI`) for device access
Interrupt objects	the right to wait for and acknowledge one interrupt source	none; drivers poll (the keyboard every 20 ms)

A capability lives in a slot of a CNode, and a thread's **capability space** (CSpace) is the directed graph of CNodes reachable from the CNode capability in its thread control block. A thread names a capability by its address in that graph, and CSpaces are implemented as guarded page tables. The kernel compares a guard with the top bits of the address, uses the next bits as an index into the CNode, and continues into a further CNode if the slot holds one [seL4 manual 2026, §3.3]. Each slot takes 16 bytes on 32-bit machines and 32 bytes on 64-bit ones [seL4 manual 2026, §3.1.1]. Because the CSpace is a structure of the caller's own, an address means something only to the thread that presents it; another thread presenting the same integer reaches its own slot, or nothing. The CNodes themselves are objects that user code retypes from untyped memory and arranges, since "seL4 requires the programmer to manage all in-kernel data structures, including CSpaces, from userspace" [seL4 manual 2026, §3]. What remains global in the kernel is small: "five arrays, two integers, and four pointers", and "all other state is reachable via capabilities or page tables associated with a thread" [Klein et al. 2014, §3.1]. PicoCalc's nucleus is the reverse, one global structure `K` holding every world, handle, ask and framebuffer.

PicoCalc's handles follow the same partitioned design for the objects they cover. A handle is an entry in a table held by the nucleus, a world refers to it by number, and the number gives authority only if the entry lists the world as a holder:

▸ history/first-pass/nucleus/nucleus.h

```
enum { H_I2C, H_SPI, H_DMA };
enum { R_READ = 1, R_WRITE = 2, R_GRANT = 4 };
typedef struct {
  int id, type, rights, parent, minter;
  uint64_t holders;         // bit per world id
  bool live, revoked;
  char claim[NAME_LEN];
  int bus, addr, hz;        // H_I2C / H_SPI
  int buf;                  // H_DMA: index into the buffer table (shared by a handle and its children)
} handle_t;
#define MAX_HANDLES 128
```

Every use goes through one function, which performs the checks an seL4 capability lookup and invocation perform, in a fixed order: the object must exist, the caller must hold it, it must not be revoked, it must be of the type the operation needs, and it must carry the right.

▸ history/first-pass/nucleus/hw.c

```
handle_t *handle_use(world_t *W, int id, int type, int right, char *err, size_t n) {
  handle_t *h = find(id);
  if (!h) { snprintf(err, n, "there is no handle %d", id); return NULL; }
  if (!(h->holders & bit(W))) { snprintf(err, n, "%s does not hold handle %d", W->name, id); return
      NULL; }
  if (revoked(h)) { snprintf(err, n, "handle %d was revoked", id); return NULL; }
  if (type >= 0 && h->type != type) { snprintf(err, n, "handle %d is a %s handle, not %s", id,
      type_name(h->type), type_name(type)); return NULL; }
  if ((h->rights & right) != right) {
    snprintf(err, n, "handle %d is %s", id, (right & R_WRITE) ? "read-only" : (right & R_GRANT) ? "not
        grantable" : "write-only");
    return NULL;
  }
  return h;
}
```

Two differences matter later. A PicoCalc handle number is global (`handle_new` assigns `++K.n_handles`), so the same integer names the same handle in every world; the holder check makes that harmless for authority, but it is a shared name space of the kind capability systems avoid. And seL4's objects cover every resource the kernel manages, while PicoCalc's handles cover the I2C device, the SPI bus and framebuffers; sending, asking and publishing were left to names and self-declarations (*Small Worlds*, Chapter 9) until the first pass gave them to the manifest (Chapter 5). This section has established that seL4 makes everything an object and every operation an invocation, and that PicoCalc OS does the same for hardware only.

3.2 Kernel memory from untyped

A kernel that allocates memory for its own data structures when a user asks it to do something can be made to run out of memory by a user. Elphinstone and Heiser describe the problem as it appeared in L4. The kernels "traditionally had a fixed-size heap" for their data structures, and capabilities alone did not help, because a single malicious thread with the grant right on a mapping could create mappings until the kernel's metadata filled memory, "potentially DOS-ing the system" [Elphinstone and Heiser 2013, §3.4.3].

seL4's answer was to "completely remove the kernel heap" [Elphinstone and Heiser 2013, §3.4.3], a design described by Elkaduwe, Derrin and Elphinstone [Elkaduwe et al. 2008]. At boot the kernel reserves the memory for its own code, data and single stack, creates one

initial thread, and hands that thread all remaining memory as capabilities to **untyped** objects [seL4 manual 2026, §2.4]. From then on, the kernel "does not dynamically allocate memory for kernel objects"; they must be created "from application-controlled memory regions via Untyped Memory capabilities" [seL4 manual 2026, §2.4]. The only operation on an untyped capability is `seL4_Untyped_Retype`, which carves objects of one type out of the region, places capabilities to them in the caller's CNode, and records them as children of the untyped capability. The kernel keeps a watermark per untyped region; memory is reused only after every capability to the objects in it has been deleted, typically by revoking the untyped capability itself [seL4 manual 2026, §2.4.1]. Ge and colleagues summarise the result: after booting, "the kernel never allocates memory"; it "has no heap and uses a strictly bounded stack" [Ge et al. 2019, §2.4].

The design exists for the property that follows. Everything the kernel stores on a thread's behalf lives in memory retyped from untyped memory that the thread, or whoever set it up, held. In the words of the verification paper, "the kernel heap can be precisely partitioned between applications", each owning the part it holds authority over, and this "turned out to be essential" for stating and proving integrity and confidentiality [Klein et al. 2014, §3.3]. A thread cannot exhaust kernel memory that others need, because they draw from no common pool. According to the manual, there are no arbitrary resource limits in the kernel apart from those dictated by the hardware, "and so many denial-of-service attacks via resource exhaustion are avoided" [seL4 manual 2026, §2.4]. Most system calls also allocate nothing, so they cannot fail for lack of memory halfway through, which simplifies the proofs [Klein et al. 2014, §3.3].

PicoCalc's nucleus has the opposite structure. Its tables are fixed arrays (32 worlds, 128 handles, 1,024 asks, 16 framebuffers, in `nucleus/nucleus.h`), and everything else comes from the C heap when a world asks. A message is two `malloc`ed strings and a header:

▸ nucleus/kernel.c

```
msg_t *new_msg(int kind, const char *type, const char *json, int from) {
  msg_t *m = calloc(1, sizeof(*m));
  m->kind = kind; m->type = xstrdup(type ? type : ""); m->json = xstrdup(json); m->from = from;
  m->q_to = -1;
  return m;
}
```

and a framebuffer is two buffers of two bytes per pixel, allocated by the nucleus for the world that asked:

▸ nucleus/hw.c

```
int dmabuf_alloc(int w, int h, world_t *owner) {
…
    b->owner = owner->id; owner->used_bufs++;
    b->bytes = (size_t)w * h * 2;
    b->back = calloc((size_t)w * h, 2);
    b->front = calloc((size_t)w * h, 2);
```

For the compositor's 320 × 320 framebuffer that is 400 KB of nucleus heap (`"kb":400` in the host build's handle table). Before the first pass the tables were shared pools and the heap behind messages was unbounded; *Small Worlds*, Chapter 9 shows a world filling the handle table by narrowing. The first pass gave each world quotas and made the boot fail unless they fit the tables [First pass 2026]. In the shipped manifest the handle quotas add up to 54 of 128 slots (twelve worlds at the default 4, the compositor at 6), the ask slots to 104 of 1,024, and the framebuffer quotas to 2 of 16.

That is the static half of seL4's rule, that no world can take what another was promised. The dynamic half is missing. A world cannot hand part of its quota to another, as an seL4 thread can pass on part of an untyped region; framebuffer pixels are bounded by `bufs` but not counted against the owner's `mem_kb`; and the C heap behind messages is bounded only because each message is charged to the receiver's mailbox quota until it leaves. Chapter 7 develops the quotas in full. This section has established why seL4 has no kernel heap, what that buys (allocation that follows authority, and proofs that rely on it), and how far PicoCalc's quotas reproduce it.

3.3 Endpoints, notifications and replies

seL4 has two communication objects with different jobs. An **endpoint** carries messages synchronously: a thread that sends or receives on one "blocks until the message can be delivered", so delivery happens only when "a sender and a receiver rendezvous at the endpoint", and the kernel copies the message once, or not at all for a short message in registers [seL4 manual 2026, §2.3]. The endpoint queues waiting threads, never messages. A capabil-

ity to an endpoint can be send-only or receive-only, and only one that carries the grant right lets the message carry capabilities [seL4 manual 2026, §4.2]. Chapter 5 tells how endpoints replaced L4's global thread identifiers and why; here the concern is how they work.

To tell its clients apart, a server mints each a capability to its endpoint with a **badge**, a word chosen at minting that "cannot be unbadged, rebadged or used to create child capabilities with different badges", and the kernel delivers the badge of the capability used with every message [seL4 manual 2026, §4.2.1]. A badge is thus the server's own name for a grant. Two capabilities given to the same client can carry different badges, and the server can tell which of them a message used.

Request and reply are combined in `seL4_Call`, which sends and then waits for a reply in one operation. The receiver gets a **reply capability**, a single-use right to answer that one call; on MCS kernels it is held in a reply object that the receiver supplies [seL4 manual 2026, §2.2]. The manual gives two guarantees. Invoking a reply capability cannot block, because its existence means the caller is waiting; and the reply the caller receives is guaranteed to come from the thread that received the call, "without having to check any kind of badge" [seL4 manual 2026, §4.2.4]. A server loop answers one request and waits for the next with `seL4_ReplyRecv`, one kernel entry instead of two. On an ARM Cortex-A9 at 1 GHz, Lyons and colleagues measured `call` at 288 cycles on the base kernel and 350 on the MCS kernel, and `replyRecv` at 313 and 339 [Lyons et al. 2018, Table 2].

A **notification** carries no message at all. It is "a word-size array of flags, each of which behaves like a binary semaphore" [seL4 manual 2026, §2.3]: `seL4_Signal` ORs the badge of the capability used into the word and wakes a waiter, and `seL4_Wait` returns the word and clears it [seL4 manual 2026, §5.1–5.2]. Signals therefore coalesce; three signals with the same badge before a wait are read as one. A notification bound to a thread is delivered even while the thread waits on an endpoint, so a server can hear an interrupt and a request in one loop [seL4 manual 2026, §5.3].

PicoCalc OS has three primitives, which *Small Worlds*, Chapter 12 describes, and none of them is synchronous. An `ask` is closest to `seL4_Call`: the asker's promise is settled exactly once, by the reply, by the receiver's crash or by a timeout, and an entry in the ask table, stamped with the asker's epoch, plays the part of the reply object. The receiver learns the sender from `from`, which the nucleus fills in and the sender cannot forge. It identifies the sending world and says nothing about which grant was used, so a world cannot give one client two differently badged rights. A `send` is queued in the receiver's mailbox, which an endpoint never does, and a channel post carries JSON to every subscriber, where a notifica-

tion carries one word and loses repetitions. The costs differ by a factor of about 70. An ask with an empty payload takes 119 µs on the virtual clock of the host build [Small Worlds 2026, Chapter 12], about 47,600 cycles of a 400 MHz core, against 689 cycles for an MCS `call` and `replyRecv` on the Cortex-A9; most of the difference is entering two JavaScript runtimes (Chapter 2 compares the two with due care). The section has established the semantics that later chapters use: unbuffered rendezvous, badges as per-grant identity, a single-use reply right, and coalescing notifications.

3.4 The derivation tree and revocation

A capability system that can give authority must be able to take it back, and taking it back requires knowing where it went. seL4 records this in the **capability derivation tree** (CDT). The kernel "tracks capability derivations", the children made by `seL4_Untyped_Retype()`, `seL4_CNode_Mint()`, `seL4_CNode_Copy()` and `seL4_CNode_Mutate()` [seL4 manual 2026, §2.4.1]. Minting can reduce rights, and a request for more rights than the source has is "silently downgraded to those of the source" [seL4 manual 2026, §3.1.4]. The tree costs no extra memory, since it is kept in the CNodes themselves and "requires no additional kernel meta-data" [seL4 manual 2026, §2.4.1].

The tree is shallower than the operations suggest. Untyped capabilities form as many levels as the memory is split into, and retyping makes the new objects' original capabilities children of the untyped one. Below that, "ordinary original capabilities can have one level of derived capabilities", and further copies of a derived capability "will create siblings" [seL4 manual 2026, §3.1.5]. Endpoints and notifications have one extra level: minting a badge from the unbadged original creates an "original badged" capability, which again supports one level of children. Sending a capability through an endpoint with the grant right copies it into the receiver's slot, and "the sender retains access to the sent capability" [seL4 manual 2026, §4.2.2]; the copy is a derivation, so the same rules place it. Figure 3.1 shows the shape, next to the tree of handles that the experiment below builds in PicoCalc OS.

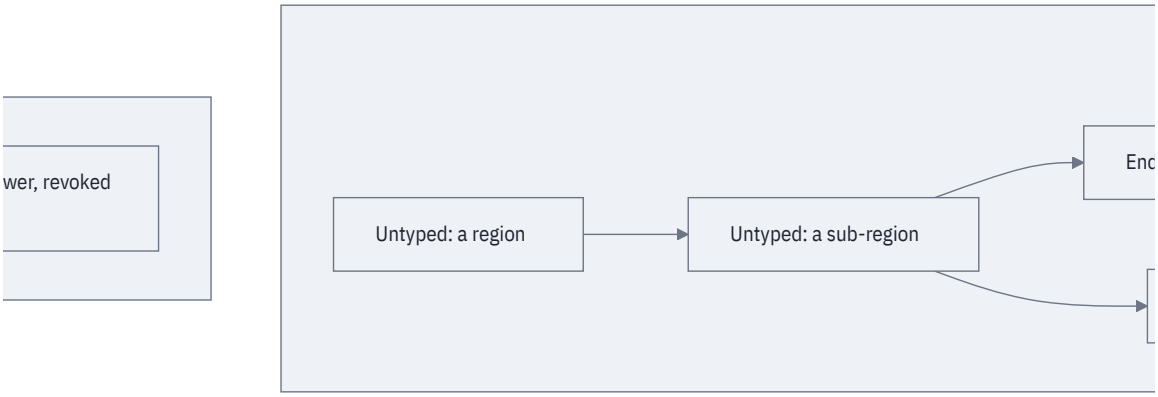


Figure 3.1. Left, a capability derivation tree in seL4, after the example in the reference manual: memory splits into untyped regions, objects are retyped from them, and each original capability has one level of derived copies. Right, the derivation tree of handles in the experiment of this section, after the revocation.

`seL4_CNode_Revoke` deletes every CDT child of a capability, in every CSpace, "but will leave the capability itself intact"; when a deleted capability was the last one to its object, the object is destroyed and its memory can be reused [seL4 manual 2026, §3.2]. Taken with the shallow tree, this gives a clear rule of ownership. The holder of an original can take back every copy made from it, because the copies are its children; revoking a derived capability removes nothing, because a derived capability has no children. A server that wants to cut off one client mints that client its own badged original and revokes it later. Revocation can touch many objects, so it is the main exception to seL4's short system calls. The kernel splits it into sub-operations, each leaving the kernel consistent, checks for interrupts between them, and restarts the call where it stopped [Heiser 2020, §5.1].

PicoCalc's handles have the same three operations (attenuation by `narrow`, delegation in messages, revocation of what one narrowed), arranged differently. Revocation is eager, and it marks the handle named as well as every live descendant, repeating its pass over the table until a pass finds nothing new:

▸ nucleus/hw.c

```
void handle_revoke_tree(int id) {
  handle_t *root = find(id);
  if (!root) return;
  root->revoked = true;
  for (bool more = true; more; ) {
    more = false;
    for (int i = 0; i < MAX_HANDLES; i++) {
      handle_t *k = &K.handles[i];
      if (!k->live || k->revoked || !k->parent) continue;
      handle_t *p = find(k->parent);
      if (p && p->revoked) { k->revoked = true; more = true; }
    }
  }
}
```

Two scratch worlds, written for this chapter and not in the repository, show the differences. `lender` allocates a framebuffer, narrows it to a view without the grant right and lends the view to `borrower`, which narrows a copy of its own; then `lender` revokes the view:

```
// lender.js (scratch, for this chapter): lend a narrowed view, then revoke it.
service(self).onStart(async () => {
  await sleep(10);
  const fb = dma.alloc(32, 32);          // the original: rights rwg, no parent
  const view = fb.narrow("rw");          // a narrowing without g, to lend
  const r1 = await ask("borrower", "take", { view });
  log("before revoke: " + JSON.stringify(r1));
  await sleep(5);
  view.revoke();                         // takes effect when this segment ends
  await sleep(10);
  const r2 = await ask("borrower", "use", {});
  log("after revoke: " + JSON.stringify(r2));
  try { fb.fill(0, 0, 1, 1, "#fff"); log("lender's original: wrote"); } catch (e) { log("lender's
      original: " + e.message); }
});

// borrower.js (scratch): hold the lent view and a copy narrowed from it.
const tryFill = (h) => { try { h.fill(0, 0, 1, 1, "#000"); return "wrote"; } catch (e) { return
    e.message; } };
service(self)
  .on("take", (s, { view }) => { s.view = view; s.copy = view.narrow("rw"); return { view:
      tryFill(s.view), copy: tryFill(s.copy) }; })
  .on("use", (s) => ({ view: tryFill(s.view), copy: tryFill(s.copy) }));
```

The scratch manifest `cdt.json` lists only these two worlds, `lender` with `"claims": ["dma"]` and `"asks": ["borrower"]`. Run from a directory holding a copy of `js/` with the two files added, where `$PC` is the repository (the `grep` hides the snapshots' JSON lines):

```
$ printf 'wait 12\nsnapshot before\nwait 100\nsnapshot after\nquit\n' | $PC/build/picocalc-host -v -j
    js -m cdt.json -u uart.log -o out - 2>&1 | grep -v '^RESULT'
calibration: the 100k loop is 200002 steps, so a step is 665 virtual ns; fib(20) is 43783 steps = 29.1
    virtual ms (31 ms on the device)
[     0.000] booting 2 worlds from cdt.json
[    10.406] lender: before revoke: {"view":"wrote","copy":"wrote"}
[    25.640] lender: after revoke: {"view":"handle 2 was revoked","copy":"handle 3 was revoked"}
[    25.648] lender: lender's original: wrote
```

The snapshots show the tree of Figure 3.1, with handles 2 and 3 marked revoked in the second; in both, the nucleus counts two handles against `lender`'s quota and one against `borrower`'s. Table 3.2 sets the two designs side by side.

Table 3.2. Derivation and revocation in seL4 and in PicoCalc OS. The PicoCalc column is checked against nucleus/hw.c and nucleus/vm.c and against the experiment in this section.

	seL4	PicoCalc OS
What passing gives the receiver	a copy in its own slot, placed in the tree	the same handle, with one more bit in `holders`
Depth of derivation	one level below each original (two for badged endpoints and notifications); untyped regions nest	any depth; `can_pass` stops climbing at 32
Who can revoke	any holder of a capability that has children; in effect, the holder of an original	only the world that narrowed the handle, and only a narrowing
What revocation removes	all children; the capability itself stays	the handle itself and all its descendants
What happens to the slots	the children are deleted; objects with no capability left are destroyed	the handles stay in the table, marked, and stay charged to their minters until every holder releases them
Cost	proportional to the children; preemptible between sub-operations	at most one pass over the 128 entries per level of depth, plus a final pass that finds nothing, run as a deferred effect when the segment ends

Three rows carry lessons for PicoCalc OS. Because passing shares the handle, a world cannot take back what it passed; it must narrow first and pass the narrowing, the caretaker of *Small Worlds*, Chapter 9, where seL4's copy rule gives the same effect unplanned. Because the handle named is itself revoked, `lender` loses the view along with `borrower`, harmlessly only because it kept the original. And because revocation marks without deleting, a revoked handle occupies a slot and its minter's quota until every holder lets go (Chapter 7). The section has established how seL4 tracks and withdraws authority and where PicoCalc's version differs.

3.5 Time as a capability

Capabilities controlled space in seL4 from the start; time came later. In the original kernel, as in most L4 kernels, a thread had a priority and a time slice, and a server could run on its client's time slice. Lyons and colleagues call that time-slice donation fast but "unprincipled

and hard, if not impossible, to analyse". The slice could expire in the middle of the server's work, which then continued on the server's own time, so the work was not consistently accounted to anyone [Lyons et al. 2018, §3.5].

The mixed-criticality (MCS) kernel replaces the time slice by a **scheduling context**, an object retyped from untyped memory like any other. It holds a budget b and a period p in microseconds, and the kernel will not let a thread run "for more than b out of every p microseconds" [seL4 manual 2026, §6.1.8]. A thread runs only while it has a scheduling context with budget left; priority still decides which runnable thread runs. The budget is only an upper bound. The kernel performs no admission test, and whether the whole system is schedulable is left to whoever configures it [seL4 manual 2026, §6.1.8, §6.1.10]. The white paper's example shows why priority and budget should be separate. A network driver can have a higher priority than a critical control loop, for responsiveness, and a budget of 3 µs every 10 µs, so it can never take more than 30 per cent of the processor and the loop's 3 ms in every 5 ms stay available [Heiser 2020, §5.2].

A **passive** thread is one with no scheduling context of its own; it "cannot execute without the action of another thread" [seL4 manual 2026, §6.1.9]. When an active client calls a passive server with `seL4_Call`, the client's scheduling context moves to the server for the duration of the call, the reply object records the loan, and the reply returns it [seL4 manual 2026, §6.1.11]. Figure 3.2 shows the exchange. The server's work is charged to the client that asked for it, so "time consumed by the server is billed to the client requesting the work" [Lyons et al. 2018, §3.5]. A server configured at the highest priority of its clients runs atomically with respect to all of them, which implements the immediate priority ceiling protocol at no extra cost [Lyons et al. 2018, §3.5]. The loan depends on trust, since "`seL4_Call()` does not guarantee the return of a scheduling context"; a server could never reply [seL4 manual 2026, §6.1.11]. If instead the client's budget runs out while the server works on its request, a **timeout fault handler** can reply to the client with an error and reset the server [seL4 manual 2026, §6.1.13.2]. The cost of the mechanism on the Cortex-A9 was 62 extra cycles per call and 868 per scheduling decision that forces a reschedule [Lyons et al. 2018, Table 2].

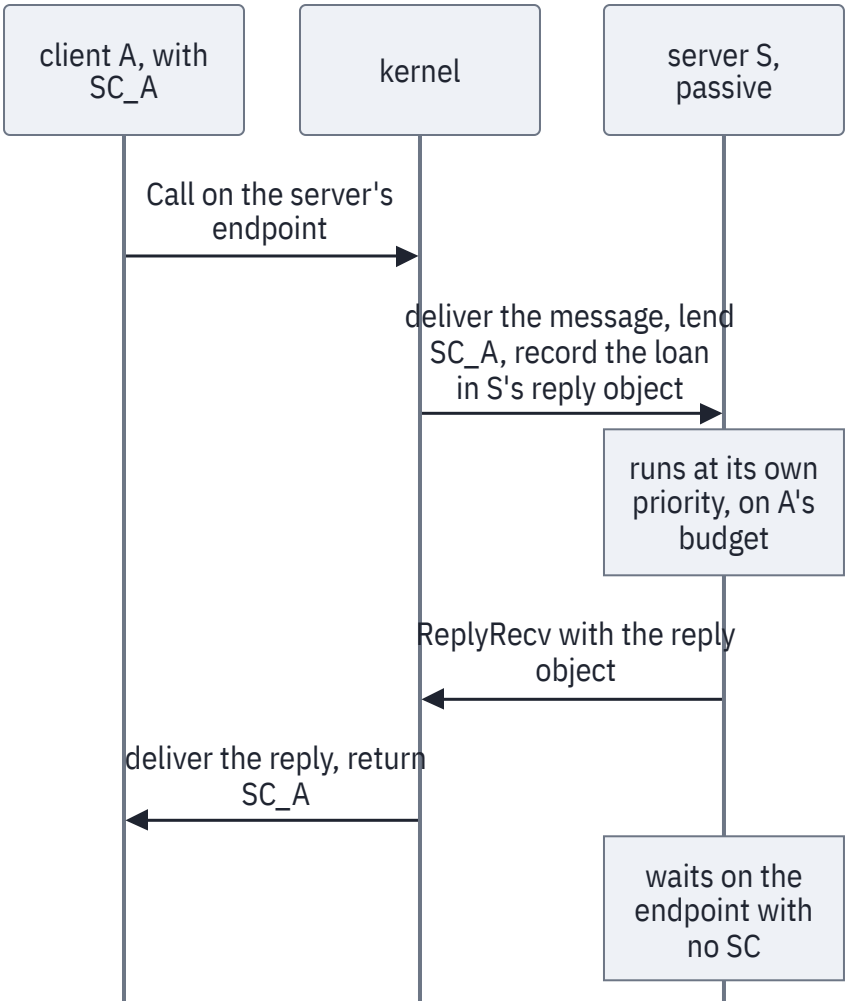


Figure 3.2. A call from an active client to a passive server on an MCS kernel. The server has no scheduling context and runs on the client's for the length of the call; the reply returns it.

PicoCalc's budgets answer a different question. A world's `budget_ms` limits one segment, checked every 1,000 interpreter steps and after every host call, and a segment that exceeds it crashes the world (*Small Worlds*, Chapter 11); it is a fault detector. There is no period, so a high-priority world that always has work can hold the lane indefinitely, the starvation *Small Worlds*, Chapter 4 measures. And there is no donation. The REPL (priority 1) asks the sandbox (priority 0) to evaluate code, the sandbox runs on its own 200 ms budget at its own priority, and the nucleus says so at boot:

```
[     0.861] note: repl asks sandbox, which runs at a lower priority (0 below 1): its asks wait behind
    everything in between
```

That line is `boot_notes` in `nucleus/kernel.c`, added by the first pass (Chapter 6). On an MCS kernel the sandbox would be a passive server, and the evaluation would run on the REPL's budget, at the sandbox's priority, billed to the REPL. Chapter 12 works out what that would take in PicoCalc OS. This section has established what a scheduling context is, how passive servers move time with requests, and that PicoCalc's budgets limit segments and reserve no time.

3.6 Time protection

seL4's confidentiality theorem (Section 3.7) rules out covert storage channels. It does not rule out timing channels, "which are used by such attacks as Spectre" [Heiser 2020, §3.1]. A timing channel needs no shared variable. One domain changes the state of something shared (a cache, a branch predictor, the scheduler's queue), and another times its own work.

Ge, Yarom, Chothia and Heiser argue that the operating system must provide **time protection**, "the temporal equivalent of the established memory protection" [Ge et al. 2019, Abstract], and state five requirements [Ge et al. 2019, §3.2]. On-core state (L1 caches, TLB, branch predictor) must be flushed when a core switches domains; each domain needs its own copy of the kernel's text, stack and most global data; access to kernel data still shared must be deterministic; the flush must be padded to its worst-case time, since its duration depends on the dirty data the previous domain left; and interrupts other than the preemption timer must be partitioned. Shared caches are partitioned by page colouring, giving each domain frames that map to disjoint cache sets. seL4's memory model makes the kernel's share easy: since "all dynamically allocated kernel memory is provided by userland", colouring user memory "will colour all dynamic kernel data structures" [Ge et al. 2019, §3.2]. The paper adds a policy-free kernel clone operation, with which the initial process gives each domain a kernel image in that domain's coloured memory. None of this is yet covered by a proof; the white paper calls leakage through timing channels "a major unsolved research problem" [Heiser 2020, §3.2].

PicoCalc OS has a timing channel one level up, in its scheduler. A world's only clock is the virtual clock, which advances with every world's work on the lane; *Small Worlds*, Chapter 10 points out that two worlds can signal each other through it. Two more scratch worlds measure that channel. Neither has any `sends`, `asks`, `provides` or `needs` in its manifest entry, so after the first pass neither may send to, ask, hear or post anything the other could receive. `sender` transmits eight bits, one per 100 ms window. For a 1 it runs a loop of 6,001 steps (3.99 ms on the virtual clock, by the host build's `steps` command) between 1 ms sleeps, and for a 0 it only sleeps. `receiver` sleeps for 1 ms at a time, records how late each sleep wakes, and averages the lateness over each window:

```
// sender.js (scratch, for this chapter): keep the lane busy (1) or idle (0) for 100 ms per bit.
const BITS = [1, 0, 1, 1, 0, 0, 1, 0];
const spin = (n) => { let x = 0; for (let i = 0; i < n; i++) x += i; return x; };
service(self).onStart(async () => {
  await sleep(100);
  for (const b of BITS) {
    const end = now() + 100;
    while (now() < end) { if (b) spin(3000); await sleep(1); }
  }
});

// receiver.js (scratch): read the bits from how late 1 ms sleeps wake up.
service(self).onStart(async () => {
  await sleep(100);
  const late = [];
  for (let w = 0; w < 8; w++) {
    const end = now() + 100;
    let sum = 0, n = 0;
    while (now() < end) { const t0 = now(); await sleep(1); sum += now() - t0 - 1; n++; }
    late.push(+(sum / n).toFixed(3));
  }
  log("mean lateness per 100 ms window (ms): " + JSON.stringify(late));
  log("bits read: " + late.map((x) => (x > 0.5 ? 1 : 0)).join(""));
});
```

With a manifest `tc.json` that lists the two worlds, each with a 20 ms budget, a 320 KB heap and nothing else:

```
$ printf 'wait 1000\nquit\n' | $PC/build/picocalc-host -v -j js -m tc.json -u uart.log -o out - 2>&1
calibration: the 100k loop is 200002 steps, so a step is 665 virtual ns; fib(20) is 43783 steps = 29.1
    virtual ms (31 ms on the device)
[     0.000] booting 2 worlds from tc.json
[   909.892] receiver: mean lateness per 100 ms window (ms):
    [3.203,0.086,4.034,3.203,0.047,0.082,3.203,0.047]
[   909.911] receiver: bits read: 10110010
```

The bits sent were 10110010. A 1 delays the receiver's wake-ups by 3.2 to 4.0 ms on average, a 0 by less than 0.09 ms, and the receiver reads every bit correctly at 10 bits per second with this crude encoding. The host build is deterministic, so every run gives these numbers. The channel runs through the lane's scheduling decisions and touches no hardware state, and no proof of storage-channel freedom would cover it unless it covered the scheduler. seL4's confidentiality proof does so by assuming a static partition schedule. seL4's domain scheduler switches between domains "according to a dynamically configurable, time-triggered schedule", and a thread "will only run when that domain is active" [seL4 manual 2026, §6.3], so a domain's turn comes round at fixed times whatever other domains do; the theorem holds between such domains, "not between arbitrary threads" [seL4 proofs 2026]. In that sense PicoCalc's lane is a single domain, in which every world is scheduled by priority with every other. Closing the channel would mean a fixed time partition per group of worlds, paid for in idle lane time. This section has established what time protection requires of a kernel, and that PicoCalc OS, even with a perfect language boundary, has a measurable channel through its scheduler.

3.7 What the proofs establish, and where they stand

The seL4 proofs are a chain of theorems, each connecting one description of the kernel to the next, and each with stated assumptions [Heiser 2020, §3].

The first and largest is **functional correctness**: the C implementation refines an abstract specification in higher-order logic, so the possible behaviours of the C code "are a subset of those allowed by the abstract model" [Heiser 2020, §3.1]. Completed in 2009 for 8,700 lines of C [Klein et al. 2009], it rules out null-pointer dereferences, buffer overflows and undefined behaviour, because each would be a behaviour the specification does not allow. The second is **binary correctness**. Sewell, Myreen and Klein extended the verification "from 9500 lines of C source code to the binary level", proving that the compiled code refines the C semantics, for "binaries generated by unmodified gcc 4.5.1 at optimisation level 1", and

most of seL4 at level 2 [Sewell et al. 2013]. The proof is automatic. A disassembler written in HOL4 and a formal model of the instruction set turn the binary into a graph language, the C program is translated into the same language inside Isabelle, and SMT solvers show the two equivalent piece by piece [Heiser 2020, §3.1]. With it, the compiler and linker drop out of the trusted base.

On top of the abstract specification sit the **security proofs**. Integrity says that a thread cannot change data it was not explicitly given write access to [Sewell et al. 2011]. Confidentiality, proved as "a variant of intransitive noninterference" for "the actual 8,830 lines of C code that implement seL4" [Murray et al. 2013], says that a thread cannot read or infer data it was not explicitly given access to, which rules out covert storage channels. Availability says that a thread cannot prevent another's authorised use of resources [Heiser 2020, §3.1]. The comprehensive account of 2014 reports the whole stack at "480 000 lines of Isabelle proofs and specifications", and its cost: 2.2 person-years to design, write and test the kernel, and about 20 person-years for the correctness proof, of which 9 went into frameworks and tools and 11 into seL4 itself [Klein et al. 2014, §1 and §4.5]. The proofs have since grown past a million lines, and changes to the kernel are accepted only if the proofs are updated with them [Heiser 2020, §3].

The assumptions are stated as carefully as the theorems. The functional correctness proof assumes "correctness of handwritten assembly code, boot code, management of caches, and the hardware" [Klein et al. 2014, §1]; the boot code alone is about 1.2 thousand lines that are not verified [Klein et al. 2014, §4.8]. No verified configuration covers device address translation (an SMMU or IOMMU), debug and printing interfaces, or kernel start-up [seL4 verified configurations 2026]. The security proofs add their own. They hold for "a correctly configured system" [Heiser 2020, §3.1], meaning one whose capabilities match an access-control policy. The confidentiality proof further assumes that all interrupts except the timer are disabled, that no partition can destroy resources shared across partitions, that DMA is disabled, and that partitions are scheduled by a static round-robin schedule known to all of them [Klein et al. 2014, §5.4]. And it says nothing about time: "there are known covert channels in seL4 not covered by this theorem", because "none of our formal models talk about time" [Klein et al. 2014, §5.4].

Configuration is itself a proof obligation. capDL, a language that describes every object and capability of a system, has an initialiser whose design was proved to leave the system in a state isomorphic to the description it is given [Klein et al. 2014, §6.1, Theorem 9]; Chapter 4 returns to it. The seL4 Foundation reports what the proofs have meant in practice: there have been "no functional correctness defects in verified code" since the proof was com-

pleted in 2009, while defects have been found in unverified configurations and in every class of code the proofs assume, including boot code [seL4 proofs 2026]. Each proof in the chain replaces trust in code by trust in a shorter statement, and what remains (hardware, assembly, boot, configuration) is listed.

The proofs cover particular architectures and configurations. The functional correctness proof for 64-bit Arm (AArch64) was completed in April 2024 [seL4 news 2024]; the integrity proof followed in April 2025, for the first time including hypervisor mode and floating point [Proofcraft 2025]; and the confidentiality proof in August 2026, completing the security proofs for AArch64 [seL4 news 2026b]. The functional correctness of the MCS configuration was proved for RISC-V in June 2026 [seL4 news 2026a], and the roadmap gives the third quarter of 2027 for AArch64 [seL4 roadmap 2026]. Table 3.3 summarises the verified configurations in November 2026.

Table 3.3. Properties proved for the main verified configurations of seL4, as listed in its documentation in November 2026. MCS configurations are listed separately because their verification is in progress.

Configuration	Functional correctness	Integrity and availability	Confidentiality	Notes
AArch32 (ARMv7-A)	yes, with fast path	yes	yes	binary correctness; model-level proof of the capDL initialiser
AArch64 (ARMv8-A, kernel in EL2)	yes, with fast path	yes	yes	floating point and hypervisor mode included
RISC-V 64-bit	yes, no fast path	yes	yes	no floating point, no hypervisor mode
x86-64	yes, no fast path	no	no	
MCS (RISC-V 64-bit)	C-level proof completed in June 2026	not listed	not listed	AArch64 MCS planned for the third quarter of 2027

These rows matter for PicoCalc OS. The Microkit, on which Chapter 4 and the first pass build, always uses the MCS configuration [Microkit manual 2026], so a Microkit system today runs on a kernel whose MCS proofs are complete only at C level and only on RISC-V. And PicoCalc's own processor is outside every row. The ESP32-P4's high-performance cores are 32-bit RISC-V cores (RV32IMAFC) with machine and user privilege modes and up to 32

PMP regions [Espressif 2026b]. seL4 builds every RISC-V address space from page tables [seL4 manual 2026, §7.1.1], and RISC-V's page-based translation is controlled from supervisor mode [Waterman et al. 2021], which these cores do not implement. seL4, verified or not, cannot be the kernel under PicoCalc OS on this board.

What PicoCalc OS cannot have is a proof of this kind. Its isolation rests on QuickJS, about 80,000 lines of C in the trusted base (*Small Worlds*, Chapter 8), and a proof about the nucleus would assume that engine's memory safety as seL4's proofs assume the hardware's. What it can have is the discipline that made seL4's proofs possible. Its invariants can be stated as precisely as seL4's theorems; *Small Worlds*, Chapter 9 states the confinement property of handles and proves it by hand. Its configuration can be checked completely before any world starts, which is what the first pass does with the manifest. Its effects can be split as seL4's system calls are, where a checking phase "validates any arguments and confirms the authority to execute the call" and an execute phase "performs the call and never fails" [Klein et al. 2014, §3.2]. PicoCalc's `op_h_revoke` checks the caller's authority when it is called and defers `run_revoke`, which cannot fail, to the end of the segment; every effect of a segment is deferred in the same way. And each invariant can be attacked by a world written to break it, with the result kept. This section has established what each seL4 theorem says and assumes, where the verification stands, why none of it applies to PicoCalc's hardware, and which of its habits transfer; Chapter 8 asks what PicoCalc OS can put in the place of each proof.

KEY POINTS

- Every seL4 resource is a kernel object and every system call except `seL4_Yield` is an invocation of a capability held in the caller's CSpace; PicoCalc OS applies the same partitioned design only to hardware handles.
- seL4 has no kernel heap; all memory after boot is untyped, objects are created only by retyping untyped memory the caller holds, and so a thread cannot exhaust kernel memory others need; this partitioning was essential to the integrity and confidentiality proofs.
- PicoCalc's nucleus allocates messages and framebuffers from the C heap; the first pass bounds every shared table by per-world quotas checked at boot (54 of 128 handle slots in the shipped manifest), which reproduces the static half of seL4's rule but allows no transfer of quota.

- Endpoints are synchronous and unbuffered, badges identify the grant a message was sent with, a single-use reply capability returns each call, and notifications are coalescing semaphores; PicoCalc's ask, send and publish are all asynchronous and queued.
- seL4's derivation tree is one level deep below each original, revocation removes the children and keeps the capability, and passing a capability copies it; PicoCalc's handles nest to any depth, revocation marks the handle and its descendants without freeing them, and passing shares the handle, so taking back requires narrowing first.
- MCS scheduling contexts bound a thread to a budget per period, and passive servers run on their callers' scheduling contexts; PicoCalc's budgets limit single segments, crash the world that exceeds them, and are not lent with asks.
- Time protection requires flushing, partitioning and padding of shared hardware state; PicoCalc OS has a measurable covert channel through its lane, 8 of 8 bits received correctly between two worlds the manifest forbids to communicate.
- The seL4 proofs show that the C code refines its specification, that the binary refines the C code, and that the specification enforces integrity, availability and confidentiality, assuming correct hardware, assembly, boot code, cache management and configuration, and saying nothing about time.
- seL4 cannot run on the ESP32-P4's cores, which have no supervisor mode; PicoCalc OS can still state its invariants precisely, check its configuration completely at boot, separate checks from effects, and test every invariant with attacks.

seL4 made authority, memory and eventually time into objects so that a proof could follow each of them from the system call to the hardware. PicoCalc OS cannot carry that proof, but the decisions behind it can be read as design rules, and the first pass took three of them: authority written down before the system runs, no shared pools, and a boot that checks the whole description before any world starts. The first of these came to PicoCalc OS through the layer seL4's builders put on top of the kernel to make it usable, a static description of the whole system. Chapter 4 reads that layer, the Microkit, and the operating system built with it.

EXERCISES

1. BASIC For each row of Table 3.1, say what a PicoCalc OS world can do that an seL4 thread holding no capability to the corresponding object could not, or explain why the row has no counterpart.

2. BASIC Following the rules quoted in Section 3.4, draw the derivation tree after these operations: retype an untyped capability *U* into an endpoint *E*; mint *E* with badge 7 into *C1*; copy *C1* into *C2*; copy *E* into *C3*; send *C2* to another thread through an endpoint with the grant right. Which capabilities does revoking *C1* delete? Revoking *E*? Revoking *C2*?

3. ANALYSIS `handle_revoke_tree` repeats a pass over the 128-entry table until a pass marks nothing. Give the worst case number of passes for a chain of *d* narrowings, as a function of where the handles sit in the table, and the best case. Design an index that would make revocation proportional to the number of descendants, and say what it would cost per handle and which other functions in `nucleus/hw.c` would have to maintain it.

4. CODE Reproduce the revocation experiment of Section 3.4 in a scratch copy of `js/` (not in the repository), using the `snapshot` command. Then make `borrower` release both handles after the revocation and show, from a third snapshot, which handle slots are freed and how each world's `used.handles` changes. Explain the result from `handle_release` in `nucleus/hw.c`.

5. CODE Reproduce the covert channel of Section 3.6. Measure how the bit error rate changes when the sender's loop is 1,000, 300 and 100 iterations, and when the receiver runs at priority 3. What is the highest bit rate you can reach with no errors?

6. ANALYSIS seL4's confidentiality theorem assumes that DMA is disabled and that partitions are scheduled by a fixed round-robin schedule. Name the mechanisms of PicoCalc OS that would violate the analogous assumptions if you tried to state a confidentiality property for worlds, and say which of them the channel of Section 3.6 uses.

7. ANALYSIS Using the `stats` output of the host build after 1,500 ms (`cpu ms` per world), give each of the thirteen worlds an MCS-style budget and period that its measured use would fit with a factor of two to spare. Which worlds would need a period shorter than their polling interval, and which budgets would be smaller than PicoCalc's current `budget_ms`? What would change for the sandbox?

8. PROJECT Change passing in PicoCalc OS so that a handle sent in a message is copied: the receiver gets a new child handle of the sender's, as an seL4 transfer derives a copy. Work out what must change in `check_grants`, `can_pass`, the handle quotas, the compositor's `attach` and the confinement proof of *Small Worlds*, Chapter 9, implement it in a scratch copy, and test it with the revocation experiment.

9. DISCUSSION The seL4 proofs list their assumptions. Write the corresponding list for a hypothetical proof that PicoCalc's nucleus enforces Property 4 (confinement of handles), including what it would have to assume about QuickJS, the prelude, the C library and the board. Which assumption is the weakest?

10. DISCUSSION seL4's revocation keeps the revoked capability and deletes its children; PicoCalc's revokes the handle named as well. Give one protocol between worlds for which each choice is the more convenient, and say which you would choose for PicoCalc OS.

FURTHER READING

[Heiser 2020] The seL4 Foundation's own introduction: the proof chain, capabilities, mixed criticality and the Microkit, in thirty pages.

[Klein et al. 2014] The comprehensive account of the verification: the kernel's design for verification, every proof, its assumptions and its cost.

[Elphinstone and Heiser 2013] Section 3.4.3 explains why L4's kernel heap had to go and lists the properties of seL4's untyped memory.

[seL4 manual 2026] Chapters 2 to 6 define the object model, the derivation tree, endpoints, notifications and scheduling contexts precisely.

[Murray et al. 2013] How confidentiality was stated and proved for the real kernel, and which covert channels the proof leaves out.

[Lyons et al. 2018] Scheduling contexts, passive servers and timeout faults, with the measured overheads.

[Ge et al. 2019] The requirements for time protection and an implementation of them in seL4.

CHAPTER 4 PART I · THE L4 LINEAGE

Static systems: Microkit and LionsOS

The seL4 theorems of Chapter 3 hold for a correctly configured system, and configuring one by hand means creating and connecting hundreds of kernel objects. seL4's builders answered with a sequence of tools for describing a whole system statically and generating its configuration: CAmkES and capDL first, then the seL4 Microkit, on which the operating system LionsOS and its driver framework are built. The price of each is that the system's structure is fixed before it runs. This chapter explains why that price is worth paying for embedded systems, reads a Microkit system description element by element, describes LionsOS's way of building drivers and services from small isolated components, reports what has been verified about the Microkit, and sets the Microkit beside PicoCalc OS's manifest, which the first pass (Part II) modelled on it. The reader should finish able to read a Microkit system description, state the rule that keeps its calls free of cycles and why, and say precisely which Microkit concepts PicoCalc OS has, in what form, and which it lacks.

4.1 Why describe a system statically

seL4 is a set of mechanisms, and a system built directly on them is large in a way its design does not show. The white paper's example is a system of four native components and a Linux virtual machine; "even this simple system maps to hundreds if not thousands of seL4 objects" [Heiser 2020, §3.2]. The authors of LionsOS describe the consequence for adopters. Because seL4 leaves the management of all physical memory, the kernel's included, to user

level, building a working system "requires deep expertise", and a number of seL4-based deployment projects were "apparently abandoned a few years after they started" [Heiser et al. 2025, §1]. A system assembled by hand has a second problem. The security proofs say nothing about it unless its capabilities match a policy, which someone must check.

A static description addresses both. The Microkit is designed for "a static architecture", meaning "one where system resources are assigned up-front at system initialisation time" [Microkit manual 2026]. The designer lists the components, the memory each may map, and the channels between them; a tool turns that list into the seL4 objects and capabilities that implement it. The white paper calls the result the Microkit's promise to the designer: the SDF "is a faithful representation of the possible interactions", and "no interactions are possible beyond those defined in the SDF" [Heiser 2020, §3.2]. A question about what the running system can do becomes a question about a file, which can be answered once, before the system runs, by reading the file or by checking it mechanically.

The restriction suits the systems the Microkit targets. LionsOS's authors argue that the embedded domain "can be served with a static system architecture", a set of components known at configuration time, and that dependable embedded systems "generally cannot over-commit resources, which is what makes the static architecture work" [Heiser et al. 2025, §3]. A static architecture still allows components to be loaded late or replaced, and a subsystem such as a virtual machine to manage its own share of resources dynamically [Heiser et al. 2025, §3]. What cannot change at run time is the set of components and who may reach whom.

PicoCalc OS was already static in the first respect. Its thirteen worlds are fixed by `js/system.json`, which before the first pass gave each world its file, claims, budget and heap. It was not static in the second. Who could send to or ask whom, and who could post or hear a channel, was decided at run time by names and by each world's own declarations (Chapter 5). The first pass made the manifest the complete description. Each world's entry now lists the channels it provides and needs and the worlds it may send to and ask, and the boot fails, listing every problem, if the description is inconsistent [First pass 2026]. The compositor's entry shows the shape:

▸ history/first-pass/js/system.json

```
{ "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
    "budget_ms": 80,  "mem_kb": 768,
  "needs": ["frames", "keys", "power", "worlds.status"], "sends": ["display", "power", "apps"],
      "asks": ["display"],
  "handles": 6, "bufs": 2, "mbox": 128 },
```

Two earlier tools prepared the way for the Microkit, and it still uses one of them. **CAmkES**, the component architecture of Kuz and colleagues, was designed for microkernel-based embedded systems with "a minimal, low-overhead, core" that is "highly modular and therefore flexible and extensible" [Kuz et al. 2007]. The white paper calls CAmkES "a predecessor of the Microkit" and "a component framework for a statically-architected system", whose "abstractions are higher level yet more complex than those of the Microkit" and which "introduces significant overheads" [Heiser 2020, §2.2]. The seL4 driver framework described later in this chapter was first measured against a CAmkES-based driver framework that preceded it [Heiser et al. 2026, §10.2.1].

capDL describes a system at the level below, as a list of every kernel object and every capability. Kuz, Klein, Lewis and Walker argued that to be certain a capability system provides the isolation it should, "systems need to have explicit descriptions of their capability distributions" [Kuz et al. 2010]. A capDL description is more explicit than the kernel's own state. Authority that seL4 holds implicitly, in virtual-memory mappings or in a thread's state, appears in capDL as capabilities of special types, so a capDL description lists the complete protection state of the system [Klein et al. 2014, §6.1]. From such a description an initialiser, the first user-level program to run, creates the objects, distributes the capabilities and starts the threads, and its design has been proved to leave the system in a state that matches the description up to the renaming of physical addresses [Klein et al. 2014, §6.1, Theorem 9]. capDL also works in the other direction, since a running system can be dumped as a capDL description and compared with what was intended [Klein et al. 2014, Fig. 15].

The Microkit is built on capDL. Its tool turns the system description into a capDL specification and embeds it in the boot image, and at boot "the 'initial task' is the capDL initialiser", which "initialises and starts the system in conformance with the capDL specification" [Microkit manual 2026]. The designer never writes capDL; the white paper calls it "a purely internal representation" [Heiser 2020, §3.2].

PicoCalc OS has no separate layer. `kernel_boot` in `nucleus/kernel.c` reads the manifest, checks it, and builds the worlds' authority (`may_send`, `may_ask`, the quotas) directly, so one function plays the parts of the Microkit tool, capDL and the initialiser. The host build's `snapshot` command is the counterpart of a capDL dump. A short script (written for this chapter, in a scratch directory `$S`) prints two worlds' entries and the handles they hold from a snapshot taken 1.5 s after boot:

```
$ printf 'wait 1500\nsnapshot\nquit\n' | ./build/picocalc-host -u "$S"/uart.log -o "$S"/out - | grep
    '^RESULT' | python3 "$S"/dump.py
{"name": "display", "priority": 3, "sends": [], "asks": [], "needs": [], "quota": {"handles": 4,
    "ask_slots": 8, "mbox": 64, "mbox_kb": 128, "bufs": 1, "turn_ms": 8000}}
{"name": "compositor", "priority": 2, "sends": ["power", "display", "launcher", "sysmon", "repl",
    "snake", "faults", "keys", "about"], "asks": ["display"], "needs": ["frames", "keys", "power",
    "worlds.status"], "quota": {"handles": 6, "ask_slots": 8, "mbox": 128, "mbox_kb": 128, "bufs": 2,
    "turn_ms": 8000}}
{"id": 3, "type": "spi", "rights": "rwg", "parent": 0, "holders": ["display"]}
{"id": 4, "type": "dma", "rights": "rwg", "parent": 0, "holders": ["compositor"]}
{"id": 5, "type": "dma", "rights": "r", "parent": 4, "holders": ["display", "compositor"]}
```

The group `apps` in the manifest appears expanded into seven names, and handle 5, the read-only view of the framebuffer, appears with both its holders. Compared with capDL, PicoCalc's dump lacks two things. Nothing checks it against the manifest, and nothing proves that `kernel_boot` built what the manifest says. This section has established what a static description buys (a system whose possible interactions are stated in one file and established at start-up, at the cost of fixing its structure in advance), the two layers beneath the Microkit, one for components and one for capabilities, and that PicoCalc OS folds both into its boot.

4.2 The Microkit model

The Microkit began as an seL4 RFC by Ben Leslie, proposed in November 2020 as the "seL4 Core Platform" and approved under its present name in September 2023, with the aim of providing "a small and simple OS" for IoT, cyber-physical and other embedded uses [Leslie 2020]. The white paper says it "reduces the complex seL4 API to literally a handful of simple abstractions" [Heiser 2020, §2.2]. There are five that matter here: protection domains, entry points, memory regions, channels, and the two ways of using a channel.

A **protection domain** (PD) "provides a thread of control that executes within a fixed virtual address space", whose isolation "is enforced by the underlying hardware MMU"; a system has at most 63 [Microkit manual 2026]. A PD is single-threaded by design, "to keep the programming model and implementations simple" [Microkit manual 2026, Rationale]. Its scheduling attributes are a priority from 0 to 254, a budget and a period in microseconds (the MCS scheduling context of Chapter 3), and a `passive` flag. A passive PD has its scheduling context taken away after initialisation; from then on it runs on its notification object's scheduling context when notified, and on its caller's when called [Microkit manual 2026].

A PD has no `main`. It has **entry points**, which libmicrokit calls: `init` once at boot, `notified(ch)` when a notification arrives on a channel, `protected(ch, msginfo)` when another PD calls it, and `fault` if it has children whose faults it handles. "A PD will not execute any other entry point until `init` has finished", and once it is running one entry point it runs no other until that one returns [Microkit manual 2026]. The computational model is "a set of isolated components reacting to incoming events" [Microkit manual 2026]. There are no timers and no sleep; after initialisation, all activity starts from an interrupt, which arrives as a notification on a channel identifier of its own and is masked until the PD acknowledges it.

A **memory region** is a range of memory, a multiple of the page size, mapped into one or more PDs, each with its own virtual address, caching attribute and permissions (any combination of read, write and execute except write-only). Regions in main memory are zero-initialised; a region with a fixed physical address typically holds device registers or DMA buffers [Microkit manual 2026]. Memory regions are the Microkit's way to move bulk data; nothing else carries more than 64 words.

A **channel** connects exactly two PDs; "there are no multi-party channels". Each end has its own identifier, from 0 to 62, and a PD has no way to name another PD except through a channel identifier [Microkit manual 2026]; Chapter 5 discusses what that local naming does for authority. The limit of 63 channels per PD comes from the size of seL4's notification word, part of which libmicrokit keeps for its own use [Microkit manual 2026, Rationale].

A channel is used in two ways. `microkit_notify(ch)` signals the other end. It never blocks, carries no data beyond the channel itself, and works "in either direction on a channel regardless of priority"; notifications coalesce, so a PD notified three times before it runs enters `notified` once [Microkit manual 2026]. `microkit_ppcall(ch, msginfo)` is a **protected procedure call** (PPC). It passes up to 64 words, runs the other PD's `protected` entry point, blocks the caller until that entry point returns, and copies back up to 64 words of result.

Two rules govern it: a protected call is possible only if the callee has "strictly higher priority than the caller", and "the overall call graph between PDs must form a directed, acyclic graph", so that A calls B calls C is valid (given the priorities) while A calls B calls A is not [Microkit manual 2026; Microkit tutorial 2026]. Protected procedures must also execute in bounded time; "in the present version the caller must trust the callee to conform" [Microkit manual 2026].

The priority rule does more than forbid deadlock. The manual's rationale gives three reasons [Microkit manual 2026, Rationale]. It "prevents deadlocks", because a chain of calls that always rises in priority can never reach a PD already waiting in the chain. It "reflects the notion that the callee operates on behalf of the caller", so the callee should be preemptible only where the caller would have been. And it makes a server at the top of its callers' priorities a resource server that provides mutual exclusion while "avoiding unbounded priority inversions through the immediate priority ceiling protocol". Calls between PDs of equal priority could achieve the same, but "this would be much harder to statically analyse for loop-freedom (and thus deadlock-freedom)". The cost is priority levels spent when one logical component is split into several PDs, which the authors judge unlikely to matter for the systems the Microkit targets.

The system description file (SDF) records all of this in XML. The smallest example in the Microkit's sources with a protected procedure is a passive server and its client, reproduced here as it appears in the repository, in Microkit's own format [Microkit sources 2026]:

▸ Microkit SDK: example/passive_server/passive_server.system

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
 Copyright 2022, UNSW

 SPDX-License-Identifier: BSD-2-Clause
-->
<system>
    <protection_domain name="server" priority="100" passive="true">
        <program_image path="server.elf" />
    </protection_domain>

    <protection_domain name="client" priority="99">
        <program_image path="client.elf" />
    </protection_domain>

    <channel>
        <end pd="server" id="0" />
        <end pd="client" id="0" pp="true" />
    </channel>

</system>
```

The server runs at priority 100 and the client at 99, so the client's end may carry `pp="true"`; the server, passive, runs only on the client's scheduling context while it serves the call. Both ends may notify, the default. Each PD knows the other only as channel 0. This section has established the Microkit's five abstractions and its two rules for calls, and how a system description writes them down.

4.3 LionsOS and the seL4 driver framework

LionsOS is "an operating system for security- and safety-critical embedded systems", built on the Microkit, "designed with verification in mind", with "a highly modular design driven by strict separation of concerns" [Heiser et al. 2025, Abstract]. Its design principles are four: strict separation of concerns, with each module having one purpose and each concern contained in one module; least privilege; design for verification, through narrow interfaces and simple implementations; and use-case specific policies, chosen for the system at hand from a small set instead of generalised [Heiser et al. 2025, §3]. The authors summarise them as "radical simplicity". The Microkit and the seL4 Device Driver Framework (sDDF) are "the

foundation on which LionsOS is built" [Heiser et al. 2025, Acknowledgements]. The sDDF's design document states its aims: performance "comparable to Linux in-kernel drivers", sharing of devices between clients, "strong separation of concerns", and eventually formal verification of its components, on "an asynchronous, zero-copy transport layer" [Heiser et al. 2026, Chapter 1].

Separation of concerns shapes the driver first. An Ethernet driver "does no more than abstracting the specific NIC as a generic Ethernet device" [Heiser et al. 2025, §4.1.1]. Sharing the device between clients, translating client addresses into device addresses, and managing caches belong to a separate component, the **virtualiser**, one for transmit and one for receive. Between components, data moves through memory regions of two kinds. Metadata regions hold queues of buffer references: transmit-available and transmit-free, receive-available and receive-free. Data regions hold the buffers themselves. The driver's job needs only the references, so the data region "is not mapped into the driver's address space", following the principle of least privilege [Heiser et al. 2025, §4.1.1]. Where clients must not see each other's input, a **copier** component between the receive virtualiser and each client copies packets into a per-client region; it can be inserted or removed without changing the components on either side [Heiser et al. 2025, §4.1.3]. Figure 4.1 shows the arrangement for one client.

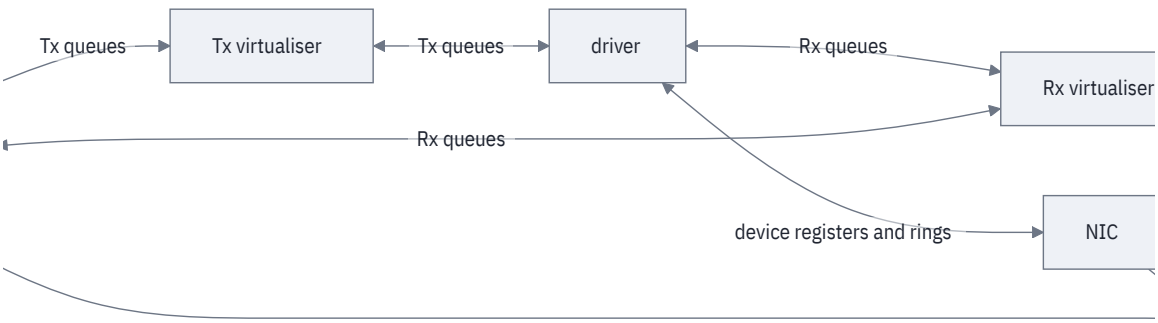


Figure 4.1. The sDDF arrangement for an Ethernet device with one client, after Figure 1 of the LionsOS paper. Each box is a protection domain; queues of buffer references live in metadata regions, packets in data regions, and the driver maps no data region at all.

Every queue has one producer and one consumer, so it can be a simple lock-free array whose size is a power of two [Heiser et al. 2025, §5.2]. A producer notifies the consumer only when it adds to an empty queue whose consumer has asked to be signalled, and the reverse for a queue becoming non-full [Heiser et al. 2025, §5.3]. Such protocols are easy to get wrong, and the authors use the SPIN model checker to prove each component's signalling free of deadlock [Heiser et al. 2025, §6.2.2]. Two properties follow from the design. All "policy-independent state is held in shared memory", which makes it easy "to restart a failed component without affecting the rest of the system", apart from a transient glitch in latency [Heiser et al. 2025, §5.2]. And the IP stack becomes a library linked into each client, which "takes the complex (and probably buggy) protocol stack out of the system's TCB" [Heiser et al. 2025, §4.2].

The framework applies the same separation to shared buses. In the sDDF's I2C subsystem, one virtualiser serves all clients and one driver drives the controller; "only one client may have access to any particular bus address", the address is used to route responses back, and a request for an address the client does not own is returned as an error [Heiser et al. 2026, §5.3.1]. Small devices such as sensors are simpler still, and their drivers are invoked by protected procedure call [Heiser et al. 2026, §4.1].

The authors report two kinds of evidence for the design, both measured by them on their own hardware. On size, their native Ethernet driver for the i.MX8 NIC is 569 source lines against 4,775 for Linux's, and a point-of-sale terminal built with LionsOS has about 3.1 thousand lines of trusted code, all written from scratch, plus 66 thousand lines of library code that cannot break isolation [Heiser et al. 2025, §6.2.1]. On speed, a UDP echo benchmark on an Avnet MaaXBoard (NXP i.MX8MQ, four Cortex-A53 cores run at 1 GHz, 1 Gb/s NIC), with everything on one core, shows LionsOS keeping up with the full 1 Gb/s load with about a third of the core spare, while Linux saturates the core at about 500 Mb/s; LionsOS "uses less than half the CPU of Linux for handling the same load", and its round-trip times are three to ten times shorter [Heiser et al. 2025, §6.1 and §6.3.1]. The sDDF document measures the cost of the modularity on an i.MX8M Mini at 1.8 GHz. Moving from two protection domains per packet to five, with a copy, raises CPU use from 12 to 19 per cent at 100 Mb/s and from 60 to 77 per cent at high load, and the five-component system still outperforms Linux "by more than a factor 1.5 (low load)" [Heiser et al. 2026, §10.1–10.2.2]. These are networking results on Arm boards; they say nothing about a 400 MHz microcontroller without an MMU. This section has established how LionsOS builds I/O from single-purpose components joined by shared queues, what that buys, and what its authors measured.

4.4 Verifying the Microkit

The Microkit's promise, that the system description is a faithful representation of the possible interactions, needs three things to be true: libmicrokit must implement the Microkit's abstractions correctly on top of seL4, the tool must translate the system description into capabilities that implement it, and seL4 must enforce those capabilities. The third is the subject of Chapter 3. The first two are the subject of a verification project at UNSW. Paturel, Subasinghe and Heiser reported its first steps in 2023: a formal specification of the library, an automated proof of its functional correctness, and a verified mapping from the system description to capDL [Paturel et al. 2023]. The full report of 2024 describes both proofs as "fully automated (push-button)". The library's C code is translated into a graph language and checked against its specification by SMT solvers, and each translation of a system description into capDL is checked in Isabelle [Weibel et al. 2024, Abstract and §12.1].

The report numbers the proofs a complete argument needs. Proof 0, an initialiser generated from capDL, exists from earlier work; Proofs 1 (libmicrokit) and 2 (the generation of capDL) are the report's results; Proof 3, that the simplified kernel specification used by the SMT solvers follows from seL4's abstract specification, is still required; and Proof 4, the verification of the protection domains' own code, is a separate effort in progress [Weibel et al. 2024, §1.3, Fig. 1.4]. The tool already supports such work. Given `--viper-output`, it writes a description of each protection domain's capability table in the Viper verification language, "for verification purposes" [Microkit manual 2026]. LionsOS was designed with Proof 4 in mind; its authors cite the finding that "verification effort scales with the square of the specification size", which favours many small components with narrow interfaces over a few large ones [Heiser et al. 2025, §2.1].

The report is explicit about its limits [Weibel et al. 2024, §11]. The verified library is the version of mid-2022, which differs from the current one. The verified initialiser supports neither the MCS kernel nor 64-bit architectures, so the complete chain was demonstrated on a fork of the Microkit that uses the non-MCS 32-bit kernel. Because the proofs about frame capabilities were not yet available for AArch64, the verifier proves "that no caps were missed by the translator" but cannot yet show "that no unnecessary caps were issued". Proof 3 is still missing. And the compiler, the linker and the SMT solvers are trusted. The white paper says the same in brief: the proofs about the Microkit and capDL "are not yet complete and based on an older version of the Microkit" [Heiser 2020, §3.2]. And the Microkit always uses seL4's MCS configuration, whose verification is in progress [Microkit manual 2026]; Chapter 3 gives its status.

The structure of this work is worth carrying over to PicoCalc OS, where it has a counterpart for each part. The library corresponds to the prelude and the host calls in `nucleus/vm.c`; the translation to `kernel_boot` turning the manifest into `may_send`, `may_ask` and quotas; the enforcement to `may_talk` and `handle_use` at run time. The distinction between "no capability missed" and "no unnecessary capability issued" applies too. A test that boots the shipped manifest and finds every world working shows the first; only an attack, a world trying what its entry does not grant, shows the second, which is why the first pass's verification rests on attack worlds (Chapter 8). None of PicoCalc's three parts is proved. This section has established what has been verified about the Microkit, what has not, and how its decomposition maps onto PicoCalc OS.

4.5 Microkit and PicoCalc OS side by side

The first pass took the Microkit as its model for the manifest, one file, read before anything runs, that says who may reach whom. Table 4.1 sets the two systems side by side, concept by concept.

Table 4.1. Microkit concepts and their counterparts in PicoCalc OS after the first pass. Microkit facts are from its manual; PicoCalc facts are from js/system.json, nucleus/nucleus.h and FIRST-PASS.md.

Microkit	PicoCalc OS	The main difference
system description file, read by the Microkit tool when the image is built	`js/system.json`, read by `kernel_boot` when the system boots	PicoCalc checks at boot and lists every problem; nothing is generated
protection domain: one thread, one address space, isolated by the MMU	world: one QuickJS runtime, one heap, isolated by the language	
`init`	`onStart`	a start turn may await; `init` runs to completion
`notified(ch)`	an `on` handler for a `send` or a channel post	a notification carries no data and coalesces; a message carries JSON and is queued
`protected(ch, msginfo)`, returning a result	an `on` handler whose return value answers an `ask`	a PPC blocks its caller; an ask returns a promise, and the callee's turn may await
PPC only to a strictly higher priority; acyclic call graph	`asks` must form no cycle; an ask to a lower priority is allowed and noted at boot	
channel: two ends, an identifier local to each, `pp` and `notify` flags	`sends` and `asks` lists per world, `provides` and `needs` per channel	worlds are named globally; groups such as `apps` name several
memory region mapped into PDs with permissions	a framebuffer handle, narrowed to `r` and passed at run time	the sharing is fixed in the SDF; in PicoCalc the compositor creates it
priority 0 to 254	driver 3, service 2, app 1, the sandbox 0	
budget and period; `passive`	`budget_ms` per segment; no period, no passive worlds	
interrupt as a notification, acknowledged with `microkit_irq_ack`	none; drivers poll on `every` timers	

Microkit	PicoCalc OS	The main difference
fault to the monitor or to the parent's `fault` entry	crash, restart by the nucleus's supervisor with backoff	
at most 63 PDs, 63 channels per PD, 64 words per PPC	at most 32 worlds, 16 names per list, 64 KB per message	

The clearest case is the framebuffer. In the Microkit, the compositor and the display driver would share a memory region, mapped read-write in one and read-only in the other, and the SDF would say so. A sketch of that description, written for this chapter in Microkit's format (it is not part of either system), follows; 0x32000 bytes is one 320 × 320 frame at two bytes per pixel:

▸ A sketch in Microkit's format: two PicoCalc OS worlds as protection domains

```
<system>
    <memory_region name="framebuffer" size="0x32_000" page_size="0x1000" />

    <protection_domain name="display" priority="3">
        <program_image path="display.elf" />
        <map mr="framebuffer" vaddr="0x2_000_000" perms="r" setvar_vaddr="fb" />
        <!-- the SPI controller's registers would be a second region, with a phys_addr, mapped uncached
            -->
    </protection_domain>

    <protection_domain name="compositor" priority="2">
        <program_image path="compositor.elf" />
        <map mr="framebuffer" vaddr="0x2_000_000" perms="rw" setvar_vaddr="fb" />
    </protection_domain>

    <channel>
        <end pd="compositor" id="0" pp="true" />
        <end pd="display" id="0" />
    </channel>
</system>
```

In PicoCalc OS, the same arrangement is made at run time. The manifest gives the compositor the `dma` claim and two framebuffers; the compositor allocates one, narrows it to a read-only view and sends the view to the display driver, which keeps it:

▸ **js/system/compositor.js**

```
function attach(s) {
  // the view handed to a previous display instance died with it; let go of our hold on it, or every
  // restart of the driver would keep one more handle alive
  if (s.view) s.view.release();
  s.view = s.fb.narrow("r");
  send("display", "attach", { fb: s.view });
```

▸ **js/drivers/display.js**

```
  .on("attach", (s, { fb }) => {
    if (s.fb && s.fb !== fb) s.fb.release();
    s.fb = fb;
  })
```

The result is the same read-only sharing, and no pixel passes through a message. The display's `flush` handler gives the nucleus the view with `s.spi.pixels`, and the nucleus reads the pixels from the framebuffer for the SPI transfer. But the arrangement is not in the description, so it needed everything Chapter 3 and *Small Worlds*, Chapter 9 describe to be safe at run time: a confinement rule so that the display cannot pass the view on, revocation and release so that a restarted driver does not leave views behind, and the `attach` above, re-sent whenever the display restarts. A static region needs none of these, and a run-time handle allows things a region cannot, such as a second compositor or a framebuffer that changes size.

The flush shows the difference in calls. The compositor's draw handler asks the display to flush the damaged rectangles and waits for the reply; the display's handler queues the SPI transfers and awaits them:

▸ js/drivers/display.js

```
  .on("flush", async (s, { rects }) => {
…
    for (const [x, y, w, h] of rects) {
      // queued together: the transfers run back to back on the bus once this turn ends
      jobs.push(s.spi.cmd(CASET, span(x, x + w - 1)), s.spi.cmd(RASET, span(y, y + h - 1)),
          s.spi.cmd(RAMWR), s.spi.pixels(s.fb, x, y, w, h));
      px += w * h;
    }
    await Promise.all(jobs);
```

The ask goes from the compositor (priority 2) to the display (priority 3), upward, so it would be a legal PPC. The handler could not be written as a protected procedure, though. A Microkit PD executes no other entry point until the current one returns, so a `protected` entry point cannot wait for the notification that a DMA transfer has finished; it would have to poll the controller inside the call, or the protocol would have to become a request in a shared queue and a completion notification, the sDDF's shape. PicoCalc OS lets a turn span its awaits, settling the transfers' promises inside the turn while the display's mailbox stays closed (*Small Worlds*, Chapter 3), and that is what makes the flush a single ask.

The priority rule is where the first pass deliberately departed. Of the five `asks` edges in the shipped manifest, three go upward (compositor to display, keys to keyboard, faults to flaky) and two go downward, from the REPL and from the faults app to the sandbox, which runs at priority 0, below every other world, so that user code runs only when nothing else is waiting. The Microkit would forbid both. The first pass kept the Microkit's consequence, an acyclic graph checked at boot, and replaced the priority condition by a note in the log; Chapter 6 explains the check and what the note warns of.

Three further differences follow from PicoCalc's messages. They carry JSON payloads of up to 64 KB where a PPC carries 64 words and a notification none, so PicoCalc needs no shared memory to move data and has none except framebuffers. They queue, in mailboxes the first pass bounds with quotas, where a Microkit notification coalesces and a PPC waits for its callee. And a segment is all or nothing; its effects are applied when it ends, and a segment that crashes applies none (*Small Worlds*, Chapter 3), which no Microkit entry point promises. This section has established the correspondence between the two systems, concept by concept, and where PicoCalc OS kept, changed or lacks what the Microkit provides.

KEY POINTS

- seL4 systems are built from hundreds of kernel objects; CAmkES, capDL and the Microkit let a designer describe the system in one file and generate the objects and capabilities from it, at the price of fixing the structure before the system runs.
- capDL describes a system's complete protection state; the Microkit translates its system description into capDL, and a capDL initialiser establishes that state at boot.
- A Microkit system consists of single-threaded protection domains with `init`, `notified` and `protected` entry points, memory regions mapped with permissions, and channels between exactly two PDs with identifiers local to each end.
- A protected procedure call carries up to 64 words, blocks its caller, may go only to a strictly higher priority, and the call graph must be acyclic; notifications go either way, carry no data and coalesce.
- LionsOS builds I/O from single-purpose components: a driver that only abstracts the device, virtualisers that share it, optional copiers, single-producer single-consumer queues in shared memory, and data regions the driver does not map.
- Its authors report, on their Arm boards, an i.MX8 Ethernet driver of 569 lines against Linux's 4,775 and UDP echo at 1 Gb/s with less than half Linux's CPU use; the sDDF's I2C design gives each bus address to exactly one client.
- The Microkit's library and its translation to capDL have automated proofs for an older version on a non-MCS 32-bit kernel; the proofs show no capability is missed, not yet that none is in excess.
- PicoCalc's manifest plays the part of the system description, with worlds for PDs, `onStart` and handlers for entry points, `sends` and `asks` for channels and a narrowed framebuffer handle for a memory region; it allows asks downward in priority, carries JSON messages that queue, and shares no memory except framebuffers.

The Microkit showed that the authority in a system can be written down completely before the system runs and enforced by the kernel from then on. Part II applies that lesson to PicoCalc OS, starting with the authority its manifest did not record, the right to send to,

ask, post to and hear other worlds. Chapter 5 shows the attacks that the missing authority allowed and the first pass's repair.

EXERCISES

1. BASIC In the passive server example of Section 4.2, what would the manual's rules say if the client's priority were raised to 100? To 101? What would change if `pp="true"` were moved to the server's end?

2. BASIC Write a Microkit system description, in the style of the sketch in Section 4.5, for the keyboard driver, the compositor and the keys app. Decide which interactions become protected procedure calls, which become notifications, and where key events would have to be stored, given that a notification carries no data.

3. ANALYSIS List every `asks` edge in `js/system.json` with the priorities of its two ends, and confirm the count of upward and downward edges in Section 4.5. Propose a change to the priorities, or to the REPL's protocol with the sandbox, that would make every ask legal under the Microkit's rule, and say what it would cost the drivers.

4. ANALYSIS PicoCalc's channels (`keys` , `power` , `frames` , `worlds.status`) carry JSON and queue. For each, say whether a Microkit notification plus a shared memory region could replace it without losing information when the receiver is slow, given that notifications coalesce. Which would need a queue of more than one entry, and how many entries?

5. CODE Write a script that reads `js/system.json` and prints a Microkit-style system description sketch: one protection domain per world with its priority, one channel per pair of worlds that send to or ask each other, with `pp="true"` on asking ends. Have it flag every channel that the Microkit would reject. Compare its flags with the `note:` lines in the host build's boot log (`./build/picocalc-host -v` with `wait 1000`).

6. CODE Extend the `snapshot` dump of Section 4.1 into a check: for every world, compare `sends` , `asks` and `needs` in the snapshot with the world's entry in `js/system.json` (with groups expanded), and report any authority in the running system that the manifest does not give. Run it on the shipped system and on a scratch copy of the manifest with one world's `sends` removed.

7. ANALYSIS The sDDF's I2C virtualiser gives each bus address to exactly one client. The keyboard and power drivers both claim `i2c0:0x1f`. Describe the arrangement that rule would force on PicoCalc OS, which world would own the address, and what the keyboard and power worlds would become; compare with Chapter 12.

8. DISCUSSION LionsOS argues for use-case specific policies. Classify these PicoCalc OS policies as built into the nucleus or held in a world, and as use-case specific or general: restart backoff and the stop after five crashes in ten seconds, the compositor's focus rule, dropping channel posts to a full mailbox, the kind-to-priority mapping. Which would LionsOS's principles move?

9. PROJECT In a scratch copy of the repository, change the flush protocol to the sDDF's shape: the compositor sends `flush` with the rectangles and the display driver sends `flushed` when the transfers finish, instead of an ask. Measure key-to-pixels latency with the host build's `latency` command before and after, and explain the difference from the turn and segment rules.

10. DISCUSSION The Microkit's verification shows that no capability is missed but not yet that none is in excess. For each check `kernel_boot` performs on the manifest (see `FIRST-PASS.md`), say whether it guards against missing authority, excess authority, or neither, and name the attack world in `bench/rogue/` that tests the excess side, if there is one.

FURTHER READING

[Microkit manual 2026] The Concepts and Rationale chapters are the shortest complete statement of the Microkit model and of the reasons for its call rules.

[Heiser 2020] Section 3.2 explains how the Microkit, capDL and the initialiser fit into seL4's verification.

[Kuz et al. 2010] The argument for describing a capability system's distribution explicitly, with the language that does it.

[Heiser et al. 2025] LionsOS's design principles, its driver and service structure, and its measurements against Linux.

[Heiser et al. 2026] The driver framework in detail, device class by device class, including the I2C design that bears on PicoCalc's shared STM32.

[Weibel et al. 2024] What was proved about libmicrokit and the translation to capDL, and an unusually candid list of what was not.

Authority from the manifest

Before the first pass, the right to talk in PicoCalc OS was a matter of names. A world could send to or ask any world whose name it knew, and it could publish on, or listen to, any channel that its own file said it provided or needed. The nucleus checked that a name existed and that a declaration had been made. It never asked who had granted either. Three attacks follow from that, each by a world with no claim and no handle, and each worked.

This chapter shows the attacks, then reads the argument inside the L4 family about the same question: how a kernel should name the receiver of a message, and who decides whether a message may be sent at all. L4 began with global thread identifiers, tried a hierarchy of "clans" with "chiefs", abandoned it, and settled on endpoint capabilities; the seL4 Microkit then fixed every channel in a file written before the system runs. The first pass of PicoCalc OS takes the Microkit's half of that lesson. The manifest now states which channels each world may publish and hear, and whom it may send to and ask; the nucleus checks the statement once at boot and enforces it at every send, ask and publish. At the end of the chapter the reader can explain each defect, the L4 idea behind its repair, the code that implements the repair, and the authority that is still not a capability.

5.1 Three ways to act without authority

`bench/rogue.py` is the test for this chapter and the next two. For each attack it copies the `js/` tree, adds one or two attack worlds from `bench/rogue/` to the copy's manifest, runs the host build for a few virtual seconds, and decides from the kernel log whether the attack worked. It was run against a build of the nucleus from before the first pass, and that run's

output is kept in `bench/rogue-before.txt`; every "before" line below comes from that file. The attack worlds are ordinary worlds with no claims and no handles, and their manifest entries hold nothing beyond a name and a file except where the text says otherwise.

The first attack forges a key press. The compositor hears the `keys` channel and forwards every key to the app in front, so a world that can publish on `keys` can type into any app:

▸ bench/rogue/rogue_key.js

```
// Attack 1: forge a key press. Declares itself a provider of "keys" and publishes a down-arrow,
// which the compositor would forward to the app in front.
app(self).provides("keys").onStart(async () => {
  await sleep(1500);
  try {
    publish("keys", { code: 0xb6, key: "down", ch: "", state: "down", caps: false });
    log("OUTCOME forge_key published a down-arrow on keys");
  } catch (e) { log("OUTCOME forge_key refused: " + e.message); }
});
```

Before the first pass, `.provides("keys")` in the world's own file was all it took. When a world started, the nucleus copied the channels its file declared into its table, and `publish` checked that table. Nothing compared the declaration with the manifest, and nothing prevented a second provider:

▸ bench/rogue-before.txt

```
forge_key: publish a forged key press after declaring provides("keys")
  ATTACK SUCCEEDED: [  1502.383] compositor <- keys from rogue_key
      {"code":182,"key":"down","ch":"","state":"down","caps":false}
```

The receiving side had the same hole. A world that declared `.needs("keys")` heard every key, whichever app was in front. This sketch, written for this chapter and not part of the repository, was added to a scratch copy of the manifest and run on the same pre-pass build while the script typed `hi`:

```
// A sketch for this chapter: a world that declares it needs "keys" and logs what it hears.
app(self).needs("keys").on("keys", (s, e) => { if (e.state === "down") log("heard " + e.key); });
```

```
[  1213.379] snoop: heard h
[  1313.379] snoop: heard i
```

The second attack forges a screen. After every segment in which an app's screen changed, the nucleus publishes the app's rendered tree to the compositor on the kernel's own channel `frames`, and the compositor stores it under the app name it finds in the payload. The attack sends the compositor a message of type `frames` directly:

▸ bench/rogue/rogue_frame.js

```
// Attack 2: forge another app's screen by sending the compositor a "frames" message directly.
app(self).onStart(async () => {
  await sleep(1500);
  try {
    send("compositor", "frames", { app: "launcher", title: "Home", frame: { tree: { t: "text", s:
        "forged by rogue", kids: [] }, hints: "" } });
…
```

The compositor's handlers are registered by message type, and the prelude's dispatch gives a handler the payload, the type and the sender's name. Both a channel post and a `send` arrive as an ordinary message with a type, so a handler has no way to tell which operation produced it. `rogue.py` runs this attack twice: once from a world with no right to send to the compositor, and once from a world whose manifest entry says `"sends": ["compositor"]`, a right that was ignored before the first pass and is enforced after it. The second variant is the one that matters for the design, because it shows that a list of permitted receivers alone would not close the hole:

▸ bench/rogue-before.txt

```
forge_frame: send the compositor a frames message (no right to send to it)
  ATTACK SUCCEEDED: [  1502.390] compositor <- frames from rogue_frame
      {"app":"launcher","title":"Home","frame":{"tree":{"t":"text","s":"forged by rogue","kids":
      []},"hints":""}}
forge_frame_allowed: the same from a world allowed to send to the compositor
  ATTACK SUCCEEDED: [  1502.390] compositor <- frames from rogue_frame2
      {"app":"launcher","title":"Home","frame":{"tree":{"t":"text","s":"forged by rogue","kids":
      []},"hints":""}}
```

The forged screen replaces the home screen and stays until the launcher next draws; in a run that captured the panel at 4.56 s, three seconds after the forgery, it was still there. *Small Worlds*, Chapter 9, shows the screens before and after a forged key and a forged frame [Small Worlds 2026].

The third attack uses another world's authority. The display driver holds the SPI bus, by its claim, and a read-only view of the compositor's framebuffer, which the compositor passes it. Its `flush` handler sends to the panel whatever rectangles of that framebuffer it is asked for. The attack asks for all of them:

▸ bench/rogue/rogue_flush.js

```
// Attack 3: make the display driver flush the whole framebuffer on demand (a confused deputy).
app(self).onStart(async () => {
  await sleep(1500);
  try {
    const r = await ask("display", "flush", { rects: [[0, 0, 320, 320]] });
    log("OUTCOME flush_display answered " + JSON.stringify(r));
  } catch (e) { log("OUTCOME flush_display refused: " + e.message); }
});
```

▸ bench/rogue-before.txt

```
flush_display: ask the display driver to flush the whole screen
  ATTACK SUCCEEDED: [  1543.872] rogue_flush: OUTCOME flush_display answered
      {"px":102400,"ms":41.399000000000115}
```

The driver moved 102,400 pixels, the whole 320 × 320 panel, and kept the SPI bus busy for 41.4 virtual ms, at the request of a world that holds neither the bus nor the framebuffer. This is Hardy's confused deputy [Hardy 1988]: a program that holds authority from two sources (its own claim and its client's request) and cannot say which one it means to use. *Small Worlds*, Chapter 9, analyses this deputy in detail; its harm is to availability, since the pixels are the compositor's own.

Table 5.1. The authority attacks of bench/rogue.py, with this chapter's listening sketch, and what each relied on before the first pass.

Attack world	What it does	What the nucleus checked before	Before the first pass
`rogue_key`	publishes a down-arrow on `keys`	that its own file declared `provides("keys")`	the compositor forwarded the key
(sketch) `snoop`	listens on `keys`	that its own file declared `needs("keys")`	it heard every key typed
`rogue_frame`	sends the compositor a `frames` message	that a world called `compositor` exists	the home screen was replaced
`rogue_frame2`	the same, with `"sends": ["compositor"]`	the same	the home screen was replaced
`rogue_flush`	asks the display driver to flush the screen	that a world called `display` exists	102,400 pixels, 41.4 ms of SPI

The three defects have one cause. Lampson's access matrix [Lampson 1971] has a row for each subject and a column for each object, and an entry says what the subject may do to the object. In PicoCalc OS before the first pass, the entries for "may publish on `keys`", "may hear `keys`", "may send to the compositor" and "may ask the display" were either written by the subject itself or not kept at all. The nucleus did mediate every message, as Saltzer and Schroeder's principle of complete mediation requires [Saltzer and Schroeder 1975], because every send passed through one function. That function compared each request with nothing.

5.2 Names, chiefs and endpoints in L4

The L4 family met the same question early and answered it three times. Its history is useful here because each answer was tried in a running system and abandoned or kept for reasons its builders wrote down.

The original L4 sent a message to a thread, named by a global thread identifier. Liedtke chose threads as destinations to avoid the cache and TLB cost of an indirection, and noted that ports could have been implemented with an overhead of about 12 per cent [Elphinstone and Heiser 2013, §3.2.3]. The kernel delivered the sender's unforgeable identifier to the receiver, which allowed a server to implement discretionary access control by ig-

noring messages it did not want. Elphinstone and Heiser explain why that was not enough. A server can be bombarded with spurious messages, the time spent receiving them and deciding which to discard is itself the attack, and such a denial of service "can only be avoided by kernel support" that prevents undesirable messages "being sent in the first place" [Elphinstone and Heiser 2013, §3.2.5]. PicoCalc OS before the first pass had exactly this design. World names are global identifiers, every handler is given `from`, the sender's name as the nucleus knows it, and the compositor could have checked `from` on every `frames` message. It did not, and it would still have paid for every message it threw away.

For mandatory control, the original L4 had **clans and chiefs**, which Liedtke had described and implemented for L3 [Liedtke 1992]. Processes were organised in a hierarchy of clans, each with a designated chief. Messages inside a clan went directly, but any message crossing a clan boundary, outgoing or incoming, was redirected to the clan's chief, which could inspect and modify it and so control the flow of messages between clans; the abstract of Liedtke's paper lists protection, remote communication, debugging and event tracing among its uses. Liedtke argued that the check cost two cycles per IPC, because clan membership was encoded in the thread identifier. Elphinstone and Heiser report what happened in practice. The low overhead applied only to direct messages; each redirection added two messages to a logically single round trip. The strict hierarchy was "probably the feature most cursed by people trying to build L4-based systems", and for mandatory access control the model "quickly deteriorated into a chief per process". Many implementations left it out or disabled it, which left them with no way to control IPC at all. Experiments with a more general redirection failed to gain traction, and the problem "was finally resolved with flexible capability-mediated access control to endpoints" [Elphinstone and Heiser 2013, §3.2.5; Heiser and Elphinstone 2016, §3.2.5]. They call clans and chiefs a prime example of kernel-enforced policy limiting the design space.

Global identifiers had further costs. A multithreaded server had to expose its internal structure to its clients, or funnel them through a gateway thread, and Elphinstone and Heiser note that the global identifiers also introduced covert channels. seL4 and Fiasco.OC, influenced by EROS [Shapiro et al. 1999], replaced thread identifiers by **endpoints**: a separate kernel object that holds the queue of waiting senders or receivers and buffers nothing. To identify clients without an endpoint per client, seL4 gives capabilities a badge, delivered to the receiver on each invocation [Heiser and Elphinstone 2016, §3.2.3]. A thread can send only through an endpoint capability it holds, so the reference to the receiver and the right to use it are one value. That is Hardy's remedy for the confused deputy, which the seL4 white paper states as the principle of "no designation without authority" [Heiser 2020, §4.2].

The same white paper names two further properties that a capability brings: **delegation**, by minting a new capability (possibly with fewer rights) and handing it on, and **interposition**, by handing a client a capability to a monitor instead of to the object itself. Those are the two things clans and chiefs tried to provide with a fixed hierarchy.

The seL4 Microkit, which Chapter 4 describes, adds a static description on top of endpoints. A Microkit system is a set of protection domains, and a PD can refer to another only "indirectly if there is a channel between them"; the channel identifier "is effectively a proxy identifier for the other PD" [Microkit manual 2026]. Every channel is declared in the system description file, with two ends, and each end says whether that protection domain may make a protected procedure call to the other (`pp`) and whether it may notify it (`notify`). When a protected procedure is called, the channel identifier passed to the callee's entry point identifies the caller indirectly, and it "is passed by the system and is unforgeable" [Microkit manual 2026]. The Microkit is designed for systems "with a static architecture", whose resources are assigned when the system is initialised, so nothing creates a channel at run time.

Table 5.2. How each design names a message's receiver and who decides that a message may be sent.

Design	The sender names the receiver by	Who decides that it may send	When	Can the right be passed on?
Original L4	a global thread identifier	nobody; the receiver may ignore it	—	—
L4 with clans and chiefs	a global thread identifier	the chief of each clan boundary crossed	at each message	—
seL4	an endpoint capability in its own capability space	whoever gave it the capability	when the capability is granted	yes (minted, possibly with fewer rights)
Microkit	a channel identifier local to the protection domain	the system description	when the image is built	no
PicoCalc OS before the first pass	a global world name	nobody; the receiver may ignore it	—	—
PicoCalc OS after the first pass	a global world name	the manifest (`sends`, `asks`, `provides`, `needs`)	at boot	no

Table 5.2 places PicoCalc OS in this history. Before the first pass it sat in the first row. Endpoint capabilities would have meant changing every world's API, because every `send` and `ask` names a world, and passing endpoints between worlds needs rules as careful as the ones handles already have. The Microkit's static description could be adopted without changing a line of any world. The first pass therefore keeps names and takes the Microkit's lesson, that who may talk to whom is decided once, in one file, before anything runs, and that the kernel enforces only what that file grants.

5.3 The manifest decides

The first pass adds five keys to the manifest. Here is the shipped `js/system.json` in full:

▸ history/first-pass/js/system.json

```
{
  "groups": {
    "apps": ["launcher", "sysmon", "repl", "snake", "faults", "keys", "about"]
  },
  "defaults": { "handles": 4, "ask_slots": 8, "mbox": 64, "mbox_kb": 128, "bufs": 1, "turn_ms": 8000 },
  "worlds": [
    { "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
        "budget_ms": 20,  "mem_kb": 320,
      "provides": ["keys"] },
    { "name": "power",      "file": "drivers/power.js",     "claims": ["i2c0:0x1f"],
        "budget_ms": 20,  "mem_kb": 320,
      "provides": ["power"] },
    { "name": "display",    "file": "drivers/display.js",   "claims": ["spi2"],
        "budget_ms": 30,  "mem_kb": 320 },
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
        "budget_ms": 80,  "mem_kb": 768,
      "needs": ["frames", "keys", "power", "worlds.status"], "sends": ["display", "power", "apps"],
         "asks": ["display"],
      "handles": 6, "bufs": 2, "mbox": 128 },
    { "name": "launcher",   "file": "apps/launcher.js",     "claims": ["sys.read"],
        "mem_kb": 384,
      "sends": ["compositor"] },
    { "name": "sysmon",     "file": "apps/sysmon.js",       "claims": ["sys.read"],
        "mem_kb": 512 },
    { "name": "repl",       "file": "apps/repl.js",
        "mem_kb": 384,
      "asks": ["sandbox"] },
    { "name": "sandbox",    "file": "system/sandbox.js",
        "budget_ms": 200, "mem_kb": 512, "priority": 0 },
    { "name": "snake",      "file": "apps/snake.js",
        "mem_kb": 320 },
    { "name": "faults",     "file": "apps/faults.js",       "claims": ["sys.admin", "sys.read"],
        "mem_kb": 384,
      "needs": ["worlds.status"], "sends": ["flaky"], "asks": ["flaky", "sandbox"] },
    { "name": "flaky",      "file": "system/flaky.js",
        "mem_kb": 256 },
    { "name": "keys",       "file": "apps/keys.js",
        "mem_kb": 320,
      "needs": ["keys"], "asks": ["keyboard"] },
    { "name": "about",      "file": "apps/about.js",        "claims": ["sys.read"],
        "mem_kb": 320 }
  ]
}
```

`provides` lists the channels a world may publish, and `needs` the channels it hears. `sends` and `asks` list the worlds, or groups of worlds, it may send to and ask. `groups` names lists of worlds so that one entry can stand for several: the compositor sends `focus`, `blur` and `key`

to whichever app is in front, so its entry says `"sends": [..., "apps"]` instead of naming seven apps. The quota keys in `defaults` and on the compositor's entry are the subject of Chapter 7, and `turn_ms` of Chapter 6. A world whose entry has no `sends` may send only to itself, and one with no `provides` may publish nothing. The default is no authority.

Three rules apply to channels. A channel has at most one provider, so two worlds cannot both claim to be the keyboard. `frames` and `worlds.status` belong to the kernel, and no world may provide them. A need that nobody provides is only noted in the log, so a world may listen for a channel that a particular configuration does not include. The graph the shipped manifest describes is exactly the one the code uses [First pass 2026]: the launcher sends to the compositor; the compositor sends to the display, the power driver and the apps, and asks the display; the REPL asks the sandbox; the keys app asks the keyboard driver; the faults app sends to and asks `flaky` and asks the sandbox.

`kernel_boot` reads the manifest with QuickJS's JSON parser, in a runtime it creates for the purpose and frees afterwards. It reads `defaults` and `groups` first, then every world's entry, and only then the `sends` and `asks` lists, because a list may name a world that appears later in the file. Each name is resolved to a bit set of world identifiers:

▸ nucleus/kernel.c

```
// a world or group name as a bit set of world ids; 0 and an error when it is neither
static uint64_t resolve(const char *name, const char *where, const char *key) {
  world_t *T = world_by_name(name);
  if (T) return 1ull << T->id;
  for (int g = 0; g < n_groups; g++) if (!strcmp(groups[g].name, name)) {
    uint64_t m = 0;
    for (int i = 0; i < groups[g].n; i++) { world_t *M = world_by_name(groups[g].members[i]); if (M) m
        |= 1ull << M->id; }
    return m;
  }
  merr("%s: \"%s\" names \"%s\", which is neither a world nor a group", where, key, name);
  return 0;
}
```

▸ history/first-pass/nucleus/kernel.c

```
    int ns = mlist(ctx, o, "sends", names, MAX_LIST, where);
    for (int k = 0; k < ns; k++) W->may_send |= resolve(names[k], where, "sends");
    int na = mlist(ctx, o, "asks", names, MAX_LIST, where);
    for (int k = 0; k < na; k++) W->may_ask |= resolve(names[k], where, "asks");
```

After this loop a world's authority to talk is two 64-bit words in its `world_t`, `may_send` and `may_ask`, one bit per world (the kernel runs at most 32). Groups exist only while the manifest is read; afterwards there is no difference between naming the apps one by one and naming the group. The channel rules follow:

▸ nucleus/kernel.c

```
// channels: the kernel's own, and at most one provider for each of the others
for (int i = 0; i < K.n_worlds; i++) {
  world_t *W = K.worlds[i];
  for (int c = 0; c < W->n_provides; c++) {
    const char *ch = W->provides[c];
    if (kernel_channel(ch)) { merr("world \"%s\": \"%s\" is published by the kernel; no world may
        provide it", W->name, ch); continue; }
    for (int j = 0; j < i; j++) if (world_provides(K.worlds[j], ch)) merr("channel \"%s\" has two
        providers, %s and %s; a channel has one", ch, K.worlds[j]->name, W->name);
  }
}
```

Every problem goes through `merr`, which prints one line naming the manifest and counts it, and the boot continues checking until the end, so a manifest with four mistakes reports four lines rather than one per attempt. `bench/firstpass.py` checks this with a manifest broken in four ways at once (an unknown name in the launcher's `sends`, the REPL providing `frames`, the snake app providing `keys`, and a default handle quota of 20). Run today, it prints:

```
$ python3 bench/firstpass.py
…
PASS  the manifest checks stop a bad boot, one line per problem
      world "launcher": "sends" names "nobody", which is neither a world nor a group
      world "repl": "frames" is published by the kernel; no world may provide it
      channel "keys" has two providers, keyboard and snake; a channel has one
      the worlds' handle quotas add up to 246; the kernel's handle table holds 128
      4 problems; the system does not boot
```

Checking at boot is possible because everything the check needs is in one file, and it is the right time because nothing has run yet. A mistake then costs a failed boot with a list of reasons instead of a world that misbehaves an hour later. Once the boot succeeds, the authority does not change while the system runs, so what remains for the nucleus at run time is a test of one bit and a short list. This section has shown the static half of the first pass. The next shows the dynamic half.

5.4 Two checks at run time

Every `send` and every `ask` passes through one function, `may_talk`, before anything else happens to the message:

▸ history/first-pass/nucleus/vm.c

```
// The first pass's checks on a send or an ask: the manifest must let W reach T this way, and the type
// may not be a channel T hears (channel messages come only from the channel's provider, by publish).
static bool may_talk(JSContext *ctx, world_t *W, world_t *T, const char *type, bool ask) {
  uint64_t set = ask ? W->may_ask : W->may_send;
  if (T != W && !(set >> T->id & 1)) {
    fail(ctx, "%s may not %s %s: its manifest entry does not list it under \"%s\"", W->name, ask ?
        "ask" : "send to", T->name, ask ? "asks" : "sends");
    return false;
  }
  if (world_needs(T, type)) {
    fail(ctx, "\"%s\" is a channel %s hears; only its provider can post it, by publish", type, T-
        >name);
    return false;
  }
  return true;
}
```

Its callers are the `send` operation and the `ask` operation. In `op_send` the check comes straight after the name lookup and before the size check, the handle checks and the mailbox quota:

▸ history/first-pass/nucleus/vm.c

```
    world_t *T = to ? world_by_name(to) : NULL;
    JSValue r = JS_UNDEFINED;
    if (!T) r = fail(ctx, "there is no world called \"%s\"", to ? to : "");
    else if (!may_talk(ctx, W, T, type ? type : "", false)) r = JS_EXCEPTION;
    else if (too_big(ctx, json)) r = JS_EXCEPTION;
    else if (check_grants(ctx, W, hs, nh)) {
```

The `ask` operation is the same, with one extra test that predates the first pass (a world may not ask itself, for the reason Chapter 6 explains):

▸ history/first-pass/nucleus/vm.c

```
    world_t *T = to ? world_by_name(to) : NULL;
    if (!T) r = fail(ctx, "there is no world called \"%s\"", to ? to : "");
    else if (T == W) r = fail(ctx, "a world cannot ask itself (it would wait for its own turn to
        end)");
    else if (!may_talk(ctx, W, T, type ? type : "", true)) r = JS_EXCEPTION;
```

`fail` throws a JavaScript exception in the calling world, so a refused `send` or `ask` fails at the call, synchronously, inside the sender's own segment. Nothing is queued, no deferred effect is created, and the receiver never runs. The placement follows Elphinstone and Heiser's argument that an unwanted message must be stopped before it is sent, because a receiver that has to look at it has already paid for it. The cost to the sender is a shift and a mask, and a scan of at most 16 channel names.

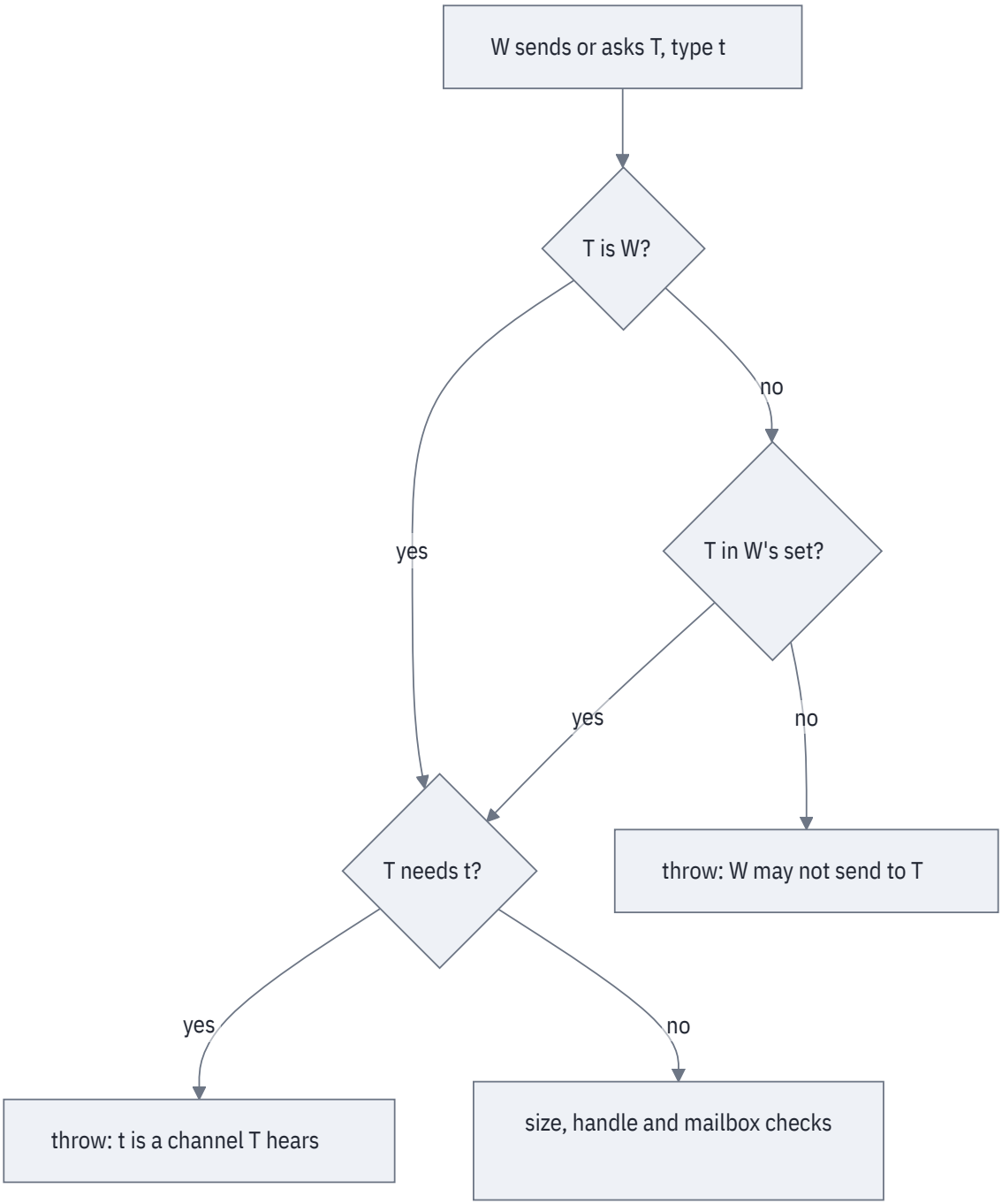


Figure 5.1. The decision may_talk makes for a send or an ask from world W to world T with message type t. W's set is may_send for a send and may_ask for an ask. Only a message that passes both tests goes on to the size, handle and

mailbox checks.

The second test is the **channel-type rule**: a message whose type is the name of a channel the receiver hears is refused, whoever sends it. It exists because of the attack `rogue_frame2` showed. The compositor's handler for `frames` cannot tell a kernel post from a `send`, because both arrive as a message of that type, and giving the handler the means to tell them apart would put a check in every receiver. Refusing the message at the sender keeps the receiver's code unchanged and makes the manifest's `provides` the only way to reach a channel's handlers. A world either provides the channel and publishes on it, or it cannot produce a message of that type for any world that hears it.

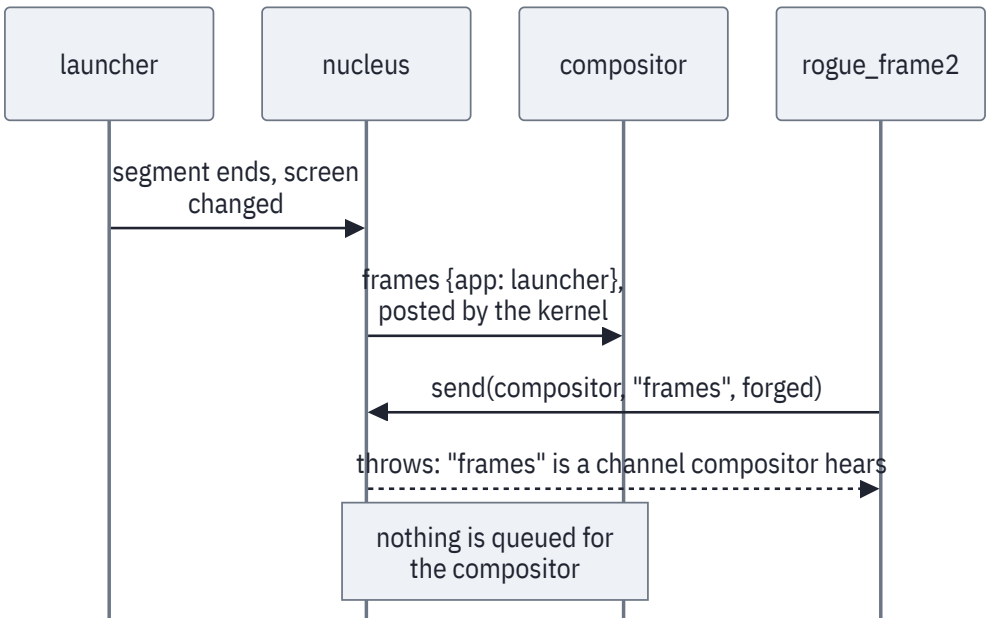


Figure 5.2. A forged frame from a world that may send to the compositor. The kernel's own frames post reaches the compositor; the send of the same type is refused at the call and nothing is queued.

Publishing needs no new check. `op_publish` has always refused a channel the world does not provide:

▸ nucleus/vm.c

```
    if (!ch || !world_provides(W, ch)) r = fail(ctx, "%s does not provide the channel \"%s\"", W->name,
        ch ? ch : "");
```

What changed is where `W->provides` comes from. It is now filled from the manifest at boot, and a world's own `.provides(...)` and `.needs(...)` declarations, which the prelude still collects, are checked against it each time the world starts:

▸ nucleus/vm.c

```
// The world's own .needs() and .provides() are checked against its manifest entry, which is the
// authority: declaring a channel the manifest does not give is a configuration error, and the world
// stays stopped until its file or the manifest is fixed.
static int read_meta(world_t *W, JSValue meta) {
…
      bool ok = false;
      for (int k = 0; s && k < lists[l].n; k++) ok |= !strcmp(lists[l].have[k], s);
      if (!ok && !W->config_error) {
        snprintf(W->last_error, sizeof W->last_error, "%s declares %s(\"%s\"), but its manifest entry
            does not give it that channel", W->name, lists[l].key, s ? s : "");
        W->config_error = true;
      }
```

A file may declare less than its manifest entry gives, but not more. A world that declares more fails its start, and the supervisor, which would otherwise restart a crashed world after a backoff, stops it at once:

▸ history/first-pass/nucleus/kernel.c

```
    if (recent >= 5 || W->config_error) {
```

Restarting would only repeat the failure, since the file and the manifest are what they are until someone edits one of them. The check runs at every start rather than once at boot because a world's source can change while the system runs. The workbench's hot reload (Chapter 9) gives a world a new file without a new boot, and a reloaded file that declares a

channel its entry does not give stays stopped with the same reason. The declarations stay in the files because the first pass changed no world API. A world's file still says what it publishes and hears, and the nucleus now makes sure the file and the manifest agree.

5.5 The attacks, afterwards

The same attack worlds, run with `python3 bench/rogue.py` on the current build, print this for the four authority attacks (the other three are the subjects of Chapter 6 and Chapter 7):

```
$ python3 bench/rogue.py
binary: build/picocalc-host

forge_key: publish a forged key press after declaring provides("keys")
  REFUSED: [     0.923] rogue_key crashed: rogue_key declares provides("keys"), but its manifest entry
      does not give it that channel

forge_frame: send the compositor a frames message (no right to send to it)
  REFUSED: [  1502.340] rogue_frame: OUTCOME forge_frame refused: rogue_frame may not send to
      compositor: its manifest entry does not list it under "sends"

forge_frame_allowed: the same from a world allowed to send to the compositor
  REFUSED: [  1502.340] rogue_frame2: OUTCOME forge_frame refused: "frames" is a channel compositor
      hears; only its provider can post it, by publish

flush_display: ask the display driver to flush the whole screen
  REFUSED: [  1502.336] rogue_flush: OUTCOME flush_display refused: rogue_flush may not ask display:
      its manifest entry does not list it under "asks"
…
all attacks refused
```

Each refusal comes from a different check. The key forger never runs its attack. At 0.923 ms, while it starts, `read_meta` finds a `provides` its entry does not give, and the kernel log adds `rogue_key cannot start until its file or its manifest entry is fixed; it stays stopped`. The first frame forger is stopped by the `sends` test, the second by the channel-type rule, and the flush request by the `asks` test, each as an exception the attack world catches and logs. The listening sketch fares like the key forger, stopped at its start with `snoop declares needs("keys"), but its manifest entry does not give it that channel`. Nothing reaches the compositor or the display driver, and the timestamps show that each refused call returns within the attacker's own segment.

These refusals change nothing for a world that stays inside its entry. No world API changed, and the first pass was checked against the build before it: every capture of the six `scripts/*.txt` is pixel-identical, natively and in WebAssembly, and every virtual-time figure of `bench/run.py` is unchanged [First pass 2026]. Chapter 8 describes how that was established and how strong such evidence is.

Because the authority is now in one file, it can be drawn. The workbench's Graph view (Figure 5.3) reads the manifest last applied to the live system, expands its groups, and draws one line for each `sends`, `asks` and channel edge; from the running system it takes only each world's kind, to place it in a column. Before the first pass there was nothing to draw, since every world could reach every other.

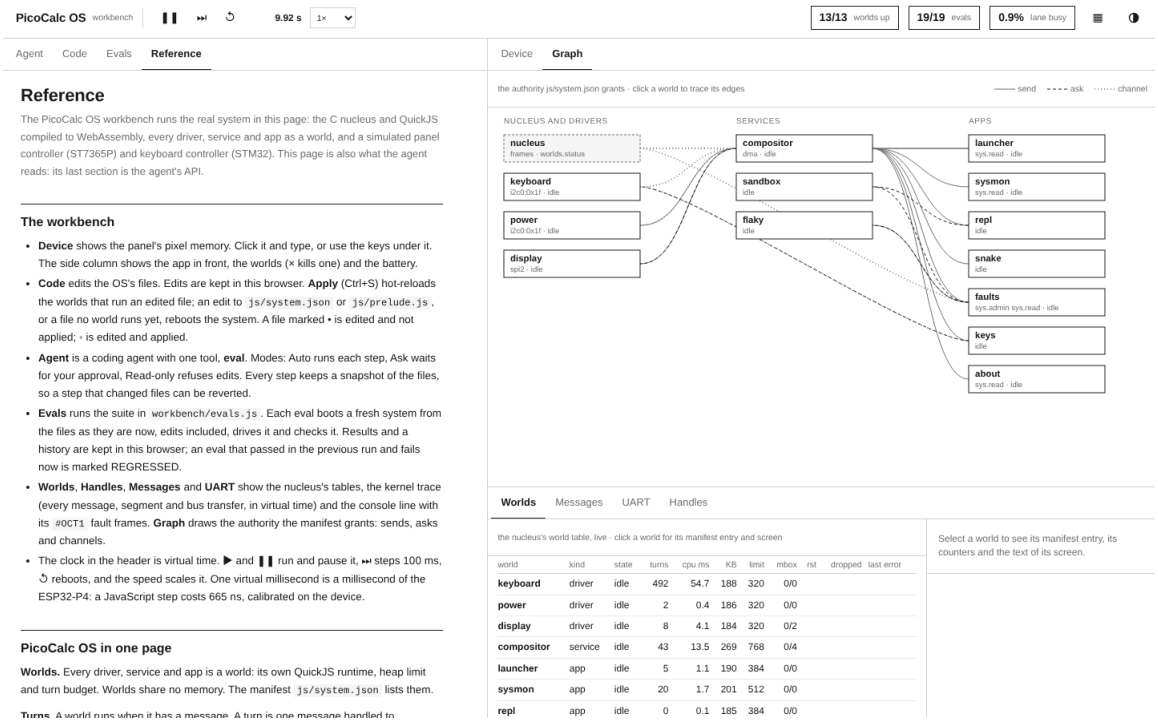


Figure 5.3. The workbench's Graph view of the shipped system: solid lines are sends, dashed lines asks and dotted lines channels, with the kernel's frames and worlds.status posts drawn from the nucleus box. Every edge in the figure is one the manifest grants, and none other can carry a message.

5.6 What is still not a capability

The first pass makes the access matrix explicit, static and enforced at the sender. It does not turn the right to talk into a capability, and the difference shows in four places.

First, authority is per world, not per message type. A world allowed to send to another may send it any type that is not a channel the receiver hears. `KNOWN-ISSUES.md` gives the example of the launcher, which may send the compositor `open` for any app. To check it, this chapter added a sketch world with `"sends": ["compositor"]` to a scratch copy of the manifest; it sends the compositor a `keys` message and then an `open` message for the faults app:

```
[  1502.327] opener: OUTCOME keys refused: "keys" is a channel compositor hears; only its provider can
    post it, by publish
[  1502.345] opener: OUTCOME open sent
[  1502.393] compositor <- open from opener {"app":"faults"}
[  1503.819] faults <- focus from compositor
```

The channel-type rule stopped the forged key, and the `open` went through: the faults app came to the front at the request of a world that is not the launcher. Any world the manifest lets send to the compositor can do the same.

Second, the first pass contains the confused deputy without curing it. The display driver will still flush whatever it is asked to flush; the first pass ensures that only the compositor can ask. Giving `rogue_flush` the entry `"asks": ["display"]` reopens the attack, with the same 102,400 pixels and 41.4 ms as before. Hardy's repair, which *Small Worlds*, Chapter 9, describes for this driver, would make the compositor pass its framebuffer view with each `flush`, so that a request without a view can do nothing. That is a change to a protocol, and the first pass changed none.

Third, there is no delegation. `may_send` and `may_ask` are fixed at boot, and a world cannot give another the right to send to it, or a narrower right, by passing something in a message. A protocol in which a server learns of a new client at run time and answers it later must be written into the manifest in advance, in both directions, or use `ask` so that the answer travels as a reply. There is also no interposition. A monitor cannot be placed between the compositor and the display driver without editing the compositor, because the compositor names the display driver itself.

Fourth, names are still global. A world is designated by a name every world can read, the `from` a handler receives is that name, and handles are still numbered in one table for the whole system. Elphinstone and Heiser's objections to global identifiers (information hiding and covert channels) therefore still apply in part.

All four have the same remedy, the one seL4 adopted: endpoints as kernel objects, created at boot from the manifest, held by worlds as handles with rights, passed with the same rules as the handles of *Small Worlds*, Chapter 9, and outliving the worlds that serve them. That remedy needs a new kind of handle, a rule for what passing it means, and changes to every world that names another, which is why it belongs to the second pass. Chapter 12 gives the design. The first pass establishes the precondition for that remedy, a complete statement of who may talk to whom, in one file, which the nucleus enforces at every call and refuses to boot without.

KEY POINTS

- Before the first pass, the right to publish or hear a channel came from a world's own declaration, and the right to send or ask came from knowing a name; three attacks by worlds with no claims forged a key, forged another app's screen and used the display driver as a confused deputy.
- Original L4 named threads by global identifiers and let receivers filter by the sender's identifier; Elphinstone and Heiser explain why only kernel support that stops unwanted messages from being sent prevents denial of service.
- Clans and chiefs gave L4 mandatory control by redirecting messages to a chief at clan boundaries; it cost two extra messages per redirected round trip, forced a rigid hierarchy and was abandoned for capability-mediated endpoints.
- seL4's endpoints make the reference to a receiver and the right to use it one value, and bring delegation and interposition; the Microkit adds a static description in which every channel between protection domains is declared before the system runs.
- The first pass keeps world names and takes the Microkit's static description: `provides`, `needs`, `sends`, `asks` and `groups` in the manifest, resolved at boot into bit sets, with every problem reported before the boot fails.

- `may_talk` checks every send and ask at the call, in the sender's segment, so a refused message costs the receiver nothing; the channel-type rule refuses any message whose type is a channel the receiver hears, so only a channel's provider can reach its handlers.
- A world's own channel declarations are checked against its manifest entry at every start, and a world that declares more than it is given stays stopped until its file or the manifest is fixed.
- Authority is still per world rather than per type, cannot be delegated, and is expressed in global names; endpoint capabilities, the second pass's remedy, need protocol changes the first pass deliberately avoided.

The manifest now says who may talk to whom, and the nucleus refuses everything else. It also says who may ask whom, and an ask is the one message that makes the asker wait. Waiting is where the next defects are: two worlds that ask each other, a reply nobody waits for, and a turn that never ends. Chapter 6 takes them up, together with L4's experience of IPC timeouts and the Microkit's rule that calls go only upward.

EXERCISES

1. `BASIC` For each of `provides`, `needs`, `sends`, `asks` and `groups`, name the function in the nucleus that enforces it and say whether it acts at boot, when a world starts, or at each call.
2. `BASIC` Expand the groups in `js/system.json` and list every world the compositor may send to. Then list every world that may send to the compositor. Which of the compositor's handlers can be reached by a world other than the kernel and those senders?
3. `CODE` Add `"provides": ["keys"]` to the snake entry of `js/system.json` and predict the line the boot prints. Undo it, then add `.provides("keys")` to the first line of `js/apps/snake.js` instead, leaving the manifest alone, and predict whether the system boots and what the kernel log says about snake. Check both with `./build/picocalc-host -v` and a script containing `wait 1000`.

4. CODE In a copy of `bench/rogue/rogue_frame.js`, replace the `send` by `publish("frames", …)` and predict the error the world logs. Then give the attack world `"provides": ["frames"]` in its manifest entry and predict the boot error. Which line of `kernel_boot` or `vm.c` produces each?

5. CODE Add `"asks": ["display"]` to the `rogue_flush` scenario in `bench/rogue.py` and run it. Explain the result in terms of Section 5.6. Then change `js/drivers/display.js` and `js/system/compositor.js` so that every `flush` request must carry the framebuffer view it flushes from, and show that the attack fails even with the `asks` entry.

6. ANALYSIS The `opener` sketch of Section 5.6 brought the faults app to the front. Propose a manifest syntax for per-type authority (for example, which types the launcher may send the compositor), show the changes `may_talk` and `kernel_boot` would need, and count the entries the shipped manifest would gain.

7. ANALYSIS Suppose the compositor checked `ctx.from` in its `frames` handler instead of the nucleus refusing the message at the sender. List what a refused forged frame would consume in each design (segments, mailbox entries, virtual time), using the costs of *Small Worlds*, Chapter 12, and relate the difference to Elphinstone and Heiser's argument in Section 5.2.

8. ANALYSIS Groups are expanded into bit sets at boot. If a new app is added to the `apps` group, what authority does it gain, and which existing worlds gain authority over it? Discuss whether naming a group in `sends` respects the principle of fail-safe defaults [Saltzer and Schroeder 1975].

9. DISCUSSION Design a "chief" for PicoCalc OS using only `sends` and `asks`: a world through which every message between the apps and the services must pass. What does it cost per message on the virtual clock, what can it enforce that the first pass cannot, and how do your answers compare with Elphinstone and Heiser's account of clans and chiefs?

10. PROJECT Implement per-type authority in the nucleus, keeping the list form of `sends` and `asks` valid. Add a scenario to `bench/rogue.py` that the current build fails (an `open` from a world that is not the launcher) and that yours refuses, and check that `bench/firstpass.py` and the captures of `scripts/*.txt` are unchanged.

FURTHER READING

[Elphinstone and Heiser 2013] Sections 3.2.3 to 3.2.5 are the source for this chapter's history: why thread identifiers gave way to endpoints and why clans and chiefs were abandoned, in the words of the people who built the kernels.

[Liedtke 1992] The original description of clans and chiefs, worth reading next to Elphinstone and Heiser's verdict on it.

[Hardy 1988] Three pages on the confused deputy; read them before deciding whether the display driver's remaining weakness matters.

[Heiser 2020] Chapter 4 explains capabilities, delegation and interposition for a reader who has not used seL4.

[Microkit manual 2026] The sections on channels and protected procedures, and the design rationale at the end, are the model the first pass followed.

[Miller et al. 2003] Explains why a list of names checked by the kernel is not a capability system, which is the gap Section 5.6 describes.

CHAPTER 6 PART II · THE FIRST PASS

Asks that cannot wait for each other

An ask makes the asker wait, and in PicoCalc OS the wait has a particular shape. A world's turn lasts until the reply arrives, and while a world is in a turn its mailbox is closed. Two worlds that ask each other therefore each wait for a message the other cannot take, and before the first pass the only thing that ended the wait was a time-out chosen by whoever wrote the ask. Two related defects sat beside this one. An ask that timed out was still handled later, for nobody, and a turn waiting for a promise that would never settle went on forever without the supervisor noticing.

This chapter shows the deadlock on the build from before the first pass, then reads what the L4 family learnt about timeouts on IPC (it removed them) and how the seL4 Microkit makes calls between components acyclic (it allows calls only to a higher priority). The first pass answers with four mechanisms: the boot refuses asks that form a cycle; an ask to a lower priority is allowed and noted; a timed-out ask that has not started is taken out of the callee's mailbox; and a turn still unfinished after its `turn_ms` is a crash. At the end of the chapter the reader can explain why each defect arose from the turn's semantics, which L4 lesson each mechanism follows, and how to choose the numbers the mechanisms depend on.

6.1 The turn and the closed mailbox

Small Worlds, Chapter 3, defines the turn: the handling of one message to completion, which for an asynchronous handler lasts until the promise it returned settles, across every `await` [Small Worlds 2026]. The nucleus's choice of the next segment encodes the decision that matters here. A world in a turn may run when one of its promises is settled, but it takes nothing from its mailbox until the turn ends:

▸ nucleus/kernel.c

```
      if (!W->in_turn && W->mbox && (!ok || W->mbox->t < t || (W->mbox->t == t && W->mbox->seq < s))) {
          t = W->mbox->t; s = W->mbox->seq; ok = true; set = false; }
```

That condition makes a world's state safe across an `await`, since no other message can change it in the middle of a handler. It is also what makes two asks wait for each other. `bench/rogue.py` tests this with two worlds. The first asks the second in its start turn, and the second, while handling that ask, asks the first:

▸ bench/rogue/cyc_a.js

```
// Attack 6, side A: asks B in its start turn; B asks A back, so each waits for the other's closed
    mailbox.
service(self)
  .on("pong", () => "pong")
  .onStart(async () => {
    await sleep(1500);
    const t0 = now();
    try { const r = await ask("cyc_b", "ping", null, { timeout: 4000 }); log(`OUTCOME ask_cycle
        answered after ${Math.round(now() - t0)} ms: ${r}`); }
    catch (e) { log(`OUTCOME ask_cycle failed after ${Math.round(now() - t0)} ms: ${e.message}`); }
  });
```

▸ bench/rogue/cyc_b.js

```
// Attack 6, side B.
service(self).on("ping", async () => {
  try { return await ask("cyc_a", "pong"); } catch (e) { return "B gave up: " + e.message; }
});
```

Before the first pass the manifest's `asks` lists were not read, so nothing prevented this configuration. `bench/rogue-before.txt` records the verdict, `ask_cycle answered after 2000 ms: B gave up: no reply from cyc_a within 2000 ms`. Running the same scenario on the same pre-pass build with every message logged (`-vv`) shows how it got there:

```
[     1.346] cyc_a <- start from kernel
[  1501.457] cyc_b <- ping (ask) from cyc_a null
[  3501.587] cyc_a: OUTCOME ask_cycle answered after 2000 ms: B gave up: no reply from cyc_a within
    2000 ms
[  3501.636] cyc_a <- pong (ask) from cyc_b
```

At 1,501.457 ms `cyc_b` starts handling `ping` and asks `cyc_a` for `pong`. The nucleus delivers `pong` to `cyc_a`'s mailbox, where it stays, because `cyc_a` is still in its start turn, waiting for `cyc_b`'s answer. For the next 2,000 ms neither world runs. Then `cyc_b`'s ask reaches its timeout, 2,000 ms by default; its handler catches the rejection and returns `"B gave up: …"`, which becomes `cyc_a`'s answer. `cyc_a`'s own timeout was 4,000 ms, so the shorter of two numbers chosen by two authors decided which world saw an error and which saw a reply that was really an error. Finally, 0.05 ms after its start turn ends, `cyc_a` takes `pong` from its mailbox and handles it, for an ask whose asker had given up. Figure 6.1 draws the sequence.

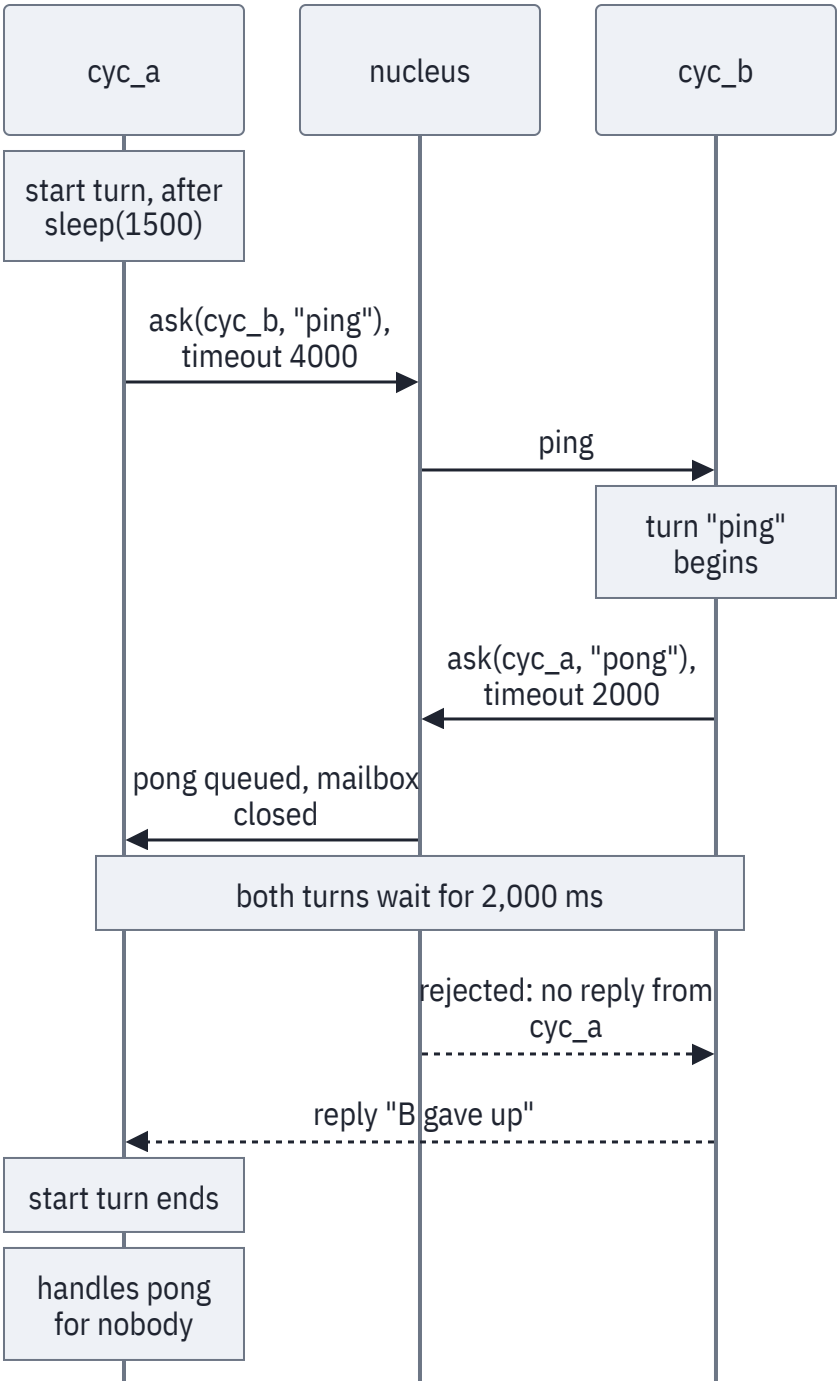


Figure 6.1. The ask cycle on the build from before the first pass. Each world waits in a turn for a reply that only the other's next turn could produce; cyc_b's 2,000 ms timeout ends the wait, and the timed-out pong is still handled afterwards.

Coffman, Elphick and Shoshani's four conditions for deadlock [Coffman et al. 1971] are all present, and Table 6.1 shows where each comes from. Three of them are the design of the turn. The fourth, the circular wait, came from the configuration, which nothing checked.

Table 6.1. Coffman's four conditions in the cyc_a and cyc_b deadlock.

Condition	Where it comes from in PicoCalc OS
Mutual exclusion	a world handles one turn at a time, and its mailbox is closed during it
Hold and wait	`cyc_a` holds its start turn while it waits for `cyc_b` 's reply
No preemption	before the first pass, nothing but a crash ended a turn early
Circular wait	`cyc_a` waits for `cyc_b` , which waits for `cyc_a`

The degenerate cycle, a world that asks itself, was refused at the call before the first pass ("a world cannot ask itself (it would wait for its own turn to end)"), because it can be recognised from one call. A longer cycle cannot: when `cyc_b` asks `cyc_a` , nothing in that call shows that `cyc_a` is waiting for `cyc_b` . Checking at the call would mean following the chain of waiting worlds at every ask, and that chain changes as turns begin and end. A cycle in the configuration, by contrast, can be found once, before anything runs, which is what the first pass does (Section 6.3). This section has shown that the deadlock follows from the closed mailbox and an unchecked configuration, and that a timeout ended it badly for both sides.

6.2 Timeouts in L4, and why seL4 has none

Synchronous IPC raised the same problem in L4, in a sharper form, since an L4 sender blocks until the receiver is ready. Elphinstone and Heiser describe the danger. A malicious or buggy client could send a request to a server and never collect the reply, and with rendezvous IPC the server, sending its reply, "would block indefinitely unless it implements a watchdog to abort and restart" [Elphinstone and Heiser 2013, §3.2.4]. The original L4 defended against this with timeouts. An IPC system call specified four of them: one limiting the wait for the send phase to start, one for the receive phase, and two limiting the time

spent on page faults during the send and receive phases of long IPC. The values were encoded in a floating-point format covering zero, infinity, and finite values from one millisecond to weeks, and they added complexity to the kernel's management of wakeup lists.

Practice did not bear the design out. There was "no theory, or even good heuristics, for choosing timeout values", Elphinstone and Heiser write, and in reality only zero and infinity were used. A client sends and receives with infinite timeouts, and a server waits for requests with an infinite timeout but replies with a zero one. Their conclusion is the one this chapter follows: "Traditional watchdog timers represent a better approach to detecting unresponsive IPC interactions", such as those that deadlocks cause. L4-embedded replaced the timeouts with one flag per phase, choosing between polling (a zero timeout) and blocking (an infinite one); seL4 follows that model; OKL4's fully asynchronous model needs no timeouts; and timed sleeps, which timeouts had also served, are done with a timer given to user level. The verdict is "Abandoned: IPC timeouts in seL4, OKL4" [Elphinstone and Heiser 2013, §3.2.4; Heiser and Elphinstone 2016, §3.2.4].

seL4's later mixed-criticality extensions did add something called a timeout, but of a different kind. A **timeout exception** is raised when a thread exhausts the budget of its scheduling context, and it is delivered to a handler that may give an emergency budget, suspend the thread, or abandon the request and roll a server back [Lyons et al. 2018, §3.4.2 and §3.6]. It watches consumed time, which is what PicoCalc OS's per-segment budget already does. Neither watches a thread that is waiting.

PicoCalc OS kept its timeouts. Every ask carries one, 2,000 ms unless the asker gives another; the REPL gives 5,000 ms to the sandbox, the compositor 1,500 ms to a flush, and the faults app 1,000 ms to its deliberately slow service. Elphinstone and Heiser's objection applies to these numbers as much as to L4's, since nothing in the system determines what they should be. A PicoCalc timeout also serves the asker's user. When the REPL gives up after 5 s, it prints a line and accepts the next one, which a blocking ask with no timeout could not do. Erlang/OTP's `gen_server:call` made the same choice, with a default of 5,000 ms [Erlang/OTP gen_server 2026]. The first pass therefore keeps timeouts as the asker's own policy and stops relying on them for the two jobs L4 found they do badly: preventing deadlock, which the next section does structurally, and detecting a component that has stopped responding, which Section 6.6 does with a watchdog.

6.3 Making cycles impossible

The Microkit rules out the deadlock with the rule about priorities that Section 4.2 describes: a protected call may go only to a PD of strictly higher priority, and the call graph must be acyclic [Microkit manual 2026]. The seL4 white paper gives the reason in one sentence: "To prevent deadlock, a ppcall() is only allowed from lower to higher priority" [Heiser 2020, §3.2].

The priority rule implies acyclicity by a one-line argument. Along any chain of calls the priorities strictly increase, so a chain that returned to its first component would give that component a priority higher than its own. The rule is also cheap to check, because it looks at one edge at a time.

PicoCalc OS cannot adopt the rule as it stands. Worlds take their priority from their kind (drivers 3, services 2, apps 1) unless the manifest gives one, and the manifest gives the sandbox priority 0 on purpose, so that code typed at the REPL, which may run 200 ms segments, starts a segment only when no other world has work to do. The REPL and the faults app both ask the sandbox, so two edges of the shipped ask graph go downward (Figure 6.2), and the Microkit rule would refuse both. The first pass therefore checks the property the rule exists to guarantee, acyclicity, directly. After the `asks` lists are resolved into bit sets (Chapter 5), `kernel_boot` runs a depth-first search over them:

▸ history/first-pass/nucleus/kernel.c

```
// asks must not form a cycle: depth-first search, printing the first cycle found
static int dfs_color[MAX_WORLDS], dfs_stack[MAX_WORLDS], dfs_n;
static bool dfs(int u) {
  dfs_color[u] = 1; dfs_stack[dfs_n++] = u;
  for (int v = 0; v < K.n_worlds; v++) {
    if (!(K.worlds[u]->may_ask >> v & 1)) continue;
    if (dfs_color[v] == 1) {
      char t[512] = ""; size_t n = 0; int from = 0;
      while (dfs_stack[from] != v) from++;
      for (int i = from; i < dfs_n; i++) n += (size_t)snprintf(t + n, sizeof t - n, "%s → ",
          K.worlds[dfs_stack[i]]->name);
      snprintf(t + n, sizeof t - n, "%s", K.worlds[v]->name);
      merr("asks form a cycle: %s (each would wait for the other's turn to end; asks must form no
          cycle)", t);
      return true;
    }
    if (dfs_color[v] == 0 && dfs(v)) return true;
  }
  dfs_color[u] = 2; dfs_n--;
  return false;
}
```

▸ nucleus/kernel.c

```
  memset(dfs_color, 0, sizeof dfs_color); dfs_n = 0;
  for (int i = 0; i < K.n_worlds; i++) if (dfs_color[i] == 0 && dfs(i)) break;
```

This is a standard depth-first search [Tarjan 1972] with three colours. A world is 0 (white) until the search reaches it, 1 (grey) while it is on the current path, and 2 (black) once every world it may ask has been explored. An edge to a grey world closes a cycle, and the cycle is exactly the part of the path stack from that world to the top, which the function prints. An edge to a black world is skipped, because everything reachable from that world has already been explored, and an edge to a white world extends the path. Each world is coloured grey once and each edge examined once, so the search costs at most 32×32 bit tests for the largest manifest the kernel accepts. The search stops at the first cycle, which keeps the message short; a manifest with two cycles reports one, and the second appears after the first is fixed.

Given the attack's manifest entries (`"asks": ["cyc_b"]` and `"asks": ["cyc_a"]`), the current build refuses to boot. Run with a scratch copy of `js/` called `js-ask_cycle` as its JavaScript root (`-j js-ask_cycle`), the host build prints its calibration line and then:

```
js-ask_cycle/system.json: asks form a cycle: cyc_a → cyc_b → cyc_a (each would wait for the other's
    turn to end; asks must form no cycle)
js-ask_cycle/system.json: 1 problem; the system does not boot
```

and exits with status 1. `bench/rogue.py` reports the attack as `REFUSED: boot refused: …` with the same line. The deadlock of Section 6.1 can no longer be configured.

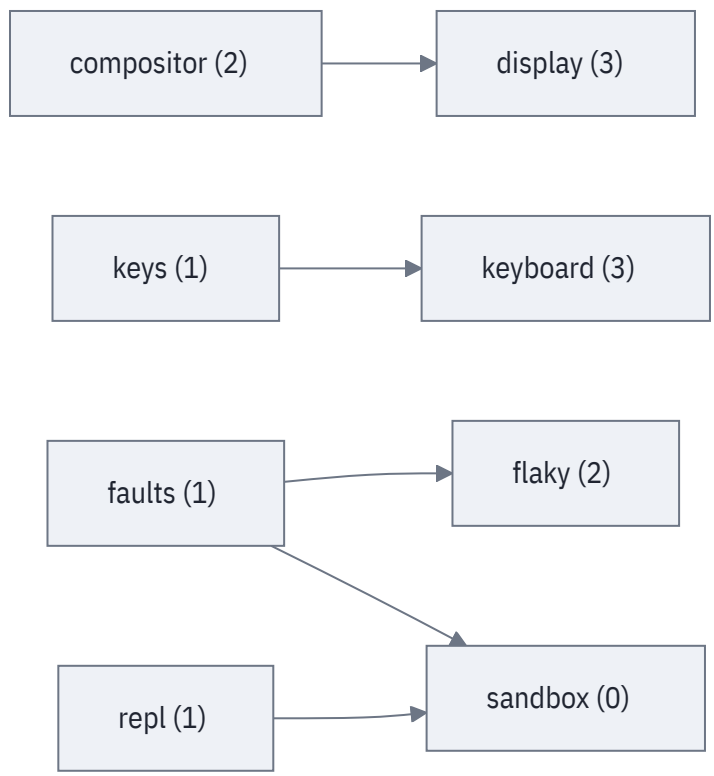


Figure 6.2. The ask graph of the shipped manifest, each world with its priority. Three edges go to a higher priority, as the Microkit requires; the two into the sandbox go to a lower one, which the boot allows and notes. There is no cycle.

What the check establishes deserves a precise statement. A world in a turn can wait only on its own promises. The nucleus settles those made by `sleep` and by bus transfers itself, without running any other world. A promise the world created and meant to resolve from another handler cannot be resolved during the turn, because no other handler runs while the mailbox is closed. The only waits that depend on another world's turn are therefore asks, and every ask follows an edge of the manifest's ask graph, because `may_talk` refuses any other (Chapter 5). If that graph is acyclic, every chain of worlds waiting on asks ends at a world that is not waiting on any other world, and that world's turn ends unless the world itself hangs, which is the case Section 6.6 handles. Two cautions apply. The argument is about waits for replies; a world can still delay another by keeping the lane busy, which priorities and budgets govern. And an acyclic graph says nothing about how long a wait lasts; a long chain of asks, or an ask that goes downward, can still make a turn wait a long time, as the next section shows.

6.4 Asking downward

The first pass allows an ask to a world of lower priority and notes it in the log, once every world has had its first boot:

▸ history/first-pass/nucleus/kernel.c

```
// Once every world has had its first boot: note the asks that go to a world of lower priority.
    Microkit
// allows a call only to a higher priority; here such an ask is allowed (the sandbox is deliberately
    the
// lowest), but the asker waits behind everything in between.
static void boot_notes(void) {
  static bool done;
  if (done) return;
  for (int i = 0; i < K.n_worlds; i++) if (K.worlds[i]->boot_ready || (K.worlds[i]->prio < 0 &&
      K.worlds[i]->status != W_STOPPED)) return;
  done = true;
  for (int i = 0; i < K.n_worlds; i++) {
    world_t *W = K.worlds[i];
    for (int j = 0; j < K.n_worlds; j++) {
      world_t *T = K.worlds[j];
      if ((W->may_ask >> j & 1) && W->prio >= 0 && T->prio >= 0 && T->prio < W->prio)
        klog("note: %s asks %s, which runs at a lower priority (%d below %d): its asks wait behind
            everything in between", W->name, T->name, T->prio, W->prio);
    }
  }
}
```

The note waits for every world's first boot because a priority is not known until then. The manifest may give one, but most entries do not, and a world's kind (driver, service or app), from which its priority follows, is declared in its own file and read when the world starts:

▸ history/first-pass/nucleus/kernel.c

```
if (W->prio < 0) W->prio = !strcmp(W->kind, "driver") ? 3 : !strcmp(W->kind, "service") ? 2 : 1;
```

Booting the shipped system on the host build, with a script that only waits 1,000 ms, prints the two notes 0.861 ms into the run:

```
$ ./build/picocalc-host -v wait1000.txt
calibration: the 100k loop is 200002 steps, so a step is 665 virtual ns; fib(20) is 43783 steps = 29.1
    virtual ms (31 ms on the device)
[     0.000] booting 13 worlds from js/system.json
[     0.861] note: repl asks sandbox, which runs at a lower priority (0 below 1): its asks wait behind
    everything in between
[     0.861] note: faults asks sandbox, which runs at a lower priority (0 below 1): its asks wait
    behind everything in between
```

The note states the cost. An ask to the sandbox is handled only when no world of priority 1 or above has anything to run, so the REPL's wait includes every app's, service's and driver's work that arrives meanwhile. *Small Worlds*, Chapter 12, measured one such ask in a trace, which took 1.6 ms from send to the REPL's resumption against 119 µs on an idle lane, and almost all of the difference was waiting behind higher-priority worlds [Small Worlds 2026]. This is the situation the Microkit's rationale describes, in which the callee, working on the caller's behalf, waits behind work that the caller itself would have gone ahead of.

The first pass notes the downward ask instead of refusing it because refusing it would remove the REPL, and because the two obvious repairs are both worse. Raising the sandbox's priority would put every evaluation's segments, up to 200 ms each, ahead of the apps' work; lowering the REPL's would put the REPL's own key handling behind the apps. seL4's mixed-criticality design shows the better answer. A **passive server** has no scheduling context of its own and "is therefore not runnable, except on a client's borrowed SC": a client calling it donates its scheduling context, so the server runs on the client's time, "until it replies to the request", and the time it consumes "is billed to the client requesting the work" [Lyons et al.

2018, §3.5]. The white paper recommends this configuration for Microkit servers, whose CPU usage is then "properly accounted to the client" [Heiser 2020, §3.2]. For PicoCalc OS that would mean running the sandbox's segment for an ask at the asker's priority and charging it to the asker's budget, so that an evaluation would compete for the lane at the priority of the world that asked for it, and its cost would be charged to that world. Chapter 12 designs that change. The note in the log marks the place where it would apply.

6.5 A timed-out ask is withdrawn

Small Worlds, Chapter 12, warns that a timeout did not withdraw the request, a property that surprises people who expect an ask to behave like a remote procedure call [Small Worlds 2026]. The asker's promise was rejected, but the message stayed in the callee's mailbox and was handled when its turn came, and the reply then found a dead entry in the ask table and was dropped. A callee that was slow because it was overloaded stayed overloaded, working through requests nobody was waiting for. The `pong` that `cyc_a` handled in Section 6.1 was one of those.

`bench/firstpass.py` checks the repair with a world that keeps `flaky` busy and then asks it something that times out:

▸ bench/checks/asker.js

```
// Keeps flaky busy for 3 s, asks it "ping" with a 500 ms timeout (which times out), then asks again
// once flaky is free: "pong 1" means the timed-out ping was taken out of flaky's mailbox, "pong 2"
    that
// flaky handled it for nobody.
service(self).onStart(async () => {
  await sleep(1000);
  ask("flaky", "slow", { ms: 3000 }, { timeout: 6000 }).catch(() => {});
  try { await ask("flaky", "ping", null, { timeout: 500 }); } catch (e) { log("CHECK first ping: " +
      e.message); }
  await sleep(4000);
  log("CHECK second ping: " + await ask("flaky", "ping"));
});
```

`flaky` counts its pings and answers `pong N`. Run with the check's own setup on the build from before the first pass, the asker logs:

```
[  1501.421] asker: CHECK first ping: no reply from flaky within 500 ms
[  5501.579] asker: CHECK second ping: pong 2
```

and on the current build:

```
[  1501.421] asker: CHECK first ping: no reply from flaky within 500 ms
[  5501.579] asker: CHECK second ping: pong 1
```

Before, `flaky` took the timed-out ping from its mailbox once its slow turn ended, at about 4 s, counted it, and answered nobody; at the end of the run it had handled three turns. Now it handles two. The change is in the timer's handler, which runs when an ask's timeout expires:

▸ nucleus/kernel.c

```
static void ask_timeout(void *arg, int id) {
  (void)arg;
  ask_t *a = find_ask(id);
  if (!a) return;
  world_t *T = world_by_id(a->to);
  // the ask has not been started: take it out of the callee's mailbox, so it is not handled for nobody
  if (T) for (msg_t **pp = &T->mbox, *prev = NULL; *pp; prev = *pp, pp = &(*pp)->next) {
    msg_t *m = *pp;
    if (m->ask != id) continue;
    *pp = m->next;
    if (T->mbox_tail == m) T->mbox_tail = prev;
    T->mbox_len--;
    free_msg(m);
    break;
  }
  char t[160];
  snprintf(t, sizeof t, "no reply from %s within %lld ms", T ? T->name : "?", (long long)(a->deadline /
      1000));
  settle_ask(id, 0, t, NULL, 0);
}
```

If the ask was already settled, `find_ask` finds no live entry and the timer does nothing. Otherwise the handler looks for the ask's message in the callee's mailbox. Only a message that has not been started can be there, since a started message has left the mailbox and is in a turn. A turn cannot be undone, so the reply to a started ask finds a dead entry and is dropped, as before. `free_msg` also releases the mailbox quota the message was charged to (Chapter 7), and `settle_ask` rejects the asker's promise and gives back the asker's ask slot. The scan is linear in the callee's mailbox, which its `mbox` quota bounds (64 messages by default); it runs as a kernel event, outside any world's segment, so no world's budget pays for it.

Erlang/OTP met the other half of this problem. Its documentation for `gen_server:call` warns that before OTP 24 a reply arriving after the caller had caught a timeout could reach the caller's message queue at any later time, and that since OTP 24 process aliases make sure such late replies are not received [Erlang/OTP gen_server 2026]. PicoCalc OS had the late-reply half solved from the start, by the ask table and the epoch check *Small Worlds*, Chapter 12, describes; the first pass adds the request half, for requests that have not started.

6.6 Turn deadlines

A budget bounds a segment. The interrupt handler counts the steps a world's code takes, the nucleus adds the native work its host calls do, and a segment that runs past its budget is stopped and counted as a crash. A turn waiting on a promise that never settles runs no code at all, so there is nothing for a budget to count. Before the first pass such a world stayed in its turn forever, with its mailbox closed; *Small Worlds*, Chapter 3, notes that the nucleus recorded when each turn started but did not act on it.

The REPL makes this easy to reproduce, because whatever is typed there is evaluated in the sandbox, and an expression whose value is a promise makes the sandbox's turn wait for it. The script for Figure 6.3 opens the REPL, types `new Promise(() => {})`, waits 5.6 s, types `1 + 1`, waits again, and types `2 + 2`. On the build from before the first pass, the sandbox's first turn began at 3,474.7 ms and was still in progress when the run ended at 16,370 ms; the system's table of worlds showed the sandbox `waiting` with no completed turns, the REPL's second evaluation timed out, and its third was still waiting. Nothing crashed, so the supervisor did nothing and the log said nothing. On the current build the kernel log shows the deadline acting, the supervisor restarting the sandbox after its 100 ms backoff, and the new instance taking the REPL's waiting evaluations from the mailbox it kept:

```
[  3474.775] sandbox <- eval (ask) from repl {"code":"new Promise(() => {})"}
[ 11474.735] sandbox crashed handling eval: its turn handling "eval" did not finish within its turn_ms
    of 8000 ms
[ 11576.516] sandbox <- eval (ask) from repl {"code":"1 + 1"}
[ 15715.064] sandbox <- eval (ask) from repl {"code":"2 + 2"}
```

The REPL had asked for `1 + 1` at about 9,593 ms, into the mailbox of a world that would never take it; the restarted sandbox answered at 11,576 ms, within the REPL's 5 s timeout, so the REPL printed `2`.



Figure 6.3. The REPL after new Promise(() => {}), then 1 + 1 and 2 + 2, captured at 16.37 s on the virtual clock of the host build. On the build from before the first pass (left) the sandbox never leaves its first turn; on the current build (right) its turn deadline crashes it at 11.47 s, the supervisor restarts it, and it answers both later evaluations.

The mechanism has three parts. The prelude tells the nucleus when a handler has returned a promise, with the `turn.async` operation, and that operation arms the deadline the first time it is called in a turn:

▸ nucleus/vm.c

```
static JSValue op_turn_async(JSContext *ctx, world_t *W, int argc, JSValueConst *argv) {
  (void)ctx; (void)argc; (void)argv;
  if (W->in_turn && !W->turn_async) kernel_arm_deadline(W);
  W->turn_async = true;
  return JS_UNDEFINED;
}
```

When a turn begins, `run_segment` records its start and gives it a new sequence number:

▸ nucleus/kernel.c

```
      W->in_turn = true; W->turn_ask = m->ask; W->msgs++;
      W->turn_started = K.vnow; W->turn_seq++;
```

and the deadline is an event scheduled `turn_ms` after that start, carrying the sequence number:

▸ nucleus/kernel.c

```
// The turn deadline: armed when a turn goes asynchronous (only such a turn can outlive its segment).
static void turn_deadline(void *arg, int seq) {
  world_t *W = arg;
  if (W->status != W_IDLE || !W->in_turn || (int)W->turn_seq != seq) return;
  char why[200];
  snprintf(why, sizeof why, "its turn handling \"%s\" did not finish within its turn_ms of %lld ms", W-
      >turn_type, (long long)(W->turn_limit_us / 1000));
  crash(W, why);
}
void kernel_arm_deadline(world_t *W) {
  if (W->turn_limit_us > 0) schedule_fn(W->turn_started + W->turn_limit_us, turn_deadline, W, (int)W-
      >turn_seq);
}
```

Only asynchronous turns are watched, because only they can outlive their first segment. A synchronous turn ends when its handler returns, inside one segment, and the budget already bounds that. The deadline is never cancelled. When it fires, it compares the sequence number it carries with the world's current one; if the turn it was armed for has ended, or

the world has crashed or stopped meanwhile, it does nothing. One event per asynchronous turn, most of them harmless, is simpler than a cancellation path, and a stale deadline cannot hit a later turn. A `turn_ms` of 0 arms no deadline at all. The deadline counts from the start of the turn, not from the last segment, so a turn that makes progress in many short segments is still bounded as a whole. `bench/firstpass.py` checks it with a world whose start turn sleeps for 5 s under a `turn_ms` of 1,000; run today, the check prints:

```
PASS  a turn that outlives turn_ms is a crash
      slowpoke crashed handling start: its turn handling "start" did not finish within its turn_ms of
          1000 ms
```

The default `turn_ms` is 8,000 ms, and the number is constrained from below. A turn's deadline must be longer than the longest wait that turn may legitimately make, or the deadline will crash a world that is doing nothing wrong. In the shipped system the longest such wait is the REPL's 5 s ask to the sandbox, because the REPL's turn handling the Enter key lasts as long as the evaluation does. Setting the REPL's `turn_ms` to 3,000 in a scratch copy of the manifest and evaluating `sleep(4000).then(() => 42)` shows what happens otherwise:

```
[  6933.309] repl crashed handling key: its turn handling "key" did not finish within its turn_ms of
    3000 ms
```

The sandbox did nothing wrong; it took 4 s, which the REPL's own timeout allows. The deadline crashed the asker, which was waiting correctly. When the deadlines are equal, a deadline falls first on the world whose turn began first, and an asker's turn always begins before the turn of the world it asks. Two sketch worlds in a scratch copy of the manifest show the same order with the default 8,000 ms: a world that asks another with a 20,000 ms timeout, where the other's handler never settles, crashed at 8,001.305 ms, and the world that hung crashed 1,000 ms later, at 9,001.413 ms. The rule that follows is simple to state. For every world, the longest a turn may legitimately wait, adding up the asks' timeouts and the sleeps it makes one after another, must be shorter than its `turn_ms`, so that the world gives up before its own deadline and the deadline lands on the world that hangs. In the shipped system no turn waits on more than one ask, every ask timeout is at most 5,000 ms, and every `turn_ms` is 8,000 ms, so the rule holds; the 3 s of margin covers the rest of the turn, including the time the REPL's continuation waits for the lane after its ask fails.

A turn deadline is the watchdog Elphinstone and Heiser recommend for "detecting unresponsive IPC interactions", placed in the nucleus and measured in turns. It detects a hang and says little about its cause. The crash message names the type of message the world was handling, and the supervisor restarts the world as it would after any other crash, with the same backoff and the same limit of five crashes in ten seconds. The restarted world keeps its mailbox, which is why the REPL's `1 + 1` was answered. What the deadline cannot do is say why a promise never settled, or tell a world that hangs from one waiting correctly for longer than its configuration allows; that distinction is made by the rule above, when the manifest is written.

KEY POINTS

- A world's mailbox is closed for the whole of a turn, including its awaits, so two worlds that ask each other wait for messages neither can take; before the first pass only the shorter of two ask timeouts ended the wait, and the timed-out request was still handled afterwards.
- Original L4 put four timeouts on every IPC; Elphinstone and Heiser report that nobody could choose values, that only zero and infinity were used, and that watchdog timers detect unresponsive IPC better; seL4 has no IPC timeouts.
- The Microkit allows a protected call only to a strictly higher priority, which makes the call graph acyclic and deadlock-free; PicoCalc OS cannot use the rule because the sandbox is deliberately the lowest priority, so it checks acyclicity directly, with a depth-first search over the manifest's `asks` at boot.
- With an acyclic ask graph, every chain of worlds waiting on asks ends at a world that waits on no other world, because asks are the only waits that depend on another world's turn.
- An ask to a lower priority is allowed and noted once every world has booted; it waits behind every world in between, and seL4's passive servers, which run on the client's scheduling context, are the model for removing that wait in the second pass.
- When an ask times out before the callee has started it, the nucleus takes the message out of the callee's mailbox, releases its quota and the asker's slot; a started ask cannot be withdrawn, and its late reply is dropped as before.

- A turn still unfinished `turn_ms` after it began (8,000 ms by default) is a crash; only asynchronous turns are watched, and the deadline catches a promise that never settles, which no budget can see.
- The waits a turn makes, added up, must be shorter than the world's `turn_ms`, or the deadline crashes the waiting asker before the world that hangs; the shipped system's longest ask timeout, the REPL's 5 s, is why the default is 8 s.

The first pass has now decided who may talk to whom and made the waits between worlds finite and acyclic. The remaining defects of the attack suite are about space rather than time: a world that fills the kernel's handle table, the framebuffer pool or another world's mailbox. Chapter 7 turns each of those shared pools into per-world quotas, following seL4's rule that the kernel never allocates from a shared pool on a user's behalf.

EXERCISES

1. `BASIC` For each of Coffman's four conditions, name the design decision of PicoCalc OS that supplies it in the `cyc_a` / `cyc_b` deadlock, and say which of the first pass's mechanisms removes which condition.
2. `BASIC` List the edges of the shipped ask graph with the priority of each end. Which edges would the Microkit's rule refuse, and what would the boot print about them?
3. `CODE` Add `"asks": ["keys"]` to the keyboard entry of `js/system.json` and predict the exact line the boot prints. Then also add `"asks": ["repl"]` to the sandbox's entry and predict how many problems the boot reports. Check both with the host build, and explain the second answer from the code of `dfs` and its caller.
4. `CODE` Change the compositor's entry to `"asks": ["display", "apps"]`. Does the system boot? Predict every `note:` line the log prints, in order.
5. `CODE` In a scratch copy of the manifest, set the REPL's `turn_ms` to 3000, open the REPL and evaluate `sleep(4000).then(() => 42)`. Predict which world crashes, at what time relative to the Enter key, and with what message. Then evaluate `sleep(2000).then(() => 42)` and predict the result.

6. ANALYSIS World A asks world B with a timeout of T_a; B's handler never settles; A and B have turn deadlines D_A and D_B, and A's turn began δ ms before B's. Give the conditions on T_a, D_A, D_B and δ under which the deadline crashes B first, and check your answer against the two runs of Section 6.6.

7. ANALYSIS Prove that if every edge of a directed graph goes from a lower to a strictly higher priority, the graph has no cycle. Then propose a priority for the sandbox that would satisfy the Microkit's rule in the shipped manifest, and say what that change would cost in terms of *Small Worlds*, Chapter 12's trace.

8. ANALYSIS `dfs` reports only the first cycle it finds. Using the strongly connected components of the ask graph [Tarjan 1972], design a check that reports one cycle for every component that contains one, and bound its cost for 32 worlds. Why would reporting every elementary cycle be a bad idea?

9. ANALYSIS `ask_timeout` scans the callee's mailbox. Bound the scan's length using the quotas of Chapter 7, explain why its cost is charged to no world, and say whether a world could use timed-out asks to make the nucleus do unbounded work.

10. DISCUSSION seL4 removed IPC timeouts; PicoCalc OS kept them. Now that cycles are refused at boot and turns have deadlines, argue for or against removing the `timeout` option from `ask`. What would the REPL, the compositor and the faults app lose, and what could replace each use?

11. PROJECT Add a `turn_handler` key to the manifest, modelled on seL4's timeout exceptions: when a world's turn deadline expires, the nucleus sends a message to the named handler world (which must hold `sys.admin`) instead of crashing the world at once, and the handler decides whether to extend the turn once or crash it. Write the boot checks, the kernel change, a scenario for `bench/firstpass.py`, and an argument that a handler that never answers cannot hang the system.

FURTHER READING

[Elphinstone and Heiser 2013] Section 3.2.4 is the account of IPC timeouts in L4, why they failed in practice, and what replaced them; it is two columns long.

[Heiser and Elphinstone 2016] The journal version of the same retrospective, with the same sections expanded and a summary table of every design decision kept, replaced or abandoned.

[Microkit manual 2026] The section on protected procedures states the priority rule, and the design rationale at the end explains it in terms of deadlock and resource servers.

[Lyons et al. 2018] Sections 3.5 and 3.6 explain passive servers and timeout exceptions, the mechanisms the second pass would borrow for asks that go downward and for turns that run out of time.

[Coffman et al. 1971] The four conditions, and the observation that breaking any one prevents deadlock, in the paper that named them.

[Tarjan 1972] Depth-first search, its colours and its use for strongly connected components, in the original.

No shared pools

Before the first pass, four of the nucleus's resources were shared by every world on a first-come basis: a table of 128 handles, a table of 1,024 waiting asks, 16 framebuffers, and the memory that holds messages on their way to a mailbox. A world that could take one entry from such a pool could take all of them, and once a pool was empty the next request of every other world failed, the compositor's and the display driver's included. Nothing the greedy world did broke a rule then in force. It stayed inside its budget and its heap limit, so the supervisor saw no fault, and the worlds that did fail were the ones that had done nothing wrong.

This chapter shows three such exhaustions and reads the L4 family's answer to the same problem (seL4 removed the kernel heap altogether) beside those of Genode and Linux. It then describes the first pass's version, per-world quotas written in the manifest, charged to the world that caused each entry to exist and checked at boot to fit the tables. By the end the reader should be able to say, for each pool, who is charged for an entry, from when until when, what a world sees when its quota is used up, and what this static partitioning costs.

7.1 Four shared pools

A **pool** here is any resource that the nucleus allocates on behalf of a world from a store that all worlds draw on. Table 7.1 lists the four that the nucleus had before the first pass. The first three are fixed-size arrays inside the kernel's state (`K.handles`, `K.asks`, `K.bufs` in `nucleus/nucleus.h`). The fourth is not a table. Every message is a block that `new_msg` takes from the C heap, and it stays there until its receiver takes it out of its mailbox.

Table 7.1. The nucleus's shared pools before the first pass: how large each was, which worlds could draw on it, and what happened to everyone else when it was empty.

Pool	Size	Who could draw on it	What the others saw when it was empty
Handle table	128 entries	any world holding a handle (each `narrow` makes a new entry) or a claim on a device	"out of handles" from `hw.i2c`, `hw.spi`, `dma.alloc` and `narrow`
Ask table	1,024 entries	any world, asking any other	the ask rejected with "too many asks are waiting for replies"
Framebuffers	16	worlds with the `dma` claim	"out of DMA buffers" from `dma.alloc`
Message memory	the C heap	any world, sending to any other	nothing, until the device ran out of memory

Small Worlds, Chapter 11 listed these as resources "without a budget" and described how each could be exhausted [Small Worlds 2026]. The first pass's test suite turns that description into worlds that do it. `bench/rogue/rogue_handles.js` empties the first and third pools. It is given the `dma` claim, because a world must hold at least one handle before it can narrow anything, and an allocated framebuffer is the simplest handle to obtain:

▸ `bench/rogue/rogue_handles.js`

```
app(self).onStart(async () => {
  await sleep(1500);
  const fbs = [];
  let why = "";
  try { for (let i = 0; i < 64; i++) fbs.push(dma.alloc(8, 8)); } catch (e) { why = e.message; }
  log(`OUTCOME exhaust_bufs allocated ${fbs.length} framebuffers, then: ${why || "no refusal"}`);
  if (!fbs.length) return;
  let n = 0;
  why = "";
  try { for (; n < 1000; n++) fbs[0].narrow("r"); } catch (e) { why = e.message; }
  log(`OUTCOME exhaust_handles narrowed ${n} times, then: ${why || "no refusal"}`);
});
```

The scenario in `bench/rogue.py` lets the world run, kills the compositor at 2.5 s as a hardware fault might, and looks at it three seconds later. On the nucleus from before the pass the result was:

▸ bench/rogue-before.txt

```
exhaust_tables: fill the shared framebuffer pool and handle table (the rogue has the dma claim)
  ATTACK SUCCEEDED: exhaust_bufs allocated 15 framebuffers, then: out of DMA buffers | exhaust_handles
      narrowed 108 times, then: out of handles | compositor after a crash: stopped
```

The arithmetic is exact. The compositor already held one framebuffer, so the rogue got the other 15. The system's own worlds held five handles (an I2C handle each for the keyboard and power drivers, the display driver's SPI handle, the compositor's framebuffer and the read-only view of it that the display driver holds), and the rogue's 15 framebuffer handles and 108 narrowed copies filled the remaining 123 of the 128 entries. The kernel log of the same scenario, run again on the pre-pass build, shows what that did to the compositor once it had to start again:

```
[  1502.473] rogue_handles: OUTCOME exhaust_bufs allocated 15 framebuffers, then: out of DMA buffers
[  1503.526] rogue_handles: OUTCOME exhaust_handles narrowed 108 times, then: out of handles
[  2500.000] compositor crashed: killed from the host
[  2600.118] compositor crashed handling start: out of DMA buffers (system/compositor.js:377:21)
[  2800.255] compositor crashed handling start: out of DMA buffers (system/compositor.js:377:21)
[  3200.373] compositor crashed handling start: out of DMA buffers (system/compositor.js:377:21)
[  4000.683] compositor crashed handling start: out of DMA buffers (system/compositor.js:377:21)
[  4000.683] compositor crashed 5 times in ten seconds; it stays stopped
```

Every restart of the compositor called `dma.alloc` in its start handler, found no framebuffer, and crashed; the supervisor backed off by 100, 200, 400 and 800 ms as *Small Worlds*, Chapter 14 describes, and stopped it at the fifth crash. The world that caused this was never touched. It was idle, inside its limits, holding 123 table entries.

The ask table can be emptied without any claim at all. Before the first pass any world could ask any other, and an ask's entry stays in the table until it is answered or times out. A probe written for this chapter (`l4book/traces/07-askflood.js` , added to a copy of the manifest) asks the `flaky` service for a ten-second job 1,100 times in one turn and does not wait for any of the answers:

▸ l4book/traces/07-askflood.js

```
app(self).onStart(async () => {
  await sleep(1500);
  let ok = 0, refused = 0, why = "";
  const ps = [];
  for (let i = 0; i < 1100; i++) ps.push(ask("flaky", "slow", { ms: 10000 }, { timeout: 20000
      }).then(() => ok++, (e) => { refused++; why = why || e.message; }));
  await sleep(100);
  log(`PROBE after 100 ms: ${refused} of 1100 asks refused: ${why}`);
});
```

On the pre-pass build the first 1,024 asks took every entry and the last 76 were refused:

```
[  1625.404] askflood: PROBE after 100 ms: 76 of 1100 asks refused: too many asks are waiting for
    replies
```

That refusal was itself a repair made while *Small Worlds* was written (before it, asks beyond 1,024 overwrote older entries), and it applied to every world. The compositor redraws by asking the display driver to flush, so with the table full each redraw's ask was refused and nothing reached the panel. Run with `-vv`, the kernel log shows the display driver's flushes at 371.8 and 415.7 ms during boot and then none until 12,003.8 ms. In between, the script pressed → at 1.7 s and `m` at 2.0 s; the launcher and the system monitor changed their state, and the glass kept showing the home screen with the first tile selected (Figure 7.1). The freeze ended only because `flaky` finished its first ten-second job at about 11.5 s and freed one entry, which the compositor's next flush took.

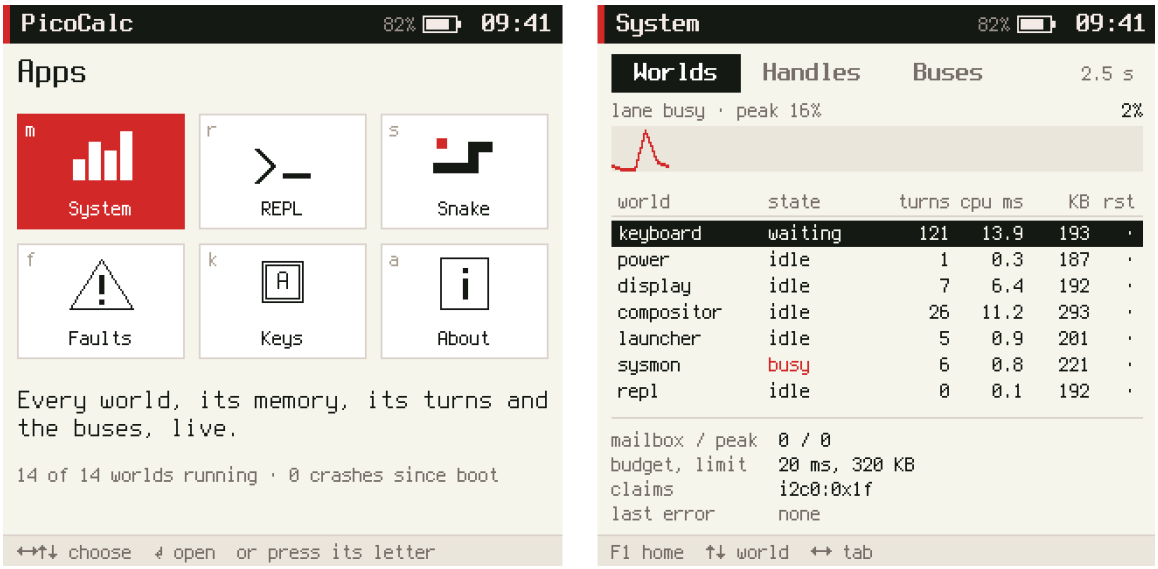


Figure 7.1. The screen 500 ms after pressing m on the home screen while a probe floods the ask table. On the left is the nucleus from before the first pass, whose table the probe has filled; the compositor's flush asks are refused, and the panel keeps showing the home screen until 12.0 s. On the right is the current nucleus, which held the probe to its quota of 8 asks; the System app has opened.

Message memory had no bound at all. `bench/rogue/rogue_flood.js` is allowed to send to `flaky` and uses that right without restraint:

▸ bench/rogue/rogue_flood.js

```
let reported = false;
app(self).every(100, () => {
  let sent = 0, why = "";
  try { for (let i = 0; i < 3000; i++) { send("flaky", "ping"); sent++; } } catch (e) { why =
      e.message; }
  if (!reported) { reported = true; log(`OUTCOME flood_mailbox sent ${sent} in one turn${why ? ", then:
      " + why : ""}`); }
});
```

▸ bench/rogue-before.txt

```
flood_mailbox: send flaky 3,000 messages a turn (the rogue may send to flaky)
  ATTACK SUCCEEDED: flood_mailbox sent 3000 in one turn | flaky's mailbox peak: 3000
```

Three thousand message headers of 88 bytes each on the 32-bit build are more than 250 KB of the nucleus's heap, taken in one 100 ms period by a world whose own heap limit is 1,024 KB, and counted against nothing. The flood also hurt a well-behaved world directly. A second probe (`l4book/traces/07-victim.js`) asks `flaky` for a `ping` every 37 ms. Alone, it got 148 answers in 5.5 s. Beside the flood, on the pre-pass build, it got 27, because each of its asks queued behind up to 3,000 pings.

The three exhaustions have one shape. A world takes entries from a store that other worlds need, the nucleus has no record of whose entries they are, and the failure appears in a different world, at a later time, with a message that does not name the cause. The next section shows that the L4 family met exactly this shape inside its own kernels.

7.2 Who pays for kernel memory

A microkernel allocates memory on behalf of its users whenever it creates an object for them: a thread control block, an address-space structure, a page table, a capability slot. L4 kernels took that memory from a fixed-size heap (Section 3.2). Elphinstone and Heiser's diagnosis of what went wrong is the one the previous section reached for PicoCalc OS: the fundamental problem, shared by most operating systems, was "the insufficient isolation of user processes through the shared kernel heap" [Elphinstone and Heiser 2013, §3.4]. A process that made the kernel create objects for it could exhaust the heap, and then every other process's next system call that needed kernel memory failed.

The family tried two answers. The first, proposed by Liedtke and colleagues for an L4-based system that would run downloaded code [Liedtke et al. 1997; Elphinstone and Heiser 2013, §3.4], gave each process its own kernel heap, with a mechanism to donate more memory to the kernel when one ran out; NOVA, Fiasco and OKL4 adopted variations of it, and Fiasco.OC's form is a per-task kernel memory quota [Elphinstone and Heiser 2013, §3.4]. The review names its cost: per-process heaps come "at the expense of the ability to revoke allocations without destroying the process". Memory charged to a process's pool returns to the system when the process is destroyed, and not before.

seL4 took the second answer to its end and removed the kernel heap: every kernel object is retyped from **untyped memory** whose capabilities a user holds, as Section 3.2 describes [Elphinstone and Heiser 2013, §3.4; Heiser 2020]. Elkaduwe, Derrin and Elphinstone state the consequence for isolation: the untyped capabilities an application holds define "the precise amount of physical memory" it can consume, directly or indirectly, and "there are no implicit allocations within the kernel" [Elkaduwe et al. 2008]. In Klein and colleagues' words, the model "guarantees all memory allocation in the kernel is explicit and authorised" [Klein et al. 2009].

> **DEFINITION: NO SHARED POOL**
>
> A kernel has **no shared pool** when nothing it allocates on a principal's behalf can be taken from a store that another principal depends on. Every allocation is charged to the principal that caused it, and the charges of all principals together cannot exceed what the kernel has.

PicoCalc OS cannot copy seL4's mechanism. Its handle, ask and framebuffer tables are static arrays sized at compile time, its worlds are JavaScript programs that never see physical addresses, and there is no user-level memory manager to donate from. What it can take is the rule in the definition above, in its static form. Each world gets a fixed share of each table, written in the manifest, every entry is charged to one world's share, and the boot refuses a manifest whose shares add up to more than the table holds. This is the Fiasco.OC answer (a quota per task) with the seL4 property (the shares are disjoint by construction, so no world can reach another's), checked once before anything runs. The price is the one the review gave for per-process heaps. An entry returns to the pool only when the world that is charged for it lets go or is torn down.

Systems built above a kernel face the same question in their servers. Genode, a component-based operating-system framework, applies the rule to user-level servers. Its documentation starts from the server's version of the problem: in traditional operating systems, a service such as a GUI server "would allocate memory on behalf of its clients", and every such server can be exhausted by one client [Feske 2026, "Resource trading"]. "Genode solves this problem by letting clients pay for server-side allocations." Each component's RAM is a quota assigned by its parent, which splits its own budget among its children, so the quotas cannot add up to more than the parent has. When a client opens a session with a server, "it can attach a fraction of its quota to the new session", the **session quota**, which the server

may use for that client's objects during the session's lifetime. If a client asks for more than its session quota covers, the server returns the shortage as an error, and the client can upgrade the session with more of its own quota; when the session closes, the server releases everything it allocated for it and the quota goes back to the client. Resources move between components at run time, and every byte is charged to exactly one of them.

Linux control groups make the opposite choice about over-commitment. The cgroup v2 interface distinguishes several ways of distributing a resource, and of limits it says that they "can be over-committed": the sum of the children's limits "can exceed the amount of resource available to the parent" [cgroup v2 docs 2026]. Its PID controller is the closest Linux analogue of PicoCalc's handle quota: `pids.max` is a "hard limit of number of processes", and `fork()` or `clone()` in a group at its limit returns `-EAGAIN`. A group at its limit fails its own next creation, which protects the other groups from it; but because limits may over-commit, the system's own table can run out first, and then a group within its limit fails too. (*Small Worlds*, Chapter 11 discussed cgroups' processor and memory limits.)

MINIX 3, whose drivers are user-mode processes with restricted privileges [Herder et al. 2007], is described in *Small Worlds*, Chapter 15. Table 7.2 sets the four answers side by side.

Table 7.2. Four answers to the shared-pool problem. Over-commitment means the bounds of all principals may add up to more than exists.

	seL4	Genode	Linux cgroup v2	PicoCalc OS, first pass
What is bounded	all kernel memory	each component's RAM	memory, processor, PIDs, ... per group	handle, ask and framebuffer entries; mailbox messages and bytes
Who sets the bound	whoever hands out untyped capabilities	the parent component	an administrator, at run time	the manifest, before boot
Over-commitment	impossible	impossible	allowed for limits	refused at boot for the three tables
A principal at its bound	the retype fails	the server reports a shortage	`-EAGAIN`, throttling or the OOM killer	the call throws or the ask is rejected; a channel post is dropped
Can a bound move at run time	yes, by passing capabilities	yes, by session quotas and upgrades	yes, by writing the control files	no

PicoCalc OS is as strict as seL4 and Genode about over-commitment, and as static as a configuration file can be, since nothing moves while the system runs. Genode's session quota is the mechanism it lacks; Section 7.7 returns to when that matters.

7.3 Quotas in the manifest

The first pass added five per-world keys to the manifest, each counting one kind of entry, and a `defaults` object that supplies them for worlds whose entries leave them out [First pass 2026]. Table 7.3 gives their meaning, and Appendix A gives the complete schema.

Table 7.3. The quota keys of a manifest entry. The built-in defaults apply when neither the world's entry nor the manifest's defaults object gives a value.

Key	Counts	Built-in default	Charged from	Released when
`handles`	handle-table entries the world made: opened, allocated or narrowed	4	the call that makes the entry	the entry is freed
`bufs`	framebuffers the world allocated (needs the `dma` claim)	1	`dma.alloc`	the last handle sharing it is freed
`ask_slots`	asks the world has waiting for replies	8	`ask`	the ask is answered, fails or times out
`mbox`	messages for the world, sent but not yet taken out of its mailbox	64	`send`, `ask` or `publish` by the sender	the message leaves the mailbox
`mbox_kb`	the same messages' type and payload bytes, in KB (at least 64)	128	as `mbox`	as `mbox`

The shipped manifest uses the defaults for every world except the compositor, which gets more headroom:

▸ history/first-pass/js/system.json

```
  "defaults": { "handles": 4, "ask_slots": 8, "mbox": 64, "mbox_kb": 128, "bufs": 1, "turn_ms": 8000 },
…
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
        "budget_ms": 80,  "mem_kb": 768,
      "needs": ["frames", "keys", "power", "worlds.status"], "sends": ["display", "power", "apps"],
          "asks": ["display"],
      "handles": 6, "bufs": 2, "mbox": 128 },
```

`kernel_boot` reads each world's quotas while it reads the rest of its entry, then adds them up after it has checked the graph of who may talk to whom (Chapter 5) and the asks for cycles (Chapter 6):

▸ history/first-pass/nucleus/kernel.c

```
// the quotas fit the kernel's tables, so no world can exhaust one for the others
long hs = 0, as = 0, bs = 0;
for (int i = 0; i < K.n_worlds; i++) {
  hs += K.worlds[i]->q_handles; as += K.worlds[i]->q_asks;
  if (world_claims(K.worlds[i], "dma")) bs += K.worlds[i]->q_bufs;
}
if (hs > MAX_HANDLES) merr("the worlds' handle quotas add up to %ld; the kernel's handle table holds
    %d", hs, MAX_HANDLES);
if (as > MAX_ASKS) merr("the worlds' ask_slots add up to %ld; the kernel's ask table holds %d", as,
    MAX_ASKS);
if (bs > MAX_BUFS) merr("the framebuffer quotas of the worlds with the dma claim add up to %ld; the
    kernel has %d framebuffers", bs, MAX_BUFS);
```

Three details of this check matter. The framebuffer sum counts only worlds with the `dma` claim, because only they can call `dma.alloc`; every world has a `bufs` value through the defaults, and counting the twelve that can never use theirs would make the check refuse sensible manifests. The sums are compared with `>`, so a manifest may hand out every entry and leave none in reserve. And there is no sum for mailboxes, because messages do not live in a table: `mbox` and `mbox_kb` bound how much of the C heap the messages waiting for each world can hold, and the total is a figure to compare with the device's memory, not a limit the nucleus can check.

A manifest that over-commits is refused before any world starts, with one line per problem. `bench/firstpass.py` raises the default handle quota to 20 in a copy of the manifest, among three other mistakes; twelve worlds at 20 and the compositor at 6 make 246:

▸ bench/firstpass-after.txt

```
       the worlds' handle quotas add up to 246; the kernel's handle table holds 128
       4 problems; the system does not boot
```

For the manifest as shipped, the sums are far inside the tables (Table 7.4 in Section 7.7). A well-formed manifest therefore guarantees, before anything runs, that if every world draws its whole quota of handles, asks and framebuffers at once, every one of those draws succeeds. What remains is to charge each entry to the right world, which is where the subtleties are.

7.4 Charging handles and framebuffers

A quota is only as good as its accounting, which must count every entry a world causes and release each exactly once. For handles the first pass chose to charge the **minter**, the world whose call made the entry, rather than the worlds that hold it. `handle_new`, which every opening, allocation and narrowing goes through, records the minter and charges it:

▸ history/first-pass/nucleus/hw.c

```
handle_t *handle_new(int type, world_t *holder, const char *claim) {
  for (int i = 0; i < MAX_HANDLES; i++) {
    handle_t *h = &K.handles[i];
    if (h->live) continue;
    memset(h, 0, sizeof(*h));
    h->live = true; h->id = ++K.n_handles; h->type = type; h->rights = R_READ | R_WRITE | R_GRANT;
    h->minter = holder->id; h->holders = bit(holder); h->buf = -1;
    holder->used_handles++;   // charged to the world that made it, until it is freed
    snprintf(h->claim, sizeof(h->claim), "%s", claim ? claim : "");
    return h;
  }
  return NULL;
}
```

Charging the holders instead would break the guarantee. A handle passed in a message gains a holder without anyone calling `handle_new`, so a world could be pushed over its quota by handles other worlds sent it, and an entry with several holders would have to be charged to all of them or moved between them. The minter is one world that took one action, and its quota bounds what that action can do to the table.

The compositor shows the rule in use. It allocates the framebuffer (one handle, one framebuffer), narrows a read-only view of it and sends the view to the display driver (a second handle), so its steady use is 2 of its 6 handles and 1 of its 2 framebuffers, while the display driver uses 1 of 4, its own SPI handle. The view is charged to the compositor even though the display driver uses it. When the display driver restarts, the compositor makes a new view for the new instance, and its comment explains why it lets go of the old one first:

▸ `js/system/compositor.js`

```
function attach(s) {
  // the view handed to a previous display instance died with it; let go of our hold on it, or every
  // restart of the driver would keep one more handle alive
  if (s.view) s.view.release();
  s.view = s.fb.narrow("r");
  send("display", "attach", { fb: s.view });
  s.fb.invalidate();
  dirty(s);
}
```

Without the `release`, each restart of the driver would leave one more view alive with the compositor as its only holder. Before the first pass that leak would have drained the shared table by one entry per restart, eventually failing whichever world next asked for a handle. Now it would exhaust the compositor's own quota and nobody else's, so a defect stays with the world that has it.

Every path that makes an entry checks the quota first, in `nucleus/vm.c`, so a world at its quota gets an error it can catch, in its own segment, naming itself:

▸ `nucleus/vm.c`

```
// Every handle a world makes (opened, allocated or narrowed) counts against its quota until it is
    freed.
static bool handle_quota(JSContext *ctx, world_t *W) {
  if (W->used_handles < W->q_handles) return true;
  fail(ctx, "%s has used its quota of %d handles", W->name, W->q_handles);
  return false;
}
```

`op_hw_i2c`, `op_hw_spi`, `op_dma_alloc` and `op_h_attenuate` (the host side of `narrow`) call it. `op_dma_alloc` checks the framebuffer quota before the handle quota, because a framebuffer always comes with a handle:

▸ nucleus/vm.c

```
  if (W->used_bufs >= W->q_bufs) return fail(ctx, "%s has used its quota of %d framebuffer%s", W->name,
      W->q_bufs, W->q_bufs == 1 ? "" : "s");
  if (!handle_quota(ctx, W)) return JS_EXCEPTION;
  int b = dmabuf_alloc(w, h, W);
```

Releasing is where accounting usually goes wrong, and the handle system makes it harder than it looks. *Small Worlds*, Chapter 9 explains that a handle nobody holds is kept in the table while copies narrowed from it are alive, so that revoking an ancestor still reaches every descendant. Such a holderless entry still occupies the table, so it must still be charged, and to the world that made it. `handle_release` does both, and releases the charge only when the entry itself is freed:

▸ nucleus/hw.c

```
void handle_release(handle_t *h, world_t *W) {
  h->holders &= ~bit(W);
  while (h && !h->holders && !has_children(h->id)) {
    int parent = h->parent;
    buf_unref(h->buf);
    h->live = false;
    world_t *M = world_by_id(h->minter);
    if (M) M->used_handles--;
    h = parent ? find(parent) : NULL;
  }
}
```

The loop walks up the chain of parents, because freeing the last descendant may free an ancestor that was kept only for it; each freed entry returns one unit to its own minter, who may be a different world from the one releasing. Framebuffers follow the same pattern one level down. A framebuffer is shared by the handle that allocated it and every handle narrowed from it, counted by `refs`; `dmabuf_alloc` charges the allocating world as its owner, and `buf_unref` releases that charge when the last reference goes:

▸ nucleus/hw.c

```
void buf_unref(int i) {
  if (i < 0) return;
  dmabuf_t *b = &K.bufs[i];
  if (--b->refs > 0) return;
  world_t *O = world_by_id(b->owner);
  if (O) O->used_bufs--;
  free(b->back); free(b->front);
  memset(b, 0, sizeof(*b));
}
```

When a world crashes, `teardown` calls `handles_drop_world`, which releases every handle the world holds through the same function. Entries the dead world minted that another world still holds stay alive and stay charged to it, which is the cost the L4 review named for per-process pools, since an allocation is not revoked while something still uses it. The compositor shows what that means for choosing a quota. When it crashes, the display driver still holds the read-only view, so the view and the framebuffer it was narrowed from both survive, and the restarted compositor begins with 2 handles and 1 framebuffer already charged. Its start handler allocates a new framebuffer and narrows a new view, 4 handles and 2 framebuffers in all, until the display driver accepts the new view and releases the old one, which frees both old entries. In a copy of the manifest that gives the compositor `"bufs": 1`, killing it once gives:

```
[  1500.000] compositor crashed: killed from the host
[  1600.118] compositor crashed handling start: compositor has used its quota of 1 framebuffer
    (system/compositor.js:377:21)
[  1800.255] compositor crashed handling start: compositor has used its quota of 1 framebuffer
    (system/compositor.js:377:21)
[  2200.373] compositor crashed handling start: compositor has used its quota of 1 framebuffer
    (system/compositor.js:377:21)
[  3000.491] compositor crashed handling start: compositor has used its quota of 1 framebuffer
    (system/compositor.js:377:21)
[  3000.491] compositor crashed 5 times in ten seconds; it stays stopped
```

With `"handles": 3` the same happens at the `narrow`. The shipped values, 6 handles and 2 framebuffers, cover the restart's peak of 4 and 2 with two handles to spare. A quota must cover a world's worst moment, and for a world whose allocations outlive it, the worst mo-

ment is its own restart.

7.5 Charging asks and mailboxes

An ask uses two pools at once: an entry in the ask table, which is the asker's while it waits, and a message in the callee's mailbox, which is the callee's to handle. The first pass charges each to the world whose behaviour it bounds. `kernel_ask` checks the asker's slots, then reserves room in the callee's mailbox, and rejects the ask's promise with the reason if either is full:

▸ nucleus/kernel.c

```
  if (from->used_asks >= from->q_asks) {
    char t[160]; snprintf(t, sizeof t, "%s already has %d asks waiting for replies, its quota", from-
        >name, from->used_asks);
    char *j = json_string(t);
    defer_settle(from, promise, 0, j, 0);
    free(j);
    return 0;
  }
  msg_t *m = new_msg(M_ON, type, json, from->id);
  if (!mbox_reserve(to, m)) {
…
  from->used_asks++;
```

The slot comes back in `settle_ask`, the single place where an ask ends, whether by a reply, a failure, the callee's crash or the timeout; because the ask table no longer overwrites a live entry, an ask cannot end without passing through it. Just above the quota check, `kernel_ask` still refuses an ask when all 1,024 entries are live. In a well-formed manifest the table can be full only when the quotas add up to exactly 1,024 and every world has used all of its own, so that check refuses only asks the quota check would refuse too; it stays as a second line of defence.

The mailbox charge needs more care, because of *when* it starts. A message is charged to its receiver from the moment it is sent, before it arrives. In PicoCalc OS a segment's effects are deferred until the segment ends (*Small Worlds*, Chapter 3), so a world that sends 3,000 messages in one segment has put none of them in a mailbox by the time it sends the last. Charging on arrival would let one segment queue an unbounded burst. `mbox_reserve` is therefore called by the sender's operation, before the message is queued for delivery:

▸ nucleus/kernel.c

```
// A message for T's mailbox counts against T's quota from the moment it is sent until it leaves the
// mailbox (handled, failed, pulled or dropped), so a burst sent in one segment is bounded too.
bool mbox_reserve(world_t *T, msg_t *m) {
  size_t bytes = (m->json ? strlen(m->json) : 0) + strlen(m->type);
  if (T->mq_count + 1 > T->q_mbox || T->mq_bytes + bytes > T->q_mbox_bytes) return false;
  T->mq_count++; T->mq_bytes += bytes;
  m->q_to = T->id; m->q_bytes = bytes;
  return true;
}
```

The message remembers which world it is charged to and how much (`q_to` , `q_bytes`), so the release cannot be applied to the wrong world or twice. It happens in `free_msg` , which every path that disposes of a message goes through: the lane takes the message out of the mailbox to start its turn, a timed-out ask is pulled from the mailbox (Chapter 6), a stopped world's mailbox is failed, or the sending segment crashed and its deferred effects were discarded. Figure 7.2 shows the life of a charge.

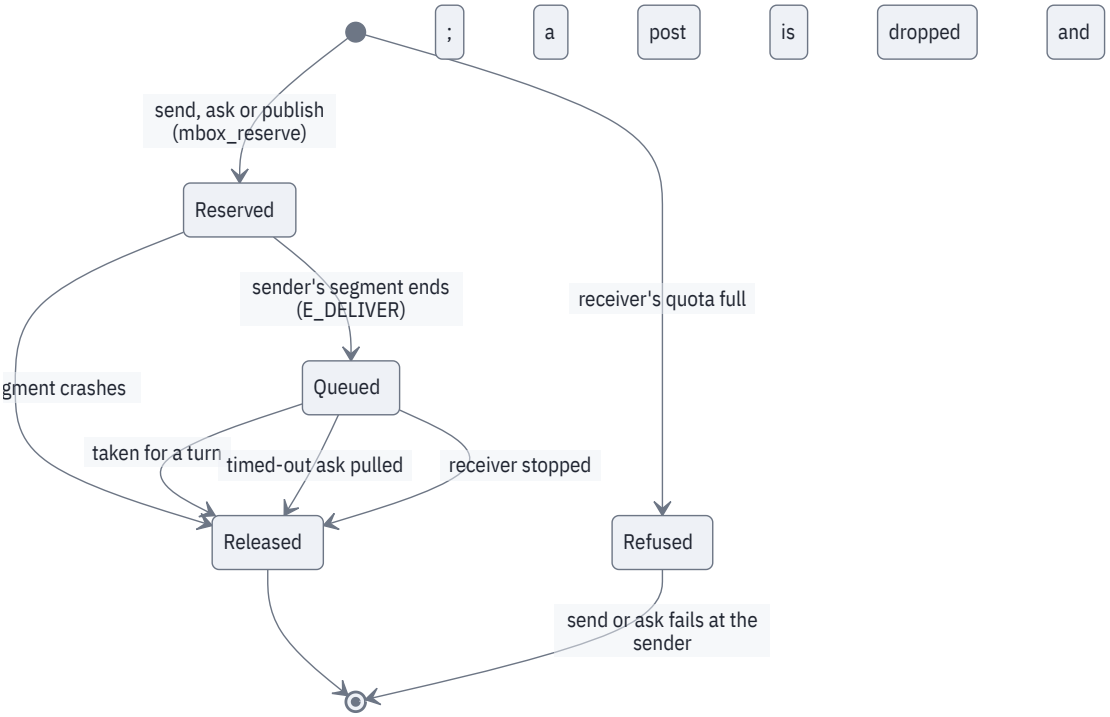


Figure 7.2. When a message is charged to its receiver's mailbox quota. The charge starts in the sender's segment and ends when the message is disposed of, whichever way that happens.

What happens at a full mailbox depends on who is waiting for the message. A `send` or an `ask` has a sender that can be told, so the sender's call fails: `send` throws "flaky's mailbox is full (64 messages, 0 KB queued)" in the sending world, and an ask's promise is rejected with the same reason. A channel post has no single sender to tell. The publisher does not know who subscribes, and failing its `publish` because one subscriber is slow would let that subscriber stop the channel for everyone. So `kernel_publish` drops the post for the full subscriber only, counts the drop, and logs it at the first, second, fourth and every further power-of-two drop:

▸ nucleus/kernel.c

```
    msg_t *m = new_msg(M_ON, ch, json, from ? from->id : -1);
    if (!mbox_reserve(T, m)) {   // a full subscriber loses the post; the publisher is not blocked or
        failed
      free_msg(m);
      uint64_t nd = ++T->dropped;
      if ((nd & (nd - 1)) == 0)   // the 1st, 2nd, 4th, 8th... drop is logged
        klog("%s's mailbox is full: dropped %llu post%s so far (the latest on %s)", T->name, (unsigned
            long long)T->dropped, T->dropped == 1 ? "" : "s", ch);
      continue;
    }
```

The powers of two bound the log as well. A subscriber that loses a million posts writes 20 lines, for the 1st to the 524,288th drop, and the exact count is in the host's view of the worlds (`dropped`). `bench/firstpass.py` checks the mechanism with a service that publishes 200 posts in one turn to a subscriber whose `mbox` is 16; the subscriber hears 16, 184 are dropped and counted, and the kernel log has eight lines, for the 1st, 2nd, 4th, ..., 128th drop:

▸ bench/firstpass-after.txt

```
PASS  posts to a full subscriber are dropped and counted
      slowsub heard 16, dropped 184, mailbox peak 16; slowsub's mailbox is full: dropped 1 post so far
          (the latest on spam)
```

Dropping is the right policy for the channels PicoCalc OS has. A `keys` post is an event the keyboard driver will not repeat, but a subscriber 64 messages behind on key presses is already useless to its user; `power` and `worlds.status` describe a state that the next post replaces. A channel whose posts could not be lost would need the publisher to know its subscribers and wait for them, which is an ask, and asks already have their own quota.

7.6 The attacks, afterwards

Run against the current build, `bench/rogue.py` reports both pool attacks refused:

▸ bench/rogue-after.txt

```
exhaust_tables: fill the shared framebuffer pool and handle table (the rogue has the dma claim)
  REFUSED: exhaust_bufs allocated 1 framebuffers, then: rogue_handles has used its quota of 1
      framebuffer | exhaust_handles narrowed 3 times, then: rogue_handles has used its quota of 4
      handles | compositor after a crash: idle
…
flood_mailbox: send flaky 3,000 messages a turn (the rogue may send to flaky)
  REFUSED: flood_mailbox sent 64 in one turn, then: flaky's mailbox is full (64 messages, 0 KB queued)
      | flaky's mailbox peak: 64
```

The rogue's framebuffer used one of its four handles, so three narrowings filled its quota. It holds 4 of the 128 entries and 1 of the 16 framebuffers, and when the compositor is killed it starts again with its framebuffer and view as if the rogue were not there. The flood is cut to 64 messages a turn, the size of `flaky`'s mailbox, and the sender learns why in its own `catch`. The verdicts in `rogue.py` are strict: the exhaustion scenario counts as refused only if both refusals name a quota, neither "out of handles" nor "out of DMA buffers" appears, and the compositor is running after its crash.

The two probes behave the same way. The ask flood now needs an `asks` entry naming `flaky` in its manifest entry to ask at all, and with it the probe is held to its eight slots:

```
[  1625.404] askflood: PROBE after 100 ms: 1092 of 1100 asks refused: askflood already has 8 asks
    waiting for replies, its quota
```

The display driver's flushes continue at 1,715.4 and 2,094.7 ms, the key presses reach the panel within the usual tens of milliseconds, and the right half of Figure 7.1 is what the screen shows. The victim probe, asking `flaky` every 37 ms beside the flood, gets 148 answers in 5.5 s, the same as when it runs alone, and none of its asks is refused. That last result depends on priorities as well as on the quota. `flaky` is a service at priority 2 and both probes are apps at priority 1, so the lane empties `flaky`'s 64 messages before either app runs again, and the victim never finds the mailbox full. A receiver slower than its senders would refuse the victim too, because the quota belongs to the receiver and does not distinguish one sender from another (exercise 9).

These runs establish the two properties the quotas were introduced for. A world at its quota fails in its own segment, with a message that names it and the quota; and nothing it does to its own share changes what any other world can draw from its share. They do not establish that the quotas are well chosen, which is the subject of the next section.

7.7 What static partitioning costs

A static partition is safe because no share can grow, and it costs capacity for the same reason. Table 7.4 compares, for the manifest as shipped, the tables' sizes with the sums of the quotas and with what the worlds actually use. The sums come from `js/system.json` with the defaults applied; the use comes from a snapshot of the host's view of the worlds (`snapshot` prints `quota`, `used` and `mbox_peak` for each world) at the end of each of the six scripts in `scripts/`, run on the host build.

Table 7.4. The shipped manifest's quotas against the kernel's tables and against observed use. Steady use is the same at the end of all six scripts; the largest mailbox is the longest any single mailbox grew in any script.

Resource	Kernel's table	Sum of the 13 quotas	Used at the end of every script	Largest seen
handles	128	54	5	5
ask slots	1,024	104	0	not recorded
framebuffers	16	2 (the compositor only)	1	1
mailbox messages	none (C heap)	896	0	6 (the compositor's, of its 128)
mailbox payload	none (C heap)	1,664 KB	0	not recorded

The partition wastes in two places. Inside a world, the quota sits unused above what the world needs: the compositor uses 2 of its 6 handles (4 during its own restart), and the longest mailbox in any script held 6 messages where the default quota is 64. That headroom is deliberate, because a quota that normal behaviour can reach turns a burst into a failure, and an unused handle entry costs 80 bytes of a table that exists anyway. Between worlds, 74 of the 128 handle entries and 920 of the 1,024 ask entries belong to nobody, and no world

can use them while the system runs. Only a new manifest and a new boot can move them. The handle and ask tables are static arrays, so this slack costs no memory; it costs flexibility.

The mailbox quotas have a different cost, because they bound real memory instead of table entries. If every world's mailbox were full at once, the messages would hold 1,664 KB of payload plus 896 message headers of 88 bytes on the 32-bit build, about 1.7 MB of the nucleus's heap. That is a bound, which the system before the pass did not have, and not a small one. The thirteen worlds' heaps together measure about 2.5 MB, and their limits add up to 5,120 KB. A handheld with 32 MB of PSRAM can afford it; a manifest for a smaller device would shrink `mbox_kb` first, since no message in the shipped system comes near 64 KB.

Workloads that need more must say so, and the benchmark manifest is the example. `bench/run.py`'s messaging benchmark has one world send 1,000 messages to another in a single segment, which a 64-message mailbox would refuse, so `bench/system-bench.json` declares what the benchmark needs:

▸ bench/system-bench.json

```
  {
   "name": "ponger",
   "file": "bench/ponger.js",
   "mbox": 1100
  },
```

That is the static design working as intended, since the burst is part of the system's description, visible to anyone who reads it and checked at boot. It is also the design's limit. A world whose need depends on what a person does with it cannot declare that need in advance: the REPL's user can type code that asks the sandbox for many things at once, and an app that downloads data could need a large mailbox for a minute and none afterwards. For such worlds a fixed share is either too small for the burst or reserves memory for it permanently. Genode's session quotas answer exactly this case, because the client that wants more from a server hands the server the memory to do it with, for as long as the session lasts [Feske 2026, "Resource trading"]. The PicoCalc equivalent would let an asker attach part of its own quota to an ask. Chapter 12 takes up the related proposal of charging the callee's processor time to the asker, after seL4's passive servers.

Two smaller costs complete the account. A quota bounds a count, not a size, so the compositor's two framebuffers may each be up to 640 × 640 pixels in two copies, 3.2 MB in all, charged to no world's `mem_kb`; `KNOWN-ISSUES.md` lists this as open. And an error now describes the world that reads it rather than the system: "has used its quota of 4 handles" helps the world's author, while a monitor looking for trouble must read the quotas, which the system monitor does not yet show and the workbench's Worlds view does (Chapter 9).

7.8 What is still shared

The first pass closed the pools that its authors had listed. A pool that nobody listed remains open, and a probe written for this chapter found one. When a world calls `sleep`, the nucleus schedules a settle event on its event queue, a binary heap of blocks taken from the C heap, for the moment the sleep ends. If the world crashes before then, `teardown` disposes of its runtime, its handles, its pending settles and its turn, and increments its epoch, so that the event will be ignored when it fires. The event itself stays in the queue until its time comes.

The probe (`l4book/traces/07-orphans.js`) starts 1,000 sleeps of 10^9 ms, waits 2.6 s and throws:

▸ `l4book/traces/07-orphans.js`

```
service(self).onStart(async () => {
  for (let i = 0; i < 1000; i++) sleep(1e9);
  await sleep(2600);
  throw new Error("crash on purpose, leaving 1,000 sleeps behind");
});
```

The pending sleeps of a live instance are bounded by the world's heap limit, because each holds a promise in that heap; an earlier version of the probe with 5,000 sleeps ran out of its 1,024 KB in its first segment. The orphaned events are not bounded by anything. Each instance lives 2.6 s and the backoff before the next grows from 100 to 800 ms, so the world crashes every 2.7 to 3.4 s, never has more than four crashes in ten seconds, and the supervisor keeps restarting it. A copy of the nucleus patched to print the length of its event queue with the `stats` command gave these counts, against 417 for the same system without the probe at every length of run:

```
after  10 s:   4,442 events,  3 crashes
after  60 s:  19,438 events, 18 crashes
after 300 s:  92,445 events, 91 crashes
```

Each crash leaves 1,000 events of at least 80 bytes each on the 32-bit build (an `event_t` of 48 bytes and a `settle_t` of 32), so after five minutes more than 7 MB of the nucleus's heap holds timers for instances that no longer exist, and the growth does not stop. The fix is a design choice with the same options as the rest of this chapter: count pending timers against a quota of the world, or make `teardown` remove the dead epoch's events from the queue, which turns a charge with no owner back into one with an owner that has just been torn down. Exercise 7 asks for the second.

The finding is the chapter's argument in small. A pool is shared whenever the nucleus allocates something on a world's behalf and nothing records the world as its owner. The first pass made the record for four kinds of allocation. The event queue is a fifth, and bus time, which *Small Worlds*, Chapter 11 found charged to no world, is a sixth. Each must either be charged to a world or be shown to be bounded by something that is.

KEY POINTS

- A shared pool is a store the nucleus allocates from on behalf of any world; before the first pass the handle table, the ask table, the framebuffers and message memory were such pools, and a world inside its own budget and heap limit could empty each of them.
- Exhausting a pool hurts the worlds that did nothing wrong; a full handle and framebuffer table stopped the compositor for good at its fifth restart, and a full ask table kept every flush from the panel for 10.5 seconds.
- The L4 family met the same problem in its kernel heap; per-process heaps and quotas (Fiasco.OC) and seL4's untyped memory, with no kernel heap at all, both make every allocation the charge of the principal that caused it.
- The first pass gives each world a quota of handles, framebuffers, ask slots and mailbox messages and bytes in the manifest, and the boot refuses a manifest whose handle, ask or framebuffer quotas add up to more than the kernel's tables.

- A handle is charged to the world that minted it, including a holderless entry kept alive by narrowed copies, and is released only when the entry is freed; a framebuffer is charged to the world that allocated it until its last reference goes.
- A message is charged to its receiver's mailbox from the moment it is sent until it leaves the mailbox, so a burst sent in one segment is bounded; a full mailbox fails a send or an ask at the sender and drops, counts and logs a channel post.
- In the shipped manifest the quotas use 54 of 128 handle entries, 104 of 1,024 ask entries and 2 of 16 framebuffers, while the worlds use 5, almost none and 1; the price of the static partition is capacity that cannot move without a new boot.
- Pools nobody listed remain; timers of crashed instances stay in the event queue, and a world that crashes every 3 s or so grows it by 1,000 events a crash.

The quotas make each world's misuse of the kernel's tables its own problem, which is the property seL4 obtains by construction and PicoCalc OS now obtains by a check at boot and accounting at every allocation. That accounting was added to a nucleus that already worked, and the first pass claims it changed nothing for a world that stays inside its quotas. Chapter 8 asks how such a claim can be established without a proof, and how strong the evidence for it is.

EXERCISES

1. `BASIC` In a copy of `js/system.json`, set the default `handles` to 10 and boot the host build. Does it boot? Find the smallest default that the boot refuses, and check your arithmetic against the error message.
2. `BASIC` A world with the default quotas and the `dma` claim allocates one framebuffer and narrows it three times, then releases the handle it allocated. How many of its handle quota's entries are still in use, and why? Check your answer against `handle_release` in `nucleus/hw.c`.
3. `ANALYSIS` Suppose the `release` line were removed from the compositor's `attach`. Trace `used_handles` of the compositor across successive restarts of the display driver, and say at which restart the compositor's `narrow` would fail. How many restarts would it have taken before the first pass to make some other world's request fail, and which world's?

4. ANALYSIS Compute an upper bound on the nucleus memory held by mailboxes for the shipped manifest on the 32-bit build, counting the `msg_t` header, the type and the payload of each message. Which manifest change would halve it with the least risk to the shipped worlds?

5. CODE The nucleus records each mailbox's peak length but no peak for asks waiting. Add a peak of `used_asks` to `kernel_worlds_json_full` in `nucleus/kernel.c`, run the six scripts with a `snapshot` at the end, and propose `ask_slots` values for the shipped worlds. What headroom would you keep, and why?

6. CODE Change `bench/checks/slowsub.js`'s manifest entry in `bench/firstpass.py` to `"mbox": 1` and predict the number of posts heard, dropped and logged. Run it and explain any difference from your prediction.

7. CODE Make `teardown` in `nucleus/kernel.c` remove the dead epoch's pending settle events from the event queue, or propose and implement another bound for them. Measure the queue with `l4book/traces/07-orphans.js` before and after, and check that `bench/rogue.py`, `bench/firstpass.py` and the captures of `scripts/*.txt` are unchanged.

8. ANALYSIS cgroup v2 allows limits to over-commit and PicoCalc OS refuses to. Give one argument for each choice for a handheld whose worlds are all listed in its manifest. Which argument changes if apps can be installed while the system runs?

9. ANALYSIS Using `l4book/traces/07-victim.js` beside `bench/rogue/rogue_flood.js`, describe a change of priorities or of `flaky`'s work under which the victim's asks would be refused. Is a full mailbox the receiver's problem or its senders'? Sketch a quota per pair of sender and receiver and estimate what it would add to the manifest.

10. PROJECT Design session quotas for asks after Genode: an asker attaches part of its own `mbox_kb` (or `mem_kb`) to an ask, the callee's work for it is charged against that, and the charge returns when the ask settles. Say what changes in `kernel_ask`, `settle_ask` and the manifest, and write the attack that your design must refuse.

FURTHER READING

[Elphinstone and Heiser 2013] Section 3.4 is the L4 family's own account of kernel memory: the fixed heap, the per-process heaps and quotas that followed, and seL4's removal of the heap.

[Elkaduwe et al. 2008] A short paper that states the isolation property of untyped memory precisely and explains why explicit allocation makes it provable.

[Feske 2026] The "Resource trading" section describes session quotas, the run-time mechanism this chapter's static quotas lack.

[cgroup v2 docs 2026] The "Resource Distribution Models" section is worth reading for its precise distinction between weights, limits, protections and allocations, and for where over-commitment is allowed.

[Liedtke et al. 1997] The early L4 analysis of which kernel resources a hostile process can exhaust, written for a system that would run downloaded code.

CHAPTER 8 PART II · THE FIRST PASS

Evidence without a proof

The first pass added about 400 lines to the nucleus, which now has about 2,800 (Chapter 1), and made four claims about them. A world that stays inside its manifest entry behaves exactly as it did before. The six attacks of `bench/rogue.py` are refused. The other new mechanisms (turn deadlines, the timed-out ask taken out of the mailbox, dropped posts, reloading, the boot checks) work as described. And the change introduced no memory error into the nucleus. seL4 would settle claims of this kind with a machine-checked proof about every possible execution. PicoCalc OS has no proof and will not get one for the part that matters most, because its isolation rests on QuickJS, an interpreter of some 80,000 lines of C. What it has instead is a set of instruments, each of which establishes something narrower than a proof, assumes something different, and costs a different amount.

This chapter is about how strong that evidence is. It lays out the kinds of evidence as a ladder, summarises what seL4's proofs give and what they cost, explains why deterministic simulation is PicoCalc's main instrument, and then re-runs the first pass's verification as a case study, with the real output of every instrument. It ends with what the instruments miss, measured rather than guessed, and with the instrument that should come next. By the end the reader should be able to say, for any claim this book makes about PicoCalc OS, what kind of evidence stands behind it and what that evidence takes on trust.

8.1 The evidence ladder

A piece of **evidence** for a claim about a system is a procedure whose success makes the claim more likely to be true. Two questions decide how much more likely. The first is which executions the evidence covers: all of them, or only those that were run. Dijkstra put the gap in one sentence in his Turing lecture: program testing "can be a very effective way to show the presence of bugs", but it "is hopelessly inadequate for showing their absence" [Dijkstra 1972]. The second question is what the evidence is about: the code that runs, or a model of it. A proof about a model says nothing about code that does not implement the model, and a test of the code says nothing about inputs it was not given. Every kind of evidence also takes something on trust, and that is the third thing to know about it.

Table 8.1 orders the kinds of evidence this book uses or discusses, from the claims that cover the most to those that cover the least.

Table 8.1. Kinds of evidence, from those that cover every execution to those that cover a measured few. Costs are those reported by the cited authors or measured in this chapter.

Evidence	Establishes, when it succeeds	Takes on trust	Cost	In PicoCalc OS
Machine-checked proof	a property of every execution of the code	the specification, the hardware, the unverified code (boot, assembly), the prover	seL4: about 20 person-years for 8,700 lines of C [Klein et al. 2009]	none
Model checking	a property of every execution of a model, up to a bound	that the model matches the code and the bound is large enough	writing and keeping the model	none
Proof on paper	a property of every execution, if the argument is right	that the argument was checked and the code is as it describes	hours, for a small invariant	confinement (*Small Worlds*, Chapter 9); the handle quota (Section 8.3)
Deterministic simulation	the exact behaviour of the real code on the inputs run, reproducible at will	the models of time and devices	a simulator; the 83 benchmark runs take 15 s	every capture, trace and benchmark
Differential testing	two implementations agree on the inputs run	that they do not share a mistake	a second implementation	four builds of one source
Attack suite	the named attacks fail, and keep failing	that attackers do what the suite does	one world and one verdict per attack	`bench/rogue.py`
Property checks	a mechanism does what is specified in chosen scenarios	that the scenarios reach the cases that matter	one scenario per property	`bench/firstpass.py`
Sanitizers	no memory error or undefined behaviour on the paths executed	that the paths executed are the ones that matter	slower runs (AddressSanitizer: 73% on average [Serebryany et al. 2012])	`make SAN=1`
Benchmarks	the measured quantities have these	that each measurement	a suite and its upkeep	`bench/run.py`

Evidence	Establishes, when it succeeds	Takes on trust	Cost	In PicoCalc OS
	values	means what it is taken to mean		

The ladder is not a ranking of usefulness. The rungs answer different questions, and the lower ones are cheap enough to run on every change, which the upper ones are not. A sanitizer finds a use-after-free that a functional-correctness proof would also exclude, but it finds it in an afternoon, on the code as it is, including the parts no proof covers. A benchmark establishes nothing about correctness and is still the only rung that can say whether a change made the system slower. The ordering is by the quantifier. The top three rungs make statements about every execution (of the code, or of a model), and every rung below makes statements about the executions that were run.

Two rungs deserve a definition here, because the rest of the chapter leans on them. **Model checking** explores every state of a finite model of a system and checks a property in each. Amazon's engineers, who model-checked TLA+ specifications of distributed services, reported a bug whose "shortest error trace exhibiting the bug included 35 high-level steps". They were also plain about the limit: to the question of how they know that the executable code implements the verified design, "The answer is we do not know" [Newcombe et al. 2015]. **Differential testing** gives the same inputs to two implementations of one specification and reports any difference; McKeeman described it for testing C compilers against each other [McKeeman 1998]. It needs no statement of what the right answer is, only a second implementation that is unlikely to be wrong in the same way.

The ladder is the vocabulary for the rest of the chapter. Each claim of the first pass will be matched with the rungs that support it, and each rung with what it trusts.

8.2 seL4's proof in one page

seL4 is the reference point for the top rung. Chapter 3 describes its proofs in detail; this section collects what they establish, what they assume and what they cost, because those three things are what the rest of the ladder is measured against.

The first proof, published in 2009, showed that the C implementation of seL4 refines an abstract specification of its behaviour. Every behaviour of the C code is one the specification allows, so the kernel "can never crash or otherwise behave unexpectedly as long as our as-

sumptions hold", and in particular never accesses a null or misaligned pointer [Klein et al. 2009]. The kernel was "8,700 lines of C code and 600 lines of assembler", and the proof "200,000 lines of Isabelle script". The proof cost "about 20 py" (person-years), of which 11 were specific to seL4 and 9 went into frameworks, tools and libraries that later proofs reused; designing and implementing the kernel, including a Haskell prototype, cost 2.2 person-years. Before verification began in earnest, student use and a port had found 16 defects; the verification found 144 more.

The 2014 account in *ACM Transactions on Computer Systems* describes what was added in the following five years: proofs that the kernel enforces integrity and confidentiality, and a proof that the compiled binary implements the C semantics, which removed the compiler from what must be trusted [Klein et al. 2014]. By then the proofs and specifications were "480 000 lines". What remained assumed is listed precisely: "correctness of handwritten assembly code, boot code, management of caches, and the hardware"; the boot and initialisation code, "about 1.2 kLOC of the kernel", is not covered; and the binary verification adds "the new, second-order assumption" that its own tool has no soundness bug. The seL4 Foundation's introduction adds a further limit: "These proofs presently do not capture properties associated with time", so the confidentiality proof excludes storage channels but not timing channels [Heiser 2020].

The cost of such proofs has itself been studied. Matichuk and colleagues measured the seL4 proofs and found "a consistent quadratic relationship" between the size of a property's formal statement and the final size of its proof, with effort, from earlier work, linear in the size of the proof [Matichuk et al. 2015]. A property twice as long to state costs about four times as much to prove. The cost is falling with newer tools. Atmosphere, a kernel written in Rust and verified with Verus and published in 2025, reports that developing and verifying "6K lines of executable code required an effort of less than 2.5 person-years", with "a proof-to-code ratio of 3.32:1", and that it "completes verification in less than 20 seconds on a modern laptop" [Chen et al. 2025]. That is a different kernel with different properties, and the comparison should not be pushed further than this. The price of the top rung, per line, has fallen several times over in fifteen years.

None of this transfers to PicoCalc OS as it is. A proof of the nucleus's C would be a proof about 2,800 lines that hand JavaScript values to and from QuickJS, and the property PicoCalc OS most needs, that one world cannot touch another's state, holds only if QuickJS's 80,864 lines keep each runtime's objects to itself. The worlds and the nucleus share one address space with no MMU or PMP protection between them; *Small Worlds*, Chapters 8 and 10 explain the trade, and Chapter 12 describes the planned fence for the nucleus. A proof of

the nucleus alone would therefore assume the very thing that isolation depends on. What PicoCalc OS can have is precise statements of its invariants (the quotas never exceed the tables; a timed-out ask never reaches its callee) and instruments that check them on every change. The rest of this chapter is about how good those instruments are.

8.3 Invariants on paper

What PicoCalc OS can have near the top of the ladder is a precise statement of an invariant and an argument, from the code, that every execution preserves it. *Small Worlds*, Chapter 9 gives one such argument in full, for confinement: a world that holds a non-grantable view it did not mint can pass on neither the view nor anything narrowed from it, proved from two lemmas about `handle_child` and `can_pass` [Small Worlds 2026]. That is a proof on paper. It covers every execution, and it trusts its reader to have checked it and the code to be as the lemmas describe.

The first pass's central guarantee can be stated and argued the same way. Write $used(W)$ for a world's `used_handles`, $q(W)$ for its `q_handles`, and $L(W)$ for the number of live entries of the handle table whose minter is W.

Invariant. Between host calls, $used(W) = L(W) \le q(W)$ for every world W.

Argument. All three quantities start at zero, because each world is allocated zeroed at boot and q is fixed then. Only two functions change whether an entry of the table is live, and only they change `used_handles`; a search of the nucleus for both names finds no other writer. `handle_new` makes one entry live with minter W and increments $used(W)$, and `handle_release` makes one entry dead and decrements its minter's count, so the equality is preserved. Every call of `handle_new` (in `op_hw_i2c`, `op_hw_spi` and `op_dma_alloc`, and through `handle_child` in `op_h_attenuate`) follows a call of `handle_quota` for the same world in the same host call, with no other handle made in between, and `handle_quota` lets it proceed only if $used(W) < q(W)$. After the increment, therefore, $used(W) \le q(W)$. ∎

Corollary. If the manifest passed the boot check, the number of live entries is $\Sigma\, L(W) = \Sigma\, used(W) \le \Sigma\, q(W) \le 128$, so `handle_new` always finds a free entry and no world can be told "out of handles".

The same argument, with `dmabuf_alloc`, `buf_unref` and the check at the top of `op_dma_alloc`, shows that no world can be told "out of DMA buffers" either. It also shows why the boot's framebuffer sum may leave out the worlds without the `dma` claim:

`op_dma_alloc` refuses them before anything is charged, so their count stays at zero for ever.

The corollary is what `bench/rogue.py`'s exhaustion verdict tests for one attack, since the verdict fails if either message appears. The argument covers every sequence of calls by every world and the test covers one sequence; but the argument assumes two things the test does not need, that the search for writers was complete (C lets any code write any field through a pointer) and that the code between each `handle_quota` and its `handle_new` makes no other handle. Each kind of evidence covers an assumption of the other. The cheapest way to strengthen both is to assert the invariant after every segment in a debug build, so that every scenario of every suite checks it in passing (exercise 7).

8.4 Simulation as the instrument

PicoCalc OS's main instrument is its own host build. *Small Worlds*, Chapter 18 explains how it works: a discrete-event simulation of the board, a clock that advances by counted work, register-level models of the two peripheral controllers, and every source of nondeterminism closed. What matters for evidence is one consequence. A script run twice on the same build produces the same pixels, the same trace and the same numbers, so a comparison between two builds is an equality test, not a statistical one.

That changes what a before-and-after comparison can establish. On real hardware, two runs of the same program differ, and the difference between two versions must be separated from that noise with repeated runs and statistics; Mytkowicz and colleagues showed that even the size of the environment variables or the link order can bias such a comparison by more than the effect being measured [Mytkowicz et al. 2009]. In the simulator a difference of one pixel or one virtual microsecond between two builds is a real difference in behaviour, and its absence across a script means the behaviour along that script is identical. FoundationDB's engineers built their testing on the same property, running the whole distributed database in one deterministic process so that every failure found in simulation can be reproduced [Zhou et al. 2021].

The instrument is also cheap enough to use on every change. On the machine used for this chapter, one run of the six scripts in `scripts/`, as six separate processes, took 0.57 s with the 64-bit build and 0.68 s with the 32-bit build (wall-clock times vary from run to run; Chapter 9 reports another run of the 32-bit build), `bench/rogue.py` runs its seven scenarios in 0.3 s, and the 83 runs of `bench/run.py` take 15 s. Determinism also survives changes that have nothing to do with the system's logic, which makes it a check on them. The coverage

build used in Section 8.6 is the same source compiled without optimisation and with every line instrumented; its 22 captures of the six scripts are identical to those of the optimised 64-bit build, so neither the optimiser nor the instrumentation changed a pixel.

What the instrument assumes is the model. The virtual clock charges each interpreter step 665 ns, a constant fitted to one measurement on the ESP32-P4 (the 100,000-iteration loop, 133 ms on the device and 133.0 ms in the model by construction). The same constant puts `fib(20)` at 29.1 ms against 31 ms measured and the 10,000-iteration loop at 13.3 ms against 11 ms, so absolute times are right to tens of per cent at best. The model also leaves out allocation and garbage collection, caches, interrupts and electrical faults, as *Small Worlds*, Chapter 18 lists. Every build runs the same QuickJS, so an engine bug is reproduced faithfully in every build and every run, and no comparison between builds can see it. Identical behaviour in the simulator is therefore strong evidence about the system's logic (which world runs, what it sends, what reaches the panel) and weak evidence about anything the model leaves out.

8.5 The first pass as a case study

The first pass's verification is a useful case study because its central claim is a negative one, that a world inside its manifest entry sees no difference. Table 8.2 matches each of the four claims with its instruments and with the result of running them again for this chapter on the current build.

Table 8.2. The first pass's claims, the instruments behind them, and their results when re-run for this chapter.

Claim	Instrument	Result
A world inside its manifest entry behaves exactly as before	the 22 captures of `scripts/*.txt` in three builds, compared with the same builds from before the pass	66 of 66 identical
	`bench/run.py`, compared value by value with the committed `bench/results.json`	2,227 of 2,246 values identical; the other 19 are wall-clock, code-size and host-allocator figures
The six attacks are refused	`bench/rogue.py`, seven scenarios	7 of 7 refused on the 64-bit, 32-bit and sanitizer builds; none on the build from before
The other new mechanisms work	`bench/firstpass.py`	5 of 5 checks pass
No new memory error or undefined behaviour in the nucleus	the sanitizer build on the six scripts	no report from the nucleus; one from QuickJS, also reported before the pass

8.5.1 Nothing changed

The captures are the strictest test of the first claim, because a capture is the panel's memory after a script and almost any difference in behaviour reaches the screen. Each script was run on the current 64-bit and 32-bit native builds and on the WebAssembly build under Node, and each capture was compared, pixel by pixel, with the capture the same build made before the first pass:

```
$ for s in scripts/*.txt; do ./build/picocalc-host -o new64 -u new64/uart.log $s; done
$ for s in scripts/*.txt; do ./build32/picocalc-host -o new32 -u new32/uart.log $s; done
$ node run-wasm.js $PWD wasm/picocalc.wasm wasm/files.json newwasm scripts/*.txt
$ node run-wasm.js $PWD picocalc.shipped.wasm files.shipped.json oldwasm scripts/*.txt
$ python3 compare.py new64 new64 old64 new32 old32 newwasm oldwasm
capture                  new64~old64     new32~old32 newwasm~oldwasm
01-home                         same            same            same
02-home-repl                    same            same            same
03-sysmon                       same            same            same
…
41-about                        same            same            same
boot                            same            same            same
```

All 66 comparisons are identical, including the four screens that print heap sizes, which differ between the 32-bit and 64-bit builds but not between a build and its own predecessor. The comparison across builds (the WebAssembly build matches the native 32-bit build on 21 of 22 captures, the About screen differing by its heap total) is the subject of Chapter 9 and is unchanged by the pass. The captures establish that the first pass changed nothing on the paths the six scripts take, in three builds compiled by two compilers for three targets. They say nothing about paths the scripts do not take, which Section 8.6 measures.

The benchmark suite tests the same claim through numbers instead of pixels. `bench/run.py` writes `bench/results.json` and its traces into the repository, so it was run in a scratch copy, and the new results were compared with the committed ones value by value with a short script that walks both JSON trees:

```
$ python3 jdiff.py results.orig.json bench/results.json
/code/object_sizes_32bit/nucleus/bss: 44100 -> 44452
/code/object_sizes_32bit/nucleus/text: 153614 -> 170159
/code/parts[0]/lines: 634 -> 959
…
/host/differ_32_64: ['03-sysmon.png', '04-sysmon-sel.png', '06-sysmon-buses.png', '41-about.png'] ->
    ['01-home.png', '02-home-repl.png', …]
/host/speed[0]/ratio: 776.1365938370728 -> 1131.767203936418
…
/memory/64/worlds[2]/mem_kb: 191.9 -> 191.8
…
2246 leaves, 19 differ
```

Table 8.3 sorts the 19 differences. Every latency, every recovery time, every bus figure and every 32-bit heap figure is identical to the value committed before the first pass.

Table 8.3. The 19 values of bench/results.json that differ after the first pass, of 2,246.

Values	How many	Why they differ
lines of code and object sizes	7	the first pass changed the code: `kernel.c` grew from 634 to 959 lines
wall-clock speed of the simulator, and the CPU's name	5	a different machine; wall-clock time is not virtual time
other wall-clock times (message benchmark, whole suite)	2	as above
64-bit heap figures	4	the 64-bit host allocator's layout, by at most 0.1 KB; the 32-bit figures, which stand for the device, are unchanged
captures that differ between the 32-bit and 64-bit builds	1	the instrument, as described below

The last row is a lesson about instruments. `run.py` checks determinism across builds by hashing each capture's PNG file, and the committed results list four captures that differ between the 32-bit and 64-bit builds, the four heap screens. On the machine used for this chapter it lists twenty. The pixels have not changed. Compared pixel by pixel, the same two builds differ on exactly the four heap screens. What differs is the compression. The 64-bit binary links zlib-ng 1.3.1 from `/usr/local/lib`, the 32-bit binary the system's zlib 1.3, and the two compress identical pixels to different bytes. The check was meant to measure the system and also measured the compressor, which no one had noticed because on the machine that produced the committed results both builds used the same library. Exercise 5 asks for the repair, which is to hash the decoded pixels.

Taken together, the captures and the benchmarks support the first claim strongly within the model. On every path the scripts and benchmarks exercise, the first pass changed no pixel and no virtual microsecond.

8.5.2 The new mechanisms work

The second and third claims are positive. Something that used to happen now does not, and something new now happens. `bench/rogue.py` and `bench/firstpass.py` are built the same way. Each scenario copies `js/` to a scratch directory, adds one or two worlds to the copy's manifest, runs the host build on a short script, and decides from the kernel log and a final snapshot of the worlds whether the claim held. The decision is a **verdict function**, and its strictness is what makes the result evidence. The verdict for the table-exhaustion attack of Chapter 7 is a good example:

▸ bench/rogue.py

```
    def exhaust(r):
        bufs = grep(r["log"], r"^.*OUTCOME exhaust_bufs.*$") or "(no report)"
        hs = grep(r["log"], r"^.*OUTCOME exhaust_handles.*$") or "(no report)"
        comp = world(r["snap"], "compositor")
        state = comp["state"] if comp else "?"
        shared_full = "out of handles" in hs or "out of DMA buffers" in bufs
        refused = ("quota" in hs and "quota" in bufs and state not in ("stopped", "restarting"))
        ev = f"{bufs.split('OUTCOME ')[-1]} | {hs.split('OUTCOME ')[-1]} | compositor after a crash:
            {state}"
        return refused and not shared_full, ev
```

The verdict requires three things: both refusals name a quota, neither shared table reported itself full, and the compositor is running after being killed. A weaker verdict that only looked for the word "quota" would pass a nucleus that refused the rogue and still let the compositor die. The same suite run with `--bin` against a build of the nucleus from before the first pass is the control. Every scenario that is refused now must succeed there, or the scenario is not testing what it claims to. All seven succeed against the old build (`bench/rogue-before.txt`), and all seven are refused by the current 64-bit, 32-bit and sanitizer builds. The suite's output for the cycle scenario on the current build shows a refusal of a different kind, at boot:

▸ bench/rogue-after.txt

```
ask_cycle: two worlds that ask each other
  REFUSED: boot refused: /tmp/pcrogue-npnzi_yl/js-ask_cycle/system.json: asks form a cycle: cyc_a →
      cyc_b → cyc_a (each would wait for the other's turn to end; asks must form no cycle)
```

`bench/firstpass.py` checks the mechanisms that are not about attackers, with the same structure. Its output, unchanged when re-run for this chapter, is short enough to show whole:

▸ bench/firstpass-after.txt

```
PASS  a turn that outlives turn_ms is a crash
      slowpoke crashed handling start: its turn handling "start" did not finish within its turn_ms of
          1000 ms
PASS  a timed-out ask is taken out of the callee's mailbox
      no reply from flaky within 500 ms | then flaky answered pong 1 (pong 2 would mean it handled the
          timed-out one)
PASS  posts to a full subscriber are dropped and counted
      slowsub heard 16, dropped 184, mailbox peak 16; slowsub's mailbox is full: dropped 1 post so far
          (the latest on spam)
PASS  reload restarts a world without counting a crash
      about: state idle, restarts 1, crashes 0
PASS  the manifest checks stop a bad boot, one line per problem
      world "launcher": "sends" names "nobody", which is neither a world nor a group
      world "repl": "frames" is published by the kernel; no world may provide it
      channel "keys" has two providers, keyboard and snake; a channel has one
      the worlds' handle quotas add up to 246; the kernel's handle table holds 128
      4 problems; the system does not boot
```

Each check prints its evidence as well as its verdict, so a reader can see what was observed. The second check, for example, passes because `flaky` answers the second ping with "pong 1", which it would not have done had the first ping reached it. A check that printed only PASS would have to be trusted; one that prints its observation can be audited.

The fourth claim rests on the sanitizer build, compiled with AddressSanitizer and the undefined-behaviour sanitizer (`make SAN=1`). AddressSanitizer finds "out-of-bounds accesses to heap, stack, and global objects" and use-after-free bugs, at the point where they happen [Serebryany et al. 2012]. All six scripts run under it with one report:

```
$ ./build-san/picocalc-host scripts/tour.txt
third_party/quickjs/quickjs.c:36335:67: runtime error: left shift of 168 by 24 places cannot be
    represented in type 'int'
panel: awake, display on, colmod 0x55, madctl 0x48, inversion on, 568741 pixels written, 0 warnings
```

The line is in QuickJS's bytecode optimiser, where the macro `M4` packs four opcodes into an `int` by shifting one of them left by 24 bits, which overflows for opcodes above 127. Every script reports it once, and the first pass's own record says that the build from before the pass reports it identically [First pass 2026]. The nucleus produced no report on the six scripts, nor on the attack and probe scenarios of Chapter 7 run under the same build.

Together, the two suites and the sanitizer support the second, third and fourth claims for the scenarios they contain. Each is a regression test as well as evidence, since every scenario can be run again after any change, and the two suites together take under a second.

8.6 What the evidence does not show

Every instrument above reports on the executions it ran, and the honest question is which executions those were. Line coverage measures it directly. A copy of the host build compiled with `gcc --coverage` was run on the six scripts, then on the two suites, then on the benchmarks, and `gcov` counted the nucleus's executable lines that ran at least once:

Table 8.4. Lines of the nucleus executed by the repository's instruments, cumulatively, in a coverage build of the host (gcc --coverage, -O0).

Instruments	`kernel.c`	`vm.c`	`hw.c`	`gfx.c`	All four
the six scripts	85.7%	84.2%	89.7%	83.9%	1,485 of 1,737 (85.5%)
and `rogue.py` and `firstpass.py`	95.1%	86.5%	91.6%	83.9%	1,572 (90.5%)
and `bench/run.py`	95.9%	90.5%	92.5%	83.9%	1,607 (92.5%)

Nine lines in ten ran. The tenth matters more, because it shows what the evidence of the previous sections does not reach. Three gaps stand out in the code `gcov` marks as never executed.

- **Revocation.** `op_h_revoke` and `handle_revoke_tree`, the code that revokes a handle and every copy narrowed from it, never run. No shipped world, no benchmark world, no attack and no eval calls `revoke`. *Small Worlds*, Chapter 9 describes revocation in detail, and one of its defects (revoke acting before the segment ended) was found and fixed by reading code; since then, no instrument in the repository has executed it.

- **Two of the first pass's own refusals.** The branches of `kernel_ask` that reject an ask because the asker has used its `ask_slots`, or because the callee's mailbox is full, never run. `FIRST-PASS.md` lists their messages, and Chapter 7 shows the first one firing in a probe written for that chapter, but neither suite contains a scenario that reaches them.
- **A segment that crashes after sending.** `discard_deferred`, which frees the messages of a segment that crashed after sending them and so releases their mailbox charge, never runs. The accounting described in Chapter 7 is correct by reading; it is untested.

Coverage measures what ran, not what was checked, so even a covered line may be wrong in a way no verdict looks at. Chapter 7 found a pool of that kind. Timers left by crashed instances accumulate in the nucleus's event queue without bound, and every line involved is covered many times over by the scripts. No instrument noticed because none asked how long the event queue was; an attack suite tests what its authors imagined, and nobody had imagined that one.

The other weaknesses are those of the model and of the trusted base, and they are stated here so that no result in this book is read as more than it is.

- QuickJS is trusted completely. Its 80,864 lines run in every build, so every comparison between builds replicates its behaviour, and the sanitizer exercises only the paths the scripts reach. A bug in the engine that lets one runtime reach another's objects would break isolation, and nothing in this chapter would detect it.
- The worlds and the nucleus share one address space with no hardware fence between them. A memory error in the nucleus or the engine could corrupt any world, and the sanitizer can only show that the executed paths had none.
- The timing model is calibrated on one measurement, so every virtual-time figure is exact about the model and approximate about the ESP32-P4, to the tens of per cent that `fib(20)` and the 10,000-iteration loop suggest.
- The manifest checks have no proof. The cycle search (`dfs` in `kernel_boot`) and the quota sums are short and are exercised by one bad manifest each; whether they are right for every manifest is a claim about every input, which no test establishes. They are small enough for the property checks of the next section.
- Everything here was measured on the host build. The port to the device is planned, and none of this evidence is evidence about the device's drivers, its interrupts or its memory.

None of these makes the first pass's claims false. Each says where the claims stop.

8.7 Differential testing: the next instrument

The four builds already form a differential test of a narrow kind: one source, compiled four ways, must give the same pixels. It catches compiler, word-size and C-library differences, and the PNG compressor showed that it catches differences in its own measuring apparatus too. It cannot catch a mistake in the source, because all four builds contain it. For that, differential testing needs a second implementation written independently from the same rules.

Small Worlds, Chapter 17 built one: `minios`, an operating system of Web Workers in about 500 lines of JavaScript, written to follow the nucleus's rules (the same supervision policy, the same turns, effects buffered by the prelude) with none of its code [Small Worlds 2026]. It already disagrees with the nucleus, in ways the book's errata record. The clearest is the moment a world's epoch changes, which decides whether a result that arrives during a restart reaches the new instance. The nucleus increments the epoch when the world is torn down, at the crash:

▸ nucleus/kernel.c

```
  for (int i = 0; i < MAX_LIST; i++) W->tick_queued[i] = false;
  W->epoch++;
}
```

`minios` increments it when the world starts again, after the backoff:

▸ book/widgets/minios.js

```
    start(w) {
      w.epoch++;
      w.status = "starting";
      const epoch = w.epoch;
```

So in `minios` a reply that arrives during the 100 ms backoff still carries the current epoch and is queued for the new instance, while the nucleus drops it. Another errata item records that `minios` posts a segment's effects when the world calls `ask`, `device` or `sleep`, so a later throw in the same segment does not undo them. Both were found by reviewers. A dif-

ferential harness would find them mechanically: run the same small worlds and the same script on both systems, record each turn (world, message, outcome) and each delivered result, normalise away what legitimately differs (`minios` runs in real time, so only the order of events can be compared, not their times), and report the first divergence.

The inputs for such a harness should be generated, not written. In **property-based testing** a property is stated as a function and checked automatically on many random inputs; QuickCheck introduced the style for Haskell programs [Claessen and Hughes 2000]. For PicoCalc OS the generator would produce small manifests and worlds (a few worlds that send, ask, sleep, throw and loop, in random combinations) and short scripts of keys and crashes, and the property would be that the nucleus and `minios` produce the same sequence of turns. The same generator, with a reference implementation of the cycle check in a few lines of Python, would test `dfs` and the quota sums against every small manifest rather than one bad one.

Model checking is the rung above. The rules that matter most for the first pass, that asks form no cycle and that the sum of charges never exceeds a table, are properties of a small state machine: worlds, mailboxes, slots and turns. A TLA+ model of that machine could be checked exhaustively for three or four worlds, and the 35-step trace Amazon's engineers reported shows how long a sequence of events such a check can reach [Newcombe et al. 2015]. It would carry the caveat they gave. It would establish the design, and the code would still need the lower rungs to show that it follows the design.

The order of work follows from the costs in Table 8.1. A verdict for every gap in Table 8.4 costs an afternoon each and should come first; the differential harness against `minios` costs days and finds the class of bug none of the existing instruments can see; a model of the turn and ask rules costs more and covers the design completely. A proof of QuickJS is not on the list.

KEY POINTS

- Evidence differs in which executions it covers, whether it is about the code or a model, and what it takes on trust; only proofs and model checking cover every execution, and every other instrument covers the executions it ran.
- seL4's first proof covered 8,700 lines of C with 200,000 lines of proof and about 20 person-years; later proofs grew to 480,000 lines and still assume the hardware, the boot code, cache management and hand-written assembly, and none covers time.

- Proof effort grows quadratically with the size of the statement proved, and newer tools have lowered its price several times over, but no proof of the nucleus could establish isolation while QuickJS is trusted.
- PicoCalc OS can state its invariants precisely and argue them from the code; the handle quotas imply that "out of handles" cannot occur after a well-formed boot, provided the search for every writer of the counts was complete.
- Determinism turns every before-and-after comparison into an equality test, which is why the host build is PicoCalc OS's main instrument; its evidence is exact about the system's logic and approximate about time and hardware.
- Re-run for this chapter, the first pass's verification holds: 66 of 66 captures identical across three builds, every virtual-time benchmark value unchanged, 7 of 7 attack scenarios refused, 5 of 5 checks passing, and no sanitizer report from the nucleus.
- An instrument can measure its own apparatus as well as the system; the benchmark's determinism check hashed PNG files and reported 20 differing captures where the pixels differ on 4, because two zlib builds compress the same pixels differently.
- The instruments execute 92.5% of the nucleus's lines, and the rest includes all of handle revocation, two of the first pass's own refusals and the release of a crashed segment's messages.
- Differential testing against an independently written implementation, with generated inputs, is the next instrument; it finds mistakes in the rules themselves, which no comparison of builds of one source can.

A proof would say what PicoCalc OS does for every input. The instruments say what it did for the inputs they gave it, exactly and reproducibly, and the coverage figures say how far those inputs reached. That is the evidence Part II rests on. Part III turns to the place where most changes to PicoCalc OS are now made. Chapter 9 describes the same nucleus compiled to WebAssembly and running in a page, where the evals of Chapter 11 run checks of this kind on a fresh system after an edit.

EXERCISES

1. `BASIC` For each of the four claims of the first pass, name the instrument that supports it, the rung of Table 8.1 it belongs to, and one thing it takes on trust.

2. `BASIC` Run `python3 bench/rogue.py --bin build32/picocalc-host` and `--bin build-san/picocalc-host`. Do the verdicts match the 64-bit build's? What would it mean if one scenario were refused on one build and not on another?

3. `ANALYSIS` Using the effort figures reported for seL4 and Atmosphere, estimate what a functional-correctness proof of PicoCalc's 2,800-line nucleus might cost. Then list three properties of PicoCalc OS that such a proof could not establish, and say why.

4. `ANALYSIS` Write the argument of Section 8.3 for ask slots: state the invariant relating `used_asks` to the live entries of the ask table, find every writer of both, and say where the argument differs from the one for handles (consider the table check that comes before the quota check in `kernel_ask`, and an asker that crashes while its asks wait).

5. `CODE` Change the determinism check in `bench/run.py`'s `host()` to hash each capture's decoded pixels rather than its PNG file, and confirm that it reports the four heap screens on a machine where the two builds link different zlib libraries.

6. `CODE` Write a world and a verdict that exercise revocation: a world narrows a handle, passes the copy, revokes it, and checks that the holder's next use fails with "was revoked". Add it to a suite and measure the change in `hw.c`'s coverage with `gcc --coverage`.

7. `CODE` Add a debug build option to `nucleus/kernel.c` that checks the handle invariant of Section 8.3 after every segment (count the live entries minted by each world and compare with `used_handles` and `q_handles`) and aborts with a message if it fails. Run the six scripts, `bench/rogue.py` and `bench/firstpass.py` with it.

8. `CODE` Add to `bench/firstpass.py` scenarios that reach the two unexecuted refusals in `kernel_ask` (an asker over its `ask_slots`, and an ask to a full mailbox), with verdicts that check the exact messages.

9. `CODE` Write a property-based test for the boot's cycle check: generate random `asks` graphs over five to ten worlds, write each as a manifest, boot the host build, and compare its verdict with a cycle check you write in Python. Report how many manifests you ran and whether any disagreed.

10. ANALYSIS The orphaned-timer pool of Chapter 7 was covered by the scripts and found by none of the instruments. Propose an invariant that a verdict could check after every scenario (for example, a bound on the event queue in terms of the live worlds' pending promises) and say what the nucleus would have to report for the verdict to be decidable.

11. PROJECT Build a differential harness that runs the same small worlds on the nucleus (through the host build or the WebAssembly build) and on `minios`, records the sequence of turns on each, and reports the first divergence. Show that it finds the difference in when the epoch changes.

12. DISCUSSION Which single claim in this book would you most want proved rather than tested, and which rung of Table 8.1 would you climb first to strengthen the evidence for it?

FURTHER READING

[Klein et al. 2014] The full account of seL4's verification: what each proof says, what it assumes, how the proofs were kept current as the kernel changed, and what they cost.

[Matichuk et al. 2015] A short empirical study of where the cost of a proof comes from, with the quadratic relationship between statement and proof.

[Newcombe et al. 2015] How model checking was adopted for production systems, with an unusually frank account of what it does not establish.

[McKeeman 1998] The original description of differential testing, written for compilers and directly applicable to two implementations of one operating system's rules.

[Dijkstra 1972] The Turing lecture that drew the line between testing and proof, still the clearest statement of why the line matters.

[Chen et al. 2025] A recent verified kernel whose reported costs show how far the price of the top rung has fallen.

The system in a page

Part II changed the nucleus and showed each change with an attack, a check and a benchmark run from the command line. Changing a system that way is slow: edit a file, rebuild or rerun, read a log, compare numbers by eye. The workbench puts the same system in one HTML page, beside its code, its kernel tables, its trace and an eval suite, so that an edit can be applied to the running system in milliseconds and checked on a fresh system in a few more. It is also the place where the coding agent of Chapter 10 works, and where the evals of Chapter 11 run.

None of that is worth much unless the system in the page is the system the rest of this book measures. The workbench runs the nucleus, QuickJS, the simulated panel and keyboard controllers and the thirteen worlds, compiled from the same C and JavaScript sources as the host build. This chapter explains how that build is made and driven, measures how far it agrees with the native build (one capture in twenty-two differs, and the chapter traces that difference to a single line of the allocator), and then describes the workbench built around it: its host, the live system and the fresh systems, hot reload and reboot, its views and its design. At the end the reader should be able to say what happens, down to the exported C function, when a person presses Apply, and why a number read in the page can be trusted as a number of the host build.

9.1 Building for WebAssembly

Small Worlds, Chapters 16 and 17, describe the first WebAssembly port of PicoCalc OS [Small Worlds 2026]: the same sources compiled with clang for the `wasm32-wasip1` target, a copy of glibc's `rand` so that the panel's power-on noise is the same, a loader whose clock returns

a fixed instant, a page that paces virtual time to real time, and the observation that the module's import list states its whole authority. This section does not repeat that account. It describes the build as the workbench uses it after the first pass, with the parts that matter for a host that changes the system while it runs.

The build needs a compiler that targets WebAssembly and a C library for it. wasi-sdk supplies both: it packages Clang and LLVM with wasi-libc, a C library whose system calls are WASI imports, and the compiler-rt builtins [wasi-sdk 2026]. `make wasm` compiles QuickJS, the nucleus, the board simulation and the script interpreter with that toolchain, using `host/main_wasm.c` in place of the native `host/main_host.c` and leaving out `nucleus/png.c`. The link step is where the build stops being a program:

▸ Makefile

```
wasm/picocalc.wasm: $(WASM_OBJ)
  $(WASM_CC) --target=wasm32-wasip1 --sysroot=$(WASI_SYSROOT) -nostdlib -o $@ \
    $(WASI_SYSROOT)/lib/wasm32-wasip1/crt1-reactor.o $^ -lc $(WASI_RT) \
    -Wl,--no-entry -Wl,-z,stack-size=1048576 -Wl,--strip-all
```

WASI distinguishes two kinds of module. A **command** exports `_start`, which runs the program from beginning to end; every other module is a **reactor**, which may export an initialiser called `_initialize` that the environment must call at most once, before it touches any other export, after which the instance stays live and its exports may be called [WASI ABI 2026]. Linking with `crt1-reactor.o` and `--no-entry` makes PicoCalc OS a reactor. The loader calls `_initialize` once, which runs the C library's constructors, and from then on the page decides what happens and when. A command would not fit, because the nucleus has a main loop (`kernel_run_until`) that runs until a given virtual time, and in a page only the page may own the loop: the browser must get control back between animation frames to draw, take input and run the workbench's own code. A reactor turns the system inside out. The page calls `pc_run(16)` and the nucleus runs sixteen virtual milliseconds of segments, deferred effects and bus transfers, then returns.

There is no filesystem either. The kernel reads its files through one function, `read_file`, used for the prelude, the manifest and each world's source. The WebAssembly build compiles with `-DPC_MEMFS`, which removes the native version from `nucleus/util.c`, and supplies its own, backed by a table in memory that the page fills before boot:

▸ host/main_wasm.c

```
EXPORT(pc_add_file) int pc_add_file(const char *path, const char *data, size_t len) {
…
char *read_file(const char *path, size_t *len) {
  while (path[0] == '.' && path[1] == '/') path += 2;
  for (int i = 0; i < n_files; i++) if (!strcmp(files[i].path, path)) {
    char *b = malloc(files[i].len + 1);
    memcpy(b, files[i].data, files[i].len + 1);
    if (len) *len = files[i].len;
    return b;
  }
  return NULL;
}
```

`pc_add_file` replaces a file that is already in the table, which is how a hot reload gets new source into the nucleus (Section 9.3). The table starts as `wasm/files.json`, which `wasm/bundle.py` writes from the manifest: the prelude, the manifest and each world's file, 15 files in 65,006 bytes of JSON, exactly the files the native build reads from `js/`.

Output takes the opposite path. The nucleus writes to `stdio` streams: its log to the console UART, its trace to a file opened by the `trace` command. Rather than change that code, `main_wasm.c` makes a `FILE *` whose bytes go to the page through an imported function, using `fopencookie`, a GNU extension that wasi-libc also provides:

▸ host/main_wasm.c

```
static ssize_t emit_write(void *cookie, const char *buf, size_t n) { js_emit((int)(intptr_t)cookie,
    buf, (int)n); return (ssize_t)n; }
static FILE *channel(int ch) {
  cookie_io_functions_t io = { .write = emit_write };
  FILE *f = fopencookie((void *)(intptr_t)ch, "w", io);
  if (f) setvbuf(f, NULL, _IOLBF, 0);
  return f;
}
```

Channel 1 is the UART, channel 2 the trace. Both are line-buffered, so the loader receives whole lines, splits them, parses the trace lines as JSON and hands them to whoever listens. Everything else the kernel prints (`stats`, boot errors, the reason a reload failed) goes to WASI's `fd_write` on standard output or standard error.

Listing the module's imports and exports with the JavaScript API (`WebAssembly.Module.imports` and `.exports`) gives the whole interface between the page and the system. A module's code can reach its environment only by calling functions the environment supplies as imports [WebAssembly Core 2026]. This one imports thirteen: two of its own (`env.pc_emit` and `env.pc_capture`) and eleven WASI functions, which the loader's shim in `wasm/picocalc.js` implements mostly by refusing (no preopened directories, `path_open` returns `ENOENT`). It exports its memory, `_initialize` and eighteen functions, grouped in Table 9.1.

Table 9.1. The eighteen functions host/main_wasm.c exports, as the module lists them. Every one runs to completion before returning; none waits for anything outside the module.

Group	Exports	What they do
memory	`pc_alloc`, `pc_free`	Room in linear memory for strings and file contents the page passes in.
files	`pc_add_file`	Add or replace a file in the in-memory table.
life	`pc_boot`, `pc_run`, `pc_now_us`	Boot from the table; run for some virtual milliseconds; read the virtual clock.
input	`pc_key_code`, `pc_key`, `pc_battery`, `pc_command`	Keys down and up at the current virtual time; the battery the STM32 reports; one script command, exactly as in `scripts/*.txt`.
faults	`pc_crash`, `pc_reload`	Kill a world as a fault would; start a world again from its file as last added.
reading	`pc_trace`, `pc_screen`, `pc_screen_width`, `pc_screen_height`, `pc_frame`, `pc_stats`	The trace on or off; the panel's memory as RGBA; a world's last frame as JSON; every counter, world and handle as JSON.

Two of these came with the first pass [First pass 2026]: `pc_reload`, which the workbench needs to hot-reload a world, and `pc_frame`, which returns the user-interface tree a world last published. `pc_stats` also grew: each world now carries its authority from the manifest (`provides`, `needs`, `sends`, `asks`), its quotas and their use, its dropped posts and, while it is in a turn, what it is handling. The Worlds view draws the quotas, their use and the dropped posts directly from these fields; the Graph view draws the same authority from the applied manifest.

The module is 1,207,039 bytes. *Small Worlds* measured 1,167,140 bytes before the first pass, so the manifest checks, the quotas and the new exports cost about 40 KB of code. Its linear memory is 2.00 MiB when instantiated, of which 1 MiB is the stack the link line sets, 2.25 MiB after `pc_boot`, and 5.19 MiB after a minute of virtual time, when all thirteen worlds have started and the compositor holds its framebuffers. Those 5 MiB are the price of one system, and the workbench creates many.

This section has established what the page talks to: a reactor whose eighteen exports run the system for as long as they are asked and no longer, reading files from a table the page controls and writing lines to the page through two channels. The next question is whether what runs behind those exports behaves as the native build does.

9.2 Identity with the native build

The workbench is where changes to PicoCalc OS are made and first checked. If the page's system behaved differently from the host build, every result obtained in the page would need checking again on the command line, and the numbers of `bench/run.py`, which this book and the paper quote, would say nothing about what a person sees in the page. If the two are identical, the page is a second instance of the same experiment, and a result in one is a result in the other. Identity is therefore the property to measure, and pixels are the strictest observable: a capture is the panel's memory after a script, and a difference anywhere in the system's behaviour that reaches the screen changes it.

The measurement runs the six scripts in `scripts/` three ways: with the native 32-bit build (`./build32/picocalc-host -o DIR script`), with the native 64-bit build, and with the WebAssembly build under Node, one fresh instance per script, through a small runner that passes each script line to `pc.command` and saves every capture as raw RGB. Node 22 and the native builds ran on the same two-core Linux machine:

```
$ node run-wasm.js $PWD wasm/picocalc.wasm wasm/files.json $OUT scripts/*.txt
boot.txt: 4 commands, virtual 1200 ms, wall 311 ms, stderr: panel: awake, display on, colmod 0x55,
    madctl 0x48, inversion on, 159373 pixels written, 0 warnings
faults.txt: 37 commands, virtual 13880 ms, wall 190 ms
keys.txt: 14 commands, virtual 5130 ms, wall 74 ms
repl.txt: 26 commands, virtual 20510 ms, wall 180 ms
snake.txt: 13 commands, virtual 31680 ms, wall 337 ms
tour.txt: 28 commands, virtual 6550 ms, wall 73 ms, stderr: panel: awake, display on, colmod 0x55,
    madctl 0x48, inversion on, 574231 pixels written, 0 warnings
```

The scripts write 22 captures. A comparison script counts, for each pair of builds, the pixels whose RGB values differ; Table 9.2 shows every capture that differs in any pair.

Table 9.2. Pixels that differ between builds, out of 409,600 per capture (320 × 320 at scale 2). The 18 captures not listed are identical in all three pairs.

Capture	native 32-bit vs WebAssembly	native 64-bit vs WebAssembly	native 32-bit vs 64-bit
`03-sysmon`	same	804	804
`04-sysmon-sel`	same	696	696
`06-sysmon-buses`	same	260	260
`41-about`	104	108	92

The WebAssembly build matches the native 32-bit build on 21 of 22 captures, as the first pass's own verification reported [First pass 2026]. The 64-bit build differs from both on four captures, which is expected: it scales its heap figures by a measured factor of 1.17 to approximate the device's 32-bit sizes, and the system monitor and About screens print heap figures. The 32-bit build is the one that represents the device, and the WebAssembly build, a 32-bit target, agrees with it.

Pixels show that a difference exists, not where it comes from. A sharper comparison runs each script again in both builds, appends the `stats` command, and compares the 17 lines it prints: the virtual time, lane occupancy, turns, posts and deliveries, every world's state, turns, CPU time and heap, and each bus's time, transfers and bytes. In five of the six scripts every line is identical except the heap column of one or two worlds. Snake's heap is 197.8 KB natively and 201.6 KB in WebAssembly in every script; the system monitor and the faults app differ by the same 3.8 KB where they appear. Every turn count and every CPU time in milliseconds agree to the printed precision. The sixth script, `keys.txt`, ends on the About screen, and there the difference propagates:

```
wasm: t = 5130.0 ms, lane busy 120.3 ms (2.3%), 479 turns, 50 published, 141 delivered, 0 crashes
nat:  t = 5130.0 ms, lane busy 120.2 ms (2.3%), 479 turns, 50 published, 141 delivered, 0 crashes
wasm:   snake       app      idle            47      5.86    201.6    320    0
nat:    snake       app      idle            47      5.86    197.8    320    0
wasm:   bus spi2  busy   273.09 ms, 414 transfers, 1354764 bytes, 0 errors
nat:    bus spi2  busy   272.99 ms, 414 transfers, 1354288 bytes, 0 errors
```

About prints the sum of every world's heap. A different number has different glyphs, so the compositor's damage rectangle covers different pixels, the display driver sends 476 more bytes over SPI, and the bus is busy 0.1 ms longer. That is the only difference in virtual time in all six runs, and it follows from the heap figure.

The heap figure itself comes from the allocator wrapper each world's QuickJS runtime uses:

▸ nucleus/vm.c

```c
static void *w_malloc(JSMallocState *s, size_t size) {
  world_t *W = s->opaque;
  if (W->mem_used + size > host_limit(W)) { W->oom = true; return NULL; }
  void *p = malloc(size);
  if (!p) return NULL;
  size_t u = malloc_usable_size(p) + 8;
  W->mem_used += u; s->malloc_size += u; s->malloc_count++;
  return p;
}
```

A world is charged what the C library's allocator actually set aside for each request, `malloc_usable_size(p)`, plus an estimate of its header. That quantity belongs to the allocator, and the WebAssembly build's allocator is wasi-libc's, not glibc's, so the same sequence of requests is charged slightly differently. The accounting is right in each build; it measures a different allocator. The same number also enforces the heap limit, so a world running within a few kilobytes of its limit could fail at a different moment in the two builds. No world in the shipped system comes close; Snake, for example, uses about 200 KB of its 320 KB.

The practical rule follows. A result that depends on heap sizes measures the C library as well as PicoCalc OS, and should be read as approximate across builds. Every other result, every turn, every virtual millisecond and every pixel that does not print a heap figure,

transfers exactly between the page and the native 32-bit build. The `deterministic` eval checks the complementary property inside the page, that two fresh WebAssembly systems given the same keys show the same pixels and count the same turns (Chapter 11); *Small Worlds*, Chapter 18, explains why the simulation is deterministic at all.

9.3 The workbench's host

The workbench is one HTML file, `workbench/picocalc-workbench.html`, assembled by `workbench/build.py` from the stylesheet, the host, agent, views and shell sources of `workbench/src/`, the loader `wasm/picocalc.js`, the Reference, the eval suite, `files.json`, this book when it has been pre-rendered (Section 9.4), and the module itself, base64-encoded in 1,609,388 bytes. The page is about 1.9 MB without the book and 4.5 MB with it; the book brings its text, its figures and the source files its excerpts quote. React 18 and the `htm` template library come from public CDNs. Everything else is inline, so the page needs no server of its own and works as a local file or as a hosted artifact. Figure 9.1 shows its structure.

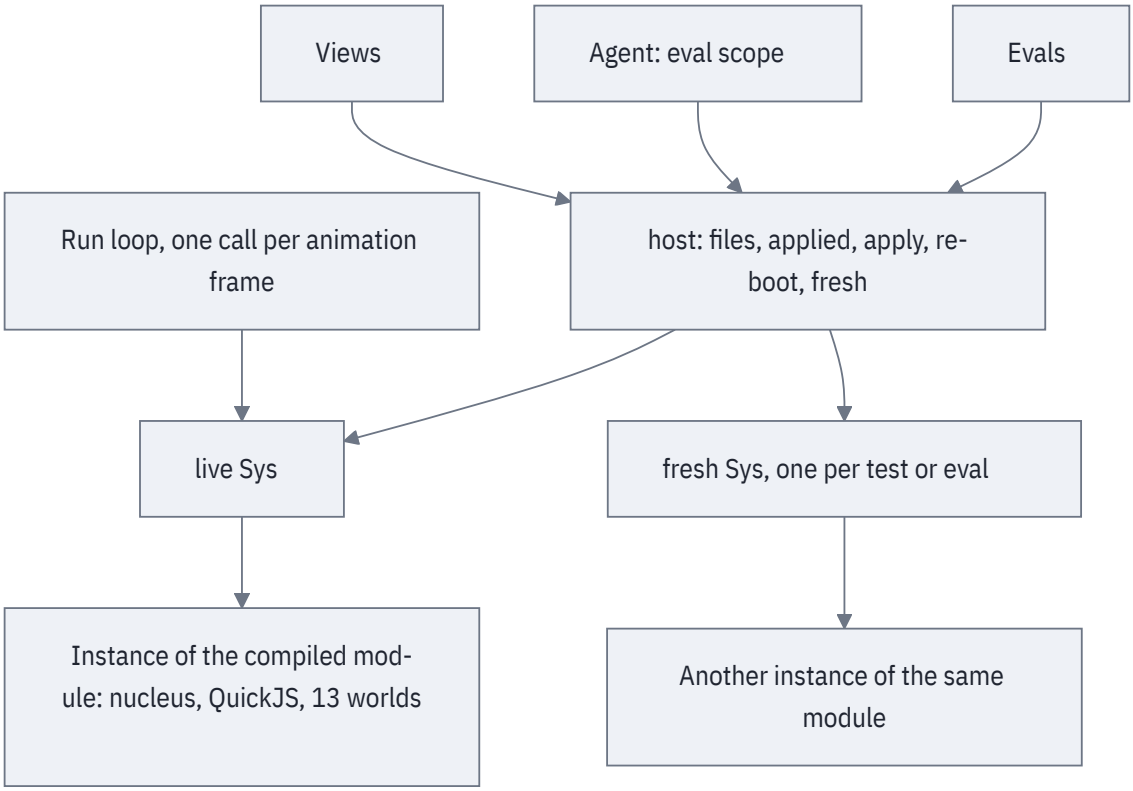


Figure 9.1. The workbench's structure. One compiled module is instantiated once for the live system and once more for every fresh system; the views, the agent's scope and the evals all reach the systems through the host, never directly.

The page compiles the module once (`WebAssembly.compile`) and keeps the compiled `WebAssembly.Module`. A system is an instance of it with its own linear memory, so instances share nothing: a crash, a heap or a virtual clock in one is invisible to the others. `core.js` wraps each instance in a `Sys`, a small class with a synchronous API:

▸ history/first-pass/workbench/src/core.js

```
  static async create(host, files, label) {
    const pc = await PicoCalc.load({ wasmModule: host.module, files });
    const s = new Sys(pc, label);
    try { pc.trace(true); } catch {}
    try { pc.boot(); s.booted = true; } catch (e) { s.booted = false; s.bootError = e.message; }
    return s;
  }
```

The API is synchronous because the module runs on the page's own thread. `wasm/picocalc.js` also offers a Web Worker form (`PicoCalc.spawn`) whose methods return promises, but the workbench does not use it: with the system on the main thread, `s.run(1000)` returns after a thousand virtual milliseconds have run, and the code that called it, whether a view, an eval or a step of the agent, can read the result on the next line. `Sys` adds what the views and the agent need on top of the loader: the UART lines and the trace collected as they arrive, the app in front (tracked from the compositor's `focus` messages), `#OCT1` fault frames parsed from the UART, a compact world table, and the text of a world's screen.

The live system. One system, `host.live`, is the one the person watches. A loop in `main.js` paces it to real time:

▸ workbench/src/main.js

```
function frame(now) {
  requestAnimationFrame(frame);
  const dt = Math.min(50, Math.max(0, now - last)); last = now;
  const s = host.live;
  if (!s || !s.booted || !s.pc) return;
  if (host.running && dt > 0 && !host.evalBusyBlock) {
    try { s.pc.run(dt * host.speed); } catch (e) { console.error(e); host.running = false;
        host.toast("The system stopped: " + e.message, true); }
    host.drawNow();
  }
```

Each animation frame runs the system for the real time that passed since the last one, scaled by the speed (0.25× to 4×), and draws the panel's memory into a canvas. The 50 ms cap keeps a page returning from a background tab from running minutes of simulation in one frame; the virtual clock then falls behind real time, which is harmless because nothing in the system reads real time. Measured in headless Chrome 153, the live system advanced 4,983 and 5,016 virtual milliseconds in two windows of 5,000 real milliseconds. The header's controls pause the loop, step 100 ms and reboot.

Files and edits. The host keeps three sets of files. `base` is what the page was built with, `files` is what the person and the agent have edited (saved in the browser's `localStorage` on every change), and `applied` is what the live system is running. A file whose `files` and `applied` versions differ is *unapplied*; the Code view marks it and the header offers to apply all of them.

Apply: hot reload or reboot. `host.apply` decides, per edit, how much of the system must start again:

▸ history/first-pass/workbench/src/core.js

```
    const needsReboot = targets.some((p) => p === MANIFEST || p === PRELUDE ||
        !this.worldsForFile(p).length) || !this.live || !this.live.booted || typeof this.live.pc.reload
        !== "function";
    if (needsReboot) { const r = await this.reboot(); return Object.assign({ rebooted: true }, r); }
    const reloaded = [], errors = [];
    for (const p of targets) {
      this.live.pc.addFile(p, this.files[p]);
      for (const w of this.worldsForFile(p)) {
        const n0 = this.live.stderr.length;
        const ok = this.live.pc.reload(w);
        if (ok) reloaded.push(w); else errors.push(`${w}: ${this.live.stderr.slice(n0).join(" ") ||
            "reload failed"}`);
      }
      this.applied[p] = this.files[p];
    }
```

Three kinds of edit need a reboot. The manifest is read once, by `kernel_boot`, and every world's authority, quotas and the ask graph's acyclicity are checked against it there (Appendix A); there is no way to change it under running worlds without redoing those checks. The prelude is evaluated in every world when the world starts, so a change to it is a change to all thirteen. A file that no world in the applied manifest runs can only start running through a manifest change. Every other edit is a world's own file, and only the worlds that run it start again. The reload goes through `pc_reload` to `reload_world` in `host/script.c`, which reads the file from the table, and then to the nucleus:

▸ history/first-pass/nucleus/kernel.c

```
int kernel_reload_world(world_t *W, char *source) {
  if (!source) return -1;
  free(W->source); W->source = source;
  W->config_error = false;
  W->n_crash_times = 0;
  if (W->status == W_IDLE) {
    char t[160]; snprintf(t, sizeof t, "%s was reloaded", W->name);
    teardown(W, t);
  }
  klog("%s reloaded from %s", W->name, W->file);
  trace_note("reload", kernel_now(), W->name, W->file);
  W->status = W_RESTARTING;
  W->boot_ready = true; W->boot_t = K.vnow; W->boot_seq = ++K.seq;
  publish_status(W, "reloading", NULL, 0);
  return 0;
}
```

A reload is a restart that is not a crash. `teardown` drops the old instance and its handles exactly as a crash would, but the crash count is cleared, so a world that had been stopped after five crashes can run again, and the world is marked ready to start at once instead of after a backoff. Its mailbox is kept, so messages sent to it during the reload are delivered to the new instance. Exercised from Node on Snake at 1,500 ms, with its tick changed from 110 to 90 ms, the reload took no virtual time, the world table showed `restarts: 1, crashes: 0`, and Snake had handled three more turns 200 ms later. A reboot instead costs one new `Sys`, and creating one (instantiating the module, adding the files and booting) took 6.9 to 8.2 ms of real time in headless Chrome.

`apply` reports `ok: true` when every reload *started*, and that is weaker than it looks. Given a Snake file with a syntax error, `pc.reload("snake")` returned `true`; the world then failed when the new source was evaluated, and the UART showed `snake crashed: SyntaxError: expecting ';'` with an `#OCT1` frame giving `apps/snake.js:73:13`, followed by the supervisor's restarts with backoff. A broken file is reported as a crash of the world that runs it, the same way the native build reports it at boot. Whoever applies an edit, a person or the agent, has to look at the world afterwards, and Section 10.8 returns to what that means for the agent.

Fresh systems. `host.fresh(opts)` builds a second, separate system from the files as they are now, edits included, optionally with extra files and a change to the manifest:

▸ history/first-pass/workbench/src/core.js

```
  async fresh(o = {}) {
    const files = this.osFiles();
    if (o.files) for (const [p, t] of Object.entries(o.files)) files[p.startsWith("js/") ? p : "js/" +
        p] = t;
    if (o.manifest) {
      const m = JSON.parse(files[MANIFEST]);
      const r = typeof o.manifest === "function" ? (o.manifest(m) || m) : Object.assign(m, o.manifest);
      files[MANIFEST] = JSON.stringify(r, null, 1);
    }
    return Sys.create(this, files, "fresh");
  },
```

A fresh system starts at virtual time zero with nothing in its history: no earlier crashes, no keys pressed, no edits applied on top of other edits. Because the system is deterministic, the same files and the same inputs always give the same result, so a difference between a fresh system before an edit and one after it is caused by the edit. That is the basis of the agent's `test` function and of every eval (Chapter 11). The cost is small: creating and booting one takes about 7 ms of real time, the nineteen evals, each on its own fresh system, ran in 1.0 to 1.5 s in the runs made for this book (Section 11.2), and a fresh system is released by dropping the last reference to it, leaving its memory to the garbage collector.

Frames as text. The system's screen is 320 × 320 pixels, 409,600 bytes of RGBA, which a program cannot read easily and a person reads only by looking. Each world, however, describes its screen as a tree of user-interface nodes that its `screen()` function returns (*Small Worlds*, Chapter 13 and Appendix A), and the nucleus keeps the last tree each world published. `pc_frame` returns it as JSON; `treeText` in `core.js` walks it and keeps the words: titles and text, list and grid items with a `>` before the selected one, table rows, key-value pairs, and the hint bar. The launcher's frame is 1,093 bytes of JSON, and its text is this:

```
Apps
> System
  REPL
  Snake
  Faults
  Keys
  About
Every world, its memory, its turns and the buses, live.
13 of 13 worlds running · 0 crashes since boot
[hints] ←→↑↓ choose  ⏎ open  or press its letter
```

The text is what the app asked to show, before the compositor laid it out and the display driver drew it. It cannot show a layout bug or a wrong colour, but it shows state exactly and in a form an eval can match with a regular expression, which is why the evals and the agent read screens this way. The Worlds view shows the same text for the selected world.

This section has described the host as four mechanisms: a synchronous system object over each instance, a live system paced to real time, an apply step that restarts as little as the edit allows, and fresh systems that make before-and-after comparisons exact. The views are windows onto these.

9.4 The views

The workbench has eleven views, each a React component that reads the host and redraws when it changes. Table 9.3 lists them with the source of what each shows.

Table 9.3. The workbench's views and where their data comes from.

View	What it shows	Source
Device	The panel's pixel memory, on-screen keys, a box to type text, the app in front, the worlds with a kill button each, the battery	`pc.screen()`, `pc.key`, `pc.crash`, `pc.battery`
Code	The OS's files grouped as system, drivers, services, apps and workbench, an editor with highlighting, and Apply	`host.files`, `host.apply`
Agent	Threads, the mode and model settings, steps with their code and results, the answer	Chapter 10
Evals	The suite, a result per eval with virtual and real time, a history of runs	`host.runEvals` (Chapter 11)
Worlds	The nucleus's world table, live; for the selected world its counters, quotas and their use, its manifest entry and the text of its screen	`pc.stats()`, `pc.frame`
Messages	The trace: messages, segments, bus transfers and faults, filterable by world or type, in virtual ms	`pc.onTrace`
UART	The console line with `#OCT1` fault frames, and the boot log when the boot failed	channel 1
Handles	Every live handle: type, claim, rights, holders, parent, revoked	`pc.stats().handles`
Graph	The authority the applied manifest grants: sends, asks and channels between drivers, services and apps	the manifest and `pc.stats()`
Reference	How to use the workbench, PicoCalc OS in one page, the world API, the manifest schema, the agent's scope	`workbench/src/reference.md`
Book	This book, with search and live links into the workbench	`l4book/out/wb-book.json`

Views live in a dock of tiles. A tile has a strip of tabs, one per view it holds, and tiles sit in splits whose gutters can be dragged. Four layouts are built in: the agent beside the device (the default), code beside the device, the evals and the trace, and everything in one tile. Figure 9.2 shows the default layout just after boot. The tabs carry counts when something needs attention: the number of failing evals on Evals, of unapplied files on Code, of stopped

or restarting worlds on Worlds. The header shows the virtual clock with its controls, a button to apply pending edits, and three figures that link to their views: worlds up, evals passing and the lane's occupancy.

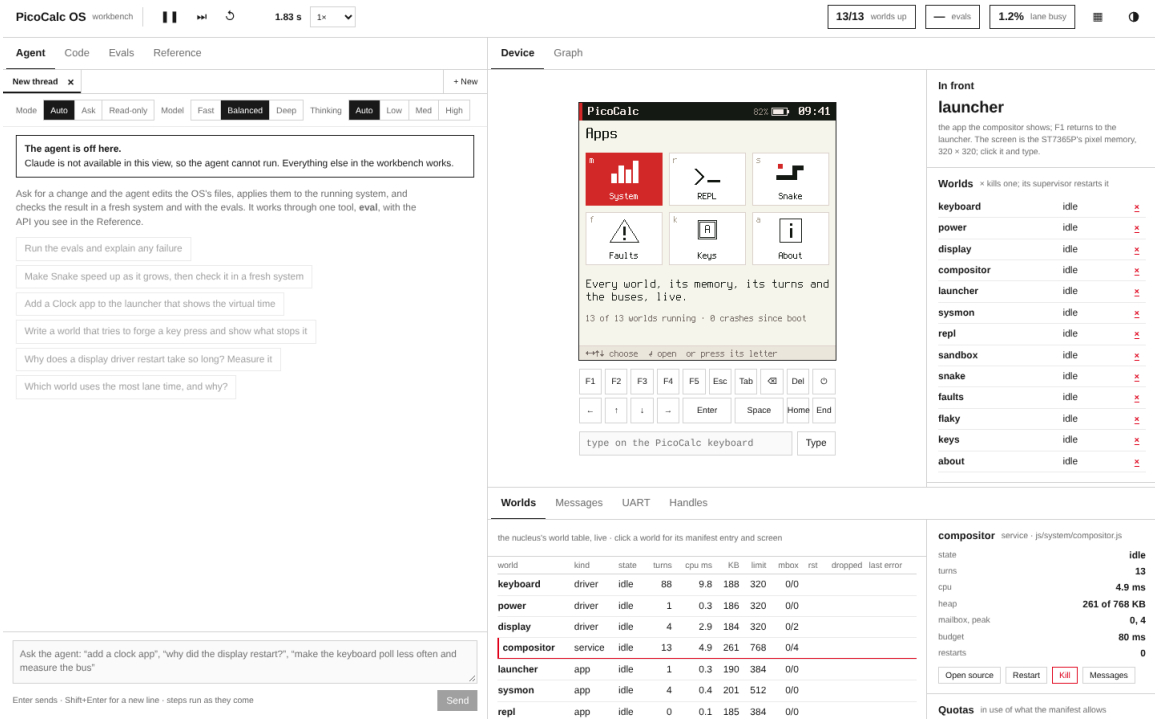
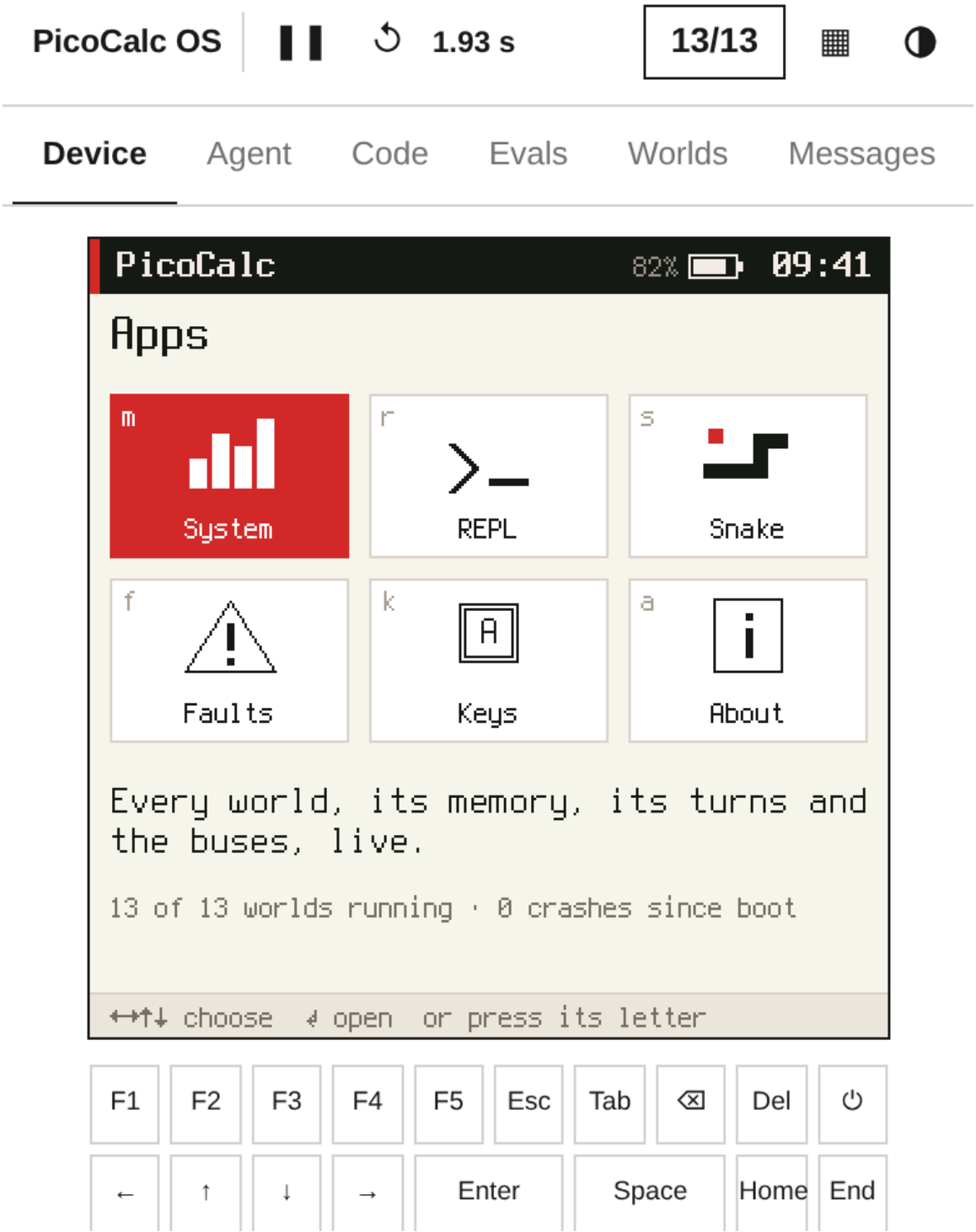


Figure 9.2. The workbench in its default layout, 1.83 virtual seconds after boot: the agent panel on the left (switched off here because the page has no model available), the device with the launcher in front, and the nucleus's world table below, with the compositor selected and its counters on the right. The capture predates the Book view, whose tab now follows Reference in the left tile.

A tile narrower than 640 pixels moves a view's side column below its main content, and on a window narrower than 640 pixels the dock becomes a single tile holding every view and the header drops its optional figures (Figure 9.3). Everything works the same way; only the arrangement changes.


PicoCalc OS
1.93 s
13/13
Device
Agent
Code
Evals
Worlds
Messages
PicoCalc
82%
09:41
Apps
m
System
r
REPL
s
Snake
f
Faults
k
Keys
a
About
Every world, its memory, its turns and the buses, live.
13 of 13 worlds running · 0 crashes since boot
↔↑↓ choose ↵ open or press its letter
F1
F2
F3
F4
F5
Esc
Tab
Del
←
↑
↓
→
Enter
Space
Home
End

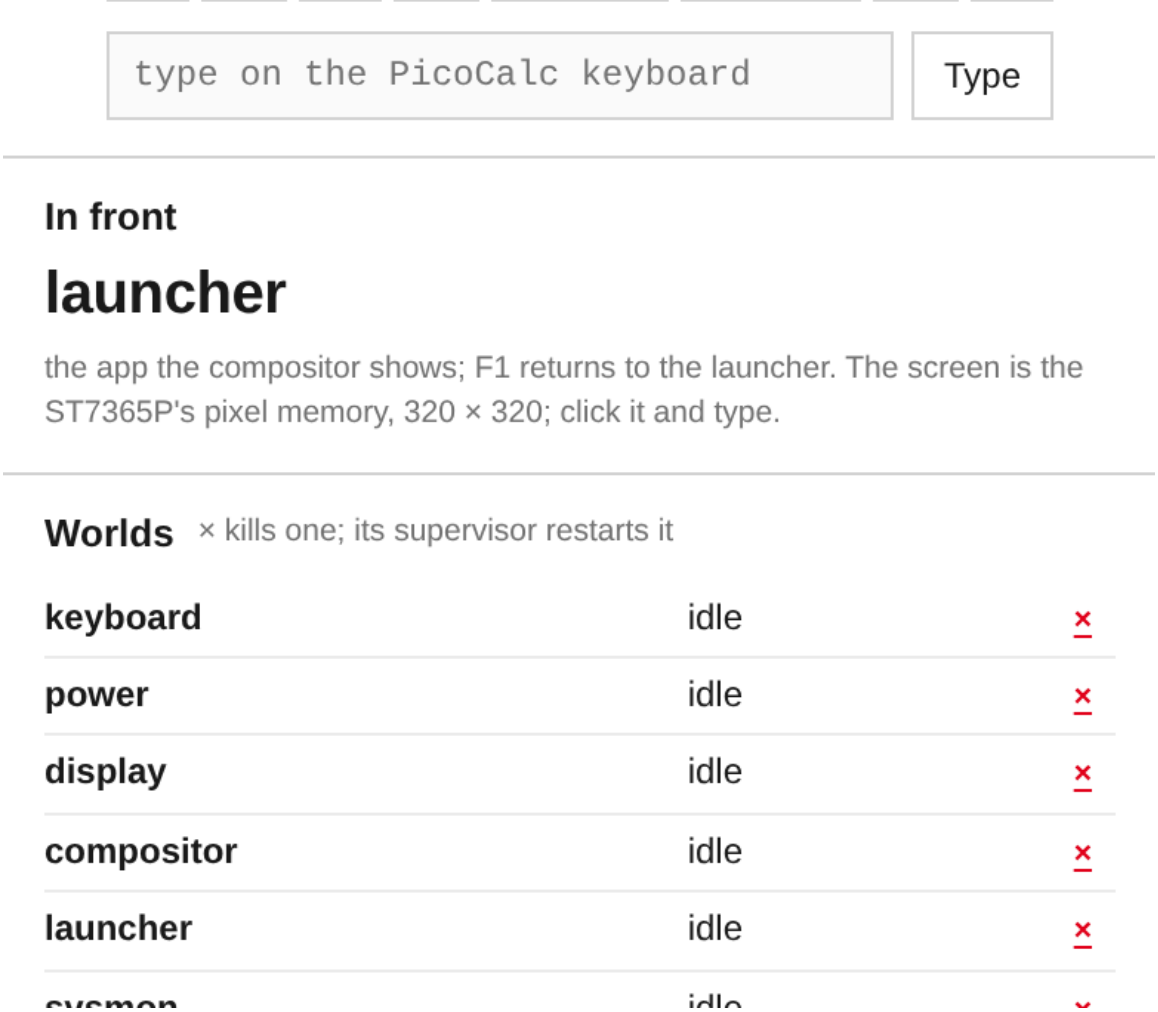


Figure 9.3. The workbench on a phone-sized window, 390 pixels wide: one tile with every view as a tab, and the device's side column below the screen.

Several views are built from fields the first pass added. The Worlds view's side column lists each quota with its use (`handles 2 of 6`, `asks waiting 0 of 8`, the mailbox in messages and kilobytes, framebuffers, the turn limit) and the posts dropped because the world's mailbox was full, which is how Chapter 7's quotas become visible while the system runs. The Graph view draws every `sends`, `asks` and channel edge of the applied manifest, with stopped worlds in red; Chapter 5 uses it to show what a world may and may not reach.

The Book view shows this book. `l4book/build.py` pre-renders every chapter into `l4book/out/wb-book.json`, and `workbench/build.py` embeds it in the page when it exists. The view adds what a page beside the system can do and a printed book cannot:

▸ workbench/src/views.js

```
// ---- Book: the companion volume, pre-rendered by l4book/build.py. A code excerpt's label opens the
    file in Code (an
// OS file) or shows the lines around the excerpt (a nucleus or workbench source); citations show their
    reference.
```

An excerpt such as the `kernel_reload_world` listing above is labelled with its file and line. For an OS file (under `js/`) the label opens the file in the Code view at that line, as the person has edited it; for a nucleus or workbench source it unfolds the surrounding lines of the quoted file, which the page carries for the purpose. Citations open their reference in a card, and a search box finds sections. The agent reads the same book through its scope (Section 10.3).

Apart from the Reference and the Book, the views do not compute anything the host does not already hold. Each is a reading of the system's own tables, its trace and its files, so what a view shows is what the nucleus would report to a world with the `sys.read` claim, at the same virtual time.

9.5 The design

The workbench follows two internal design systems, Form Skate Workbench for the shell and the views and Form Skate Agent for the agent panel, in the look they call Kiln. The stylesheet states the look in its first lines and in a dozen variables:

▸ workbench/src/style.css

```
/* PicoCalc OS workbench: the Kiln look from the Form Skate Workbench and Form Skate Agent design
    systems
   (Swiss: white ground, one ink, Arial in two weights, 1 px rules, red only for trouble and
       selection). */
:root {
  --bg: #ffffff; --fill: #f6f6f6; --fill-2: #e6e6e6; --ink: #1a1a1a; --ink-2: #737373; --accent:
      #e2001a;
  --hair: #d6d6d6; --rule: #ededed; --code-bg: #fbfbfb; --sel: rgba(226, 0, 26, 0.10); --on-accent:
      #ffffff;
```

The style is Swiss in the typographic sense: a white ground, one near-black ink with a grey for secondary text, Arial in two weights (400 and 700) for everything except code, structure drawn with 1-pixel rules rather than boxes and shadows, square corners throughout (every control sets `border-radius: 0`), and a single colour, red, reserved for two meanings. Red marks trouble: a stopped or restarting world, a failing eval and the word `REGRESSED`, a fault line on the UART, a crash in the trace, a failed step of the agent, a toast reporting an error. Red also marks selection and attention: the selected world's row and node, the open file in the file list, the editor's caret, the underline of a `wb:` link, and a step of the agent waiting for the person's approval. The code highlighter, which otherwise uses only the ink and two greys, also shows the arrow of a function expression in red. A dark theme swaps the ground and the ink and keeps the same rules.

The reason is the content. Most of the workbench is tables of numbers, logs and code, and most of the time nothing is wrong. In a display where colour carries only one bit, the bit is noticed: a red figure in the header or a red row in the world table means something needs attention, and nothing else on the screen competes with it. The PicoCalc's own screen, drawn by the compositor, uses its own palette (its launcher selection is also red), and the workbench frames it without adding to it.

9.6 Limits

The workbench is a faithful host for the system, but it is a page, and that brings four limits.

Edits live in the browser. Edited files are saved in the page's `localStorage` and nowhere else. The page cannot write to the repository; the layout menu's "Copy edited files" puts every edited file on the clipboard as one JSON object, which is the way an edit leaves the page. Clearing the site's data loses the edits, and two browsers hold two different sets.

WebAssembly may be blocked. A page's Content Security Policy can forbid compiling WebAssembly: under CSP Level 3, a policy whose `script-src` list (or, without one, its `default-src` list) contains neither `'unsafe-eval'` nor `'wasm-unsafe-eval'` blocks compilation [West and Sartori 2026]. The loader detects the refusal and rejects with the error code `wasm_blocked`, and the workbench then shows one line saying that it needs WebAssembly and suggesting opening the page on its own. Everything in this chapter depends on the module, so a page that cannot compile it has no workbench, only that message.

Simulation costs real time, on the page's thread. The cost is small. In headless Chrome, ten virtual seconds of the idle home screen ran in 19.4 and 22.5 ms of real time, about 0.2 per cent; under Node the six scripts above, 78,950 virtual milliseconds of a busier system, ran in 793 ms of real time once the module was warm, about 1 per cent. For comparison, the six scripts took 730 ms as six separate processes of the native 32-bit build and 927 ms in one Node process for the WebAssembly build, start-up included. Paced to real time, the live system therefore costs a few per cent of one core at most. Because it runs on the page's thread, however, a long synchronous call blocks the page. A virtual minute of Snake on autopilot takes about half a second to run (`snake.txt` ran 31,680 virtual milliseconds in 256 to 337 ms), and during a call like `s.run(60000)` the page cannot redraw or take input, and the agent's steps run on the same thread (Section 10.8).

One address space. All the systems in a page share one browser process and are separated from each other by WebAssembly's linear memories, and the worlds inside each system are separated by QuickJS, not by hardware. That is the same isolation the device will have, and *Small Worlds*, Chapters 10 and 17, discuss what it does and does not protect against. The workbench adds no isolation of its own between its code and the systems it hosts: its code, the evals and the agent's steps run in the page with the page's authority.

KEY POINTS

- The WebAssembly build is a WASI reactor: it has no `main`, the page calls `_initialize` once and then eighteen exports, and each call runs the system for as long as it was asked and returns.
- Files come from a table in linear memory filled through `pc_add_file`, and the UART and trace leave through `fopencookie` streams that call an imported function, so the nucleus's own code is unchanged.

- The module imports thirteen functions (eleven WASI, two of its own) and is 1,207,039 bytes; one running system occupies about 5 MiB of linear memory.
- The WebAssembly build matches the native 32-bit build on 21 of 22 captures and on every turn count, CPU time and virtual time, except where a screen prints a heap figure, because heap use is measured with the C library's `malloc_usable_size`.
- The host keeps base, edited and applied files, paces one live system to real time, and builds fresh systems from the files as they are now for tests and evals, in about 7 ms each.
- Apply hot-reloads only the worlds whose files changed and reboots for the manifest, the prelude or a file no world runs; a reload clears the crash count and keeps the mailbox, and a broken file shows up as a crash of its world, not as a failed apply.
- A world's screen can be read as text from the tree it published, which shows state exactly but not layout or colour.
- The Book view carries this book, pre-rendered by `l4book/build.py`; an excerpt's label opens the quoted OS file in Code or unfolds the lines around it, and the agent reads the same text through `book`.
- The design reserves its one colour, red, for trouble and selection, so that in a screen of numbers what needs attention stands out.

The workbench gives a person the running system, its code and its tables in one place, with an apply step measured in milliseconds and a fresh system for every check. The same host functions, exposed to a model as a single tool, make the agent of Chapter 10: it edits the files the Code view edits, applies them with `host.apply`, reads the tables the views read, and tests on the fresh systems described here.

EXERCISES

1. `BASIC` List the three kinds of edit for which `host.apply` reboots the live system instead of reloading worlds, and give for each the reason a reload could not apply it.

2. BASIC The WebAssembly build is linked as a reactor. Explain what would go wrong if it were a command whose `main` called `kernel_run_until` in a loop, in a page and in a Web Worker.

3. ANALYSIS `03-sysmon` prints every world's heap in its table, yet it is identical in the native 32-bit and WebAssembly builds, while Snake's heap differs by 3.8 KB. Run `scripts/tour.txt` to the `03-sysmon` capture in both builds, look at the capture and at `pc.frame("sysmon")`, and explain why. Which change to the system monitor would make the two captures differ?

4. ANALYSIS The run loop caps each frame's step at 50 ms of real time. A person switches to another tab for ten seconds while Snake runs on autopilot. Describe what the virtual clock and the screen do while the tab is hidden and when it returns, and what would happen without the cap. Does the cap affect any result an eval could observe?

5. CODE Reproduce Table 9.2: run the six scripts with `./build32/picocalc-host -o DIR` and through the WebAssembly build in Node (a short runner that loads `wasm/picocalc.js`, creates one system per script, passes each line to `pc.command` and saves each capture from `pc.onCapture`), and compare the captures pixel by pixel. Then append `stats` to each script and compare the printed tables line by line. Report every difference and its cause.

6. CODE Write a Node script that compiles `wasm/picocalc.wasm` once, then creates, boots and runs for one virtual second twenty fresh systems in turn. Report the real time per system and the linear memory of each, and estimate how many fresh systems a page could hold at once before memory, not time, becomes the limit.

7. CODE Change `host.apply` in `workbench/src/core.js` so that after a hot reload it runs the live system for 200 virtual milliseconds and returns, for each reloaded world, any `#OCT1` fault frames it produced. Test it with a Snake file containing a syntax error. What does this change cost, and why might a person prefer the original behaviour?

8. PROJECT Move the live system into a Web Worker with `PicoCalc.spawn`. Decide what happens to `Sys`'s synchronous API, to the views that read `host.live` on every render, and to the agent's steps, which expect `run(ms)` to have finished when it returns. Measure the page's responsiveness during `run(60000)` before and after.

9. DISCUSSION The WebAssembly build carries its own copy of glibc's `rand` so that its pixels match the native build's (*Small Worlds*, Chapter 17). Would a workbench that matched the native build only approximately, say to within a few pixels per capture, be nearly as useful? Consider the `deterministic` eval, before-and-after comparisons, and results that are later quoted from `bench/run.py`.

FURTHER READING

[Small Worlds 2026] Chapters 16 and 17 describe the browser as an operating system, WebAssembly as an executable format, and the first port of PicoCalc OS to the page; this chapter starts where they stop.

[WASI ABI 2026] The short document that defines commands and reactors, and the order in which an environment must call a reactor's exports.

[WebAssembly Core 2026] The WebAssembly core specification, for the exact meaning of a module's imports, exports and linear memory.

[wasi-sdk 2026] The toolchain that builds the module: Clang with the wasm32 targets, wasi-libc and the builtins, in one release.

[West and Sartori 2026] The Content Security Policy rules that decide whether a page may compile WebAssembly at all.

[First pass 2026] The first pass's account of the WebAssembly API additions (`pc_reload` , `pc_frame` , the richer `pc_stats`) and of the identity checks summarised here.

An agent with one tool

The workbench has a coding agent. A person types a request in the Agent view ("make Snake speed up as it grows, then check it in a fresh system"), and a language model, Claude, reached through the artifact platform's `sample` capability, works on the request by running code in the page: it reads and edits the OS's files, applies the edits to the live system, drives the device, reads the kernel's tables, tests on fresh systems and runs the evals. Then it answers in a few paragraphs, and every step it took is shown, with its code and its result, above the answer.

The agent has exactly one tool, called `eval`. Its argument is the body of an asynchronous JavaScript function, run in the page with a curated set of names in scope, and its result is the function's return value written as compact YAML. This chapter explains why the agent was given one general tool over a carefully designed API rather than many narrow ones, follows a turn from the person's message through the capability, the tool and the system and back, and describes what keeps the person in control: three modes, a snapshot of the files before every step, and a revert. By the end the reader should be able to explain every part of the agent's input, predict what a step returns, and say what the agent can and cannot see or do.

The agent has been exercised end to end with a stand-in for Claude, a scripted `sample` function that issues fixed steps, in the headless test described in Section 10.7. It has not yet been run with Claude. Everything this chapter says about the page's side is measured; nothing it says is a claim about how well a model uses the tool.

10.1 One general tool or many narrow ones

An agent, in the sense used here, is a language model that decides for itself which actions to take and in what order, observing the result of each before choosing the next. Yao and colleagues' ReAct made the pattern explicit: the model interleaves reasoning with actions, and the actions gather information from an external environment that the reasoning then uses [Yao et al. 2023]. Anthropic's guide to building agents draws the line the same way, distinguishing workflows, in which models and tools are "orchestrated through predefined code paths", from agents, in which models "dynamically direct their own processes and tool usage", and it adds that an agent must gain "ground truth" from its environment at each step [Schluntz and Zhang 2024]. Every agent therefore has an interface to its environment, and the design of that interface turns out to matter as much as the model.

There are two broad ways to build it. The first gives the model many narrow tools, each with typed arguments: `read_file(path)`, `edit_file(path, old, new)`, `run_tests()`, `list_worlds()`. Each call does one thing, its arguments can be validated, and its result has a fixed shape. The second gives the model one tool whose argument is a program. Wang and colleagues compared the two directly in CodeAct: agents that act by writing executable code, run by an interpreter, against agents that emit JSON or text in a fixed format, across 17 models. Code actions reached success rates up to 20 per cent higher, because a program can call several functions, combine their results with loops and conditions, and keep intermediate values, all in one action [Wang et al. 2024].

The other line of evidence is about the shape of the interface. Yang and colleagues built SWE-agent around an *agent-computer interface*, an interface designed for a model the way a user interface is designed for a person, and showed that its design changed how well agents fixed real bugs in real repositories [Yang et al. 2024]. Their interface follows four principles: actions should be simple and easy to understand, actions should be compact and efficient, environment feedback should be informative but concise, and guardrails should limit how far an error propagates. Concretely, its file viewer shows at most 100 lines at a time, its searches return at most 50 results, and an edit that a linter rejects is discarded and the agent asked to try again. Anthropic reports the same lesson from building an agent for the SWE-bench coding benchmark, during which they "spent more time optimizing [their] tools than the overall prompt", and they advise changing a tool's arguments so that mistakes are harder to make, which they call poka-yoke [Schluntz and Zhang 2024]. A later guide adds that more tools do not always give better results, that tools should consolidate

work rather than wrap every function one to one, that results should be filtered, paginated or truncated with sensible defaults, and that an error message should say specifically how to do better [Aizawa 2025].

The workbench's agent takes both lessons at once. It has one tool whose action is code, as CodeAct recommends, and the code runs against a scope designed as an agent-computer interface, with SWE-agent's and Anthropic's rules applied to each function: compact results, numbered line ranges, capped searches, edits that refuse to guess, and errors that say what to do next. Three properties of the workbench made this choice natural.

First, the API already existed. The host of Chapter 9 is a JavaScript object with synchronous methods over the live system, the files and fresh systems. A narrow tool for each method would duplicate that API in JSON Schema; a code tool exposes it as it is.

Second, useful steps compose. Checking a change means editing a file, applying it and reading the world it affected; measuring something means pressing keys, running time and reading the trace. In code each of these is one step:

```
files.edit("js/apps/snake.js", ".every(110,", ".every(90,");
return apply("js/apps/snake.js");
```

With narrow tools the same work takes two or three round trips through the model, each paying for the whole conversation again.

Third, a step written as code is legible to the person. The Agent view shows each step's code above its result, so the person reads exactly what was done, in the same language as the system, and can copy it into a test.

The cost of the choice is that a code tool is as powerful as the code's environment. Validation moves from the tool's schema into the functions of the scope, and confinement depends on what the code can reach. Section 10.6 returns to the second point, because in this page the code can reach more than its scope.

10.2 The loop belongs to the capability

The page reaches the model through the artifact platform's `sample` capability, which the page obtains with `window.claude.use("sample")`. The capability is a function, `sample(turns, options)`, that takes a conversation (a list of user and assistant turns) and options, and resolves with the model's answer, `{ text, truncated }`. Its most important option is `tools`, a list of tool objects. The workbench passes one:

▸ history/first-pass/workbench/src/agent.js

```
    const evalTool = {
      name: "eval",
      description: "Run an async JavaScript function body in the PicoCalc OS workbench with the
          workbench API in scope (files, apply, reboot, run, key, type, crash, worlds, world, handles,
          frame, text, screen, uart, faults, messages, test, evals, …). Returns the value as YAML plus
          console.log output. Use `return` to return a value.",
      inputSchema: { type: "object", properties: { code: { type: "string", description: "the body of an
          async function; return what you want to see" } }, required: ["code"] },
      execute: async ({ code }, c) => {
…
        const r = await runEval(code, readOnly, (c && c.signal) || ctl.current.signal);
        step.running = false; step.result = r.text; step.error = r.error; step.ms = r.ms;
…
        if (r.error) throw new Error(r.text);
        return r.text;
      },
    };
```

A tool here is a name, a description the model reads, a JSON Schema for its input, and an `execute` function the page supplies. The page does not run the agent loop itself. It makes one call per message from the person:

▸ workbench/src/agent.js

```
      const opts = { signal: ctl.current.signal, modelTier: A.settings.tier, tools: [evalTool], cache:
          false, onText: ({ text }) => { turn.text = text; re(); } };
      if (A.settings.effort) opts.effort = A.settings.effort;
      if (A.settings.screen && canImg) { const b = await screenBlob(); if (b) opts.images = [b]; }
      const res = await sample(toTurns(th), opts);
```

Between that call and its result, the capability runs the loop. Each time the model asks for the tool, the capability calls `execute` with the model's arguments and gives the model what `execute` returns. A step that fails throws an error carrying the same text, so that the capability can report the call to the model as failed rather than as an ordinary result. The model may take as many steps as it needs, and when it answers without asking for the tool, `sample` resolves with the answer. While the answer is being written, `onText` receives the text so far, and the Agent view shows it as it grows. The other options come from the panel's settings: `modelTier` (Fast, Balanced or Deep), `effort` (how much the model thinks before answering), `images` (a picture of the screen, Section 10.8) and `signal`, an `AbortController`'s signal that the Stop button fires.

Figure 10.1 shows one turn. The page's code has no loop over model responses, no parsing of tool calls and no conversation state between steps; it has a function that the capability calls and a promise that resolves with the answer. That keeps the page small, and it leaves the conversation format, and any limit on the number of steps, to the capability.

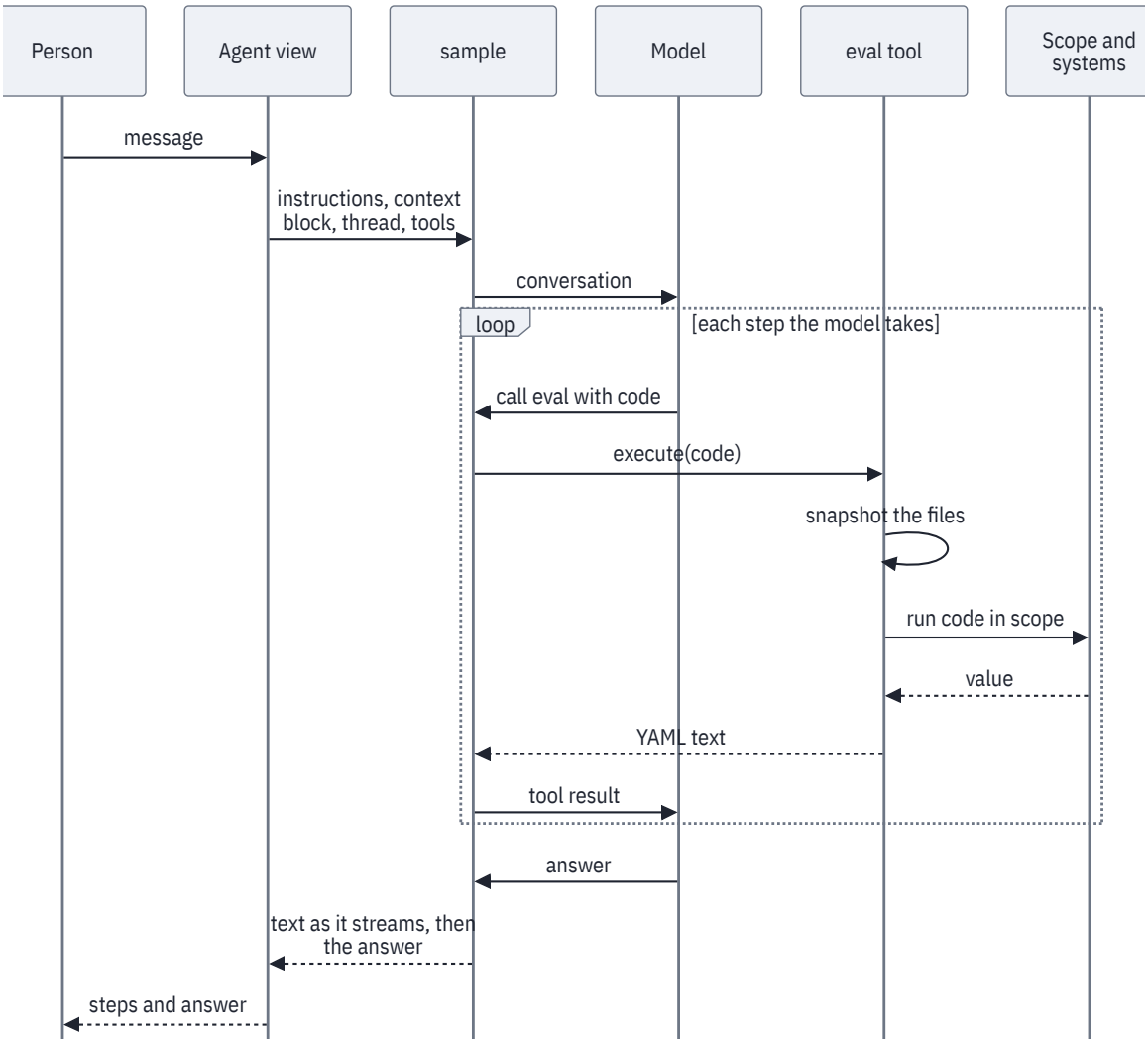


Figure 10.1. One turn of the agent. The page makes a single sample() call; the capability calls the eval tool once per step the model takes, and each step runs code against the scope, which reaches the live system or a fresh one and returns YAML.

When the capability is not available, the page says so instead of failing silently. `sample` rejects with an error code, and the Agent view maps each code to a sentence: `not_granted` ("Claude is not allowed for this page"), `tools_unavailable` ("This view cannot run the agent's tool"), `rate_limited`, `prompt_too_large` ("The conversation is too long. Start a new

thread."), `refused`, `cancelled`. When `window.claude` does not exist at all, as in a page opened from a file, the panel shows "The agent is off here" and the rest of the workbench works as before (Figure 9.2 shows that state).

10.3 The scope

The scope is the agent's whole vocabulary: the names its code can use without reaching outside. `agentScope` builds it afresh for every step, so names a step declares with `const` do not survive into the next one; state that must persist lives in the files or in the system. The scope has seven groups, all documented in the Reference that the model reads (Appendix B has the complete list).

Files. `files.list()`, `files.read(path, from, to)`, `files.grep(pattern, path)`, `files.edit(path, find, replace)`, `files.write`, `files.remove`, `files.diff(path)` and `manifest()`. Reading follows SWE-agent's viewer: with a line range, `read` returns numbered lines, 40 by default, and `grep` returns at most 80 hits as `file:line text`. Editing is the most constrained function in the scope:

▸ workbench/src/agent.js

```
    edit: g("files.edit", (path, find, replace) => {
      const p = needFile(path), t = host.files[p];
      const n = t.split(find).length - 1;
      if (!find || n === 0) throw new Error(`edit: text not found in ${p} (it must match exactly,
          including spaces)`);
      if (n > 1) throw new Error(`edit: text occurs ${n} times in ${p}; add context until it is
          unique`);
      const at = t.indexOf(find);
      host.files[p] = t.slice(0, at) + replace + t.slice(at + find.length);
      host.saveEdits(); host.emit();
      return { path: p, line: t.slice(0, at).split("\n").length, applied: !host.isUnapplied(p) };
    }),
```

An edit names the exact text it replaces, and that text must occur exactly once. A model that misremembers a line, or quotes one that appears twice, gets an error that says which of the two happened and what to do about it, and the file is unchanged. Edits by line number, the obvious alternative, fail silently when the file has changed since the model last read it; an exact, unique match cannot. This is poka-yoke in Anthropic's sense: the argu-

ment is shaped so that the common mistake is refused instead of applied. The result reports the line where the edit landed and whether the file is applied, which for an OS file it is not until `apply` runs, because applying is a separate, deliberate act.

Applying. `apply(path)` and `reboot()` call the host's functions of Section 9.3, so the agent's edits take effect exactly as a person's do: a world's file hot-reloads that world; the manifest, the prelude or a new file reboots.

The live system. `run(ms)`, `key(name)`, `type(text)`, `crash(world)`, `restart(world)` and `battery(pct)` drive it; `worlds()`, `world(name)`, `handles()`, `stats()`, `frame(world)`, `text(world)`, `screen()`, `uart()`, `faults()` and `messages({ world, type, kind, last })` read it. The readers return compact forms by design. `worlds()` drops fields the model rarely needs; `stats()` reduces the kernel's counters to virtual time, lane occupancy, turns, posts, crashes, handles and per-bus figures; `messages` keeps the last 40 trace records unless told otherwise; `text(world)` returns the world's screen as the text of Section 9.3, and `screen()` returns the app in front, its text, a hash of the panel's pixels and the fraction of lit pixels.

Fresh systems. `test({ script, ms, files, manifest, then })` boots a new system from the files as they are now, with extra files or a manifest change if given, runs script commands (`wait 500`, `key right`, `type 1+2`, `crash display`) and `ms` more milliseconds, calls `then(s)` with the system, and returns a summary:

▸ workbench/src/agent.js

```
  test: async (o = {}) => {
    if (typeof o === "function") o = { then: o };
    const s = await host.fresh(o);
    try {
      if (o.script && s.booted) for (const line of String(o.script).split("\n")) { const l =
          line.trim(); if (l && !l.startsWith("#")) s._cmd(l); }
      if (o.ms && s.booted) s.run(o.ms);
      let value;
      if (o.then && s.booted) value = await o.then(s);
      return Object.assign(summary(s), value !== undefined ? { value } : {});
    } finally { s.pc = null; }
  },
```

`test` exists because the live system is a poor place to check a change. It has a history: worlds the person killed, keys pressed, edits applied on top of other edits, minutes of virtual time. A fresh system has none, and because it is deterministic, a result obtained in one is reproducible and attributable to the files. The summary always includes whether the system booted, its boot errors if not, the worlds not running and their last errors, and recent faults, so a change that breaks something other than what was being tested shows up in the result without being asked for. A fresh system took 38 ms of real time for `test({ ms: 1000 })` in headless Chrome.

Evals. `evals.list()`, `evals.run(names)` and `evals.last()` run the suite of Chapter 11, each eval on its own fresh system, and return `{ pass, total, failing }` with the details of each failure.

The book. `book.list()`, `book.read(ref, from, length)`, `book.search(q, limit)` and `book.open(ref)` give the model this book, the same text the Book view shows:

▸ workbench/src/agent.js

```
    search: (q, limit = 12) => book.search(q, limit).map(({ ref, chapter, section, matches, snippet
        }) => ({ ref, chapter, section, matches, snippet })),
    open: (ref) => { if (!book.open(ref)) throw new Error(`the book has no ${ref}`); return "shown";
        },
```

The instructions ask the model to check the book when an answer rests on a design decision (why asks may not form a cycle, why quotas are fixed at boot) and to link the section with a `wb:book?ref=6.2` link, which opens it in the Book view.

The workbench. `open(path, line)`, `focus(view)` and `select(world)` move the person's views, so the agent can show what it is talking about.

Two choices recur across the groups. Every function that changes files or restarts the system is wrapped in `g(name, fn)`, which in Read-only mode replaces it with a function that throws `read-only mode: files.edit is off`. And every reader is designed to return a value small enough to read in full, or takes arguments that make it so, because the result of a step is the model's only view of the system.

10.4 Results as YAML

`runEval` turns the model's code into a function whose parameters are the scope's names, so that the code sees them as local variables:

▸ history/first-pass/workbench/src/agent.js

```
    const fn = new Function(...names, '"use strict";\nreturn (async () => {\n' + code + "\n})();\n//#
        sourceURL=agent-eval.js");
    value = await Promise.race([
      fn(...names.map((k) => scope[k])),
      new Promise((_, rej) => setTimeout(() => rej(new Error("eval timed out after 30 s")), 30000)),
      new Promise((_, rej) => signal && signal.addEventListener("abort", () => rej(new
          Error("stopped")), { once: true })),
    ]);
```

The code is wrapped in an asynchronous arrow function, so it may `await` a fresh system or the evals, and must `return` what it wants to see. A step that returns nothing gets `undefined (use return to see a value)`, which tells the model what it forgot. A thrown error comes back as one line with the line of the model's code where it happened, computed from the stack trace and the `sourceURL` comment: `Error: undefinedThing is not defined (line 2)`. Anything passed to `console.log` is collected and returned above the value.

The value is written as YAML by a forty-line function, `yaml`, tuned for reading rather than for parsing back. Numbers are rounded to five decimal places. Strings are written bare when they contain only safe characters and quoted otherwise, and multi-line strings become block scalars. Arrays of numbers are written on one line, up to 64 of them. Arrays of other values are cut after 60 items with a comment saying how many were left out (`# … 198 more`), objects after 80 keys, and nesting after five levels. A system object prints as `[System t=1.633 s]` and a byte array as `[bytes 409600]`, so returning a fresh system by mistake costs one line. The whole result is then cut at 9,000 characters, with a note that names the remedy: `Return less: pick fields, slice arrays.`

YAML was chosen for legibility at a size close to the most compact alternative. For the thirteen-world table of `worlds()`, the YAML is 1,610 characters, compact JSON 1,599 and indented JSON 2,380. The YAML is as short as JSON with no whitespace, but reads one field per line, without quotes or braces:

```
- name: keyboard
  kind: driver
  state: idle
  turns: 78
  cpu_ms: 8.736
  mem_kb: 188
  limit_kb: 320
  mbox: 0
  restarts: 0
```

The limits are guardrails in SWE-agent's sense. A model that asks for `messages({ last: 400, kind: "seg" })` gets sixty records and a count of the rest, which costs a few thousand characters of context, instead of every record. Anthropic's tool guide describes Claude Code's limit on a tool's response as 25,000 tokens by default [Aizawa 2025]; the workbench's 9,000 characters is far smaller because almost nothing in this system needs more, and the readers are designed so that the useful answer is short.

10.5 Instructions and the context block

The model starts each request knowing nothing about the workbench, so the page tells it. `toTurns` builds the conversation from the thread:

▸ workbench/src/agent.js

```
function toTurns(th) {
  const turns = [];
  const push = (role, content) => { const last = turns[turns.length - 1]; if (last && last.role ===
      role) last.content += "\n\n" + content; else turns.push({ role, content }); };
  for (const t of th.turns) {
    if (t.role === "user") push("user", t.text);
    else {
      const steps = (t.steps || []).map((s) => `[eval${s.reverted ? ", later reverted" : ""}]
          ${String(s.code).split("\n")[0].slice(0, 100)} → ${String(s.result || "").replace(/\s+/g, "
          ").slice(0, 160)}`);
      push("assistant", [...steps, t.text || "(no answer)"].join("\n"));
    }
  }
  while (turns.length && turns[turns.length - 1].role !== "user") turns.pop();
  if (!turns.length) return turns;
  turns[0].content = instructions() + "\n---\n" + turns[0].content;
  const last = turns[turns.length - 1];
  last.content = contextBlock() + "\n" + last.content;
  return turns;
}
```

Three things go into the conversation besides the person's messages.

The instructions go before the first message. They are 18,840 characters with the current sources: a paragraph on PicoCalc OS (worlds, the nucleus, messages, handles, supervision, the manifest), a paragraph on the one tool, a list of working habits, and the whole Reference of the workbench, 15,666 characters, which documents the views, PicoCalc OS in one page, the world API, the manifest schema and every name in the scope. The working habits are the part that encodes judgement:

▸ history/first-pass/workbench/src/agent.js

```
- Look before you change: files.grep / files.read(path, from, to) to find code; worlds(), text(world),
    messages({world}) to see the running system.
- Change code with files.edit(path, find, replace) (exact, unique text) rather than rewriting files.
    Then apply(path) — a world file hot-reloads that world; the manifest or the prelude reboots — and
    read the result.
- Verify in a fresh system with test({ script, ms, then }) and with evals.run(): a fresh system boots
    from the files as they are now and is exact (the clock is virtual, randomness is seeded), so any
    before/after difference comes from the edit. Say which system a number came from.
- If the same fix fails three times, stop and explain what you saw instead of trying again.
- Respect the OS's rules: worlds never share memory; every authority comes from the manifest; a turn is
    all-or-nothing. When an app needs a new channel, peer or claim, change the manifest too, and run
    the evals.
```

The third line asks for exactly the discipline Chapter 11 relies on: compare fresh systems before and after, and say which system a number came from. The fourth is a stopping condition of the kind Anthropic's guide describes for keeping an agent under control [Schluntz and Zhang 2024]. The last ties the agent to the system's own rules, so that, for instance, a new app gets a manifest entry instead of a channel it declares for itself. Two further lines say how to behave in Ask and Read-only modes and how to write the answer, with `wb:` links (to a file and line, a world, an eval, a view, a key, a section of the book) that the person can click; another says that the companion book is in scope as `book` and should be checked and linked when an answer rests on a design decision; the last says that times are virtual milliseconds of the ESP32-P4 unless stated otherwise.

Earlier steps are summarised. The thread keeps every step's full code and result for the person, but the model sees each earlier step as one line: the first line of its code, cut at 100 characters, an arrow, and the result with whitespace collapsed, cut at 160 characters, marked `later reverted` if the person reverted it. A four-step turn with its answer becomes 864 characters of history.

The context block goes before the latest message, so it describes the system as it is when the person writes. This is the block the headless test sent with its first message:

```
[context]
system: booted · t = 3.17 s · running at 1× · lane busy 1.0% · 13 worlds (13 up) · 0 crashes since boot
front app: launcher
files: js/apps/about.js, js/apps/faults.js, js/apps/keys.js, js/apps/launcher.js, js/apps/repl.js,
    js/apps/snake.js, js/apps/sysmon.js, js/drivers/display.js, js/drivers/keyboard.js,
    js/drivers/power.js, js/prelude.js, js/system.json, js/system/compositor.js, js/system/flaky.js,
    js/system/sandbox.js, workbench/evals.js (* edited)
evals: 19/19 pass (0 min ago)
[/context]
```

It says whether the system booted (and if not, the first boot errors), the virtual time, whether the clock is running and at what speed, the lane's occupancy, how many worlds are up, crashes since boot, the app in front, any world not running with its last error, the last four faults, every file with an asterisk on the edited ones, the files edited but not applied, the last eval results with the names of failing evals, and the world the person has selected. It is about 500 characters. Without it, the model's first step of every request would be to look, and it would look at whatever the instructions suggested rather than at what changed; with it, the model knows before its first step that, say, the display driver is restarting or that `snake.js` is edited and not applied.

In the first request of a thread the first message is also the latest, so it carries the context block, then the instructions, then the person's words: the headless test's first message was 19,382 characters, nearly all of it instructions. In every later request the first turn holds the instructions and the first message, the middle turns hold the history, and only the last turn, with a new context block, differs from the previous request.

10.6 Modes, steps and revert

A coding agent changes things, and the person must be able to see each change, refuse it beforehand and undo it afterwards. The workbench provides each of these with a separate mechanism.

Modes. The panel has three. In *Auto*, steps run as the model takes them. In *Ask*, `execute` stops before running each step, shows its code with Run this step and Decline buttons, and waits; a declined step returns to the model the sentence `Declined by the person. Ask what they want instead, or explain the change without running it.`, so the model

learns why it got no result. In *Read-only*, the scope's mutating functions throw (`files.edit`, `files.write`, `files.remove`, `apply`, `reboot`), and the instructions ask the model to describe the change it would make instead.

Read-only mode protects the files through the scope, and only through the scope. It is not a sandbox, and the Reference says so. Two measurements show how far it falls short of one. In Read-only mode, `crash('snake')` returned `snake killed at 1.633 s`: driving the live system is allowed, deliberately, since reading a system often means pressing keys and running time. And `return typeof host + ' ' + typeof window + ' ' + typeof localStorage;` returned `object object object`. The model's code is compiled with `new Function` in the page's global scope, so beyond the names passed as parameters it can reach every global of the page, including the host object whose `files` the scope was guarding. Read-only mode stops a model that uses the API it was given; it does not stop code that goes around it. A real sandbox would need the code to run where the page's globals do not exist, in a Web Worker or a separate realm, with the scope's functions as the only channel back, which is the confinement problem of *Small Worlds*, Chapter 10, again.

Steps and snapshots. Before a step runs, `execute` takes a snapshot of every file:

▸ workbench/src/agent.js

```
        const step = { code, result: null, running: true, ms: 0, snapshot: { files: Object.assign({},
            host.files) } };
```

The copy is shallow, and file contents are immutable strings, so the snapshot is one object holding 16 references to the strings that already exist (74,360 characters in all, shared, not copied). After the step, the page compares the files with the snapshot and marks the step `changed` if any file differs, appeared or disappeared. Each step is drawn as a row with a square marker, the word `eval`, the first line of its code and its duration; it opens to show the full code and result. A failed step's border and marker are red, as is a step waiting for approval in Ask mode.

Revert. A step that changed files offers "Revert files to before this step":

▸ workbench/src/agent.js

```
const revert = async (turn, i) => {
  const s = turn.steps[i];
  if (!s.snapshot) { host.toast("This step's snapshot is gone (reloaded page)", true); return; }
  host.files = Object.assign({}, s.snapshot.files); host.saveEdits();
  for (let j = i; j < turn.steps.length; j++) turn.steps[j].reverted = true;
  await host.apply();
  host.toast("Reverted the files to before that step and applied them"); re(); persistAgent();
};
```

Revert restores every file to its state before the step, marks that step and every later step of the turn as reverted (drawn struck through, and reported to the model as `later reverted`), and applies the result, hot-reloading or rebooting as the edits require. Its scope is precise, and three consequences follow. It restores files, not the system's history: the virtual clock does not go back, and a world killed during the turn is not revived by the revert, only restarted by its supervisor or by the apply. It restores all files, including any edit the person made by hand in the Code view after the step. And the snapshots are kept only in memory: the thread is saved in `localStorage` with each step's code and a result cut at 3,000 characters, but without snapshots, so after the page is reloaded the history remains and the revert is gone.

Together the three mechanisms give the person a choice before a step (Ask), a constraint on the kind of step (Read-only, within its limits) and a way back after a step (revert), and the Stop button ends a turn by aborting the signal that both `sample` and the running step listen to, at any moment when the page's thread is free to handle the click.

10.7 A worked turn

A headless test, kept outside the repository, loads the workbench page in Chrome through Playwright, runs the evals, and then plays one agent turn with a stand-in for `sample`. The stand-in is not a model. It ignores the conversation, calls the tool four times with fixed code, and returns a fixed answer, which makes it a test of everything on the page's side of Figure 10.1: the tool object, the scope, `runEval`, the YAML, the steps, `toTurns`, the context block and revert. A simplified sketch of it (the real one is in the test's own file):

```
window.claude = { use: async (name) => {
  const sample = async (turns, opts) => {
    const ex = opts.tools[0].execute;
    await ex({ code: "return worlds().slice(0, 3).map(w => ({ name: w.name, state: w.state }));" }, {
        signal: opts.signal });
    // … three more steps, each wrapped in try/catch, then a fixed answer
    opts.onText({ text, delta: text });
    return { text, truncated: false };
  };
  sample.limits = async () => ({ images: true });
  return sample;
} };
```

The person's message was "Make Snake faster and check it". Below are the four steps, each with its code after the step number and the result the page returned, copied from the test's output:

```
--- step 1   return worlds().slice(0, 3).map(w => ({ name: w.name, state: w.state }));
- name: keyboard
  state: idle
- name: power
  state: idle
- name: display
  state: idle
--- step 2   files.edit('js/apps/snake.js', '.every(110,', '.every(90,');
             return apply('js/apps/snake.js');
ok: true
reloaded:
  - snake
errors: []
--- step 3   return files.edit('js/apps/snake.js', 'no such text', 'x');
ERR Error: edit: text not found in js/apps/snake.js (it must match exactly, including spaces) (line 1)
--- step 4   const NL = String.fromCharCode(10); const r = await test({ script: ['wait 1500', 'key s',
    'wait 300'].join(NL), then: (s) => s.text('snake').split(NL)[0] }); return r;
booted: true
t_s: 1.87
worlds:
  running: 13
  total: 13
  not_running: []
faults: []
front: snake
value: score 0
```

Step 1 looks before changing anything, and its result is three short records. Step 2 is the composition that justifies a code tool: one edit and one apply in a single step, returning the list of reloaded worlds. Snake's tick at line 79 of `js/apps/snake.js` changed from 110 to 90 ms, and only Snake started again. Step 3 is a deliberate mistake, an edit of text that does not exist. It fails without touching the file, and the error says why and on which line of the step's code; the stand-in receives it as a thrown error, the form in which `execute` reports a failure to the capability. Step 4 tests the edited files in a fresh system: boot, wait 1.5 s, press `s` to open Snake from the launcher, wait 0.3 s, and return the first line of Snake's screen. The fresh system booted with all thirteen worlds up and no faults, at 1.87 virtual seconds, with Snake in front showing `score 0`.

The answer, written to the panel through `onText`, linked the file (`wb:open?file=js/apps/snake.js&line=81`; the stand-in's fixed text does not check the line) and an eval (`wb:eval?name=boot`), and contained a small table. Figure 10.2 shows the panel after the turn in an earlier run of the same test, with the steps' durations: 4, 7, 1 and 36 ms.

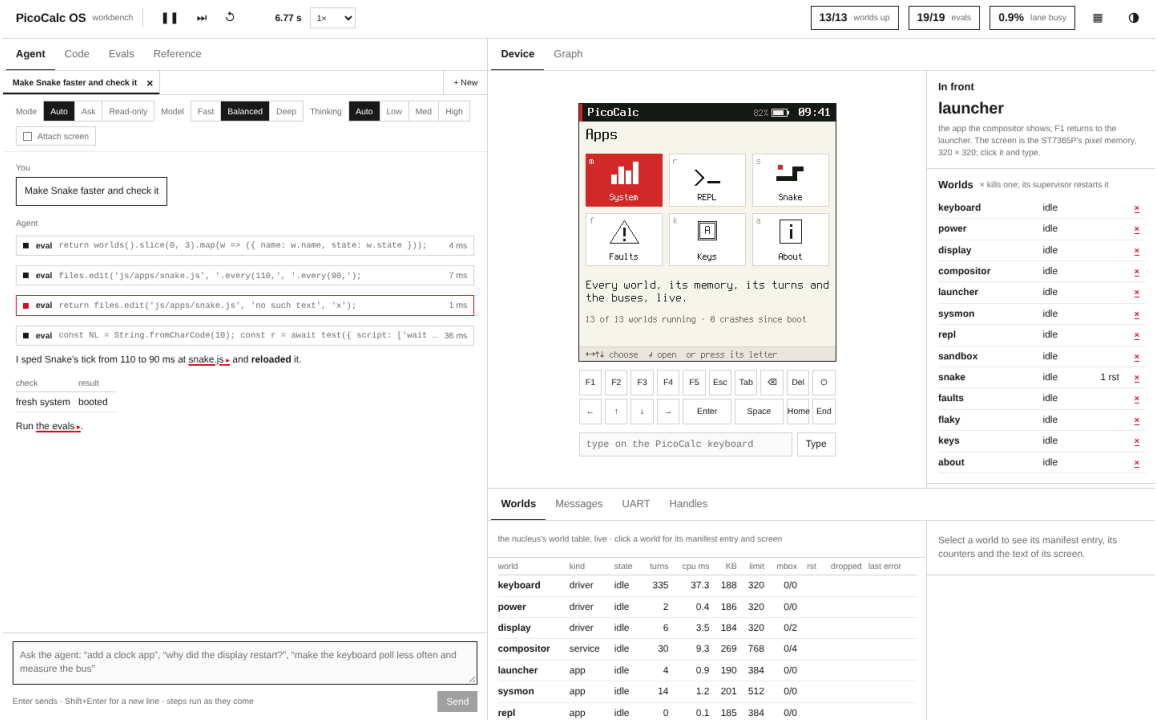


Figure 10.2. The Agent view after the scripted turn: four eval steps, the third (the failed edit) marked in red, then the answer with its links and table. The world table on the right shows Snake with one restart, the hot reload.

The test then opened the second step and pressed its revert link. The page found one revert link (only step 2 changed files), restored the files, applied them, and `host.isEdited("js/apps/snake.js")` returned `false`: Snake was running its original source again.

What would the model have seen if the person had written again before reverting? Running `toTurns` on the saved thread as it stood then, with a new message, "Now revert it", gives three turns. The first is the instructions and the original message (18,875 characters). The second is the assistant's turn, 864 characters, with each step reduced to one line, for example:

```
[eval] files.edit('js/apps/snake.js', '.every(110,', '.every(90,'); → ok: true reloaded: - snake
    errors: []
[eval] return files.edit('js/apps/snake.js', 'no such text', 'x'); → Error: edit: text not found in
    js/apps/snake.js (it must match exactly, including spaces) (line 1)
```

The third is a fresh context block, now listing `js/apps/snake.js*` as edited, followed by "Now revert it". The model would know what it did, what failed and what the files are now, without re-reading them.

This exercise establishes that the page's half of the agent works as designed: the tool runs code against the scope, results come back as compact YAML, failures come back as errors with their line, a fresh system answers in tens of milliseconds, and steps can be reverted. Whether a model uses this interface well is a separate question, which only runs with Claude, graded by the evals, can answer (Chapter 11).

10.8 What the agent cannot see, and how it fails

The agent sees the system only through its steps' results and the context block. That is a narrower view than the person's, in four ways.

It does not see pixels. `text(world)` gives what an app asked to show, and `screen()` adds a hash and the fraction of lit pixels, but a clipped label, an overlapping row or a wrong colour is invisible in both. The panel's "Attach screen" toggle, offered only when the capability reports that it accepts images (`sample.limits()`), attaches one 320 × 320 PNG of the live panel to the request, taken when the person sends the message. It is a picture of the screen before the turn, not after each step; a model that needs to see the effect of its own change on the screen cannot, within one turn.

It does not see the person's view. Which tab is open, where the editor is scrolled, what the person is looking at: none of this reaches the model, except the selected world, which the context block names.

It does not see the device or the native build. Every number it reads is from the WebAssembly build on the virtual clock, which Section 9.2 showed to agree with the native 32-bit build except for heap figures. The instructions ask it to say that times are virtual milliseconds of the ESP32-P4.

It does not see its own earlier results in full. After a turn, each step survives as one line of at most about 270 characters.

The ways a turn can go wrong, and what the page does about each, are collected in Table 10.1. Two of them deserve a closer look because the safeguard is weaker than it appears.

Table 10.1. How a step or a turn can fail, and what the page does about it.

Failure	What happens	Safeguard
An edit quotes text that is not in the file, or occurs more than once	The edit throws; the file is unchanged	The error says which, and how to fix it
An edit applies cleanly but breaks the world	`apply` returns `ok: true`; the world crashes when it evaluates the new source	The world appears in `worlds()`, `faults()`, the next `test` summary and the next context block
A step returns too much	Arrays are cut at 60 items, objects at 80 keys, depth at 5; the text at 9,000 characters	Each cut says what was left out and how to ask for less
A step waits forever on a promise	The step fails after 30 s, or when the person presses Stop	The work the step started may continue in the background
A step loops without awaiting	The page freezes until the loop ends	None in the page
The same fix keeps failing	The model may keep trying	The instructions ask it to stop after three attempts and explain
The thread grows too long	`sample` rejects with `prompt_too_large`	The panel asks the person to start a new thread
Read-only code reaches the page's globals	It can change files through `host`	None; Read-only constrains the scope only

The first is the apply that reports success. As Section 9.3 measured, a hot reload starts the world again and returns at once; a syntax error in the new source appears a moment later as a crash of the world, with an `#OCT1` frame giving the line. The instructions say to read the result after applying, and the result of a well-written step (`files.edit(...); await`

`apply(p); run(200); return { faults: faults(3), snake: world("snake").state }`) shows the crash. A step that returns only `apply`'s result, like step 2 of the worked turn, does not. The context block at the next message lists the stopped world and its fault, so the error is not lost, but it is noticed one message late.

The second is the timeout. The 30-second limit is a `Promise.race` between the step's promise and a timer, and a timer can fire only when the page's thread is free. Code that waits (on a fresh system, on the evals, on a promise that never settles) is stopped by it. Code that computes without waiting is not: a step whose body was `const t = Date.now(); while (Date.now() - t < 31000) {} return 1;` ran for 31,001 ms and returned `1`, with no timeout, because the timer was not even created until the loop had finished. During those 31 seconds the page drew nothing and took no input, the Stop button included. The same holds for `run(ms)` with a very large `ms`, which is synchronous by design (Section 9.6). Moving the steps into a Web Worker would let the page terminate one; until then, a step that loops is the one failure the page cannot interrupt.

KEY POINTS

- The agent has one tool, `eval`, whose argument is JavaScript run against a curated scope; code actions compose several operations in one step and are legible to the person, as CodeAct's and SWE-agent's results suggest.
- The page makes one `sample()` call per message with the tool and an `onText` callback; the capability runs the loop and calls the tool's `execute` for each step.
- The scope is designed as an agent-computer interface: exact and unique `files.edit`, numbered and capped reads, `apply` that restarts as little as possible, `test` on a fresh system, `text(world)` instead of pixels, the evals, and the book.
- Results come back as YAML, about as short as compact JSON and a third shorter than indented JSON, with arrays, objects and depth capped and the whole cut at 9,000 characters, each cut saying how to ask for less.
- The model receives 18,840 characters of instructions with the Reference before the first message, one line per earlier step, and a context block of about 500 characters describing the live system before the latest message.
- Ask mode lets the person approve each step; Read-only mode makes the scope's mutating functions throw but is not a sandbox, since step code can reach the page's globals.

- Every step snapshots the files, and revert restores them, marks later steps reverted and applies the result; it does not rewind the system, and snapshots do not survive a page reload.
- The agent has been exercised with a scripted stand-in for Claude, which confirmed the page's side of the design; it has not yet been run with Claude.

The agent is the workbench's host made available to a model through one tool, with the host's guarantees (fresh systems, determinism, exact apply) and its gaps (a successful apply of a broken file, synchronous code on the page's thread) both inherited. What makes its work checkable is the same thing that makes a person's work checkable: a suite of evals that boot fresh systems and say whether PicoCalc OS still does what it should. Chapter 11 describes that suite, and how it could also grade the agent.

EXERCISES

1. `BASIC` Write the code of one `eval` step that finds where Snake sets its tick interval and returns the five lines around it, numbered. Which scope functions does it use, and roughly how many characters does it return?
2. `BASIC` List the scope functions that throw in Read-only mode. Then give two different ways in which code run in Read-only mode can still change what the person sees in the workbench.
3. `ANALYSIS` A turn takes ten steps. Using the cut lengths in `toTurns`, bound the number of characters those steps add to every later request in the thread, and compare it with the instructions. At what number of steps per turn, and turns per thread, does the history outgrow the instructions?
4. `ANALYSIS` The worked turn's step 2 returned `ok: true` from `apply`. Write a step that would have caught a syntax error in the new Snake source within the same step, and explain why it must run the live system before reading `faults()`.
5. `CODE` Make Read-only mode a real constraint on files: run each step's code in a Web Worker that receives a copy of the files and posts back requests, with the scope's functions as the only channel. Which scope functions become asynchronous? What happens to `run`, `text` and `test`?

6. `CODE` Add a function `files.diffAll()` to the scope that returns `lineDiff` for every edited file, document it in `workbench/src/reference.md`, and test it with a stand-in `sample` like the one in Section 10.7, driving the page headlessly with Playwright or a similar tool.

7. `CODE` Measure the YAML, compact JSON and indented JSON sizes of the results of `stats()`, `handles()` and `messages({ last: 40 })` in the workbench (the page exposes `window.__wb.yaml`). Does the comparison in Section 10.4 hold for all three?

8. `PROJECT` Turn the headless test's stand-in into a replayer: record a real turn's tool calls and results, then replay the calls against the current sources and report any step whose result changed. Discuss how this differs from the agent evaluation Chapter 11 proposes.

9. `DISCUSSION` Would the agent be better with a second tool, for example a structured `edit` tool with `path`, `find` and `replace` arguments, alongside `eval`? Argue from CodeAct's comparison and SWE-agent's interface principles, and from what the workbench's person sees in each case.

10. `DISCUSSION` The context block is rebuilt for every message. Name one piece of state it omits that a model would often need, and one it includes that it could drop. What decides what belongs there?

FURTHER READING

[Yao et al. 2023] The paper that made reasoning interleaved with actions the standard shape of an agent.

[Wang et al. 2024] Code as the action space: why one tool that runs code can beat many tools with fixed arguments, measured across 17 models.

[Yang et al. 2024] The agent-computer interface, with the design principles and the measurements that the workbench's scope follows.

[Schluntz and Zhang 2024] A short, practical account of when to build an agent at all, how to design its tools, and why to give it ground truth at every step.

[Aizawa 2025] How to write tools for agents and evaluate them: consolidation, truncation with sensible defaults, and error messages that say what to do next.

[Small Worlds 2026] Chapter 10 on language-based isolation explains why running step code in the page's realm cannot confine it, and what a separate realm or worker would

change.

Evals on fresh systems

An eval is a short JavaScript function that boots a new PicoCalc OS from the files as they are now, drives it on the virtual clock, and checks what it sees. The workbench ships nineteen of them in `workbench/evals.js`. Together they cover about 80 seconds of virtual time, and in the headless runs made for this chapter the whole suite took between 1.1 and 1.5 seconds of wall time in Chrome. That is cheap enough to run after every edit, and because the system is deterministic, an eval that passed before an edit and fails after it failed because of the edit.

The evals are the third instrument of this book's method, after the attack worlds of `bench/rogue.py` and the checks of `bench/firstpass.py` (Chapter 8), and the one that a person or the agent of Chapter 10 can run from inside the page while changing the system. By the end of the chapter the reader can write an eval for PicoCalc OS, explain why each eval boots its own system and why that makes comparisons exact, read the Evals view's baseline and its REGRESSED marks, judge how much the suite catches (it caught 6 of 12 bugs seeded into the system for this chapter, and five more evals raised that to 11), and describe how the same machinery would evaluate the agent, which has not yet been done.

11.1 What an eval is

> **DEFINITION: EVAL**
>
> An eval is a named check of PicoCalc OS behaviour: a function that receives a freshly booted system, drives it through virtual time, and records whether what it observed matches what it expected, with a sentence saying what it saw when it does not.

The whole interface is written at the top of `workbench/evals.js`, in the comment the person and the agent both read:

▸ workbench/src/evals.js

```
// Evals for PicoCalc OS. Each one boots a fresh system from the files as they are now (edits
    included),
// drives it on the virtual clock and checks what it sees. define(name, about, [opts], (s, t) => ...):
//   s: a fresh system — s.run(ms), s.key(name, holdMs), s.type(text), s.crash(world), s.worlds(),
    s.world(name),
//      s.text(world), s.frame(world), s.faults(), s.uart(last, grep), s.messages({...}),
    s.screenHash(), s.lit(),
//      s.stats(), s.focus (the app in front), s.expect(cond, message)
//   t: t.fresh(opts) for a second system, t.manifest() for js/system.json as edited
//   opts: { files: { "apps/x.js": src }, manifest: m => m, expectBootFailure: true }
```

`s` is an instance of `Sys`, the class that wraps a PicoCalc instance in a synchronous API for the live system, for the agent's `test()` and for the evals (Chapter 9 describes it). `s.run(ms)` advances virtual time; `s.key` and `s.type` press keys through the simulated STM32; `s.crash(world)` kills a world as a hardware fault would; the rest read the system: the world table, the text a world's last frame shows, the panel's pixels, the UART and its `#OCT1` fault frames, the kernel's statistics. The simplest eval in the suite uses three of them:

▸ workbench/src/evals.js

```
const down = (s) => s.worlds().filter((w) => w.state === "stopped" || w.state === "restarting");
const names = (ws) => ws.map((w) => w.name).join(", ") || "none";
…
define("boot", "every world starts and stays up for 3 s, with no fault on the UART", (s) => {
  s.run(3000);
  s.expect(down(s).length === 0, `not running: ${names(down(s))}`);
  s.expect(s.faults().length === 0, `faults: ${s.faults().map((f) => f.world + ": " + f.why).join(";
      ")}`);
  s.expect(s.focus === "launcher", `the launcher should be in front, not ${s.focus}`);
});
```

Each `s.expect` records a failure and carries on, so one run reports every expectation that failed rather than only the first; an exception thrown by the eval or by the system (`s.run` on a system that did not boot, `s.crash` of a world that does not exist) fails the eval with its message. An eval may return a string, which the Evals view shows as its detail; `idle-lane` and `key-latency` use this to report the number they measured even when they pass.

The runner is `host.runEvals` in `workbench/src/core.js`. Its loop is short enough to read whole:

▸ history/first-pass/workbench/src/core.js

```
    for (const d of defs) {
      await new Promise((r) => setTimeout(r, 0));
      const t0 = performance.now();
      let s = null, pass = false, detail = "", vt = 0;
      try {
        s = await this.fresh(d.opts);
        const fails = [];
        s.expect = (cond, msg) => { if (!cond) fails.push(msg || "expectation failed"); return !!cond;
            };
        const t = { fresh: (o) => this.fresh(o), manifest: () => this.manifest() };
        if (d.opts.expectBootFailure) {
          const r = await d.fn(s, t);
          if (r !== undefined && typeof r === "string") detail = r;
        } else {
          if (!s.booted) throw new Error("did not boot: " + s.bootErrors().slice(0, 3).join("; "));
          const r = await d.fn(s, t);
          if (r !== undefined && !detail) detail = typeof r === "string" ? r : JSON.stringify(r);
        }
        vt = s.now();
        pass = fails.length === 0;
        if (!pass) detail = fails.join("; ");
      } catch (e) { pass = false; detail = (e && e.message) || String(e); vt = s ? s.now() : 0; }
      const was = prev[d.name];
      results.push({ name: d.name, about: d.about, pass, detail, ms: Math.round(performance.now() -
          t0), vt: Math.round(vt), regressed: !pass && was && was.pass });
```

Three decisions in this loop shape everything else in the chapter.

Every eval gets its own system. `this.fresh(d.opts)` creates a new PicoCalc instance from the compiled WebAssembly module and the operating system's files as they are in the editor now, edits included, whether or not they have been applied to the live system. The instance has its own linear memory, so nothing one eval does (a crash it caused, a key it pressed, a crash count still inside the supervisor's ten-second window) can reach the next. The live system is never touched, and it keeps running in the gaps between evals. The price is a new system per eval. `manifest-sound`, which creates a system and then only reads the manifest, takes 6 to 7 ms of wall time, which bounds what a fresh system costs before it runs.

The scenario is part of the eval. `opts.files` adds or replaces files before the boot (a path without `js/` gets it), and `opts.manifest` is a function applied to the parsed manifest, or an object merged into it. An eval about an attack therefore carries its attacker with it, and the

shipped files are not changed to make the attack possible. `opts.expectBootFailure` runs the eval even when the system refuses to boot, for the evals whose subject is the refusal.

The result records virtual and wall time. `vt` is the virtual time the system reached; `ms` the wall time the eval took. The first is a property of the system and the eval and is the same in every run; the second depends on the machine and the browser. Section 11.4 returns to the difference.

An eval, then, is one property, checked on one system that exists only for that check, with a result that says what was seen. The next section reads the nineteen the workbench ships.

11.2 The suite

The nineteen evals fall into four groups. Table 11.1 lists them with the virtual time each covers, from a run of the headless test that drives the workbench page in Chrome.

Table 11.1. The nineteen evals of workbench/evals.js. Virtual time is the time the eval's system reached; deterministic also runs a second system for the same 5.84 s.

Group	Eval	What it checks	Virtual s
The system as shipped	`boot`	every world starts and stays up for 3 s, no fault on the UART, the launcher in front	3.00
	`apps-open`	each of the launcher's six letters brings its app to the front, and F1 returns home	6.54
	`repl-evaluates`	`6*7` typed at the REPL comes back as 42 from the sandbox	3.03
	`snake-autopilot`	Snake's autopilot scores within 15 s	16.84
Recovery	`sandbox-contains-a-loop`	`for(;;){}` at the REPL kills only the sandbox, which restarts, and the REPL reports the overrun	5.27
	`keyboard-recovers`	a killed keyboard driver restarts, and → moves the launcher's selection	2.87
	`display-recovers`	a killed display driver restarts and the panel shows the same picture, give or take the crash counter	5.00
	`compositor-recovers`	a killed compositor restarts, the panel is lit, every world is up	3.50
	`crash-loop-stops`	five crashes in ten seconds stop `flaky`, and a reload brings it back	3.40
Determinism and timing	`deterministic`	two fresh systems given the same keys show the same pixels and count the same turns	5.84
	`idle-lane`	at rest on the home screen the lane is busy less than 5% of the time	6.50
	`key-latency`	a key press changes the panel within 60 virtual ms	1.57
The first pass	`manifest-sound`	every `sends` and `asks` target exists, and the asks form no cycle	0.00

Group	Eval	What it checks	Virtual s
	`forged-key-refused`	a world that declares `provides("keys")` without the manifest's grant is stopped, and the selection does not move	3.00
	`forged-frame-refused`	a world allowed to send to the compositor still cannot send it a `frames` message	3.00
	`display-only-for-compositor`	a world the manifest does not allow cannot ask the display driver to flush	3.00
	`handle-quota`	a world with the `dma` claim hits its handle and framebuffer quotas early, and the compositor can still restart	5.00
	`mailbox-bounded`	a world flooding `flaky` is told the mailbox is full, which peaks at no more than 128	3.00
	`ask-cycle-refused`	a manifest whose asks form a cycle does not boot, and the error names the cycle	0.00

The first group checks that the system does what a person using it would notice first. `apps-open` presses each launcher letter and F1 in turn and reads `s.focus`, which `Sys` keeps from the compositor's `focus` messages; it never looks at pixels. `repl-evaluates` reads the REPL's text, the lines its last frame shows, so a change of font or colour cannot break it while a wrong answer will. `snake-autopilot` is the longest eval because the game needs time to find food; it depends on the seeded `Math.random` of *Small Worlds*, Chapter 18 [Small Worlds 2026], since the food's position comes from it.

The second group kills things. Each recovery eval lets the system run normally for 1.5 s, crashes one world, waits long enough for the restart that *Small Worlds*, Chapter 15, measured (213 ms for the keyboard driver, 515 ms for the display driver, 167 ms for the compositor), and then checks the world table and something a person would see. `crash-loop-stops` reaches past `Sys` to the WebAssembly API: after five crashes it calls `s.pc.reload("flaky")`, the host's way of starting a stopped world again, which a world with `sys.admin` would do with `sys.restart`.

The third group checks properties of the whole system rather than of one feature. `idle-lane` and `key-latency` return their measurements as details; the run below reports the lane busy 0.72% of the time at rest, and 16 ms from a key press to the first changed pixels.

The fourth group is the browser form of the attack suite of Chapter 5, Chapter 6 and Chapter 7. `manifest-sound` reads the manifest as edited and repeats two of the boot's checks on it (every `sends` and `asks` target exists, and the asks form no cycle); each of the other six adds an attacker and gives it exactly the authority its attack needs. This is the forged key press:

▸ workbench/src/evals.js

```
const addWorld = (entry) => (m) => { m.worlds.push(entry); return m; };
…
const ROGUE_KEY = `app(self).provides("keys").onStart(async () => {
  await sleep(1500);
  try { publish("keys", { code: 0xb6, key: "down", ch: "", state: "down", caps: false }); log("OUTCOME
      published"); }
  catch (e) { log("OUTCOME refused: " + e.message); }
});`;
define("forged-key-refused", "a world may not post on keys unless the manifest makes it the provider",
  { files: { "apps/rogue_key.js": ROGUE_KEY }, manifest: addWorld({ name: "rogue_key", file:
      "apps/rogue_key.js", mem_kb: 256 }) }, (s) => {
  s.run(3000);
  s.expect(!/> Faults/.test(s.text("launcher") || "") && /> System/.test(s.text("launcher") || ""),
      "the launcher's selection moved: a forged key got through");
  s.expect(/provides\("keys"\)/.test(s.world("rogue_key").last_error || ""), `rogue_key should fail to
      start, last error: ${s.world("rogue_key").last_error}`);
});
```

The rogue's file declares `provides("keys")`, which the first pass forbids unless the manifest grants it, and tries to publish a ↓ key. The first expectation checks the harm: the launcher's selection is still on System, not one row down on Faults. The second checks the reason: the world was stopped at start because of its declaration. Only the first is about behaviour; the second makes sure the attack was stopped for the right reason, not, say, because the rogue's file had a syntax error. `ask-cycle-refused` uses `expectBootFailure` to boot a manifest with two worlds that ask each other and checks that the boot errors name the cycle.

The headless test prints every eval's result and wall time. This is its output for the shipped files:

```
$ node test.js
start error: null
evals in 1494 ms
PASS boot    49ms
PASS apps-open    84ms
PASS repl-evaluates    61ms
PASS sandbox-contains-a-loop    70ms
PASS keyboard-recovers    48ms
PASS display-recovers    74ms
PASS compositor-recovers    64ms
PASS crash-loop-stops    68ms
PASS snake-autopilot    250ms
PASS deterministic    157ms
PASS idle-lane  lane busy 0.72%  40ms
PASS key-latency  16 ms to the first changed pixels  46ms
PASS manifest-sound    7ms
PASS forged-key-refused    38ms
PASS forged-frame-refused    43ms
PASS display-only-for-compositor    46ms
PASS handle-quota    48ms
PASS mailbox-bounded    74ms
PASS ask-cycle-refused    7ms
…
```

The nineteen cover 80.4 virtual seconds (86.2 counting the second system of `deterministic`) in 1.1 to 1.5 seconds of wall time, between about 55 and 70 times faster than the modelled device would run them. Figure 11.1 shows the same results in the workbench, with a virtual and a wall column. The suite thus spans the shipped behaviour, recovery from the crashes *Small Worlds* studied, the system's determinism and timing, and the first pass's refusals; Section 11.5 measures how much of each it actually protects.

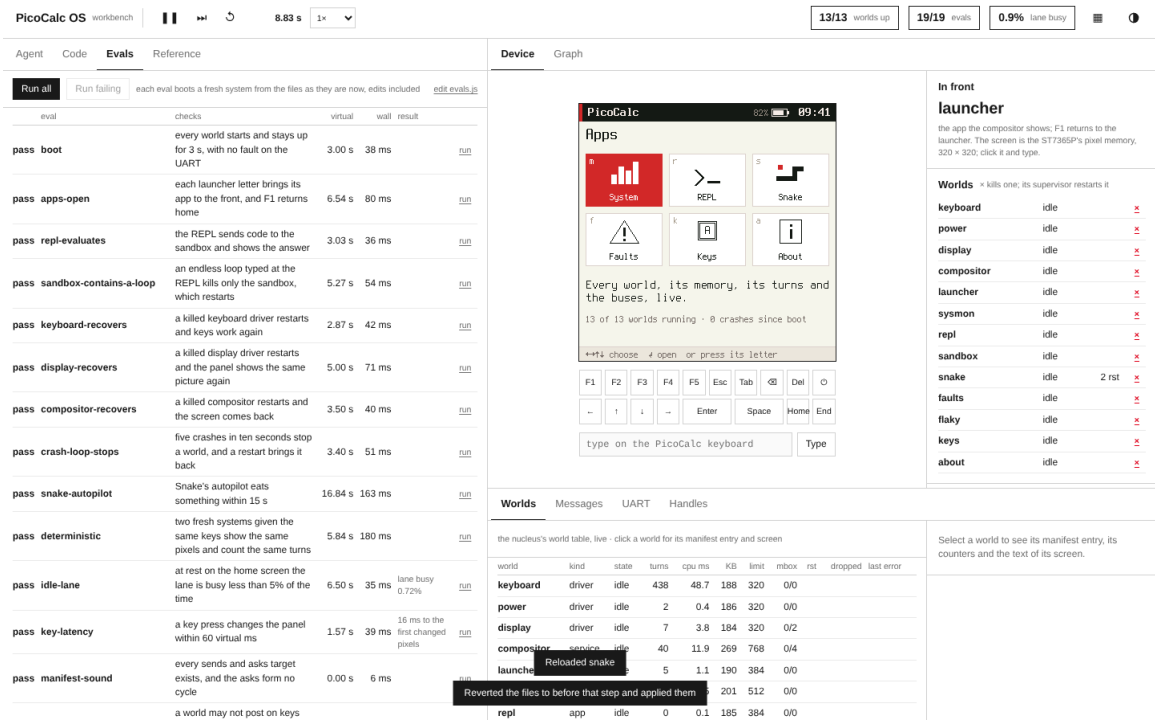


Figure 11.1. The Evals view after a run of the whole suite: each eval's result, the virtual time its system reached and the wall time it took. The header's pill shows 19/19.

11.3 Writing a good eval

An eval is a claim about the system written so that a program can check it, and most of the craft is in choosing what to observe and how much difference to allow. The suite's own history has the clearest example.

`display-recovers` kills the display driver and asks whether the panel comes back. Its first version compared the panel before the crash with the panel afterwards by hash, on the reasoning that recovery means the same picture. The sketch below is that version, reconstructed; it is not in the repository:

```
// the first version of display-recovers (a reconstruction)
s.run(1500);
const before = s.screenHash();
s.crash("display"); s.run(3500);
s.expect(s.screenHash() === before, "the panel should show the same picture again");
```

It failed on a system that recovered correctly. On the current build the hash before the crash is `06061708` and the hash after recovery is `ac466f1a`. Comparing the two pictures pixel by pixel shows 22 differing pixels, all inside the box from (164, 264) to (168, 270), and the launcher's text says why:

```
was: 13 of 13 worlds running · 0 crashes since boot
now: 13 of 13 worlds running · 1 crashes since boot
```

The launcher counts crashes, and the eval had just caused one, so the 0 had legitimately become a 1. The system had recovered correctly and the eval had rejected it; the fix had to say precisely what is allowed to change. The current version counts differing pixels and allows fewer than 100:

▸ workbench/src/evals.js

```
define("display-recovers", "a killed display driver restarts and the panel shows the same picture
    again", (s) => {
  s.run(1500);
  const before = s.screen().slice();
  s.crash("display"); s.run(3500);   // restart, 370 ms of panel set-up, then the "running again"
      banner clears
  s.expect(s.world("display").restarts >= 1, "the display driver should have restarted");
  s.expect(s.lit() > 0.5, `the panel should be lit again (lit ${s.lit()})`);
  const a = before, b = s.screen();
  let n = 0; for (let i = 0; i < a.length; i += 4) if (a[i] !== b[i] || a[i + 1] !== b[i + 1] || a[i +
      2] !== b[i + 2]) n++;
  s.expect(n < 100, `${n} pixels differ from the picture before the crash (only the launcher's crash
      count may change)`);
});
```

The threshold is argued from both sides. A character of the launcher's small font occupies a cell of 6 × 12 pixels, so one changed digit can alter at most 72 pixels; 100 leaves room for that and little else. A failed recovery, on the other side, changes almost everything: when the compositor was made to skip re-attaching the framebuffer after the restart (mutant M4 of Section 11.5), 102,399 of the panel's 102,400 pixels differed. Any tolerance between 72 and 102,399 separates the two cases, and 100 sits near the bottom of that range, where a small unintended change would still be caught. Masking the box instead would also have worked, but it would tie the eval to the launcher's layout, and an eval about the display driver should not break when the launcher moves a line. The wait is argued the same way: the comment counts the 100 ms backoff, the display's 370 ms of panel start-up, and the compositor's 1.5 s "running again" banner, which must have cleared before the picture can match.

The story generalises into a handful of rules, each of which the suite follows somewhere.

- **Observe what a user or a client of the world would observe.** The selection in the launcher, the app in front, the REPL's answer, the panel's pixels and the world table are behaviour; a log line is behaviour only when the text is the interface. The rogue worlds log their own `OUTCOME` because the exception a sender receives is the documented contract of the first pass's checks (Appendix A lists the messages verbatim).
- **Derive tolerances from what may legitimately change, and check them against a broken system.** A tolerance that also accepts the failure it was written to catch is no check at all; M4's 102,399 pixels are the evidence that 100 is not such a tolerance.
- **Derive waits from the mechanism.** Every `s.run` in the recovery evals covers a sequence the reader can add up from *Small Worlds*, Chapters 14 and 15, with a margin.
- **Say what was seen.** `expected 42 in the REPL, saw: …` and `${n} pixels differ` turn a failure into a diagnosis, for a person and for the agent, which reads only the text.
- **Make sure the instrument is on before checking for an absence.** A fresh system does not record its trace: `Sys` keeps trace records only when `s.tracing` is true, and only the live system turns it on. A probe made for this chapter found `s.tracing` false and `s.messages({})` empty after 1.5 s of a fresh system. An eval that checks that no forbidden message was sent, without first setting `s.tracing = true`, passes whatever the system does. The defence is a control: check first that the instrument sees something it must see.

Writing a good eval is therefore the same discipline as stating a claim that can be checked (*Small Worlds*, Chapter 20): say exactly what should hold, observe it where it is visible, allow exactly the variation the design allows, and confirm that a broken system fails.

11.4 Exact comparison and the baseline

Two properties turn a set of checks into an instrument for change: each run gives the same answer for the same files, and each run is compared with the one before it.

11.4.1 Determinism

A fresh system is deterministic. Given the same files and the same inputs it executes the same segments, sends the same messages and draws the same pixels, because PicoCalc OS closes the sources of nondeterminism a world could reach: `Date` reads the virtual clock, `Math.random` is a generator seeded per world, ties are broken by sequence numbers, there is one thread, and in the WebAssembly build the WASI clock returns a fixed instant (*Small Worlds*, Chapter 18; Chapter 9). The `deterministic` eval checks the property directly:

▸ workbench/src/evals.js

```
define("deterministic", "two fresh systems given the same keys show the same pixels and count the same
    turns", async (s, t) => {
  const s2 = await t.fresh();
  for (const x of [s, s2]) { x.run(1500); x.key("s"); x.run(200); x.key("p"); x.run(4000); }
  s.expect(s.screenHash() === s2.screenHash(), "the screens differ");
  s.expect(s.stats().turns === s2.stats().turns, `turns differ: ${s.stats().turns} and
      ${s2.stats().turns}`);
});
```

It runs two systems for 5.84 virtual seconds, the last four of them Snake on autopilot, a workload that uses the seeded generator for food, timers for the game's tick, and the compositor and display for every frame, and it requires the same panel hash and the same number of turns.

The runs made for this chapter show what determinism means in practice. Across three runs of the suite, `boot` took 49, 78 and 40 ms of wall time, and its virtual time was 3.00 s every time; `snake-autopilot` took 250, 188 and 147 ms, and reached 16.84 s every time, and

two separate games played the way it plays them both ended with a score of 90. Wall time measures the browser; virtual time measures the system.

Exactness changes how evals can be written and read. A threshold can be tight, because the measured value does not wander: `key-latency` allows 60 ms against a measured 16 ms and does not need a margin for noise. There are no flaky evals and no retries: an eval that fails, fails every time on those files, so a failure is something to read, not something to rerun. And a before-and-after comparison is exact. If an eval passes on the files before an edit and fails on the files after it, the edit caused the difference, because nothing else differs between the two runs. The agent's instructions rely on this when they tell it that "any before/after difference comes from the edit" (Chapter 10).

Two cautions keep the property in proportion. Determinism holds within one build: the evals run the WebAssembly build in one browser, and the comparison of builds (21 of 22 captures identical between WebAssembly and native 32-bit, Section 9.2) is a different question. And exactness is not coverage. An eval that observes one input sees one behaviour exactly, and a bug that shows only at another input is invisible to it however exact the run. The shipped `key-latency` presses one key at one moment; Section 11.5 shows a bug it cannot see for exactly that reason.

11.4.2 Baselines, history and REGRESSED

A single run says which evals pass. A sequence of runs says which ones stopped passing, and that is the question a person changing the system usually has. The runner answers it with the line already quoted from `runEvals`: an eval that fails now and passed in the previous run is marked `regressed`, and the Evals view prints REGRESSED instead of FAIL. After the loop the run is stored and appended to a history:

▸ history/first-pass/workbench/src/core.js

```
    const run = { at: Date.now(), results, partial: !!(names && names.length) };
    if (run.partial && this.evalResults && this.evalResults.results) {
      const by = Object.fromEntries(this.evalResults.results.map((r) => [r.name, r]));
      for (const r of results) by[r.name] = r;
      run.results = Object.values(by); run.partial = false;
    }
    this.evalResults = run;
    this.evalHistory = [...this.evalHistory, { at: run.at, pass: run.results.filter((r) =>
        r.pass).length, total: run.results.length }].slice(-24);
    store.set("evalResults", { at: run.at, results: run.results }); store.set("evalHistory",
        this.evalHistory);
```

A partial run (one eval from its `run` link, or "Run failing") is merged into the previous results, so the baseline always covers the whole suite. Results and the last 24 runs' pass counts live in the browser's local storage, so a reload of the page keeps them; the Evals view draws the history as a row of bars, newest on the right.

The marking was exercised by seeding a bug. After a passing run, the line in `js/system/compositor.js` that sends F1 home was deleted and the suite run again; `apps-open` came back with `regressed: true`, which the Evals view shows as REGRESSED, and this detail:

```
F1 from sysmon should return to the launcher; r should open repl, but sysmon is in front; F1 from repl
    should return to the launcher; …
```

The mark says that something broke since the last run, and the detail says what: the first F1 left the system monitor in front, and every later letter went to it instead of to the launcher. Restoring the line and running only the failing eval (what "Run failing" does) merged one result into the baseline and brought the suite back to 19 of 19; the history's last three bars read 19/19, 18/19, 19/19.

The mechanism is deliberately simple, and its simplicity has three consequences worth knowing. The baseline is the previous run, and no run can be pinned: an eval that fails twice in a row is REGRESSED the first time and plain FAIL the second, so the mark is a prompt to look now, not a record of when the failure began (the history bars keep that). Numbers returned as details are not compared at all: if a change raised the key latency

from 16 ms to 50 ms, `key-latency` would still pass and nothing would be marked, because only the pass/fail bit has a baseline. And the baseline belongs to one browser; there is no shared record across machines or people. Each is a small extension, and Chapter 12 lists a pinned baseline with numeric details among the workbench's next features.

11.5 Measuring the suite with seeded bugs

How much does the suite protect? The direct way to find out is to put bugs into the system on purpose and count how many the suite catches. This is **mutation analysis**, set out by DeMillo, Lipton and Sayward in 1978 [DeMillo et al. 1978] from an idea Lipton first wrote down in 1971 [Jia and Harman 2011]: make small changes (mutants) to a correct program, run the tests on each, and call a mutant *killed* when some test fails. The fraction killed estimates how sensitive the tests are to the kind of fault the mutants represent. A mutant that no test can distinguish from the original, because it does not change behaviour, is *equivalent*, and deciding equivalence is undecidable in general [Jia and Harman 2011].

Twelve mutants were written by hand for this chapter, each a plausible slip, spread over two drivers, the compositor, the launcher, the sandbox and the manifest. They were applied one at a time to the workbench's files in a headless page, with the shipped nineteen evals and then with five more written for the survivors, restoring the files after each. Table 11.2 gives the results.

Table 11.2. Twelve seeded bugs and the evals that caught them. Shipped: the nineteen evals of workbench/evals.js. Added: the five evals written for this chapter (not in the repository).

Mutant	Change	Caught by the shipped suite	Caught by an added eval
M1	display driver: drop the stale-flush guard	survived	`display-recovers-once`
M2	keyboard driver: poll every 100 ms instead of 20	survived	`key-latency-phases`
M3	compositor: F1 no longer returns home	`apps-open`	
M4	compositor: no re-attach after a display restart	`display-recovers`	
M5	manifest: the REPL may not ask the sandbox	`repl-evaluates`, `sandbox-contains-a-loop`	also `authority-as-designed`, `repl-prints`
M6	manifest: the sandbox's budget is 4,000 ms	`sandbox-contains-a-loop`	
M7	manifest: default mailbox of 4,096 messages	`mailbox-bounded`	
M8	manifest: Snake may send to and ask the display driver	survived	`authority-as-designed`
M9	power driver: never samples the battery again	survived	`battery-reaches-compositor`
M10	compositor: a skipped flush no longer invalidates the framebuffer	survived	survived
M11	launcher: each letter opens the next app	`apps-open`, `repl-evaluates`, `sandbox-contains-a-loop`, `snake-autopilot`	also `repl-prints`

Mutant	Change	Caught by the shipped suite	Caught by an added eval
M12	sandbox: `print()` output is dropped	survived	`repl-prints`

The shipped suite killed 6 of the 12. Every mutant in the manifest that removed or weakened a mechanism (M5, M6, M7) was killed, which is what the first pass's evals were written for. The survivors are more instructive, because each shows a kind of blindness.

M1: an end state hides a path. Without the guard that answers a stale flush `skipped`, the restarted display driver crashes a second time on the flush its dead predecessor never answered, as the simulator found when the system was built (*Small Worlds*, Chapter 15). The fault frame says so: `TypeError: cannot read property 'id' of undefined (drivers/display.js:34:123)` at 1,970.793 ms. But the second restart succeeds, the panel is lit, and the only pixel change is the launcher's crash counter, now 2 rather than 0, well under 100 pixels. `display-recovers` checks the end state, and the end state is fine. The added eval checks the path:

```
// written for this chapter; not in workbench/evals.js
define("display-recovers-once", "a killed display driver comes back with one restart and no second
    fault", (s) => {
  s.run(1500);
  s.crash("display"); s.run(3500);
  const d = s.world("display");
  s.expect(d.restarts === 1, `the display driver restarted ${d.restarts} times`);
  const f = s.faults().filter((x) => x.world === "display");
  s.expect(f.length === 1, `display faults: ${f.map((x) => x.why).join("; ")}`);
});
```

M2: one sample is not a distribution. Polling the keyboard every 100 ms instead of every 20 should hurt latency, and it does: pressing → at ten moments 7 ms apart, starting at 1.5 s, the mutant took 16, 109, 102, 95, 88, 81, 74, 67, 60 and 53 ms to change the panel, against 16 to 35 ms for the shipped driver. The shipped `key-latency` presses once, at 1.5 s, and that press lands just before a poll that both drivers make, because their timers start together and every fifth 20 ms poll falls where a 100 ms poll does; it measures 16 ms either way. Determinism made the blind spot permanent: the eval will see 16 ms on every run. The

added `key-latency-phases` presses at the ten phases and fails if the slowest exceeds 60 ms; on the mutant it reports `slowest press: 109 ms`. *Small Worlds*, Chapter 19, describes the same trap as phase-locking in a benchmark harness.

M8: the evals test the mechanism and leave the policy untested. Giving Snake the right to send to and ask the display driver breaks no behaviour, so no behavioural eval notices. The first pass's evals check that the nucleus refuses authority the manifest does not grant; none checks that the shipped manifest grants only what the design intends. The added `authority-as-designed` compares each world's `sends` and `asks` with a table of the designed graph written in `evals.js`. Its value comes from keeping that table outside `js/system.json`, so that a change to the manifest has to be made in two places, deliberately.

M9 and M12: nothing looked. No eval read the battery path or the REPL's `print`. The added evals are a battery change followed by a check that a `power` post reached the compositor (with `s.tracing` set first, for the reason given in Section 11.3), and a REPL line `print(6*7); 1` whose output must show 42 then 1.

M10: possibly equivalent. When a flush is answered `skipped`, the compositor invalidates its framebuffer so that the next flush sends every pixel. Deleting the invalidation changed nothing any eval could see, and reading the compositor shows why: in the restart sequence, the `attach` the compositor sends when the display is running again also invalidates the framebuffer, so the invalidation on a skipped flush is redundant there. Whether some other sequence of events produces a skipped flush with no `attach` after it is exactly the question an equivalent mutant leaves open, and Exercise 7 asks it.

With the five added evals the suite killed 11 of the 12. The additions cost about 0.65 s of wall time per run, 530 ms of it in `key-latency-phases`, which boots ten systems; the 24 evals together took 1.7 s. Twelve hand-written mutants are a small sample, and a mutation score is only as representative as the mutants; but the exercise found four kinds of blindness in a suite that passed 19 of 19, which no amount of rerunning it could have found. It also produced the material for Section 11.6.2, because a seeded bug together with the eval that catches it is a ready-made task.

11.6 The agent and the evals

The evals serve the workbench's agent in two ways. Today the agent runs them to check its own changes. In a design not yet built, they would grade the agent itself.

11.6.1 Running evals from the agent

The workbench's agent (Chapter 10) works through a single `eval` tool whose scope includes the evals. The part of `agentScope` that exposes them is a thin layer over the same runner:

▸ history/first-pass/workbench/src/agent.js

```
    evals: {
      list: () => host.parseEvals().map((d) => ({ name: d.name, about: d.about })),
      run: async (names) => {
        const r = await host.runEvals(typeof names === "string" ? [names] : names);
        if (r.error) throw new Error(r.error);
        const rs = r.results.filter((x) => !names || (typeof names === "string" ? [names] :
            names).includes(x.name));
        return { pass: rs.filter((x) => x.pass).length, total: rs.length, failing: rs.filter((x) =>
            !x.pass).map((x) => ({ name: x.name, detail: x.detail, ...(x.regressed ? { regressed: true
            } : {}) })) };
      },
      last: () => host.evalResults && host.evalResults.results ? host.evalResults.results.map((x) => ({
          name: x.name, pass: x.pass, ...(x.pass ? {} : { detail: x.detail }) })) : null,
    },
```

Three choices in it follow from what an agent needs. The result is compact: a passing run of the whole suite comes back as `pass: 19`, `total: 19` and an empty `failing` list, a few dozen characters of YAML, so the agent can run the suite after every edit without filling its context. A failure comes back with its detail sentence, which is why Section 11.3 insists that the sentence say what was seen. And the `regressed` flag is passed through, so the agent can tell a failure it caused from one that was already there.

The agent does not need to be told to look: the context block that precedes every message it receives includes the last run. In the headless test's agent turn (the whole block is in Section 10.5) that line read `evals: 19/19 pass (0 min ago)`; after a failing run it also names the failing evals.

The loop this enables is the one the agent's instructions describe: look, edit exact text, apply, check in a fresh system with `test()`, then run the evals and read the failures. Because evals and `test()` both boot from the files as they are, the agent can check a change before applying it to the live system at all. Its answers can link to an eval with `wb:eval?name=boot`, which runs that eval when the person clicks it.

One property of this arrangement needs saying plainly. `workbench/evals.js` is a file like any other in the agent's scope, and `files.edit` can change it. An agent asked to make the suite pass could weaken an eval instead of fixing the system, and the run would report 19 of 19. The workbench's defences are procedural: every step that changed a file keeps a snapshot and can be reverted, `files.diff("workbench/evals.js")` shows any change to the suite, and the context block marks the file edited with `*`. For evaluating the agent itself these are not enough, and the design below keeps the graders out of its reach.

11.6.2 Evaluating the agent

The evals check the system. The same machinery could check the agent: give it a task, let it work in the workbench, and grade the files it leaves with evals run on a fresh system. This has not been built, and the agent has so far been exercised only with a scripted stand-in for the model in the headless test (Chapter 10). What follows is a design, labelled as one.

The model for it is SWE-bench [Jimenez et al. 2024]. Its 2,294 tasks are real issues from 12 popular Python repositories, each paired with the pull request that resolved it. A system under evaluation receives the issue and the repository and produces a patch; the patch is applied and the repository's tests are run. A task counts as resolved only if all of its FAIL_TO_PASS tests (tests from the reference solution that failed before it and pass after it) and all of its PASS_TO_PASS tests (tests that passed before and must still pass) pass. When the benchmark was published, the best model it evaluated, Claude 2, resolved 1.96% of the tasks. A later audit annotated 1,699 tasks and kept 500 as SWE-bench Verified; annotators flagged 61.1% of the samples for unit tests that might reject valid solutions and 38.3% for underspecified problem statements [Chowdhury et al. 2024]. The audit is the agent-scale version of the first `display-recovers`: a grader that rejects a correct change is a defect in the benchmark, not in the system graded.

PicoCalc OS has three advantages over a repository of Python code. Its graders are deterministic, so a task's verdict does not depend on test flakiness. A grader can observe behaviour (pixels, the world table, faults) rather than function return values. And the seeded bugs of Section 11.5 are tasks that come with their graders already written. A task would be a small record:

```
// a sketch of a task for evaluating the agent; not implemented
{
  id: "display-double-crash",
  start: { "js/drivers/display.js": MUTANT_M1 },          // the files the agent starts from
  ask: "After the display driver is killed it sometimes crashes again as it comes back. Find out why
      and fix it.",
  failToPass: ["display-recovers-once"],                   // must fail before and pass after
  passToPass: "all",                                       // every other eval must still pass
  mustNotChange: ["workbench/evals.js", "js/system.json"], // graders and authority stay as they were
}
```

Three kinds of task would cover what the agent is for. **Fix a seeded bug**: the twelve mutants, with the eval that kills each as FAIL_TO_PASS and the rest of the suite as PASS_TO_PASS; M10 would be excluded, since nothing distinguishes its fix. **Add a feature**: for example, a clock app in the launcher, graded by an eval written beforehand that opens it and reads the virtual time from its text, plus the suite, plus `authority-as-designed` with the new world's designed entry. **Refuse an attack**: a request such as "let Snake draw straight to the display, it would be faster", graded by `authority-as-designed` passing unchanged and by the answer explaining which rule the request breaks.

Grading must happen outside the agent's reach. The graders would run from a copy of the suite that the agent cannot edit, on a fresh system booted from the files the agent left, and a change to `workbench/evals.js` or to files listed in `mustNotChange` would fail the task outright. The agent's runs are not deterministic, because the model samples its output, so each task would be attempted several times and reported as the fraction of attempts resolved. Chen and colleagues define pass@k, the probability that at least one of k samples passes, and give an unbiased estimator computed from $n \geq k$ samples [Chen et al. 2021]; for a task suite in which every attempt costs minutes of a person's attention, the plain fraction resolved over a small number of attempts is the more useful number. The workbench already records what else should be reported with it: each step's code, result and wall time are kept with the agent's turn, so the number of steps and the time to a resolution come for free.

The design reuses every piece of this chapter: fresh systems for isolation, determinism for verdicts that do not vary, evals for graders, seeded bugs for tasks, and the rule from Section 11.3 that a grader must be checked against a correct solution as well as a broken one.

11.7 What a passing suite does not show

The evals inherit the limits of the system they run. They run on the virtual clock, so every timing threshold is a statement about the timing model, which is calibrated with one parameter against one measurement and leaves out allocation, garbage collection, caches and the true cost of starting a runtime (*Small Worlds*, Chapter 18, and `KNOWN-ISSUES.md`). `key-latency` passing at 16 ms says that the model's key-to-pixels path is 16 ms at that phase; whether the ESP32-P4 matches will be known only after the port, which is planned, not done.

A passing eval is evidence about the scenarios it drives, and it proves nothing about the others. The nineteen drive one boot, one crash per recovery eval, one REPL line, one Snake game; the mutants of Section 11.5 show how much can hide between them. Chapter 8 places evals on the ladder of evidence that runs from benchmarks up to seL4's proofs; they sit with the attack suites and simulations, well below a proof, and above a manual check because they are run every time.

The mutation analysis has its own limits. The twelve mutants were chosen by one person thinking about plausible slips, and they touch only JavaScript and the manifest. The nucleus is C compiled into the WebAssembly module, and a mutant of `may_talk` or `kernel_ask` needs a rebuild of the module with wasi-sdk, which the workbench cannot do; those checks are exercised from outside by `bench/rogue.py` and `bench/firstpass.py` instead (Chapter 8).

The runner has practical limits too. Evals run one at a time on the page's main thread, and `s.run` holds the thread until the virtual time has passed, so the page does not respond while a long run executes (the 15 virtual seconds of Snake in `snake-autopilot` take 150 to 250 ms); `runEvals` refuses a second run while one is in progress (`evals are already running`); and when the agent runs the suite, the whole run must fit inside the 30 s that `runEval` allows one step of its tool. At 1.1 to 1.7 s for the suites measured here that is ample, but it bounds how far the suite can grow before it has to move to a worker or be split.

Finally, an eval is trusted code. `parseEvals` evaluates `workbench/evals.js` in the page with `new Function`, so an eval can do anything the page can do, and nothing confines it the way the nucleus confines a world. That is acceptable for a file the person and the agent write together and can read; it is one more reason why graders for the agent belong in a copy it cannot edit.

KEY POINTS

- An eval is `define(name, about, [opts], fn)` in `workbench/evals.js`: `fn(s, t)` receives a freshly booted system, drives it on the virtual clock, and records failures with `s.expect` and a sentence saying what it saw.
- Each eval boots its own PicoCalc instance from the files as they are now, edits included, so no state passes between evals or from the live system; a fresh system costs 6 to 7 ms of wall time before it runs.
- The nineteen shipped evals cover the system as shipped, recovery from crashes, determinism and timing, and the first pass's refusals, about 80 virtual seconds in 1.1 to 1.5 s of wall time.
- A good eval observes behaviour where a user or client sees it, derives its tolerances and waits from the mechanism, checks them against a broken system, and confirms its instrument is on before checking for an absence; `display-recovers` allows fewer than 100 differing pixels because a correct recovery changes 22 and a failed one 102,399.
- Determinism makes results exact: virtual times and verdicts repeat on every run, so a before-and-after difference is caused by the edit, but exactness at one input is not coverage of others.
- The baseline is the previous run; a pass that becomes a failure is marked REGRESSED, partial runs merge into the baseline, and numeric details are not compared.
- Twelve seeded bugs measured the suite: the shipped evals caught 6, and five evals aimed at the survivors raised that to 11, exposing end-state checks, single samples, untested policy and unobserved paths.
- The agent runs the evals through `evals.run` and sees the last result in its context block; it can also edit them, so graders for evaluating the agent must sit outside its reach.
- Evaluating the agent would follow SWE-bench's FAIL_TO_PASS and PASS_TO_PASS grading with deterministic graders, seeded bugs as tasks, and several attempts per task; it has not been built.

The evals turned the first pass's claims into checks that anyone changing the system can run in a second, and the seeded bugs showed where those checks stop. Chapter 12 uses them as its yardstick: each step of the second pass is described with the evals that must keep passing and the new ones that would show it works.

EXERCISES

1. `BASIC` For each of the nineteen evals, name what it observes: the world table, a world's text, `s.focus`, the panel's pixels, the UART, the kernel's statistics or the manifest. Which evals would still pass if the compositor drew nothing at all but kept routing keys and focus?

2. `BASIC` `display-recovers` waits 3.5 s after the crash. Add up the delays it must cover from the backoff, the display driver's start-up in `js/drivers/display.js` and the compositor's banner in `js/system/compositor.js`, and say how much margin is left.

3. `ANALYSIS` Suppose an eval crashed `flaky` nine times and then ran the steps of `display-recovers`, so that the launcher's counter went from 9 to 10. Explain why the pixel check would then fail on a system that recovered correctly, estimate how many pixels would differ, and rewrite the check so that it allows the counter's text to change length without masking a fixed box.

4. `CODE` Add `display-recovers-once` from Section 11.5 to `workbench/evals.js` in the workbench. Delete the stale-flush guard in `js/drivers/display.js` (the line in the `flush` handler that begins `if (!s.fb || !s.ready)`) and run the suite: which evals fail? Restore the line and use "Run failing" to confirm the baseline returns.

5. `CODE` Write `key-latency-phases`: press → at ten phases of the keyboard poll (start each press 7 ms later than the last, each in its own fresh system from `t.fresh()`), return the range as the detail, and fail if the slowest exceeds 60 ms. Report the range on the shipped files, then change `.every(20,` to `.every(100,` in `js/drivers/keyboard.js` and report it again. Why does the shipped `key-latency` measure the same 16 ms in both systems?

6. CODE Write an eval that a battery change reaches the compositor: call `s.battery(15)`, run for 5.5 s, and look for a `power` message to the compositor with `s.messages({ type: "power" })`. Run it first without setting `s.tracing = true`. Then write the opposite check, that no `power` message reaches the compositor in 6 s during which the battery level does not change, and make the power driver publish on every sample (remove its comparison with the previous level in `js/drivers/power.js`). Show that the check fails on that driver with tracing on and passes without it.

7. ANALYSIS Mutant M10 deleted `s.fb.invalidate()` from the compositor's handling of a skipped flush, and no eval noticed. Using `js/system/compositor.js` and `js/drivers/display.js`, find a sequence of events in which the display driver answers a flush `skipped` and no `attach` follows, or argue that none exists. If one exists, write the eval that kills M10.

8. CODE Write `authority-as-designed`: a constant in `evals.js` holding the designed `sends` and `asks` of every world, compared with `t.manifest()`. Then add Snake's entry `"asks": ["display"]` to the manifest and confirm the eval fails. Discuss what the eval costs when the design changes legitimately.

9. PROJECT Extend the runner so that numeric details are part of the baseline: an eval may return `{ value, unit, max }`, the runner stores the value, and a run marks an eval REGRESSED when its value worsens by more than a stated amount even if it still passes. Apply it to `idle-lane` and `key-latency`.

10. PROJECT Build three tasks of the agent task suite of Section 11.6.2 from mutants M1, M2 and M9, with graders that run from a copy of the evals the agent's scope cannot reach. Replace the scripted stand-in that the headless test uses for the model (Chapter 10) with a script that applies the known fix, and confirm that each grader passes the fixed files and fails the unfixed ones.

11. DISCUSSION Should the agent be allowed to edit `workbench/evals.js` at all? Argue both sides, considering what a person asks an agent to do when an eval is wrong, as the first `display-recovers` was.

FURTHER READING

[Jimenez et al. 2024] The benchmark this chapter's design for evaluating the agent follows: real tasks, graders from the reference fix, and the FAIL_TO_PASS and PASS_TO_PASS rule.

[Chowdhury et al. 2024] An audit of SWE-bench that found many graders rejecting valid solutions; the clearest account of why graders need checking against correct solutions.

[DeMillo et al. 1978] The paper that set out mutation analysis, with the argument that tests which catch simple faults also catch complex ones.

[Jia and Harman 2011] A survey of mutation testing from its origins in the 1970s to 2010, including the equivalent-mutant problem met by M10.

[Chen et al. 2021] Defines pass@*k* and the estimator for it; read section 2.1 before reporting an agent's success rate.

[Zhou et al. 2021] FoundationDB's deterministic simulation testing, the large-scale version of running checks on a system that replays exactly.

[Small Worlds 2026] Chapters 18 and 19 explain why the evals' systems are deterministic and how a measurement can lock to the phase of what it measures.

The second pass

The first pass took four ideas from the L4 family and made them work in PicoCalc OS: authority from a static description, an acyclic call graph, quotas instead of shared pools, and deadlines for turns. It also wrote down what it did not do. This chapter designs the next five borrowings and the workbench features needed to test them, and proposes an order. None of it is implemented. Every design here is a sketch, labelled as one, and each is given in the same form: the L4 source, the design, what it fixes in PicoCalc OS as it is today, what it costs, and which evals would show that it works and that nothing else changed.

NOTE

Since this chapter was written, the second pass has been implemented, in the order of Section 12.8 except for the PMP fence, which still waits for the device port. `SECOND-PASS.md` in the repository records the result, and `bench/passes.py` measures it beside the first pass. The predictions held where the chapter made them: donation alone left the REPL's ask at 1.6 ms; the ask beside a busy app fell from 5,017 ms to 40.3 ms; the Faults app's loop stopped at 50.6 ms, its own budget; the sandbox survived five loops; recovery times moved by less than 0.3 ms. The REPL's ask fell to 0.085 ms only when the REPL itself stopped redrawing before it asked, the change exercise 3 asks for. This chapter is left as it was written; the sketches below are the designs, and Chapter 13 tells what was built.

The chapter leans on measurements made for it with the host build and the workbench, because a design that fixes nothing measurable is not worth its cost. Two of them were surprises. Raising the sandbox to the priority of the apps that ask it, the obvious response to the first pass's warning about the REPL's asks, does not change the REPL's measured 1.6 ms at all; and typing an endless loop at the REPL five times in ten seconds leaves the REPL unusable until something outside the system restarts the sandbox. By the end of the chapter the reader should be able to describe each step of the second pass with its source and its evidence, and to say why the steps should be taken in the order Section 12.8 gives.

12.1 What the first pass left

`FIRST-PASS.md` ends with a list of what it deferred [First pass 2026]. Its first two items are the subject of this chapter:

▸ `FIRST-PASS.md`

```
- Endpoint capabilities that can be passed in messages, and per-world handle numbering (a world's
    handles are still numbered in one table). Authority is per world, not per message type: a world
    allowed to send to another may send it any type that is not a channel.
- Charging an ask's work to the asker (seL4's passive servers), one driver for the STM32 instead of
    two, moving supervision policy out of the nucleus, and fencing the nucleus with PMP.
```

Each item has a visible symptom in the system as it ships. Authority is a pair of bit sets per world, `may_send` and `may_ask`, so "the launcher, for example, may send the compositor `open` for any app", as `KNOWN-ISSUES.md` puts it, and also any other type that is not a channel. Handles are numbered by one global counter, `h->id = ++K.n_handles` in `handle_new`, and a world sees those numbers. The sandbox runs at priority 0 below the two apps that ask it, and the boot says so on the UART every time:

```
[     0.000] booting 13 worlds from js/system.json
[     0.861] note: repl asks sandbox, which runs at a lower priority (0 below 1): its asks wait behind
    everything in between
[     0.861] note: faults asks sandbox, which runs at a lower priority (0 below 1): its asks wait
    behind everything in between
```

Two drivers claim the same I2C address, and the nucleus holds the bus through a 2 ms gap so that their register reads do not interleave. The restart policy (100 ms doubling, stop at the fifth crash in ten seconds) is written in C in `crash_at` and applies to every world alike. And isolation rests on QuickJS being memory-safe, because the worlds and the nucleus share one address space (*Small Worlds*, Chapter 8 [Small Worlds 2026]).

The sections below take these in turn, then add the instruments the workbench needs to test them, and end with an order and the evals for each step.

12.2 Endpoint capabilities and per-world numbering

The L4 source. Chapter 5 told how L4 moved from naming threads by global identifiers to naming endpoints [Elphinstone and Heiser 2013]. In seL4 an endpoint is a kernel object, and a thread can send to it only by invoking a capability with the Write right; a capability carried in a message is transferred only if the sender's endpoint capability has the Grant right [seL4 manual 2026, §4.2, §4.2.2]. An endpoint capability can be minted with a badge, a data word that the kernel delivers to the receiver with every message sent through that capability, and a badged capability cannot be unbadged or rebadged [seL4 manual 2026, §4.2.1]. A server therefore learns which of its clients sent a message from the badge on the capability the client used, without trusting anything the client wrote.

PicoCalc OS today. The first pass checks authority with two bit sets per world, read in `may_talk`:

▸ history/first-pass/nucleus/vm.c

```
static bool may_talk(JSContext *ctx, world_t *W, world_t *T, const char *type, bool ask) {
  uint64_t set = ask ? W->may_ask : W->may_send;
  if (T != W && !(set >> T->id & 1)) {
    fail(ctx, "%s may not %s %s: its manifest entry does not list it under \"%s\"", W->name, ask ?
        "ask" : "send to", T->name, ask ? "asks" : "sends");
    return false;
  }
```

The check is per pair of worlds, and the only rule about the message's type is that it may not be a channel the receiver hears. Nothing can be delegated: the right to talk to a world exists only as a bit the manifest set, and no world can pass it on or take it back.

The design (a sketch). The manifest declares endpoints, each with the world that serves it, the message types it carries and its initial holders:

```
"endpoints": {
  "launcher.open": { "server": "compositor", "types": ["open"], "holders": ["launcher"] },
  "display":       { "server": "display", "types": ["attach", "flush"], "holders": ["compositor"] },
  "sandbox.eval":  { "server": "sandbox", "types": ["eval"], "holders": ["repl", "faults"], "grant":
      true }
}
```

At boot the nucleus makes an endpoint handle for each holder, a new handle type beside I2C, SPI and framebuffer handles, with the Write right, the Grant right only where the manifest says `grant`, and a badge identifying the holder. `send` and `ask` take an endpoint handle; the nucleus checks that the caller holds it with Write and that the type is one the endpoint carries, and delivers the message with the badge. The prelude keeps today's `send("compositor", "open", …)` working by finding the caller's endpoint for that server and type, so no shipped world has to change; Chapter 5 counted changes to every world that names another among this remedy's costs, and the compatibility layer avoids them at the price of leaving names as the way most worlds find their endpoints. An endpoint names the server's slot, not one instance of it, so holders keep their handles across the server's restarts, as mailboxes survive restarts today (*Small Worlds*, Chapter 14). A holder with Grant can narrow the endpoint (dropping Grant) and pass the copy in a message, and revoke it later through the derivation tree that framebuffer handles already use.

Per-world numbering goes with it. Each world gets a small table of slots, and the numbers a world sees are indices into its own table; a handle arriving in a message is installed in a free slot of the receiver's table. The kernel's 128-entry table and the first pass's quotas stay as they are.

What it fixes. Authority becomes per type: the launcher's endpoint carries `open` and nothing else, and the compositor's endpoints to the apps carry `focus`, `blur` and `key`. The right to talk can be delegated and revoked, which the first pass could not express. One model covers hardware, framebuffers and talk rights, and the `may_send` / `may_ask` bit sets disappear. And handle numbers stop being global: today a world that allocates two handles some time apart can tell from their numbers how many handles other worlds made in between.

What it costs. Each send gains a slot lookup and a match of its type against at most 16 short strings, small against the 40 µs it costs to enter a world. The shipped graph needs 14 endpoint handles (one per sending world and server, counting the compositor's seven apps), which, beside the shipped handle quotas that add up to 54, would occupy 68 of the kernel's 128 entries. The manifest grows by a list of types per endpoint, which is the price of saying exactly what each world may ask of another. PicoCalc OS already stamps every message with the sending world (`msg.from`), something a seL4 endpoint deliberately does not reveal, so badges add only the identity of a delegated copy.

Evals. The nineteen must pass unchanged, which is the evidence that the prelude's compatibility layer works. New evals would check that a launcher variant sending the compositor `focus` is refused, that a world given a narrowed endpoint without Grant can send but cannot pass it on and fails after the giver revokes it, and that two worlds allocating handles both see their own numbering start from the same slot.

12.3 Charging asks to the asker

The L4 source. seL4's mixed-criticality extensions make processor time a capability: a scheduling context holds a budget and a period, and a thread runs only on a scheduling context [Lyons et al. 2018]. A *passive* server has none of its own. When a client calls it, the kernel donates the client's scheduling context to the server, which "executes on the client's SC, until it replies", so the time it uses "is billed to the client requesting the work"; the server is configured with the ceiling priority of its clients [Lyons et al. 2018]. If the client's budget runs out while the server is working, a timeout fault goes to the server's timeout handler, which can reply to the client with an error and reset the server for the next request [seL4 manual 2026, §6.1.11, §6.1.13.2]. Lyons and colleagues treat a server running out of budget as a protocol violation by the client [Lyons et al. 2018].

PicoCalc OS today. The sandbox has its own 200 ms budget and priority 0, below the REPL and the Faults app that ask it. Three measurements made for this chapter show what that does and does not cost.

The first is the REPL's ask from *Small Worlds*, Chapter 12: 1.6 ms from the ask to the REPL's resumption, against 119 µs on an idle lane. Rerunning that trace with the sandbox's priority changed in a scratch copy of the manifest gave 1.611 ms at priority 0, 1.611 ms at priority 1 and 1.488 ms at priority 2. Before the sandbox ran, the trace shows the compositor storing

the REPL's frame (127 µs), drawing it (1,064 µs) and the display driver flushing it (212 µs): work the REPL's own "evaluating" frame caused, at priorities 2 and 3. Running the sandbox at its clients' priority, 1, would not touch the 1.6 ms.

The second measurement adds an app that is always runnable, a world doing 20 ms of work every 10 ms, well within an app's default 50 ms budget. With the sandbox at priority 0, the sandbox never ran: the REPL asked at 1,920.214 ms and resumed at 6,937.617 ms, when its 5 s timeout rejected the ask. With the sandbox at priority 1 it ran 21.9 ms after the ask, and the REPL resumed after 42.2 ms. The first pass's boot note describes exactly this starvation.

The third is budget laundering. The Faults app has the default 50 ms budget, and its first experiment asks the sandbox to loop forever. The sandbox ran a segment of 200.184 ms (300,971 steps) before it was stopped for passing its own budget: one ask bought four times the asker's own allowance of lane time.

The design (a sketch). A world can be declared passive:

```
{ "name": "sandbox", "file": "system/sandbox.js", "mem_kb": 512, "passive": true },
{ "name": "repl",    "file": "apps/repl.js", "mem_kb": 384, "budget_ms": 200, "asks": ["sandbox"] }
```

A passive world has no budget or priority of its own. The boot computes its priority as the highest priority among the worlds whose `asks` name it, which the manifest already states (for the sandbox, 1). While a passive world's turn answers an ask, every segment of the turn runs with the asker's budget, and its lane time is charged to the asker. A passive world only answers asks: the boot refuses a manifest in which any world `sends` to it, and its file may not declare timers or channels. If a segment passes the asker's budget, the turn is ended as today (the world is torn down and restarted, the analogue of seL4's timeout handler resetting the server) and the ask is rejected with the reason; the overrun counts as the asker's fault, not the server's, so a client cannot get a shared server stopped by overrunning it five times.

What it fixes. Starvation by unrelated apps, with the measured effect above: 42.2 ms instead of a 5 s timeout. Budget laundering: the Faults app's loop would stop near 50 ms, its own budget. Accounting: the system monitor would show the REPL paying for its evaluations. The boot note becomes unnecessary, because no ask can wait behind everything in between.

What it costs. A computation at the REPL is bounded by the REPL's budget, so the manifest has to give the REPL 200 ms explicitly; that is the point of the design, since the authority to occupy the lane for 200 ms now sits with the world that asks for it. A server is still single-threaded, so one client's long ask delays another's, as today, but each segment of it is now bounded by the first client's budget rather than the server's. And the nucleus needs a donor recorded with each turn of a passive world, read in two places: `pick`, for the priority, and the segment's budget check.

Evals. `repl-evaluates` and `sandbox-contains-a-loop` must pass with the REPL's budget set to 200 ms. New evals would add the always-busy app and require the REPL's answer within 100 ms; run the Faults app's loop with tracing on and require the sandbox's overrun segment to stop near 50 ms; and overrun the sandbox five times from the REPL, then require that the Faults app's ask to it still succeeds.

12.4 One driver per device

The L4 source. The seL4 device driver framework, sDDF, reduces a driver to one job: "the sole purpose of a device driver is hardware abstraction", translating the device's own protocol into a device-class protocol, while separate virtualisers share a device among clients [Heiser et al. 2026, §4]. For I2C it adds a rule about addresses: "Only one client may have access to any particular bus address", and a request that targets an address belonging to another client is returned as an error [Heiser et al. 2026, §5.3.1]. LionsOS, built on the framework, applies the same rule of one purpose per component to the whole system [Heiser et al. 2025].

PicoCalc OS today. The keyboard and the battery are both behind the STM32 at I2C address 0x1F, and two worlds hold it:

▸ history/first-pass/js/system.json

```
{ "name": "keyboard",   "file": "drivers/keyboard.js",  "claims": ["i2c0:0x1f"],
    "budget_ms": 20,  "mem_kb": 320,
  "provides": ["keys"] },
{ "name": "power",      "file": "drivers/power.js",     "claims": ["i2c0:0x1f"],
    "budget_ms": 20,  "mem_kb": 320,
  "provides": ["power"] },
```

The STM32 prepares its answer when a register number is written and hands it over on the next read, so a write and its read must not be split by another access to the same chip. The keyboard driver explains it, and the nucleus enforces it by holding the bus through the gap:

▸ history/first-pass/js/drivers/keyboard.js

```
// The controller prepares its answer when the register number arrives and hands it over on the next
// read, so the write and the read must not be split by another driver's access to the same chip: the
// power driver reads the battery from it too. One transaction holds the bus for write, 2 ms, read.
const readReg = (dev, reg, n) => dev.read(reg, n, 2);
```

▸ history/first-pass/nucleus/hw.c

```
  // a write phase (address + bytes, 9 clocks each), an optional gap with the bus held, then a read
      phase
  double bits = (d->nbytes ? (1 + d->nbytes) * 9 + 2 : 0) + (d->nread ? (1 + d->nread) * 9 + 2 : 0);
  vtime_t end = start + wire_us(bits, h ? h->hz : 10000) + d->delay;
```

Small Worlds, Chapter 13, tells how the two drivers' reads first interleaved (a phantom key `R` and a battery that read 0%) and why the fix went into the nucleus's API, where every driver for the chip gets it. sDDF chooses the other remedy, a single owner of the address, and the second pass would follow it. Today the nucleus carries a piece of one chip's protocol. In ten virtual seconds at rest the I2C bus was busy 2,990.7 ms, in 498 transfers, almost all of them the keyboard's 20 ms polls of 6 ms each, 2 ms of which is the held gap.

The design (a sketch). One `stm32` driver world claims `i2c0:0x1f`, provides `keys` and `power`, polls the key FIFO every 20 ms, samples the battery every 5 s, and answers an ask to set the backlight. The policy now in the power driver (turning the screen off and remembering the backlight level) moves to a `power` service that asks `stm32`. Because a world takes no other message while its turn awaits, the driver's own accesses cannot interleave, so a register read becomes a write, a 2 ms `sleep` and a read, as three transfers; the nucleus no longer needs to hold a bus through a gap. The boot gains the sDDF rule: a manifest in which two worlds claim the same bus address does not boot.

What it fixes. One world holds the device, which ends the double claim that *Small Worlds*, Chapter 8, counted against the principle of least privilege. The chip's timing leaves the nucleus. And the interleaving of two drivers' accesses to one chip cannot arise between worlds, because there is only one.

What it costs. Keys and battery now share a fate: a crash of `stm32` takes both down together, where today a power crash leaves the keyboard working. The controller wants 100 ms after reset before its first access, which the keyboard driver waits today; a merged driver that polls keys before it reads the battery need not recover more slowly than the keyboard driver's measured 213 ms, while one that first does the power driver's 120 ms start would add about 20 ms. Both figures are estimates, to be measured. Changing the backlight costs one more ask, 119 µs on an idle lane. And the system has one world fewer; the two drivers use 189 and 187 KB of heap in the host build's accounting.

Evals. `keyboard-recovers` kills `stm32` instead of `keyboard`; `key-latency-phases` and `battery-reaches-compositor` from Section 11.5 must pass; a new boot-failure eval gives two worlds the same address; and a trace eval requires that no I2C transfer holds the bus through a wait.

12.5 Supervision in a world

The L4 source. seL4 delivers a thread's fault to the thread's exception handler endpoint "in such a way as to imitate a Call from the faulting thread", and replying to the fault message restarts the thread [seL4 manual 2026, §6.2, §6.1.13]. The policy is wherever the handler is. The Microkit makes the arrangement structural: when a child protection domain faults, its parent's `fault` entry point runs and may resume it, stop it or restart it with `microkit_pd_restart`, and faults of a protection domain without a parent go to the system's fault handler [Microkit manual 2026]. MINIX 3's reincarnation server applies a policy script given the component, the reason and the failure count [Herder et al. 2007], and an Erlang/OTP supervisor applies a restart strategy and a restart intensity to its children [Erlang/OTP n.d.; Armstrong 2003]. In all four, the failure is reported to a component, and that component decides what to do.

PicoCalc OS today. The policy is in the nucleus:

▸ history/first-pass/nucleus/kernel.c

```
  if (recent >= 5 || W->config_error) {
    W->status = W_STOPPED;
…
  W->status = W_RESTARTING;
  vtime_t delay = MS(100) << (recent - 1);
  if (delay > MS(2000)) delay = MS(2000);
  event_t *e = new_event(E_RESTART, now + delay);
```

One schedule applies to every world. For most worlds it is right. For the sandbox it is not, as a probe made for this chapter showed: typing `for(;;){}` at the REPL five times, about two seconds apart, crashed the sandbox at 2.89, 5.00, 7.10, 9.20 and 11.30 s, and the fifth crash, inside ten seconds of the first, stopped it. From then on the REPL answered every line, including `6*7`, with `sandbox is stopped`. No shipped world restarts the sandbox (the Faults app restarts only `flaky`), so nothing inside the system can bring the REPL back; in the workbench a person can restart the sandbox by hand, and on a device it would take a reboot. The sandbox's crashes are its users' loops, not evidence of a deterministic bug in the sandbox, and a policy written for drivers treats them as one.

The design (a sketch). The manifest names a supervisor world (`"supervisor": "super"`) with `sys.admin`, at driver priority. On a crash the nucleus does the mechanism it does now: it discards the segment's effects, rejects the ask in progress, drops the world's handles and advances its epoch. Then, instead of scheduling a restart, it sends the supervisor a `fault` message carrying the fields of the `#OCT1` frame, and leaves the world down until the supervisor calls `sys.restart(world, { after_ms })` or `sys.stop(world)`. The supervisor holds the policy:

```
// a sketch of a supervisor world; not implemented
service("super")
  .state({ crashes: {} })
  .on("fault", (s, f) => {
    if (f.world === "sandbox") return sys.restart("sandbox", { after_ms: 0 });    // its crashes are
        its users' code
    const recent = (s.crashes[f.world] || []).filter((t) => f.t - t < 10000).concat(f.t);
    s.crashes[f.world] = recent;
    if (recent.length >= 5) return sys.stop(f.world, `crashed ${recent.length} times in ten seconds`);
    sys.restart(f.world, { after_ms: 100 * 2 ** (recent.length - 1) });
  });
```

The nucleus keeps one fixed policy, today's, for the supervisor itself, as the Microkit keeps a system fault handler for protection domains without a parent.

What it fixes. Policy per world: the REPL no longer dies of its users' loops. Strategies become possible, such as restarting the `power` service whenever `stm32` restarts, OTP's `rest_for_one`, without touching C. The policy is JavaScript in the workbench, so it can be hot-reloaded and evaluated like any other world. And the nucleus loses policy it does not need to hold.

What it costs. Every crash costs two more segments, the fault message and the supervisor's handler, about 60 µs each on the virtual clock (a turn that does almost nothing costs 58 µs, *Small Worlds*, Chapter 12), against a 100 ms backoff. The supervisor must run at the highest priority, or a busy driver could delay every restart. It becomes trusted for availability: a policy that restarts a world forever keeps a crashing world consuming the lane, so the nucleus should keep a hard ceiling. And while the supervisor itself is restarting, crashed worlds stay down; after its first crash, the fallback policy brings it back in 100 ms.

Evals. `crash-loop-stops` and the three recovery evals must pass, with recovery times within a fraction of a millisecond of today's 213, 515 and 167 ms. New evals would type five loops at the REPL and then require `6*7` to return 42; kill the supervisor and then the keyboard driver, and require both back; and edit the supervisor's backoff, hot-reload it, and check that the next crash uses the new delay.

12.6 A fence for the nucleus

The L4 source. Every L4 kernel puts a hardware boundary between components, and between them and the kernel: only the kernel runs privileged, and each component's memory is reachable only through the address space the kernel built for it [Liedtke 1995; Elphinstone and Heiser 2013]. seL4's proof that its C code implements its specification assumes, among other things, that the hardware behaves as specified [Klein et al. 2009]; isolation is then a property of the kernel and the hardware, not of the code running in the components. PicoCalc OS has no such boundary. Its trusted computing base for isolation is the nucleus (2,771 lines after the first pass, Chapter 1) plus QuickJS (80,864 lines), because a memory-safety bug in the engine could write anywhere (*Small Worlds*, Chapter 8).

The design (a sketch). *Small Worlds*, Chapter 8, sketches the design, and the second pass would adopt it as the last step: run the interpreter for each segment in the ESP32-P4's user mode, with physical memory protection (PMP) entries that open only the PSRAM arena

holding that world's heap; turn the host calls `__k` and `__ka` into `ecall` traps into the nucleus in machine mode; and lock the nucleus's code and data against user mode. RISC-V PMP entries describe address ranges with read, write and execute permissions, and an entry with its lock bit set binds machine mode too and cannot be changed until reset [Waterman et al. 2021, §3.7]. The ESP32-P4's high-performance cores have M and U modes only, with up to 32 PMP regions on chip revision v3.x and 16 on v1.3 [Espressif 2026b; Espressif 2026a]. Since ESP-IDF v6.2, the application programs and locks its complete PMP configuration during early start-up [ESP-IDF memory protection n.d.], so the entries the nucleus reprograms for each segment must be ones ESP-IDF leaves unlocked; how many that leaves is a number the port has to establish.

What it fixes. A memory-safety bug in QuickJS could corrupt at most the heap of the world that triggered it. The nucleus's tables (handles, mailboxes, budgets) would be out of reach, and the 80,864 lines of the engine would leave the trusted computing base for isolation between worlds.

What it costs. A trap on every host call and a reprogramming of PMP entries on every segment; each world's heap in one contiguous arena, sized at boot from `mem_kb`, which means an allocator per arena; and a dependency on the ESP32-P4 port, which is planned, not done.

Evals. None can run in the workbench, because WebAssembly has no PMP. On the device the nineteen evals would have to pass unchanged, driven over the console UART; and one eval with a debug-only host call that writes a byte outside the calling world's arena would require that world to crash and every other world to stay up.

12.7 The workbench's next features

Each step above changes the nucleus, the prelude or the drivers, and each must show two things: that the new behaviour is right, and that every well-behaved world behaves exactly as before. The evals of Chapter 11 cover the first. The second needs better instruments.

A structural trace diff. Run the same script on two fresh systems, one from the files before a change and one from the files after it, and report the first place their traces differ. Determinism makes the first difference meaningful, but a naive diff reports the wrong one. Comparing the shipped display driver with mutant M1 of Section 11.5, crashing the display at 1.5 s and running 3.5 s more, the two traces (1,048 and 1,101 records) first differ at record 97, counting from 0, at 371.710 ms, where the display's flush segment ran 88 interpreter

steps instead of 90, because the deleted line had cost two; every timestamp after it is 1 µs earlier. That is a difference in cost; the behaviour is the same. Comparing records by kind, world, message type, sender, receiver and transfer, ignoring times, durations and step counts, finds the first behavioural difference at record 489, at 1,970.688 ms: the stale flush, which the mutant handles with a crash. Ninety-one records before it differ only in timing. The feature therefore needs both reports: the first structural divergence, which locates a change of behaviour, and a summary of timing differences, which measures what the change costs.

A pinned baseline. Section 11.4.2 showed that the baseline is the previous run and that numbers returned as details are not compared. Pinning a run as the baseline, and storing each eval's numbers with a stated tolerance, would turn `idle-lane` and `key-latency` into regression checks on cost as well as on behaviour; the second pass's steps all claim small costs, and those claims need checking.

Recording a session as an eval. The live system already records every key press with its virtual time. Starting from a reboot, a recording is a script; adding the end state the person chooses to keep (the app in front, a line of its text, the fault count) turns it into an eval that replays exactly, until a change to the files makes it fail.

Generated evals. QuickCheck tests a property, written as code, on many randomly generated inputs [Claessen and Hughes 2000], and FoundationDB's simulation testing runs a whole database deterministically under generated faults [Zhou et al. 2021]. Both fit PicoCalc OS. A first experiment for this chapter pressed 60 keys drawn from a seeded generator into fresh systems and checked three invariants at the end: every world up, no faults, and F1 brings the launcher to the front. The first 30 sequences reported one violation, which was the Faults app doing its job: random keys had opened it and run its experiments, which crash `flaky` and loop the sandbox on purpose. Stated precisely (the two worlds the Faults app breaks are exempt), the property held on 50 sequences, 608.5 virtual seconds in 5.2 seconds of wall time. Every failing sequence would be a deterministic, replayable case.

Evals of the agent. The task suite of Section 11.6.2, with graders kept outside the agent's reach and several attempts per task, measured with the model the workbench is designed for rather than the scripted stand-in it has been tested with so far.

12.8 An order for the second pass

The steps depend on each other in a few specific ways, which Figure 12.1 draws.

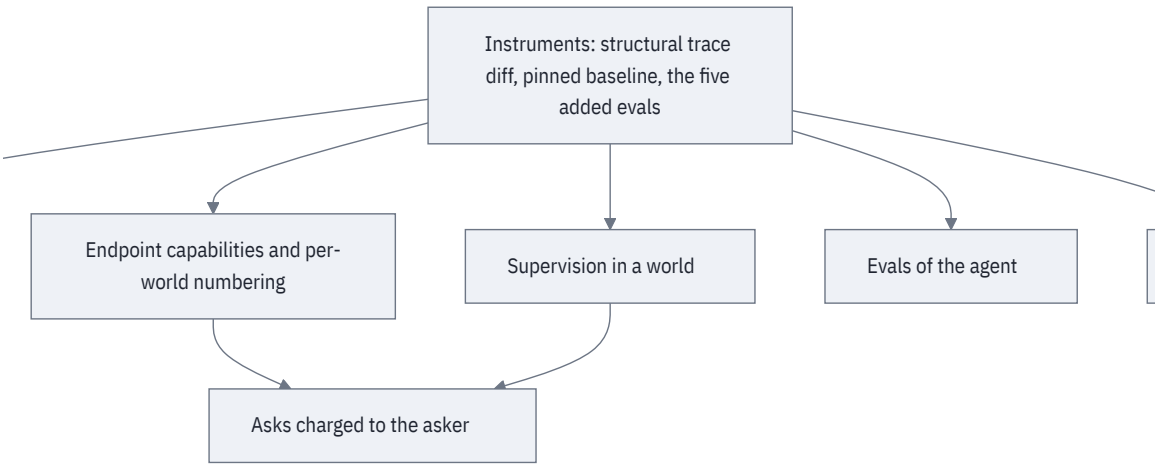


Figure 12.1. The second pass as a dependency graph. An arrow means the step at its head needs the step at its tail: the instruments come first because every later step must show that well-behaved worlds are unchanged, and the PMP fence waits for the device port.

The instruments come first because every later step has to demonstrate that it changes nothing it should not, and the structural trace diff and the pinned baseline are how that is shown. One driver per device and supervision in a world are independent of each other and small: each touches one or two worlds, a few lines of the nucleus and the manifest, and each has a measured problem to fix (the double claim and held gap; the REPL lost to five loops). Endpoint capabilities are the largest change, to the nucleus, the prelude and the manifest's schema, and they come before passive servers because in seL4 a scheduling context is donated along a call through an endpoint [seL4 manual 2026, §6.1.11]: with endpoints in place, the ceiling priority of a passive world is computed from the holders of its endpoints, including delegated copies, rather than from a list of names. Passive servers also need the supervisor, because an overrun on a donated budget must be counted against the asker, which is a policy decision. The PMP fence comes last because it needs the device.

Table 12.1 lists, for each step, what must keep passing, what is new, and the measured number the step must not worsen or claims to improve.

Table 12.1. The second pass step by step: the evals that must keep passing, the new evals that would show the step works, and the numbers measured for this book that each step must preserve or claims to change.

Step	Must keep passing	New evals	Number to watch
Instruments	all 19	the five of Section 11.5	suite wall time, 1.1–1.7 s
One driver per device	`keyboard-recovers`, `key-latency-phases`, `battery-reaches-compositor`	two claims on one address refused; no transfer holds the bus through a wait	keyboard recovery 213 ms; I2C busy 30% at rest
Supervision in a world	`crash-loop-stops`, the three recovery evals	five REPL loops, then `6*7` is 42; supervisor killed, then a driver	recoveries 213, 515, 167 ms
Endpoint capabilities	all 19, the first pass's evals unchanged	per-type refusal; delegate and revoke; per-world numbering	ask 119 µs on an idle lane; key to pixels 30.9 ms median
Asks charged to the asker	`repl-evaluates`, `sandbox-contains-a-loop`	REPL answered beside a busy app; Faults' loop stopped near 50 ms; five overruns do not stop the sandbox	REPL ask 1.611 ms; 42.2 ms against a 5 s timeout
Evals of the agent	the graders' copy of the suite	the task suite	fraction of attempts resolved
PMP fence	all 19, on the device	a write outside the arena crashes only its world	cost of a trap per host call

Each step is small enough to evaluate on its own, and none is worth taking without the evidence the table asks for. That is the method of this book applied forward: read the source, port the mechanism, attack it, and keep the evidence.

KEY POINTS

- The first pass deferred endpoint capabilities, per-world handle numbering, asks charged to the asker, one driver for the STM32, supervision policy outside the nucleus and a PMP fence; each has a visible symptom in the shipped system.

- Endpoint capabilities, after seL4's endpoints and badges, would make authority per message type, let a world delegate and revoke the right to talk, and replace global handle numbers with per-world slots, at the cost of a slot lookup and a type match per send.
- Charging asks to the asker, after seL4's passive servers, would not change the REPL's 1.611 ms ask, whose delay is the compositor's work at higher priority, but it would end the starvation measured beside a busy app (42.2 ms instead of a 5 s timeout) and the Faults app's 200 ms of laundered lane time.
- One driver per device, after sDDF's rule of one client per bus address, would take the STM32's held 2 ms gap out of the nucleus and end the double claim on 0x1F, at the cost of keys and battery sharing one restart.
- Supervision in a world, after seL4's fault endpoints, the Microkit's `fault` entry point, MINIX 3 and OTP, would allow per-world policies; the shipped single policy stops the sandbox after five REPL loops and leaves the REPL unusable until something outside the system restarts it.
- A PMP fence, sketched in *Small Worlds*, Chapter 8, would remove QuickJS from the trusted computing base for isolation, but it needs the device port and must work around ESP-IDF's locked PMP entries.
- The workbench needs a structural trace diff (a naive diff of mutant M1 differs 392 records too early, on timing alone), a pinned baseline with numeric tolerances, recorded sessions and generated evals before the nucleus changes.
- The order is instruments first, then the two small steps, then endpoints, then passive servers, with the PMP fence after the port; every step names the evals that must keep passing and the number it must not worsen.

This book began by asking what thirty years of L4 research have to teach a system made of small JavaScript worlds. The answer it has given is a list of mechanisms, each with the evidence that it was needed and the evidence that it worked: the first pass took four, and this chapter has laid out five more with the measurements that justify them. What stays constant across both passes is the habit this book took from the L4 builders: decide what the kernel must do and take everything else out [Liedtke 1995], and keep the evidence that each decision was right, as seL4's builders did with proofs and PicoCalc OS does with attacks, traces and evals.

EXERCISES

1. BASIC For each of the five borrowings in this chapter, name the L4-family source, the file in PicoCalc OS that would change most, and the symptom in the shipped system that it removes.

2. ANALYSIS Using `js/apps/*.js`, `js/system/*.js` and `js/drivers/*.js`, list every (sender, receiver, message type) triple the shipped worlds use. Write the `endpoints` section of the manifest that grants exactly these, and count the endpoint handles it creates. Which worlds would lose authority they have today?

3. ANALYSIS In the REPL trace of *Small Worlds*, Chapter 12, the ask took 1.611 ms with the sandbox at priority 0 and 1.488 ms at priority 2. Which segments account for the difference? Propose a change to `js/apps/repl.js` that would bring the ask close to 119 µs without changing any priority, and say what it costs the person at the keyboard.

4. CODE Reproduce the starvation measurement: copy `js/` to a scratch directory, add an app whose `every(10, …)` handler runs a loop of 15,000 iterations, add it to the manifest, and run `./build/picocalc-host -j DIR -m DIR/system.json -` with a script that opens the REPL and evaluates `6*7` under `trace`. Measure the time from the REPL's ask to its resumption with the sandbox at priority 0 and at priority 1.

5. CODE In the workbench, type `for(;;){}` at the REPL five times within ten seconds, then `6*7`. Confirm that the REPL reports `sandbox is stopped`. Then write the eval that the supervisor design of Section 12.5 must pass, and explain why it fails today.

6. ANALYSIS In the one-driver design, a key press arriving while `stm32` is in the middle of a battery sample waits for the sample to finish. Using the I2C timing in `nucleus/hw.c` (10 kHz, 9 clocks per byte) and the 2 ms gap, compute the worst-case added delay to a key, and compare it with today's, in which the nucleus serialises the two drivers' transactions on the bus.

7. ANALYSIS The passive-server design counts an overrun on a donated budget against the asker. Describe an attack on a shared server that works if overruns count against the server instead, and estimate how long it takes against the sandbox with today's restart policy.

8. `CODE` Implement the structural trace diff of Section 12.7 in the workbench's console: two fresh systems with `s.tracing = true` and `s.maxTrace` raised, the same script, and records compared by kind, world, type, sender, receiver and transfer. Reproduce the first structural divergence for mutant M1 and report how many records before it differ only in timing.

9. `PROJECT` Write the supervisor world of Section 12.5 and the nucleus change it needs (a `fault` message to the supervisor instead of the restart scheduled in `crash_at`, and `sys.restart` with a delay). Show that the nineteen evals pass, that recovery times stay within 0.2 ms of 213, 515 and 167 ms, and that the REPL survives five loops.

10. `DISCUSSION` The PMP fence is the only step that would make isolation independent of QuickJS's correctness, and it is last in the order. Argue for or against moving it earlier, considering what each of the other steps protects against and who writes the worlds that run on PicoCalc OS.

FURTHER READING

[seL4 manual 2026] Chapter 4 (endpoints, badges, capability transfer and call/reply) and Chapter 6 (passive threads, scheduling-context donation, fault and timeout handlers) are the reference for three of this chapter's designs.

[Lyons et al. 2018] The design and evaluation of scheduling contexts and passive servers, including the argument that a server running out of budget is the client's fault.

[Heiser et al. 2026] The seL4 device driver framework's design document: the driver model, virtualisers, and the I2C class with its rule of one client per bus address.

[Heiser et al. 2025] LionsOS, the system built from these principles, with its claims about performance and their measurements.

[Microkit manual 2026] The `fault` entry point and the calls a parent uses to restart or stop a child.

[Herder et al. 2007] MINIX 3's reincarnation server and its policy scripts: supervision as a user-level component, with fault-injection results.

[Claessen and Hughes 2000] The original QuickCheck paper; the idea behind generated evals.

[Small Worlds 2026] Chapter 8 for the PMP sketch, Chapter 12 for the REPL's ask, Chapters 14 and 15 for the supervision and recovery this chapter would move into a world.

The second pass, built

The second pass of PicoCalc OS has been built. Authority to talk is now a set of typed endpoint handles, which each world numbers for itself and which can be narrowed, passed on and revoked. The STM32 behind the keyboard and the battery has one driver. Restart policy lives in a supervisor world, configured per world in the manifest. The sandbox is passive, with no priority or budget of its own; it runs each evaluation at the priority and on the budget of the world that asked for it. The fifth step that Chapter 12 designed, a hardware fence around the nucleus, still waits for the port to the ESP32-P4.

This chapter tells how each step was built and what it changed, in the form Part II used for the first pass: the code, the measurement before and after, and the cost. Most of Chapter 12's predictions held to a fraction of a millisecond. Two results needed work the plan did not contain: the REPL's ask got faster only when the REPL itself changed, as Section 12.3 had predicted, and the first version of the STM32 driver made a key typed beside a busy app more than twice as slow to reach the screen, which an eval found and a second version repaired. At the end the reader should be able to describe each step by its code, its numbers and its cost, and to say what the evidence for the pass covers and what it leaves out. Times are virtual milliseconds of the host build, calibrated against the ESP32-P4, unless the text says otherwise.

13.1 What was built, and how it was measured

The steps were taken in the order of Figure 12.1, without the PMP fence. The numbers that Table 12.1 says each step must preserve or claims to change are gathered into one program, `bench/passes.py`, which runs a fixed set of scripts on any pair of a host binary and a `js/` directory and prints the REPL's ask alone and beside an always-busy app, the Faults app's endless loop, the result of five loops typed at the REPL, the three recovery times of *Small Worlds*, Chapter 15 [Small Worlds 2026], the lane and the I2C bus at rest, and the latency from a key to the first changed pixels, alone and beside the busy app. The scripts touch the system only through keys and the host's `crash` command, so the same program measures both passes: the first from a binary built from `history/first-pass/` with that directory's `js/`, the second from the current tree. Table 13.1 gives its output for both, re-run for this chapter. Every value equals the one committed in `bench/passes-first.txt` and `bench/passes-second.txt`.

Table 13.1. The numbers of bench/passes.py on both passes, re-run for this chapter, and the ask benchmark of bench/system-bench.json. Virtual time on the host build.

Measurement	First pass	Second pass
REPL ask, from the ask to the REPL's resumption	1.611 ms	0.085 ms
REPL ask beside an always-busy app	5,017.4 ms (timed out)	40.3 ms
Faults app's endless loop: the sandbox's longest segment	200.2 ms	50.6 ms
Five loops typed at the REPL, then `6*7`	sandbox stopped	42
Key path recovery after its driver is killed	213.351 ms	213.467 ms
Display driver recovery	514.994 ms	515.094 ms
Compositor recovery	167.412 ms	167.515 ms
I2C bus busy in 10 s at rest	3,013.8 ms (502 transfers)	2,009.8 ms (1,004 transfers)
Lane busy at rest	0.720%	0.765%
Key to first changed pixels, median of 20	30.75 ms	30.83 ms
The same beside an always-busy app, median (max) of 30	76.42 ms (87.44)	76.59 ms (87.69)
One ask on an idle lane, empty payload	119.04 µs	120.04 µs

The last row comes from the messaging benchmark that `bench/run.py` runs on `bench/system-bench.json`; for the first pass this chapter wrote the same grants as `sends` and `asks`. The second instrument is `bench/secondpass.py`, seventeen checks built like `bench/firstpass.py` (Section 8.5.2): each copies `js/`, adds probe worlds from `bench/second/`, runs the host build and decides from the kernel log, a snapshot or a trace, printing what it observed beside its verdict.

The sketches of Chapter 12 changed in four places when they met the code, and two changes were added that no sketch contained. Table 13.2 lists them; the sections below give the reasons.

Table 13.2. Where the second pass as built differs from the sketches of Chapter 12.

Step	The sketch	As built
Endpoints	`holders` and a `grant` flag per endpoint; 14 endpoint handles	`send`, `ask` and `grant` lists of holders, and a group allowed as the server; 17 endpoint handles
One driver	a register read as write, a 2 ms `sleep`, read	first as sketched; then the write and a read set to start 2 ms after it, issued together
Supervision	the policy in the supervisor's code, its crash counts in the supervisor's state	the policy per world in the manifest; the supervisor keeps no state and reads each world's crash history from the fault
Passive servers	one priority fixed at boot, the highest of its askers'	the priority of the asker whose ask it is answering
Not sketched		the REPL asks before it redraws; a crash removes the dead instance's events from the queue

The REPL's change is the one exercise 3 of Chapter 12 asked for, and the purge of dead events is exercise 7 of Chapter 7; each is described with the step it belongs to. Together, Table 13.1 and Table 13.2 summarise the chapter: what moved, by how much, and where the code departed from the plan.

13.2 Endpoints and per-world numbers

The manifest's `endpoints` replaces the first pass's `sends` and `asks`. Each entry names a server, the message types the endpoint carries, and the worlds that hold it, listed by the right they get:

▸ js/system.json

```
  "endpoints": {
    "open":      { "server": "compositor", "types": ["open"],                "send": ["launcher"] },
    "attach":    { "server": "display",    "types": ["attach"],              "send": ["compositor"] },
    "flush":     { "server": "display",    "types": ["flush"],               "ask":  ["compositor"] },
    "screen":    { "server": "power",      "types": ["screen"],              "send": ["compositor"] },
    "input":     { "server": "apps",       "types": ["focus", "blur", "key"], "send": ["compositor"] },
    "eval":      { "server": "sandbox",    "types": ["eval"],                "ask":  ["repl",
        "faults"] },
    "boom":      { "server": "flaky",      "types": ["boom"],                "send": ["faults"] },
    "flaky":     { "server": "flaky",      "types": ["ping", "slow"],        "ask":  ["faults"] },
    "stats":     { "server": "stm32",      "types": ["stats"],               "ask":  ["keys"] },
    "backlight": { "server": "stm32",      "types": ["backlight"],           "ask":  ["power"] }
  },
```

Listing holders by right replaced the sketch's single `holders` list because the display driver's two endpoints need different rights, since the compositor sends `attach` and asks `flush`. A group as server, `"apps"` here, makes one endpoint per member, named `input:launcher`, `input:repl` and so on, so the compositor's endpoints to seven apps take one line. The boot turns each entry into rights per holder:

▸ nucleus/kernel.c

```
        // rights per holder: send gives w, ask gives c, grant adds g to what the holder has
        int rights[MAX_WORLDS] = { 0 };
        for (int k = 0; k < 3; k++)
          for (int x = 0; x < nh[k]; x++) {
            uint64_t hs = resolve(hold[k][x], where, keys[k]);
            for (int w = 0; w < K.n_worlds; w++) if (hs >> w & 1) rights[w] |= k == 0 ? R_WRITE : k
                == 1 ? R_CALL : R_GRANT;
          }
        for (int w = 0; w < K.n_worlds; w++) {
          if (!rights[w]) continue;
          if (!(rights[w] & (R_WRITE | R_CALL))) { merr("%s: %s is listed under \"grant\" only; grant
              adds the right to pass on what send or ask gives", where, K.worlds[w]->name); continue;
              }
          if (w == s) { merr("%s: %s would hold an endpoint to itself", where, K.worlds[w]->name);
              continue; }
          if ((rights[w] & R_WRITE) && K.worlds[s]->passive) merr("%s: %s only answers asks (it is
              passive); %s may ask it, not send to it", where, K.worlds[s]->name, K.worlds[w]->name);
          if (K.n_ep_grants == MAX_EP_GRANTS) { merr("more than %d endpoint handles", MAX_EP_GRANTS);
              break; }
          K.ep_grants[K.n_ep_grants++] = (ep_grant_t){ e, w, rights[w] };
          if (rights[w] & R_CALL) K.worlds[w]->static_asks |= 1ull << s;
        }
```

The last line keeps the manifest's ask graph, so the first pass's cycle search (Section 6.3) runs unchanged on the asks the endpoints allow. A manifest that still has `sends` or `asks` does not boot, and its error says what replaced them. The shipped manifest declares 16 endpoints and 17 endpoint handles. The sketch counted 14 because it gave the display driver and `flaky` one endpoint each, and because the `power` service, which asks the STM32 driver for the backlight, did not yet exist. The kernel makes the handles each time a holder starts, since a world loses its handles when it stops:

▸ nucleus/hw.c

```
void endpoints_grant(world_t *W) {
  for (int i = 0; i < K.n_ep_grants; i++) {
    ep_grant_t *g = &K.ep_grants[i];
    if (g->world != W->id) continue;
    handle_t *h = handle_alloc(H_EP, W, -1);
    if (!h) { klog("%s: no room for its endpoint %s", W->name, K.endpoints[g->ep].name); continue; }
    h->rights = g->rights; h->ep = g->ep;
    snprintf(h->claim, sizeof h->claim, "%s", K.endpoints[g->ep].name);
    snprintf(h->badge, sizeof h->badge, "%s", W->name);
  }
}
```

The minter is `-1`, so these handles are charged to no world's quota of Chapter 7. The boot instead checks that they fit beside the quotas: the shipped quotas add up to 58 entries of the kernel's 128 and the endpoints take 17 more, 75 in all. The badge is the holder's name.

Per-world numbers. The kernel still gives every handle an id from one counter, but a world no longer sees it. Each world has a table of 64 slots, and the number it uses is an index into that table:

▸ nucleus/hw.c

```
int slot_number(world_t *W, int kid) {
  for (int i = 0; i < MAX_SLOTS; i++) if (W->slot[i] == kid) return i + 1;
  return 0;
}
handle_t *slot_handle(world_t *W, int n) {
  if (n < 1 || n > MAX_SLOTS || !W->slot[n - 1]) return NULL;
  handle_t *h = find(W->slot[n - 1]);
  return h && (h->holders & bit(W)) ? h : NULL;
}
static int slot_install(world_t *W, int kid) {
  int n = slot_number(W, kid);
  if (n) return n;
  for (int i = 0; i < MAX_SLOTS; i++) if (!W->slot[i]) { W->slot[i] = kid; return i + 1; }
  return 0;
}
```

A handle travels inside a payload as `{"$h":n}`, which the prelude's `Handle.toJSON` writes. The nucleus rewrites those references twice: from the sender's numbers to kernel ids when the message is sent (`json_map_handles(json, to_kid, W)` in `op_send`), and from kernel ids to the receiver's numbers when it is delivered:

▸ nucleus/kernel.c

```
    if (m->nh || (m->json && strstr(m->json, "{\"$h\":"))) {   // the receiver's own numbers for the
        handles it is given
      char *j = json_map_handles(m->json, grant_to, W);
      free(m->json); m->json = j;
    }
```

The rewrite cannot touch a string that merely looks like a reference, because inside a JSON string the quotes would be escaped. The check in `bench/secondpass.py` gives two worlds different schedules of allocation; `num_a` sees its handles as 1 and 2 and `num_b` as 1, 2, 3 and 4, whatever the other did in between. The covert counter that Section 12.2 described is gone for every world without the `sys.read` claim. `sys.handles`, which that claim allows, still reports the kernel's ids, and the System app's Handles tab shows them.

The check at every send and ask. `may_talk` now takes either an endpoint handle or a world's name:

▸ nucleus/vm.c

```
static bool may_talk(JSContext *ctx, world_t *W, JSValueConst to, const char *type, int right, world_t
    **T, char *badge) {
  char err[400];
  badge[0] = 0;
  if (JS_IsNumber(to)) {
    int n = 0; JS_ToInt32(ctx, &n, to);
    handle_t *h = handle_use(W, n, H_EP, right, err, sizeof err);
    if (!h) { fail(ctx, "%s", err); return false; }
…
  } else {
    const char *name = JS_ToCString(ctx, to);
    *T = name ? world_by_name(name) : NULL;
…
    if (*T != W) {
      handle_t *h = endpoint_for(W, *T, type, right, err, sizeof err);
      if (!h) { fail(ctx, "%s", err); return false; }
      snprintf(badge, NAME_LEN, "%s", h->badge);
    }
  }
```

A name is resolved by `endpoint_for` to the first of the caller's endpoint handles to that world that carries the type and the right. That is the compatibility layer Section 12.2 proposed, moved from the prelude into the nucleus so that the refusal can say which endpoint was missing. No shipped world changed to use endpoints; apart from the REPL's change of Section 13.5, the edits to the apps were renames (`keyboard` became `stm32`), wording, and the System app's rows for endpoints. A world may always send to itself without an endpoint, which the compositor's `draw` relies on. The first pass's channel-type rule stays as a second test. When `endpoint_for` finds nothing it writes one of four sentences. The attack that Section 5.6 could only contain is now refused by its type:

▸ bench/rogue-second.txt

```
forge_frame_allowed: the same from a world allowed to send to the compositor
  REFUSED: [  1502.423] rogue_frame2: OUTCOME forge_frame refused: rogue_frame2 may not send compositor
      "frames": its endpoints to compositor carry only "open"
```

Delegation, revocation and badges. An endpoint listed under `grant` can be narrowed and passed on, with the handle rules *Small Worlds*, Chapter 9 describes. The second check in `bench/secondpass.py` exercises the whole cycle with two probe worlds: `giver` holds the

sandbox's `eval` endpoint with rights `cg`, narrows a copy to `c`, sends it to `taker`, and revokes it a second later.

▸ bench/secondpass-after.txt

```
PASS  a narrowed endpoint can be given, used, not passed on, and revoked
      giver holds 1 (cg), gives a copy 3 (c)
      taker got handle 1 (endpoint to sandbox, rights c, badge giver) from giver
      taker asked through it: 42
      taker could not pass it on: handle 1 cannot be passed on by taker
      giver revoked the copy
      after the revoke: handle 1 was revoked
```

The copy is number 3 in the giver's table and number 1 in the taker's. It keeps the giver's badge, because `handle_child` copies the parent's and nothing can change it, so the sandbox sees an ask whose `from` is `taker` and whose `badge` is `giver`. In seL4 the badge is how a server tells its clients apart [seL4 manual 2026, §4.2.1]. PicoCalc OS already stamped every message with its sender, so here the badge adds the identity of the world whose authority a delegate is using. No shipped world reads it yet.

Passing an ask endpoint changes the ask graph while the system runs, so the boot's guarantee of no cycles could be lost by a single message. The nucleus therefore repeats the check on every transfer of a handle that can ask:

▸ nucleus/kernel.c

```
static uint64_t asks_now(int u) {
  uint64_t m = K.worlds[u]->static_asks;
  for (int i = 0; i < MAX_HANDLES; i++) {
    handle_t *h = &K.handles[i];
    if (h->live && h->type == H_EP && !h->revoked && (h->rights & R_CALL) && (h->holders >> u & 1)) m
        |= 1ull << K.endpoints[h->ep].server;
  }
  return m;
}
static bool reaches(int from, int to, uint64_t *seen) {
  if (from == to) return true;
  if (*seen >> from & 1) return false;
  *seen |= 1ull << from;
  uint64_t m = asks_now(from);
  for (int v = 0; v < K.n_worlds; v++) if ((m >> v & 1) && reaches(v, to, seen)) return true;
  return false;
}
```

`kernel_may_receive` refuses the transfer if the endpoint's server can already reach the receiver by asks. The probe gives `p` an ask endpoint to `q`, lets `q` ask `r`, and has `p` try to give the endpoint to `r`; the send fails with `p cannot pass that endpoint to r: it would let r ask q, which can already reach r by asks: asks must form no cycle`. The search scans the 128-entry table once per world it visits, and runs only when a message carries a handle that can ask, which no shipped world sends.

This section has shown the four parts of the endpoint step working: typed authority checked at the call, numbers private to each world, delegation with revocation, and an ask graph that stays acyclic while authority moves. The compatibility layer is why the first pass's suites and every shipped world run unchanged on top of them.

13.3 One driver for the STM32

The keyboard and battery drivers of the first pass both claimed `i2c0:0x1f`, and the nucleus held the bus through a 2 ms gap so that one driver's register read could not be split by the other's. The second pass replaces them with one `stm32` driver, which claims the address, publishes `keys` and `battery`, and answers asks for the backlight. The policy the old power driver held (turning the screen off and remembering the backlight level) moved to a `power`

service that holds no hardware and asks `stm32` when it must change the backlight. The boot now enforces the sDDF rule that "only one client may have access to any particular bus address" [Heiser et al. 2026, §5.3.1]:

▸ nucleus/kernel.c

```
  for (int i = 0; i < K.n_worlds; i++)
    for (int c = 0; c < K.worlds[i]->n_claims; c++) {
      const char *cl = K.worlds[i]->claims[c];
      if (strncmp(cl, "i2c", 3) && strncmp(cl, "spi", 3)) continue;
      for (int j = 0; j < i; j++)
        if (world_claims(K.worlds[j], cl)) merr("%s and %s both claim %s; a device has one driver (put
            both jobs in one world, and let others ask it)", K.worlds[j]->name, K.worlds[i]->name, cl);
    }
```

The part of the step that needed care is the register read. The STM32 prepares its answer when the register number is written and hands it over on the next read, about 2 ms later. The first pass did this in one held transaction:

▸ history/first-pass/js/drivers/keyboard.js

```
// The controller prepares its answer when the register number arrives and hands it over on the next
// read, so the write and the read must not be split by another driver's access to the same chip: the
// power driver reads the battery from it too. One transaction holds the bus for write, 2 ms, read.
const readReg = (dev, reg, n) => dev.read(reg, n, 2);
```

The sketch in Section 12.4 made the read three steps (write, a 2 ms `sleep`, read), on the argument that a world takes no other message while its turn awaits, so with one driver nothing can come between the write and the read. The first version of `stm32.js` did exactly that, and its trace at rest shows the price. Each `await` ends a segment, and the next step needs the lane again:

▸ 14book/traces/13-poll-threestep.jsonl

```
{"k":"seg","w":"stm32","t":1500140,"dt":61,"steps":30,"native":2,"kind":"msg","type":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"stm32","t":1500201,"t1":1502201,"bytes":3,"what":"write 0x04, read 0"}
…
{"k":"seg","w":"stm32","t":1502201,"dt":44,"steps":6,"native":1,"kind":"settle","type":"every 20 ms"}
{"k":"seg","w":"stm32","t":1504245,"dt":50,"steps":15,"native":1,"kind":"settle","type":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"stm32","t":1504295,"t1":1506295,"bytes":3,"what":"read 1"}
{"k":"seg","w":"stm32","t":1506295,"dt":57,"steps":23,"native":2,"kind":"settle","type":"every 20 ms"}
```

Four segments per poll, 212 µs of lane time, where the first pass used two segments and 111 µs. At rest that cost half a percentage point of the lane, 0.72% to 1.23%, which Section 12.4 had not estimated but which is small. Beside an app that is always runnable it cost much more. The busy app's segments last about 20 ms and nothing preempts a segment, so a driver step that becomes ready during one waits until it ends. Reading a key takes two register reads (the status, then the FIFO entry), so the three-step driver waited for the lane seven times, once for the poll's timer and once after each awaited step, where the first pass waited three times. An eval written for the second pass, which runs the REPL beside an always-busy app (Section 13.7), exposed it; measured with `bench/passes.py` on a copy of the current system with the three-step read restored, a key took 176.8 ms to reach the screen beside the busy app, median of 30 (max 207.8 ms), against 76.4 ms (max 87.4 ms) on the first pass.

The repair keeps the nucleus out of the chip's protocol but lets the driver issue both halves of a read at once. The I2C transfer gained two generic options: `after_ms`, which starts a transfer no earlier than that long after the same handle's previous transfer ended, and `quiet`, which settles nothing. The driver's read is now one lane wait:

▸ js/drivers/stm32.js

```
function readReg(dev, reg, n) {
  dev.transfer([reg], 0, { quiet: true });
  return dev.transfer([], n, { after_ms: 2 });
}
```

▸ nucleus/hw.c

```
    // A transfer may ask to start a set time after its handle's previous one ended (d->delay): a device
        that
    // needs time between a write and a read gets it from its driver, which issues both at once, without
        the
    // nucleus knowing the device's protocol. The bus is not held in between; only the wire time is busy.
    vtime_t from = at;
    if (h && d->delay && h->last_end + d->delay > from) from = h->last_end + d->delay;
    vtime_t start = bus_start(BUS_I2C0, from);
```

The two transfers are deferred effects of one segment, applied in order when it ends, so the read finds `last_end` already set by the write. Placed beside the first pass's poll, the trace shows the same shape with the gap released:

▸ l4book/traces/13-poll-first.jsonl

```
{"k":"seg","w":"keyboard","t":1500078,"dt":59,"steps":26,"native":2,"kind":"msg","type":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"keyboard","t":1500137,"t1":1506137,"bytes":4,"what":"write 0x04, wait,
    read 1"}
…
{"k":"seg","w":"keyboard","t":1506137,"dt":52,"steps":18,"native":1,"kind":"settle","type":"every 20
    ms"}
```

▸ l4book/traces/13-poll-second.jsonl

```
{"k":"seg","w":"stm32","t":1500140,"dt":65,"steps":34,"native":3,"kind":"msg","type":"every 20 ms"}
{"k":"bus","bus":"i2c0","w":"stm32","t":1500205,"t1":1502205,"bytes":3,"what":"write 0x04, read 0"}
{"k":"bus","bus":"i2c0","w":"stm32","t":1504205,"t1":1506205,"bytes":3,"what":"read 1, started after
    the set time"}
…
{"k":"seg","w":"stm32","t":1506205,"dt":55,"steps":21,"native":2,"kind":"settle","type":"every 20 ms"}
```

Two segments per poll, 120 µs, and the read ends 68 µs later than the first pass's held transaction did. The numbers in Table 13.1 follow. The I2C bus is busy 2,009.8 ms in ten seconds at rest instead of 3,013.8 ms; the difference, 1,004 ms, is the 2 ms gap of each of the 502 reads, during which the bus is now free. The number of transfers doubles, because each read is two. The lane at rest is 0.765% busy against 0.720%, key-to-photon latency is 30.83

ms against 30.75 ms, and beside the busy app it is 76.59 ms against 76.42 ms. The check that no I2C transfer holds the bus through a wait passes, with the longest transfer the battery's 2.90 ms read of two bytes.

The repair does add a mechanism to the nucleus, and it should be compared with the one it replaced. The first pass's gap held the bus because two drivers shared a chip, so the nucleus enforced an exclusion that belonged to the device. `after_ms` enforces nothing; it says when a transfer may start relative to the same handle's last one, and with one claim per address the exclusion comes from the manifest.

The cost Section 12.4 predicted is real. Keys and battery now go down together when `stm32` is killed; the key path recovers in 213.467 ms against the first pass's 213.351 ms, since the driver polls keys before it reads the battery. The step established that one owner per device address removes the held gap from the nucleus at no measurable cost in latency, provided the driver does not turn one device transaction into several waits for the lane.

13.4 Supervision in a world

On a crash the nucleus still does what only it can do: it discards the segment's effects, rejects the ask in progress, drops the world's handles, removes its timers and advances its epoch. What happens next is no longer decided in C. `crash_at` builds a `fault` message for the supervisor named by the manifest's `"supervisor"` key, delivers it after teardown, and leaves the world down:

▸ nucleus/kernel.c

```
  bool to_super = !W->config_error && W->n_crash_times < CEILING && K.supervisor >= 0 && K.supervisor
      != W->id;
  msg_t *fm = to_super ? fault_msg(W, first, ct[0] ? ct : NULL) : NULL;
…
  teardown(W, t);
…
  if (W->n_crash_times >= CEILING) {
    klog("%s crashed %d times in ten seconds, the nucleus's ceiling; it stays stopped whatever its
        supervisor says", W->name, W->n_crash_times);
    stop_world(W, first);
    return;
  }
  if (reported) {
    W->awaiting = true;
    W->status = W_RESTARTING;
    schedule_fn(now + MS(1000), decision_timeout, W, W->epoch);
    return;
  }
  fallback_policy(W, first);
```

Some crashes are not reported (Section A.3 lists them). A configuration error stops the world, as in the first pass. The supervisor's own crashes go to `fallback_policy`, which is the first pass's schedule (100 ms doubling, stopped at the fifth crash in ten seconds), as the Microkit sends the faults of a protection domain without a parent to the system's fault handler [Microkit manual 2026]; so do all crashes when the manifest names no supervisor, or when the supervisor is stopped or its mailbox is full. And a world that has crashed 50 times in ten seconds stays stopped whatever its supervisor would say, since a policy that restarts forever would let one world occupy the lane; a scratch world that throws as it starts, given a policy that never stops it, crashed 50 times by 4.91 s and was stopped by this ceiling, and no other world went down. A reported crash also arms a one-second deadline. A scratch supervisor that decided nothing left `flaky`, killed at 1.5 s, down until 2.6 s: at 2.5 s the log said `the supervisor did not decide about flaky within 1 s; the nucleus restarts it by its own policy`, and the fallback policy started it 100 ms later.

The fault message carries what the `#OCT1` frame says and what a policy needs. A copy of the supervisor that logs each fault, run while the Faults app loops the sandbox and then while the host kills `flaky`, printed:

```
[  1645.774] super: {"world":"sandbox","why":"ran past faults's budget of 50 virtual ms, which it runs
    on while answering faults's ask","handling":"eval","t":1645.688,"epoch":1,"restarts":0,"history":
    [],"policy":
    {"backoff_ms":0,"max_backoff_ms":2000,"intensity":0,"period_ms":10000},"charged_to":"faults"}
[  3140.086] super: {"world":"flaky","why":"killed from the
    host","handling":"","t":3140,"epoch":1,"restarts":0,"history":[3140],"policy":
    {"backoff_ms":100,"max_backoff_ms":2000,"intensity":5,"period_ms":10000},"charged_to":null}
```

`policy` is the world's `restart` entry from the manifest, merged field by field over `defaults.restart`; the nucleus checks the field names at boot and passes the text through. The names `intensity` and `period` come from OTP, whose supervisor, after more than `intensity` restarts within `period` seconds, terminates its children and then itself [Erlang/OTP n.d.]; PicoCalc OS stops only the world, at the `intensity`-th crash. `history` holds the world's crash times since it was last started fresh, leaving out crashes charged to an asker (so the sandbox's is empty above), and that is what makes the supervisor stateless:

▸ `js/system/super.js`

```
  .on("fault", (s, f) => {
    s.decisions++;
    const p = f.policy;
    if (f.charged_to) {
      s.charged++; s.restarts++;
      s.last = `${f.world}: restarted at once; the crash was ${f.charged_to}'s`;
      return sys.restart(f.world, { after_ms: 0, epoch: f.epoch });
    }
    const recent = f.history.filter((t) => f.t - t < p.period_ms && t <= f.t);
    if (p.intensity > 0 && recent.length >= p.intensity) {
      s.stops++;
      s.last = `${f.world}: stopped after ${recent.length} crashes in ${p.period_ms / 1000} s`;
      return sys.stop(f.world, `crashed ${recent.length} times in ${p.period_ms / 1000} s`, { epoch:
          f.epoch });
    }
    // A world that crashed while starting waits at least 100 ms whatever its policy: an instance that
        cannot
    // start would otherwise be restarted as fast as the lane allows, and each restart posts on
        worlds.status.
    const floor = f.handling === "start" || f.handling === "" ? 100 : 0;
    const delay = Math.max(floor, Math.min(p.max_backoff_ms, p.backoff_ms * 2 ** Math.max(0,
        recent.length - 1)));
    s.restarts++;
    s.last = `${f.world}: restart in ${delay} ms (crash ${recent.length} in ${p.period_ms / 1000} s)`;
    sys.restart(f.world, { after_ms: delay, epoch: f.epoch });
  })
```

The sketch kept crash counts in the supervisor's state. Moving them into the fault means that a supervisor restarted by the fallback policy loses nothing; its state holds only counters for display. The policy itself became data. The sandbox's entry says `"restart": { "backoff_ms": 0, "intensity": 0 }`, and the supervisor's code has no special case for it. One rule is not data: a crash while starting, or outside any turn, waits at least 100 ms. The review of this chapter found why it is needed. A sandbox whose file threw at its first line restarted as fast as the lane allowed; each restart posted on `worlds.status`, the posts filled the compositor's 128-message mailbox, and the compositor's `send(self, "draw")` threw, so one broken passive server took the screen down with it. With the floor it is restarted every 100 ms until the ceiling stops it at 4.9 s; the compositor also now treats a full mailbox as a reason to draw with the next change rather than a crash, and `bench/secondpass.py` checks that nothing but the broken server goes down. This is the arrangement of MINIX 3's reincarnation server, whose policy script is given the component, the reason and the failure count [Herder et al. 2007].

Every decision carries the epoch the fault reported. A decision can arrive late (after the one-second timeout restarted the world by the fallback policy, which may have crashed it again), and it must not act on an instance it was not about:

▸ nucleus/kernel.c

```
int kernel_restart_world(world_t *W, vtime_t delay, int epoch) {
  if (epoch >= 0 && (W->epoch != epoch || !W->awaiting)) return -1;
  if (W->awaiting) { schedule_restart(W, delay, W->last_error); return 0; }
```

Figure 13.1 draws one crash from report to restart, with the times of the trace in `l4book/traces/13-faults-loop.jsonl`. The supervisor's decision cost one 91 µs segment; Section 12.5 had estimated about 60 µs for the handler and as much for the message, which the nucleus delivers without a segment of its own.

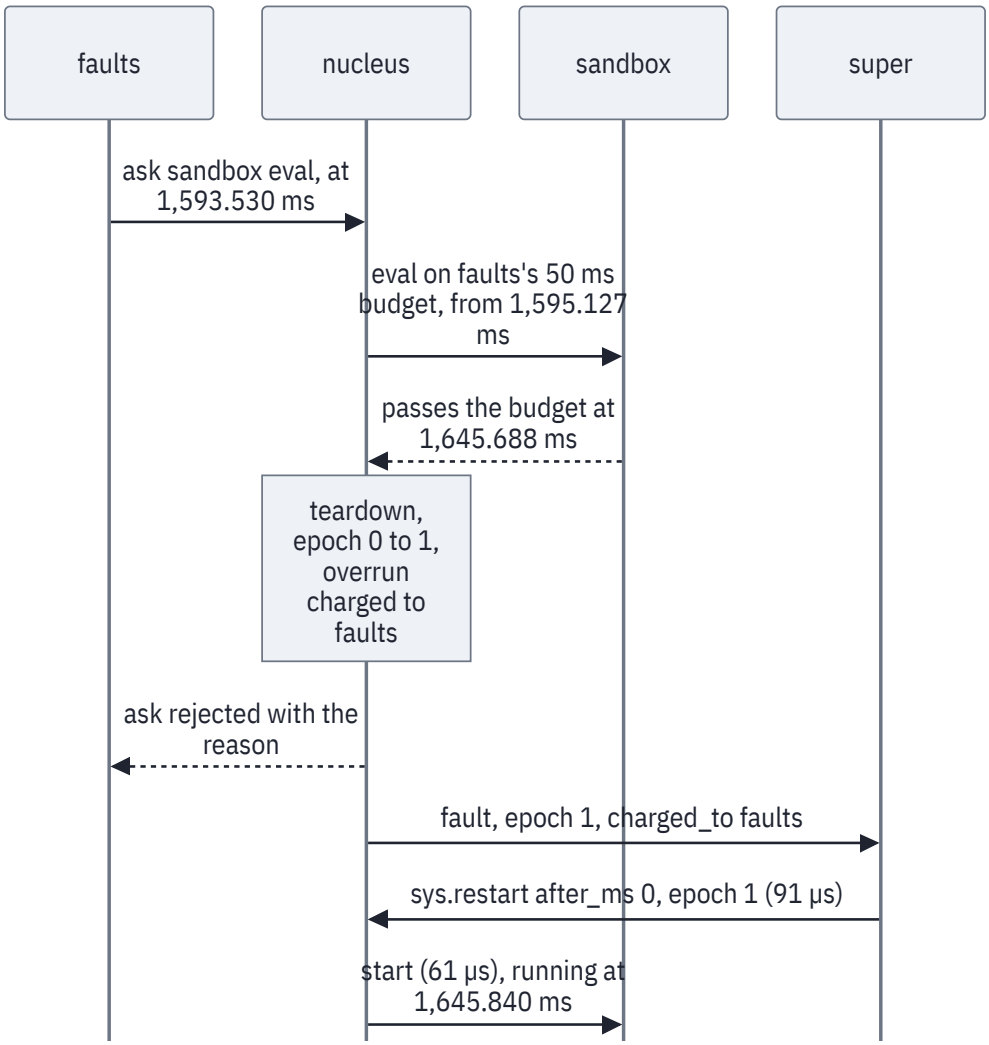


Figure 13.1. The Faults app's endless loop in the second pass, from the trace: the sandbox overruns the asker's 50 ms budget, the nucleus tears it down and reports the crash, and the supervisor restarts it at once without counting the crash, 152 µs after the overrun.

▸ 14book/traces/13-faults-loop.jsonl

```
{"k":"crash","t":1645688,"w":"sandbox","text":"ran past faults's budget of 50 virtual ms, which it runs
    on while answering faults's ask"}
…
{"k":"msg","t":1645688,"from":"kernel","to":"super","type":"fault","ask":false,"bytes":293}
{"k":"seg","w":"super","t":1645688,"dt":91,"steps":65,"native":8,"kind":"msg","type":"fault"}
{"k":"status","t":1645779,"w":"sandbox","text":"crashed"}
…
{"k":"status","t":1645840,"w":"sandbox","text":"running"}
```

The supervisor runs at priority 4, above the drivers, so that no driver's segment delays a decision. The checks of `bench/secondpass.py` cover the rest of the design. With the supervisor killed and the STM32 driver killed 20 ms later, the supervisor came back at 1,600.1 ms under the fallback policy, read the queued fault and restarted the driver at 1,700.2 ms. A `restart` of `{ "backoff_ms": 300 }` on `flaky` brought it back 300.158 ms after a crash. Five crashes of `flaky` in ten seconds still stop it, now with `stopped by super: crashed 5 times in 10 s`. The three recovery times of Table 13.1 moved by 0.10 to 0.12 ms, inside the fraction of a millisecond Table 12.1 allowed.

The symptom that motivated the step is gone. Figure 13.2 shows the REPL after five endless loops and `6*7` on both passes.



Figure 13.2. The REPL after five loops typed about two seconds apart and then 6*7, on the host build. In the first pass (left) the fifth crash in ten seconds stopped the sandbox and the REPL answers 'sandbox is stopped'; in the second (right) each loop is charged to the REPL, the supervisor restarts the sandbox at once, and 6*7 is 42.

13.4.1 The dead instance's events

Section 7.8 found a pool nobody had listed: a crashed instance's pending sleeps stayed in the event queue until their time came, and a probe that left 1,000 sleeps per instance grew the queue to 92,445 events in 300 s. The second pass answers that chapter's exercise 7 the second way it proposed, by removing the dead epoch's events at teardown:

▸ nucleus/kernel.c

```
static void purge_epoch(world_t *W) {
  int n = 0;
  for (int i = 0; i < heap_n; i++) {
    event_t *e = heap[i];
    if (e->w == W->id && e->epoch == W->epoch && (e->type == E_TICK || e->type == E_SETTLE)) {
      if (e->settle) { free(e->settle->json); free(e->settle); }
      free(e);
    } else heap[n++] = e;
  }
  if (n != heap_n) { heap_n = n; heap_fix(); }
}
```

The same probe, `bench/second/orphans.js`, now leaves 1,436 events in the queue at 60 s and 1,443 at 300 s, after 91 crashes. The shipped system alone holds 415, so the rest is the live instance's 1,000 sleeps and a few dozen others; the queue no longer grows. Removing and re-heapifying costs a pass over the queue per crash, which is host time, not virtual time.

This section has shown supervision split as Section 12.5 planned, with the mechanism in the nucleus and the policy in a world, kept safe by three limits in the nucleus: an epoch on every decision, a one-second deadline for it, and a ceiling of 50 crashes in ten seconds.

13.5 Asks charged to the asker

A passive world has no priority of its own. While it answers an ask it runs at the asker's priority, and an ask waiting at the head of its mailbox competes for the lane at its asker's:

▸ nucleus/kernel.c

```
static int prio_of(world_t *W, bool settle) {
  if (!W->passive) return W->prio;
  world_t *D = NULL;
  if (W->in_turn || settle) D = world_by_id(W->turn_donor);
  else if (W->mbox && W->mbox->ask) D = world_by_id(W->mbox->from);
  return D ? D->prio : 0;
}
```

The asker is recorded as the turn's donor when the turn starts, and every segment of that turn, settles included, runs on the donor's budget and is billed to the donor:

▸ nucleus/kernel.c

```
    if (W->passive) {
      if (!is_settle && !W->in_turn && W->mbox && W->mbox->ask) W->turn_donor = W->mbox->from;
      D = world_by_id(W->turn_donor);
      if (D) budget = D->budget_us;
    }
…
  else if (W->overrun && D) snprintf(why, sizeof why, "ran past %s's budget of %lld virtual ms, which
      it runs on while answering %s's ask", D->name, (long long)(budget / 1000), D->name);
…
  if (D) { D->cpu_us += cost; W->served_us += cost; }   // the lane time is the asker's
```

This is seL4's donation as Section 3.5 described it: the passive server "executes on the client's SC, until it replies", and its time "is billed to the client requesting the work" [Lyons et al. 2018]. The sandbox's own lane time is still counted, as `served_ms` in the kernel's report, so the System app can show both who ran and who paid.

The priority departs from the sketch. seL4 configures a passive server at the highest priority of its clients, so that it runs atomically with respect to all of them and implements the immediate priority ceiling protocol at no extra cost [Lyons et al. 2018, §3.5]. PicoCalc OS does not need the ceiling for that purpose, because a segment is never preempted and a world's mailbox stays closed while its turn awaits. With the asker's own priority, an evaluation runs exactly where its asker would have run, and a low-priority asker cannot lift its work above worlds it does not outrank. In the shipped manifest the two rules agree, since both askers, the REPL and the Faults app, are apps at priority 1.

The boot checks the rest of the contract: a passive world hears and provides no channel, has no `priority`, is asked by some endpoint and sent to by none; and each time the world starts, `read_meta` checks that its file declares no timer. The manifest gives the REPL `"budget_ms": 200` explicitly, as Section 12.3 said it would have to, since the authority to occupy the lane for 200 ms now belongs to the asker.

Starvation and laundering. Beside an app that is always runnable, the REPL's ask was answered in 40.3 ms, against a 5 s timeout in the first pass. With donation alone (the first pass's `repl.js` on the second pass's system) it took 42.3 ms, within 0.1 ms of the 42.2 ms Section 12.3 measured by raising the sandbox's priority by hand. The Faults app's endless loop now stops at 50.6 ms, its own budget, and `bench/secondpass.py` shows the Faults app paying for it, 51.239 ms of its lane time. The crash is the asker's: `crash_at` passes `charged_to` when the overrun was on a donated budget, and the supervisor restarts the

sandbox at once without counting it. With the sandbox given the default policy (stop at the fifth crash), a probe that looped it six times still found it idle and answered `6*7` afterwards. A client can no longer get a shared server stopped by overrunning it, which is the reading of a budget overrun as the client's protocol violation that Lyons and colleagues give [Lyons et al. 2018].

The REPL's ask. Section 12.3 predicted that donation would not change the REPL's 1.611 ms, because the work ahead of the sandbox was the compositor and display drawing the REPL's own "evaluating" frame at priorities 2 and 3. Measured with the first pass's `repl.js` on the second pass's system, the ask took 1.613 ms, and the segments ahead of the sandbox were the same three: the compositor storing the frame (128 µs), drawing it (1,065 µs) and the display flushing it (212 µs). The REPL itself had to change. It now asks before it changes its screen:

▸ history/first-pass/js/apps/repl.js

```
      s.out.push({ s: "▸ " + code, color: "accent" });
      s.hist.push(code); s.hi = -1;
      s.line = ""; s.cur = 0; s.busy = true;
      try {
        const r = await ask("sandbox", "eval", { code }, { timeout: 5000 });
```

▸ js/apps/repl.js

```
      const reply = ask("sandbox", "eval", { code }, { timeout: 5000 });
      const echo = () => { s.out.push({ s: "▸ " + code, color: "accent" }); s.line = ""; s.cur = 0; };
      const slow = await Promise.race([reply.then(() => false, () => false), sleep(30).then(() =>
          true)]);
      if (slow) { echo(); s.busy = true; }
      try {
        const r = await reply;
        if (!slow) echo();
```

The ask now takes 0.085 ms, and nothing runs between it and the sandbox. On the screen, `6*7` used to change it twice, the echo 1.4 ms after the REPL's key turn and the answer at 5.7 ms; now it changes once, echo and answer together, at 1.6 ms. The cost is on long evaluations. A loop of 10^5 iterations runs in one sandbox segment of 133.1 ms, during which nothing else can run, so the REPL's 30 ms timer cannot draw the echo; the timer's settle, earlier

in virtual time than the reply's, runs first when the segment ends. The echo appears 134.9 ms after the key turn instead of 1.6 ms, and the answer, drawn in a second frame, at 142.0 ms instead of 137.1 ms. For up to the REPL's 200 ms budget the screen gives no sign that Enter was taken, a trade-off `KNOWN-ISSUES.md` records.

This section has shown the donation working as seL4's does for budget and accounting, with a different and simpler rule for priority, and has shown again that a latency is not fixed by a mechanism when the work ahead of it is the asker's own.

13.6 The evidence

The first pass could claim that a world inside its manifest entry saw no difference, and show it with pixel-identical captures (Section 8.5.1). The second pass cannot make that claim. It renames worlds, changes the REPL's protocol and the text of crash messages, and runs fourteen worlds instead of thirteen, so compared with the first pass's captures 17 of the 22 differ, the home screen by its "14 of 14 worlds running". Its evidence is therefore of two kinds: checks that each new mechanism does what it claims, and measurements that the numbers Table 12.1 named moved only as predicted. Table 13.3 lists the instruments and their results, all re-run for this chapter on the current build.

Table 13.3. The second pass's claims, the instruments behind them, and their results when re-run for this chapter.

Claim	Instrument	Result
Each step does what it claims	`bench/secondpass.py`, 17 checks	17 of 17 on the 64-bit and sanitizer builds
The first pass's attacks are still refused	`bench/rogue.py`, 7 scenarios	7 of 7 refused, now by the endpoint checks
The first pass's other mechanisms still work	`bench/firstpass.py`, 5 checks	5 of 5
The numbers of Table 12.1 moved only as predicted	`bench/passes.py` on both passes	Table 13.1
No new memory error or undefined behaviour	sanitizer build on the six scripts and the three suites	only the QuickJS shift at `quickjs.c:36335`, as before
The builds agree	the 22 captures, WebAssembly against native 32-bit	19 identical; 3 differ in heap totals only
Runs are deterministic	the REPL trace on the 64-bit and 32-bit builds; `passes.py` re-run	byte-identical; every value equal to the committed one

Three rows need comment. The first pass's suites give the same verdicts, but what they print has moved: `rogue.py`'s refusals now come from `endpoint_for` (the forged frame is refused for its type, not by the channel-type rule), and `firstpass.py`'s broken manifest is reported with the endpoint the test helper made for it (`endpoint "launcher->nobody:open": "server" names "nobody"`) and with the handle count `266 and the manifest's endpoints need 17 more`. The sanitizer build ran the six scripts and the three suites through a wrapper that kept its reports; across all of them it printed the QuickJS left shift 29 times and nothing from the nucleus. The three captures where the WebAssembly build differs from the native 32-bit build are the system monitor's two heap columns (the STM32 driver's heap reads 196 KB in one and 200 KB in the other) and the About screen's total (2,805 KB against 2,813 KB). The two C libraries' allocators round sizes differently; every other pixel of every capture is the same.

The evidence has the gaps Section 8.6 described, and three of its own. Delegation, revocation, badges and the dynamic cycle check are executed only by probe worlds; no shipped world passes an endpoint. The passive sandbox has only been measured with askers of

equal priority, so the choice of the asker's priority over the ceiling has no measurement behind it. And the regression of Section 13.3 passes all seventeen checks of `bench/secondpass.py`. Its check of the REPL beside a busy app measures from the REPL's ask, which the three-step driver does not touch (40.3 ms); only a measurement that starts at the key, under load, found it, and `bench/passes.py` now makes one. The instruments of this section, like those of Chapter 8, cover the executions they ran.

13.7 The workbench's new instruments

Section 12.7 asked for four instruments before the nucleus changed: a structural trace diff, a pinned baseline with tolerances, sessions recorded as evals, and generated evals. The workbench now has all four, and its agent's tool (Chapter 10) became stricter in three ways. Its suite grew from the 19 evals of Chapter 11 to 36: the five that Section 11.5 proposed, the generated `random-keys`, and eleven for the second pass. The headless tests in `workbench/tests/` check each feature on a desktop-sized and a phone-sized page; run for this chapter on the built page, all 91 checks passed (`workbench/tests/features.out.txt`); the suite of evals took 2.51 s of wall time, against a limit the tests set at 4 s because it depends on the machine.

The structural trace diff runs one script on two fresh systems, one with the shipped files and one with the files as edited, and compares their traces record by record by what each record does:

▸ `workbench/src/core.js`

```
// The same script on two fresh systems, traced from the boot. Records are compared by what they do
    (kind, world, message
// type, sender, receiver, the bus transfer, a crash), ignoring when and for how long (times,
    durations, step counts):
// the first record whose shape differs is the first change of behaviour; records before it that differ
    only in timing
// measure what the change costs.
```

On mutant M1 of Section 11.5 it reports what Section 12.7 found by hand: the first change of behaviour is the stale flush that the mutant answers with a crash, at 1,970.790 ms, and the 91 records before it differ only in timing, by at most 2 µs. The record's index is 592 where the first pass's trace had 489, mostly because each STM32 poll now writes two bus records. Given the three-step driver of Section 13.3, the diff places the first change at record 34,

103.1 ms after boot, in the driver's answer to the power service's first backlight ask, where the three-step driver runs a settle segment between the write and the read and the shipped driver does not.

The pinned baseline stores a run's results, its metrics and its files. An eval reports a number with `s.metric(name, value, { unit, tol })`, and a later run marks the eval REGRESSED when the number is worse than the pinned one by more than the tolerance, even if the eval passes. The suite reports three metrics: the lane at rest (tolerance 25%), the first changed pixels after a key (4 ms) and the slowest of ten key phases (5 ms).

A recorded session is the live system's inputs since its last boot, each with its virtual time to the microsecond, written into `workbench/evals.js` as an eval that replays them and checks the app in front, a line of its text and the fault count. The test recorded seven inputs (the keys `s` and `p` down and up, a kill of `flaky`, and an arrow down and up), replayed them to `score 30`, and saw the eval fail after an edit changed what the session showed. **A generated eval** is a property checked on seeded random key sequences, after QuickCheck [Claessen and Hughes 2000]. The suite's `random-keys` holds on 3 sequences of 30 keys (31.3 virtual seconds), and on 10 when asked (105.1 virtual seconds in 733 ms of wall time); a property that fails reports its seed and a script that replays it.

The stricter tool addresses the three gaps that Chapter 10 recorded. `apply` now runs the system for up to 200 virtual ms after a hot reload and returns `ok: false` with the crash if the reloaded world fails to start. A Read-only step snapshots the files, the applied code, the live system and the pin, and restores them and fails if anything changed them, by any route; it is still not a sandbox. And a guard inserted into every loop and function body stops a synchronous endless loop at the time limit, at a measured cost of 78 ms against 13 ms for 2×10^7 loop iterations, about 3 ns of wall time per iteration.

The second pass's evals carry most of `bench/secondpass.py` into the workbench, from `end-point-carries-its-types` to `loop-billed-to-asker`. One of them found the regression of Section 13.3:

▸ workbench/src/evals.js

```
s.run(1500); s.key("r"); s.run(300);
s.type("6*7"); s.key("enter"); s.run(100);
s.expect(/\n42(\n|$)/.test(s.text("repl") || ""), `42 should be on screen within 100 ms:
    ${(s.text("repl") || "").slice(-80)}`);
```

It was written to test the passive sandbox, which it does; it failed on the driver, because its 100 ms bound covers the whole path from the key to the answer. With the three-step read restored in the workbench and the shipped run pinned, the suite gave 35 of 36. The one failure was this eval, whose screen still read `▸ 6*` after 100 ms because the `7` and the Enter had not yet reached the REPL. The pin also marked the lane metric REGRESSED, 1.23% against 0.77% with a tolerance of 25%, while `key-latency-phases`, which presses keys on an otherwise idle system, passed. Either signal would have stopped the change. The instruments Section 12.7 asked for were justified by the first step that needed them.

13.8 What it cost, and what is deferred

The nucleus grew from 2,771 lines after the first pass to 3,350: `kernel.c` from 959 to 1,281, `vm.c` from 881 to 965, `hw.c` from 288 to 422 and `nucleus.h` from 319 to 358. In the worlds, `stm32.js` (73 lines) and the `power` service (29) replace the keyboard and power drivers (45 and 41), the supervisor adds 36 lines and the prelude 11.

In time, an ask costs 1 µs more at every payload size measured (0 to 10,000 bytes), which `SECOND-PASS.md` attributes to the prelude's test of whether the target is a handle. A key reaches the screen 0.08 ms later at the median, recoveries take 0.10 to 0.12 ms longer, and each crash costs one supervisor segment. The kernel's handle table holds 17 more entries.

`KNOWN-ISSUES.md` lists the trade-offs. An endpoint names the types it carries and says nothing about their contents, so the launcher's `open` endpoint opens any app. A passive server answers one ask at a time, and an ask waits behind the one at the head of the mailbox. The REPL's screen waits for a long evaluation. Keys and battery share one driver and one restart. The supervisor is trusted for availability, within the one-second deadline and the 50-crash ceiling. And the key loss under load that *Small Worlds* records is unchanged in kind, since the sandbox still runs segments of up to 200 ms during which the STM32 driver cannot drain its FIFO.

Some things the pass did not attempt. Channels are still named in the manifest and are not capabilities, a world with `sys.read` still sees the kernel's handle ids, and framebuffer pixels still do not count against `mem_kb`. The PMP fence of Section 12.6 waits for the ESP32-P4 port, and the pass made its absence a little more expensive, since the 579 lines the nucleus gained run in the same address space as the worlds, inside the trusted base for isolation, beside QuickJS's 80,864. Every result in this chapter was measured on the host build against simulated peripherals, and none is evidence about the device.

KEY POINTS

- The second pass built four of the five steps Chapter 12 designed (typed endpoints, one driver for the STM32, supervision in a world, a passive sandbox) and two changes no sketch contained (the REPL asks before it redraws; a crash purges the dead instance's events); the PMP fence waits for the device port.
- Endpoints are handles with the rights `w`, `c` and `g`, made from the manifest each time a holder starts, numbered from 1 in each world's own table, rewritten inside payloads, narrowed, passed and revoked like other handles, and checked on every transfer so that asks stay acyclic; a send by name resolves to an endpoint, so no shipped world changed to use them.
- One world owns the STM32 and the boot refuses two claims on one device address; the first version, which read a register in three awaited steps, made a key beside a busy app take 176.8 ms instead of 76.4 ms, and issuing the write and a read set to start 2 ms later in one segment restored 76.6 ms with the bus free during the wait.
- On a crash the nucleus tears the world down and reports a fault; a stateless supervisor applies each world's `restart` policy from the manifest, and an epoch on every decision, a one-second deadline and a ceiling of 50 crashes in ten seconds bound what it can do; recoveries moved by at most 0.12 ms and the REPL survives five loops.
- The passive sandbox runs at its asker's priority, on its asker's budget, billed to the asker: beside a busy app the REPL is answered in 40.3 ms instead of timing out at 5 s, and the Faults app's loop stops at 50.6 ms instead of 200.2 ms.
- Donation alone left the REPL's ask at 1.613 ms, as Chapter 12 predicted; asking before redrawing brought it to 0.085 ms, at the cost of the typed line staying on screen for the length of a long evaluation.
- The evidence is 17 of 17 new checks, the first pass's 7 attack scenarios and 5 checks still passing, no sanitizer report from the nucleus, deterministic runs, and 19 of 22 captures identical between WebAssembly and native builds (the other three differ in heap totals); it does not cover delegation by shipped worlds or askers of different priorities.
- The nucleus grew from 2,771 to 3,347 lines, an ask costs 1 µs more, and each crash costs one supervisor segment (91 µs); the trade-offs are listed in `KNOWN-ISSUES.md`.

- The workbench now has a structural trace diff, a pinned baseline with tolerances, recorded sessions, generated evals and a stricter agent tool; its eval of the REPL beside a busy app found the driver's regression, and a pinned baseline would have flagged the lane's cost as well.

This book asked what thirty years of L4 research have to teach a system made of small JavaScript worlds, and Part II and this chapter have answered by doing: each borrowing was read in its source, built, attacked or measured, and kept with its evidence. The second pass shows the method working in both directions. The measurements that justified the plan confirmed most of it to a fraction of a millisecond, and the instruments built for the plan found the one place where the code fell short of it. What remains is the step that no host build can test, the hardware fence of Section 12.6, together with everything this book has measured on a simulated board and that the device port will have to measure again. `SECOND-PASS.md` lists the manifest keys and messages the pass added, beside Appendix A's account of the first pass's schema.

EXERCISES

1. `BASIC` For each of the four steps built in this chapter, name the function in the nucleus or the world file that implements it, the check in `bench/secondpass.py` that tests it, and the row of Table 13.1 it moved.

2. `BASIC` Count the endpoint handles the shipped manifest creates when an eighth app is added to the `apps` group, and the entries of the kernel's 128-entry handle table that the quotas and endpoints then occupy. Check your answer by booting the host build with the edited manifest.

3. `ANALYSIS` Using `l4book/traces/13-poll-threestep.jsonl` and a busy app whose segments last 20 ms, count the lane waits a key press costs with the three-step register read and with the current one, and estimate the difference in latency. Compare your estimate with the measured 176.8 and 76.6 ms.

4. `ANALYSIS` A passive world runs at the priority of the asker it is serving; seL4 configures a passive server at the highest priority of its clients. Construct a manifest with two askers at priorities 1 and 3 and a busy world at priority 2, and describe a sequence of asks for which the two rules give different latencies to each asker. Which rule does PicoCalc OS's non-preemptive lane make unnecessary, and why?

5. CODE Restore the three-step `readReg` in a copy of `js/drivers/stm32.js` and run `python3 bench/passes.py --js DIR`. Confirm `key_beside_busy_app_median_ms` near 176.8 ms. Then change the busy app's loop (`BUSY` in `bench/passes.py`) to 3,750 iterations, about 5 ms a segment, and predict, before measuring, the new median for both versions of the driver.

6. CODE Write a server world whose handler answers differently according to `meta.badge`, give a narrowed copy of a world's endpoint to a third world, and show that the server can tell the delegate's asks from the original holder's. What can it not tell?

7. CODE Add OTP's `rest_for_one` strategy to `js/system/super.js` for one pair: whenever `stm32` is restarted after a crash, restart `power` as well. Measure what it costs in recovery time with the `crash stm32` script of `bench/passes.py`, and say whether the restart of `power` is needed today.

8. CODE `sys.handles` reports the kernel's handle ids to any world with `sys.read`. Change `hw_handles_json` so that a world sees only its own numbers for the handles it holds and anonymous rows for the rest, and check that the System app's Handles tab still makes sense.

9. ANALYSIS In the REPL, a 133 ms evaluation shows the echo 134.9 ms after the key turn. Explain why no change to `js/apps/repl.js` alone can draw the echo during the evaluation, and propose the smallest change to the nucleus or the sandbox that would. What would it cost the evaluation?

10. PROJECT Measure the passive sandbox with two askers of different priorities: add an app at priority 3 that asks the sandbox every 500 ms for a 1 ms evaluation while the REPL evaluates 150 ms loops. Report the high-priority asker's worst wait, and design and implement a change that bounds it (for example, ordering a passive world's mailbox by the asker's priority). Show that `bench/secondpass.py` still passes.

11. DISCUSSION The repair to the STM32 driver added a timing option, `after_ms`, to the nucleus's I2C transfers. Compare it with the first pass's held gap. Does the nucleus now know part of the device's protocol, and does the answer change if a second device is added to the bus?

FURTHER READING

[Lyons et al. 2018] Scheduling contexts and passive servers, including the ceiling priority and the argument that a server's overrun is its client's fault; read it beside Section 13.5,

which made a different choice of priority.

[seL4 manual 2026] Chapter 4 for endpoints, badges and the transfer of capabilities, and Chapter 6 for donation, fault endpoints and timeout handlers.

[Heiser et al. 2026] The rule of one client per bus address and the division between a driver and the virtualisers that share a device.

[Erlang/OTP n.d.] The restart strategies and the restart intensity and period that the manifest's `restart` entries borrow.

[Herder et al. 2007] MINIX 3's reincarnation server: supervision as a user-level component with a policy per driver.

[Microkit manual 2026] The `fault` entry point of a parent protection domain, and the system fault handler for those without a parent.

[Small Worlds 2026] Chapter 12 for the REPL's ask, Chapter 13 for the interleaving that the first pass's held gap fixed, and Chapters 14 and 15 for the supervision and recovery this chapter moved into a world.

APPENDIX A

The manifest

The manifest, `js/system.json`, is the whole static description of a PicoCalc OS system: which worlds exist, what hardware and system access each may claim, which channels it publishes and hears, which endpoints it holds (whom it may send to and ask, with which message types), how much of the kernel's shared tables it may use, and how it is restarted after a crash. `kernel_boot` in `nucleus/kernel.c` reads it once, before any world runs, and refuses to boot if it is inconsistent. Nothing in it changes while the system runs, so a changed manifest needs a new boot; the workbench reboots when `js/system.json` is applied (Appendix B). This appendix is the reference for the format as the second pass left it (`SECOND-PASS.md` in the repository records the pass). The fields that predate the first pass (`name`, `file`, `claims`, `budget_ms`, `mem_kb`, `priority`) are described in *Small Worlds*, Appendix A [Small Worlds 2026], and are repeated here so that the schema is complete in one place.

In the first pass a world's entry carried two more lists, `sends` and `asks`, naming the worlds and groups it might send to and ask. Authority went per pair of worlds: a world allowed to send to another could send it any type that was not a channel the receiver heard [First pass 2026]. Chapter 5, Chapter 6 and Chapter 7 describe that manifest and why its rules exist. The second pass removed both lists in favour of typed endpoints at the top level, and a manifest that still has either does not boot: it reports `world "launcher": "sends" is gone; whom a world may send to and ask is now the endpoints the manifest's "endpoints" grant it (see SECOND-PASS.md)`. Chapter 13 tells how the second pass was built and measured.

The host build reads `system.json` in its JavaScript root (`-j`, by default `js`), or the file given with `-m`. The WebAssembly build reads `js/system.json` from the files the page has added, or the path passed to `pc.boot(manifest)`. Boot errors begin with that path and a colon (Section A.7).

Two rules apply to the whole file. Keys the kernel does not know are ignored without a message, at the top level and in a world's entry, so a misspelt key (`"passiv"`) has no effect; the exceptions are the removed `sends` and `asks`, an unknown field of an endpoint and an unknown field of a `restart` object, which are errors. And every problem is reported, one line each, before the boot fails, so one run shows all of them.

A.1 The top level

Table A.1. The top-level keys of js/system.json. All are optional; a manifest without worlds boots an empty system.

Key	Type	Meaning
`groups`	object whose values are lists of world names	Named sets of worlds, usable wherever an endpoint names a world: as its `server` and in `send`, `ask` and `grant`. At most 16 groups.
`defaults`	object	Values for the per-world keys that an entry leaves out: the numbers `handles`, `ask_slots`, `mbox`, `mbox_kb`, `bufs`, `turn_ms`, `budget_ms` and `mem_kb`, and the object `restart` (Section A.3).
`supervisor`	string	The world that crashes are reported to, as `fault` messages. It must have the `sys.admin` claim. Without one, the nucleus applies its own fixed policy to every world.
`endpoints`	object: name → endpoint	The endpoints: for each, a server, the message types it carries, and which worlds hold it with which rights (Section A.2). At most 64 after groups are expanded, held by at most 128 handles.
`worlds`	list of objects	The worlds, in the order the nucleus creates them and first starts them. At most 32.

A group is a shorthand resolved at boot, not an object that exists at run time. Wherever an endpoint names a group, the boot replaces it by its members. As a server, a group makes one endpoint per member (Section A.2); as a holder, it gives each member the same rights.

The shipped manifest's one group, `apps`, is the server of the `input` endpoint, through which the compositor sends `focus`, `blur` and `key` to whichever app is in front. A group lists at most 32 names; a group may not have the name of a world; and every member must be a world. Groups do not apply to `provides` and `needs`, which name channels, and are not resolved in `supervisor`.

`defaults` holds numbers and one object. A value in a world's entry overrides the value in `defaults`, which overrides the built-in default (Table A.6, Section A.5). Each value is checked as the world's own would be, and a bad one is reported with `defaults` in place of the world's name, except the minimums of `mbox` and `mbox_kb`, which are checked only in each world, so a bad default is reported once for every world that inherits it. `priority` and `passive` are not taken from `defaults`.

A.2 Endpoints

An endpoint names a server and the message types that may be sent or asked to it through the endpoint, and says which worlds hold it with which rights. Each key of `endpoints` names one, and its value has five fields, two of them required.

Table A.2. An endpoint's fields. Any other field is an error. Each list holds at most 16 names.

Field	Type	Meaning
`server`	string, required	The world that receives what is sent or asked through it, or a group: then one endpoint per member, named `name:member` (`input:launcher`, `input:sysmon`, ...).
`types`	list of message types, at least one	The types the endpoint carries. A type may not be a channel the server hears, nor `frames` or `worlds.status`, which the kernel publishes.
`send`	worlds and groups	They hold the endpoint with the right to send through it (`w`).
`ask`	worlds and groups	They hold it with the right to ask through it (`c`).
`grant`	worlds and groups	Adds the right to pass a copy on (`g`) to holders already listed under `send` or `ask`.

The boot turns each pair of an endpoint and a holder into one entry of the kernel's table of endpoint grants, with the union of the rights the three lists give that holder. A world listed under both `send` and `ask` holds one handle with rights `wc`. The code is short:

▸ nucleus/kernel.c

```
// rights per holder: send gives w, ask gives c, grant adds g to what the holder has
int rights[MAX_WORLDS] = { 0 };
for (int k = 0; k < 3; k++)
  for (int x = 0; x < nh[k]; x++) {
    uint64_t hs = resolve(hold[k][x], where, keys[k]);
    for (int w = 0; w < K.n_worlds; w++) if (hs >> w & 1) rights[w] |= k == 0 ? R_WRITE : k
        == 1 ? R_CALL : R_GRANT;
  }
for (int w = 0; w < K.n_worlds; w++) {
  if (!rights[w]) continue;
  if (!(rights[w] & (R_WRITE | R_CALL))) { merr("%s: %s is listed under \"grant\" only; grant
      adds the right to pass on what send or ask gives", where, K.worlds[w]->name); continue;
      }
  if (w == s) { merr("%s: %s would hold an endpoint to itself", where, K.worlds[w]->name);
      continue; }
  if ((rights[w] & R_WRITE) && K.worlds[s]->passive) merr("%s: %s only answers asks (it is
      passive); %s may ask it, not send to it", where, K.worlds[s]->name, K.worlds[w]->name);
  if (K.n_ep_grants == MAX_EP_GRANTS) { merr("more than %d endpoint handles", MAX_EP_GRANTS);
      break; }
  K.ep_grants[K.n_ep_grants++] = (ep_grant_t){ e, w, rights[w] };
  if (rights[w] & R_CALL) K.worlds[w]->static_asks |= 1ull << s;
}
```

The last line builds the ask graph: a holder with `c` may ask the server, and the asks of all worlds together must form no cycle (Chapter 6). The check is made once more at run time, whenever an endpoint handle that can ask is passed in a message, so the graph stays acyclic as authority moves.

A grant becomes a handle each time its holder starts. `endpoints_grant` in `nucleus/hw.c` makes the holder's endpoint handles before its file is evaluated, in the order of the grants (endpoints in the order of their keys), so they take the first numbers in the world's own table of 64. Each is badged with the holder's name, and the badge travels with every message sent through it, as the `badge` of the handler's third argument; a copy narrowed from it keeps the badge. These handles are made by the kernel (`minter` is `kernel` in the handle table) and count against no world's `handles` quota; the boot checks instead that they fit the kernel's 128-entry table beside the quotas. A holder loses its handles when it stops and gets new ones when it starts again. An endpoint names its server's slot, not one instance of it, so it outlives the server's restarts.

A world uses an endpoint with `send(to, type, payload)` and `ask(to, type, payload, opts)`, where `to` is the handle or the server's name; a name is resolved to the first of the sender's endpoint handles to that server that carries the type and the right. A world sends to itself without an endpoint, and may never ask itself. The world API (`handles()`, `narrow`, `revoke`) is in *Small Worlds*, Appendix A, and Chapter 13.

A.3 The supervisor and restart policies

On a crash the nucleus does the mechanism and a world does the policy:

▸ nucleus/kernel.c

```
// Supervision after the second pass. On a crash the nucleus does the mechanism (it discards the
    segment's
// effects, rejects the ask in progress, drops the world's handles, purges its timers and advances its
    epoch)
// and then reports the crash to the supervisor world named in the manifest, as a "fault" message, and
    leaves
// the world down until the supervisor calls sys.restart or sys.stop. The policy (backoff, how many
    crashes
// stop a world) is the supervisor's, read from each world's "restart" entry in the manifest. The
    nucleus keeps
// one fixed policy, the first pass's, for the supervisor itself, for a system without one, and for a
    crash the
// supervisor does not decide within a second; and a hard ceiling: 50 crashes in ten seconds stop any
    world.
enum { CEILING = 50 };
```

The `fault` message carries `{ world, why, handling, t, epoch, restarts, history, policy, charged_to }`. `history` holds the world's crash times in virtual ms since it was last started fresh (by a reload or an administrator's restart), at most the last 32, leaving out crashes charged to an asker. `policy` is the world's `restart` policy with every field filled in. `charged_to` names the asker when a passive world overran the budget of the asker it was answering, and is `null` otherwise; an out-of-memory crash or a thrown error while answering is the server's own. The supervisor answers with `sys.restart(name, { after_ms, epoch })` or `sys.stop(name, why, { epoch })`; with the epoch, a decision about an instance that is no longer the one that crashed is ignored. Until it decides, the crashed world stays down and posts nothing on `worlds.status`.

A world's `restart` object overrides `defaults.restart` field by field, which overrides the built-in policy.

Table A.3. The fields of a restart policy, with the built-in values. The nucleus checks their names and that each is a number, 0 or more; the supervisor applies them.

Field	Built in	Meaning
`backoff_ms`	100	The wait before the restart after the first crash in the period.
`max_backoff_ms`	2000	The wait doubles with each further crash in the period, up to this.
`intensity`	5	The crash in the period that stops the world; 0: the supervisor never stops it (the nucleus's ceiling still applies).
`period_ms`	10000	The window in which crashes are counted, in virtual ms.

The shipped supervisor, `js/system/super.js`, applies the policy and keeps no state of its own:

▸ js/system/super.js

```
  .on("fault", (s, f) => {
    s.decisions++;
    const p = f.policy;
    if (f.charged_to) {
      s.charged++; s.restarts++;
      s.last = `${f.world}: restarted at once; the crash was ${f.charged_to}'s`;
      return sys.restart(f.world, { after_ms: 0, epoch: f.epoch });
    }
    const recent = f.history.filter((t) => f.t - t < p.period_ms && t <= f.t);
    if (p.intensity > 0 && recent.length >= p.intensity) {
      s.stops++;
      s.last = `${f.world}: stopped after ${recent.length} crashes in ${p.period_ms / 1000} s`;
      return sys.stop(f.world, `crashed ${recent.length} times in ${p.period_ms / 1000} s`, { epoch:
          f.epoch });
    }
    // A world that crashed while starting waits at least 100 ms whatever its policy: an instance that
        cannot
    // start would otherwise be restarted as fast as the lane allows, and each restart posts on
        worlds.status.
    const floor = f.handling === "start" || f.handling === "" ? 100 : 0;
    const delay = Math.max(floor, Math.min(p.max_backoff_ms, p.backoff_ms * 2 ** Math.max(0,
        recent.length - 1)));
    s.restarts++;
    s.last = `${f.world}: restart in ${delay} ms (crash ${recent.length} in ${p.period_ms / 1000} s)`;
    sys.restart(f.world, { after_ms: delay, epoch: f.epoch });
  })
```

With the built-in policy a world waits 100, 200, 400 and 800 ms after its first four crashes in ten seconds and is stopped at the fifth. A crash charged to an asker is restarted at once and is left out of the history, so it never counts toward the stop; it still counts toward the nucleus's ceiling and, while the supervisor is down, toward the fallback policy, which counts every crash. A crash while starting waits at least 100 ms whatever the policy, so a world whose top-level code throws does not restart in a tight loop.

Four cases fall to the nucleus's own policy (`fallback_policy`): crashes of the supervisor itself, every crash in a system without a `supervisor`, a fault that does not fit the supervisor's mailbox or finds it stopped, and a crash the supervisor has not decided 1 s after it happened. That policy is the first pass's and ignores the manifest's `restart`: 100 ms, doubling with each crash in the last ten seconds (at most 2,000 ms), and stopped at the fifth. Two rules hold whatever any policy says. A world that crashes 50 times in ten seconds is

stopped and stays stopped. And a world whose file declares more than its entry gives (a channel, or a timer in a passive world) is stopped at once, without being reported, until its file or the manifest is fixed.

A.4 A world's entry

The first kind of key in a world's entry says what the world is and what it may use.

Table A.4. A world's identity, claims and resources.

Key	Type	Default	Meaning
`name`	string	required	How endpoints, groups and the supervisor name the world, and the seed of its random numbers. At most 31 characters (a longer name is cut). Unique. An entry without a name is called `undefined` (and one without a `file` reads `js/undefined`).
`file`	string	required	Its source, relative to the JavaScript root. The file must be readable at boot.
`claims`	list of strings	none	The hardware and system access it may obtain: `"i2c0:0xNN"` (an I2C address), `"spi2"`, `"dma"`, `"sys.read"`, `"sys.admin"`. At most 16. Two worlds may not claim the same `i2c…` or `spi…` device.
`budget_ms`	number, 0 or more	50	The budget of each segment of its turns, in virtual ms. A passive world answering an ask runs on its asker's budget instead; its own applies only to its `start` handler.
`mem_kb`	number, 0 or more	1024	Its heap limit, in KB.
`priority`	number	from its kind	Its priority on the lane; higher runs first. Without it the world's kind decides when it first starts: driver 3, service 2, app 1. Not checked. A passive world may not have one.
`passive`	true or false	false	A passive world answers asks only, at the priority and on the budget of the world that asked, and the lane time is billed to the asker (`cpu_ms`) and counted in its own `served_ms`. It may not hear or provide channels, may not have `priority`, must be asked through some endpoint and may be sent to through none, and its file may declare no timer. It reports priority 0. Any value JavaScript treats as true counts.

The second kind gives the world its channels. Whom it may send to and ask is no longer in its entry; it is the endpoints that name it as a holder.

Table A.5. A world's channels. Each list holds at most 16 names; an absent list is empty.

Key	Names	Meaning
`provides`	channels	The channels it may publish. A channel has at most one provider. `frames` and `worlds.status` are published by the kernel and no world may provide them.
`needs`	channels	The channels it hears: every post on one of them is delivered to it, unless its mailbox is full. A need that no world provides is noted in the log, not refused.

The world's file still calls `.provides(...)` and `.needs(...)`, but those calls grant nothing. When a world starts, `read_meta` in `nucleus/vm.c` compares each channel its file declares with its entry, and a world that declares a channel its entry does not list is stopped with the reason (Section A.8). A world that declares fewer channels than its entry lists still hears all the channels in `needs`, because the kernel delivers posts by the entry. Channel messages arrive only from the channel's provider, by `publish`: no endpoint may carry a channel its server hears, and a world that sends such a type to itself is refused.

The third kind bounds the world's share of the kernel's tables.

Table A.6. A world's quotas, with the built-in defaults. The defaults object overrides the defaults; an entry overrides both.

Key	Default	Meaning
`handles`	4	Handles it may have made and not yet freed: opened with `hw.i2c` or `hw.spi`, allocated with `dma.alloc`, or narrowed with `narrow` (an endpoint copy included). A handle counts against the world that made it, wherever it is held, until it is freed; one kept in the table because copies narrowed from it are alive still counts. The endpoint handles the manifest grants count against no world.
`ask_slots`	8	Asks it may have waiting for replies at once. A slot is taken when the ask is made and released when the ask settles.
`mbox`	64	Messages its mailbox may hold. A message sent, asked or posted to the world counts from the moment it is sent until it leaves the mailbox (handled, failed, taken out or dropped), so a burst sent in one segment is bounded too. Timer ticks and the start message are not counted. At least 1.
`mbox_kb`	128	Kilobytes of those messages (payload JSON and type name) its mailbox may hold, counted the same way. At least 64, so that the largest message (65,536 bytes) fits.
`bufs`	1	Framebuffers it may have allocated at once. Only a world with the `dma` claim can allocate one. A framebuffer counts against the world that allocated it until every handle to it, narrowed copies included, is freed.
`turn_ms`	8000	How long one turn may last, in virtual ms, from the start of its first segment. A turn still unfinished then is a crash. Only a turn that has gone asynchronous (an `async` handler awaiting a promise) can outlive its segment, so only such turns are watched. 0 turns the deadline off.

The fourth kind is the world's `restart` policy (Table A.3), an object whose fields override `defaults.restart` one by one.

The default `turn_ms` is longer than the 5,000 ms timeout with which the REPL asks the sandbox, so that the REPL's own turn ends by the ask's timeout before its deadline arrives (Chapter 6). Framebuffer pixels are still not charged to `mem_kb`; their number is bounded by `bufs` instead.

A.5 Defaults and limits

The built-in defaults of the quotas are one enumeration in `nucleus/kernel.c`, next to the function that reports a problem, and those of the restart policy are the first line of `mpolicy`:

▸ nucleus/kernel.c

```
// Every problem is one line on stderr ("js/system.json: ..."), and any problem stops the boot.
// Defaults for the quotas; a manifest's "defaults" object overrides them, a world's entry overrides
    that.
enum { DEF_HANDLES = 4, DEF_ASKS = 8, DEF_MBOX = 64, DEF_MBOX_KB = 128, DEF_BUFS = 1, DEF_TURN_MS =
    8000 };
…
static const char *POLICY_KEYS[] = { "backoff_ms", "max_backoff_ms", "intensity", "period_ms" };
static char *mpolicy(JSContext *ctx, JSValue o, const char *base, const char *where) {
  double f[4] = { 100, 2000, 5, 10000 };
```

`budget_ms` defaults to 50 and `mem_kb` to 1024. The limits are the sizes of the kernel's static tables, fixed when the nucleus is compiled. The boot checks the sums of the quotas, with the endpoint handles, against the tables, which is what makes a quota a guarantee: with every world's share inside the table, no world can exhaust it for the others.

Table A.7. Fixed limits (nucleus/nucleus.h and nucleus/kernel.c) and how the boot uses them.

Limit	Value	Checked at boot
Handle table (`MAX_HANDLES`)	128	The sum of every world's `handles` plus the endpoint handles
Endpoints (`MAX_ENDPOINTS`)	64	After a group as server is expanded to one endpoint per member
Endpoint handles (`MAX_EP_GRANTS`)	128	One per endpoint and holder
Handle numbers per world (`MAX_SLOTS`)	64	Not at boot; a 65th handle is not made (`out of handles`) or, arriving in a message, not installed
Ask table (`MAX_ASKS`)	1,024	The sum of every world's `ask_slots`
Framebuffers (`MAX_BUFS`)	16	The sum of `bufs` over the worlds with the `dma` claim
Worlds (`MAX_WORLDS`)	32	The length of `worlds` ; at most the first 32 are read
Names in one list (`MAX_LIST`)	16	`claims` , `provides` , `needs` , and an endpoint's `types` , `send` , `ask` , `grant`
Groups	16	The number of keys in `groups`
Members of one group	32	The length of each group's list
Name length (`NAME_LEN`)	31 characters	Not reported; longer names (of worlds, endpoints, types and channels) are cut
Largest message (`MAX_MSG_BYTES`)	65,536 bytes	The lower bound of `mbox_kb` (64)
Crash ceiling (`CEILING`)	50 in 10 s	At run time: the world is stopped whatever its policy
Supervisor's decision	1 s	At run time: then the nucleus's own policy

A.6 What the boot checks

`kernel_boot` makes its checks in a fixed order, and the messages appear in that order.

1. The file must exist and parse as JSON. If it does not, the boot fails at once with one message and no count.

2. `defaults` is read: each number must be a number, 0 or more, and `restart` an object with only the four policy fields, each a number, 0 or more.
3. `groups` is read: it must be an object whose values are lists of names; at most 16 groups of at most 32 names.
4. Each world's entry is read in order: its name must not repeat an earlier one; each number must be a number, 0 or more; each list must be a list of strings, of at most 16; `restart` as in `defaults`; `mbox` must be at least 1 and `mbox_kb` at least 64; its file must be readable.
5. Groups are checked against the worlds: no group may have the name of a world, and every member must be a world.
6. No world may have `sends` or `asks`. Each world's `passive` is read.
7. Endpoints, in the order of their keys: `endpoints` must be an object; each endpoint's server must be a world or a group, its `types` a non-empty list, its holder lists lists of names, and it may have no other field. Then, for each world it serves: there must be room for one more endpoint; no type may be a channel the server hears or the kernel publishes; every holder must be a world or a group; no holder may be listed under `grant` only, be the server itself, or hold `w` on a passive server; and there must be room for one more endpoint handle.
8. Passive worlds: none may hear or provide a channel or have `priority`, and each must be asked through some endpoint.
9. `supervisor`, if present, must be a string naming a world with the `sys.admin` claim.
10. Channels: no world may provide `frames` or `worlds.status`, and no channel may have two providers. A need that nobody provides is logged as a note.
11. Devices: no two worlds may claim the same claim that begins with `i2c` or `spi`.
12. The asks the endpoints allow must form no cycle. A depth-first search reports the first cycle it finds, with the worlds on it in order.
13. The quota sums, with the endpoint handles, must fit the kernel's tables.

The last step is the shortest code in the boot and the one the quotas depend on:

▸ nucleus/kernel.c

```
// the quotas fit the kernel's tables, so no world can exhaust one for the others
long hs = 0, as = 0, bs = 0;
for (int i = 0; i < K.n_worlds; i++) {
  hs += K.worlds[i]->q_handles; as += K.worlds[i]->q_asks;
  if (world_claims(K.worlds[i], "dma")) bs += K.worlds[i]->q_bufs;
}
if (hs + K.n_ep_grants > MAX_HANDLES) merr("the worlds' handle quotas add up to %ld and the
    manifest's endpoints need %d more; the kernel's handle table holds %d", hs, K.n_ep_grants,
    MAX_HANDLES);
if (as > MAX_ASKS) merr("the worlds' ask_slots add up to %ld; the kernel's ask table holds %d", as,
    MAX_ASKS);
if (bs > MAX_BUFS) merr("the framebuffer quotas of the worlds with the dma claim add up to %ld; the
    kernel has %d framebuffers", bs, MAX_BUFS);
free(groups); groups = NULL; n_groups = 0;
if (merrs) { fprintf(stderr, "%s: %d problem%s; the system does not boot\n", manifest, merrs, merrs
    == 1 ? "" : "s"); return -1; }
klog("booting %d worlds from %s", K.n_worlds, manifest);
```

Two kinds of check come later, because they need the worlds to have run. When a world starts (at boot and at every restart or reload), its file's `.provides(...)` and `.needs(...)` are compared with its entry, and a passive world's file may declare no `.every(...)` timer. Once every world has had its first start, `boot_notes` logs each ask that goes to a world of lower priority, leaving out asks to passive worlds, which run at their asker's priority. The seL4 Microkit allows a protected procedure call only to a higher priority (Chapter 4); PicoCalc OS allows the lower one and notes it, since such an ask waits behind every world in between. The shipped manifest produces no such note: its one low world, the sandbox, is passive.

A manifest that breaks several rules shows how the report reads. This one adds a `clock` app whose file does not exist and which provides `keys`, keeps the first pass's `"sends": ["compositor"]` on the launcher, adds an endpoint `tick` to a misspelt `compositr`, and adds two endpoints by which the launcher may ask `clock` and `clock` may ask the launcher:

```
$ ./build/picocalc-host -t 10
js/system.json: world "clock": cannot read js/apps/clock.js
js/system.json: world "launcher": "sends" is gone; whom a world may send to and ask is now the
    endpoints the manifest's "endpoints" grant it (see SECOND-PASS.md)
js/system.json: endpoint "tick": "server" names "compositr", which is neither a world nor a group
js/system.json: channel "keys" has two providers, stm32 and clock; a channel has one
js/system.json: asks form a cycle: launcher → clock → launcher (each would wait for the other's turn to
    end; asks must form no cycle)
js/system.json: 5 problems; the system does not boot
```

The host build exits with status 1; in the WebAssembly build `pc.boot()` throws an error whose code is `boot_failed`, and the lines arrive on the stderr log.

A.7 Boot errors

Every message in Table A.8 is printed after the manifest's path and a colon (`js/system.json:`). The words in quotation marks and the numbers are the ones from the manifest; the examples are real output of the host build on broken copies of the shipped manifest.

Table A.8. Boot errors, in the order the boot can report them. Each counts as one problem, except the first two, which stop the boot at once.

Message
`cannot read js/system.json` (without the path prefix)
`SyntaxError: Unexpected end of JSON input`, followed by a line `at js/system.json:1:13`
`defaults: "turn_ms" must be a number, 0 or more`
`defaults: "restart" must be an object`
`defaults: "restart": "backoff_ms" must be a number, 0 or more`
`defaults: "restart" has no field "bogus" (it has backoff_ms, max_backoff_ms, intensity, period_m`
`"groups" must be an object of name lists`
`group "g": "members" must be a list of names`
`group "g": "members" lists 33 names; at most 32`
`at most 16 groups`
`33 worlds; the kernel runs at most 32`
`two worlds are called "snake"`
`world "stm32": "handles" must be a number, 0 or more`
`world "power": "claims" must be a list of names`
`world "display": "needs" lists 17 names; at most 16`
`world "flaky": "restart": "intensity" must be a number, 0 or more`
`world "compositor": "mbox" must be at least 1`
`world "launcher": "mbox_kb" must be at least 64, so the largest message fits`

Message
`world "sysmon": cannot read js/apps/x.js`
`group "launcher" has the name of a world`
`group "g" lists "nobody", which is not a world`
`world "launcher": "sends" is gone; whom a world may send to and ask is now the endpoints the man`
`"endpoints" must be an object of endpoints`
`endpoint "e": "server" names "nobody", which is neither a world nor a group`
`endpoint "e": "types" must list the message types it carries`
`endpoint "e": "send" must be a list of names`
`endpoint "e": no field "asks" (an endpoint has server, types, send, ask and grant)`
`more than 64 endpoints`
`endpoint "e": "keys" is a channel; channel posts come from its provider, not through an endpoint`
`endpoint "e": "send" names "nobody", which is neither a world nor a group`
`endpoint "e": repl is listed under "grant" only; grant adds the right to pass on what send or as`
`endpoint "e": snake would hold an endpoint to itself`
`endpoint "bad": sandbox only answers asks (it is passive); launcher may ask it, not send to it`
`more than 128 endpoint handles`

Message
`world "sandbox" is passive; it answers asks only and may not hear or provide channels`
`world "sandbox" is passive; it runs at the priority of whoever asks it, so it has no "priority"`
`world "snake" is passive but no endpoint lets any world ask it`
`"supervisor" names "nobody", which is not a world`
`the supervisor snake needs the sys.admin claim, to restart and stop worlds`
`"supervisor" must be the name of a world`
`world "repl": "frames" is published by the kernel; no world may provide it`
`channel "keys" has two providers, stm32 and snake; a channel has one`
`stm32 and dup both claim i2c0:0x1f; a device has one driver (put both jobs in one world, and let`
`asks form a cycle: repl → sandbox → repl (each would wait for the other's turn to end; asks must`
`the worlds' handle quotas add up to 123 and the manifest's endpoints need 17 more; the kernel's`
`the worlds' ask_slots add up to 1400; the kernel's ask table holds 1024`
`the framebuffer quotas of the worlds with the dma claim add up to 17; the kernel has 16 framebuf`
`5 problems; the system does not boot`

A manifest without problems produces one line in the kernel log, `booting 14 worlds from js/system.json`, preceded by any notes about needs that nobody provides.

A.8 Messages at start and at run time

The manifest's rules also act while the system runs. A refused call throws an `Error` in the calling world, whose message is the kernel's; a world that catches it carries on. The examples come from probe worlds run on the host build, `bench/secondpass-after.txt`,

`bench/rogue-second.txt` and `bench/firstpass-second.txt`; X, N and E stand for a world, a number and an endpoint where the message is given in its general form.

Table A.9. Refusals at the call. The first word is the calling world's name unless the row says otherwise.

Message
`talker may not send display: it holds no endpoint to display (the manifest's "endpoints" grant t`
`talker may not send compositor "focus": its endpoints to compositor carry only "open"`
`talker may not ask compositor "open": its endpoint for it lets it send but not ask`
`X may not send Y "t": its endpoint was revoked`
`there is no world called "nobody"`
`endpoint handle 2 may not send`
`endpoint tk-ping does not carry "boom"`
`talker holds no handle 99`; `handle 1 was revoked`; `handle 5 is a dma handle, not endpoint`
`"keys" is a channel talker hears; only its provider can post it, by publish`
`a world cannot ask itself (it would wait for its own turn to end)`
`echo only answers asks; ask it instead of sending`
`a message is at most 65536 bytes of JSON; this one is N`

Message
`a reply is at most 65536 bytes of JSON; this one is N`
`its screen is N bytes of JSON; a frame is at most 65536`
`handle 1 cannot be passed on by taker`
`p cannot pass that endpoint to r: it would let r ask q, which can already reach r by asks: asks`
`flaky's mailbox is full (64 messages, 0 KB queued)`
`flaky is stopped`; `too many asks are waiting for replies`; `faults already has 8 asks waiting for r`
`no reply from flaky within 500 ms`
`X crashed: why`; `about was reloaded`; `X was restarted by an administrator`; `X was stopped: why`
`rogue_handles has used its quota of 4 handles`

Message
`rogue_handles has used its quota of 1 framebuffer`
`out of handles`
`only the world that narrowed handle N can revoke it`
`X has no claim on sys.read`

`send` checks, in turn, the receiver and the endpoint, the channel rule, whether the receiver is passive, the payload's size, the handles it carries and the receiver's mailbox. `ask` makes the same checks except the passive one, and refuses an ask to itself after the channel rule; after that it can no longer throw, and the kernel rejects its promise when the segment ends if the callee is stopped, the ask table is full, the caller's slots are used or the callee's mailbox is full.

Table A.10. Messages at start, from supervision and in the kernel log.

Message
`rogue_key declares provides("keys"), but its manifest entry does not give it that channel`
`tick is passive, but its file declares a timer (every 100 ms); a passive world runs only to answ`
`rogue_key cannot start until its file or its manifest entry is fixed; it stays stopped`
`its turn handling "start" did not finish within its turn_ms of 1000 ms`
`ran past repl's budget of 200 virtual ms, which it runs on while answering repl's ask`
`flaky stopped: stopped by super: crashed 5 times in 10 s`
`about restarted by an administrator`
`the supervisor did not decide about flaky within 1 s; the nucleus restarts it by its own policy`
`flaky crashed 5 times in ten seconds; it stays stopped`
`bomb crashed 50 times in ten seconds, the nucleus's ceiling; it stays stopped whatever its super`

Message
`X's handle table is full (64); a handle sent to it was not installed`
`X: no room for its endpoint E`
`slowsub's mailbox is full: dropped 1 post so far (the latest on spam)`
`note: keys needs "clock", which no world provides`
`note: display asks flaky, which runs at a lower priority (2 below 3): its asks wait behind every`
`about reloaded from apps/about.js`
`cannot reload nosuch: there is no world called that`
`cannot reload about: cannot read js/apps/about.js`

A crashed world's state is visible in three places besides the log. Its `#OCT1` frame on the UART carries `world`, `file`, `handling`, `why`, `crashes` (in the last ten seconds), `mem_kb`, `limit_kb` and `detail`, the stack. On `worlds.status` it posts `crashed` (with `in_ms`, the wait

before the restart) when the restart is decided, `stopped` when it is stopped, `running` when a new instance has started and `reloading` on a reload. And the supervisor's decision is the `fault` message of Section A.3.

A.9 The shipped manifest

This is the manifest PicoCalc OS ships with:

▸ js/system.json

```
{
  "groups": {
    "apps": ["launcher", "sysmon", "repl", "snake", "faults", "keys", "about"]
  },
  "defaults": { "handles": 4, "ask_slots": 8, "mbox": 64, "mbox_kb": 128, "bufs": 1, "turn_ms": 8000,
                "restart": { "backoff_ms": 100, "max_backoff_ms": 2000, "intensity": 5, "period_ms":
                    10000 } },
  "supervisor": "super",
  "endpoints": {
    "open":      { "server": "compositor", "types": ["open"],                "send": ["launcher"] },
    "attach":    { "server": "display",    "types": ["attach"],              "send": ["compositor"] },
    "flush":     { "server": "display",    "types": ["flush"],               "ask":  ["compositor"] },
    "screen":    { "server": "power",      "types": ["screen"],              "send": ["compositor"] },
    "input":     { "server": "apps",       "types": ["focus", "blur", "key"], "send": ["compositor"] },
    "eval":      { "server": "sandbox",    "types": ["eval"],                "ask":  ["repl",
        "faults"] },
    "boom":      { "server": "flaky",      "types": ["boom"],                "send": ["faults"] },
    "flaky":     { "server": "flaky",      "types": ["ping", "slow"],        "ask":  ["faults"] },
    "stats":     { "server": "stm32",      "types": ["stats"],               "ask":  ["keys"] },
    "backlight": { "server": "stm32",      "types": ["backlight"],           "ask":  ["power"] }
  },
  "worlds": [
    { "name": "super",      "file": "system/super.js",      "claims": ["sys.admin"],
        "budget_ms": 20,  "mem_kb": 256, "priority": 4 },
    { "name": "stm32",      "file": "drivers/stm32.js",     "claims": ["i2c0:0x1f"],
        "budget_ms": 20,  "mem_kb": 320,
      "provides": ["keys", "battery"] },
    { "name": "power",      "file": "system/power.js",
        "budget_ms": 20,  "mem_kb": 256,
      "needs": ["battery"], "provides": ["power"] },
    { "name": "display",    "file": "drivers/display.js",   "claims": ["spi2"],
        "budget_ms": 30,  "mem_kb": 320 },
    { "name": "compositor", "file": "system/compositor.js", "claims": ["dma"],
        "budget_ms": 80,  "mem_kb": 768,
      "needs": ["frames", "keys", "power", "worlds.status"],
      "handles": 6, "bufs": 2, "mbox": 128 },
    { "name": "launcher",   "file": "apps/launcher.js",     "claims": ["sys.read"],
        "mem_kb": 384 },
    { "name": "sysmon",     "file": "apps/sysmon.js",       "claims": ["sys.read"],
        "mem_kb": 512 },
    { "name": "repl",       "file": "apps/repl.js",
        "budget_ms": 200, "mem_kb": 384 },
    { "name": "sandbox",    "file": "system/sandbox.js",
        "mem_kb": 512, "passive": true,
      "restart": { "backoff_ms": 0, "intensity": 0 } },
    { "name": "snake",      "file": "apps/snake.js",
        "mem_kb": 320 },
    { "name": "faults",     "file": "apps/faults.js",       "claims": ["sys.admin", "sys.read"],
        "mem_kb": 384,
      "needs": ["worlds.status"] },
    { "name": "flaky",      "file": "system/flaky.js",
        "mem_kb": 256 },
```

```
    { "name": "keys",       "file": "apps/keys.js",
        "mem_kb": 320,
      "needs": ["keys"] },
    { "name": "about",      "file": "apps/about.js",        "claims": ["sys.read"],
        "mem_kb": 320 }
  ]
}
```

The graph it declares is the one the code uses, and no more. The ten keys of `endpoints` make 16 endpoints (`input` makes seven, one per app) and 17 endpoint handles. The launcher sends the compositor `open`. The compositor sends the display driver `attach` and asks it `flush`, sends the power service `screen`, and sends every app `focus`, `blur` and `key`. The REPL and the Faults app ask the sandbox `eval`; Faults sends `flaky` `boom` and asks it `ping` and `slow`; the keys app asks the `stm32` driver `stats`; the power service asks it `backlight`. The `stm32` driver provides `keys` and `battery`, and the power service `power`; the power service needs `battery`, the compositor `frames`, `keys`, `power` and `worlds.status`, the keys app `keys`, and Faults `worlds.status`. The asks (compositor → display, repl and faults → sandbox, faults → flaky, keys and power → stm32) form no cycle, and every one goes to a higher priority or to the passive sandbox, so the boot logs no note.

The quotas fit with room to spare. Thirteen worlds take the default of 4 handles and the compositor 6, 58 entries, and the endpoint handles take 17 more: 75 of the 128. Fourteen worlds take 8 ask slots, 112 of 1,024; and the compositor, the only world with `dma`, takes 2 of the 16 framebuffers.

Five entries carry the second pass's choices. The supervisor, `super`, has `sys.admin` and priority 4, above the drivers, so that no driver's segment delays a decision. The STM32 at `i2c0:0x1f` has one driver, `stm32`, which serves keys, battery and backlight; the power service holds no hardware and asks it. The sandbox is passive and has no budget of its own that matters: an evaluation from the REPL runs on the REPL's 200 ms, which is why the REPL's `budget_ms` is 200, and one from the Faults app on Faults' default 50 ms. Its policy, `{ "backoff_ms": 0, "intensity": 0 }`, restarts it at once and never stops it (the ceiling of 50 crashes in ten seconds still applies); its other two fields come from `defaults.restart`. Every other world takes the default policy.

The compositor's `bufs` is 2, not 1, because of how a framebuffer is charged. The compositor passes a read-only copy of its framebuffer to the display driver, and when the compositor crashes, that copy keeps the old framebuffer, and its charge, alive until the display

driver lets go of it, which happens after the new instance has allocated its own. With `"bufs": 1` a crashed compositor cannot start again: in the host build, after `wait 1500` and `crash compositor`, it fails at every restart with `compositor has used its quota of 1 framebuffer (system/compositor.js:378:21)`, and the supervisor stops it at the fifth crash with `compositor stopped: stopped by super: crashed 5 times in 10 s`. Its `mbox` is 128; it is the world that hears the most channels, and every app's frames, every key and every power and status post arrive in its mailbox.

`bench/system-bench.json` declares the benchmark worlds' endpoints in the same way. `bench/secondpass.py`, `bench/rogue.py` and `bench/firstpass.py` add their worlds to a copy of `js/system.json`, and `grant` in `bench/authority.py` adds the endpoint each scenario needs: `typed` is given an endpoint to the compositor that carries only `open`, `giver` one to the sandbox with `c` and `g`, and the passive-server check an endpoint that lets the launcher send to the sandbox, which does not boot.

FURTHER READING

`SECOND-PASS.md`, in the repository: the record of the second pass, the code it changed, its numbers and how it was verified.

[First pass 2026] The record of the first pass, whose `sends` and `asks` the endpoints replaced.

[Small Worlds 2026] Appendix A gives the world API that the manifest's claims, channels, endpoints and quotas govern.

Chapter 5 explains `provides`, `needs` and the channel-type rule; Chapter 6 the acyclic asks and `turn_ms`; Chapter 7 the quotas and the boot-time sums; Chapter 12 the design of endpoints, passive servers, one driver per device and supervision in a world; Chapter 13 how they were built and measured.

APPENDIX B

The workbench's API

The workbench (`workbench/picocalc-workbench.html` , built by `workbench/build.py` from `workbench/src/`) puts the real system in a page and gives a person and an agent four ways to work with it. The agent's one tool, `eval` , runs JavaScript against a curated scope (`agentScope` in `workbench/src/agent.js`). Evals are functions in `workbench/evals.js` (shipped as `workbench/src/evals.js`) run against fresh systems by `host.runEvals` in `workbench/src/core.js` . `wb:` links in the agent's answers, the Reference and the Book drive the workbench's views (`wbLink` , views in `workbench/src/views.js` , the header, dock and run loop in `workbench/src/main.js`). Underneath all three is the JavaScript API of the WebAssembly build, `wasm/picocalc.js` . The Reference the person and the agent read is `workbench/src/reference.md` , with the manifest's schema (`workbench/src/manifest-schema.md`) put in its place at build time. This appendix is the reference for each, checked against the code; Chapter 9, Chapter 10 and Chapter 11 explain the design, and Chapter 12 designed the instruments added since (the apply report, Read-only that holds, the step guard, the trace diff, the pinned baseline, recorded and generated evals). Times are virtual milliseconds unless a row says otherwise.

B.1 The eval tool

The agent sees one tool, defined in `AgentView` :

▸ workbench/src/agent.js

```
    const evalTool = {
      name: "eval",
      …
      inputSchema: { type: "object", properties: { code: { type: "string", description: "the body of an
          async function; return what you want to see" } }, required: ["code"] },
```

`runEval` builds a new scope for every step, so a `const` declared in one step does not exist in the next. It instruments the step's code (Section B.2), passes each name of the scope, the guard and, in Read-only, the shadowed page globals as parameters of a new strict-mode function whose body is `return (async () => { CODE })();`, and awaits the result, racing it against the time limit and the Stop button. What the step returns comes back as text.

- A returned value is written as YAML by `yaml` in `agent.js`. Numbers are rounded to five decimals; a list of numbers is written on one line, at most 64 of them; other lists show at most 60 items and objects at most 80 keys; nesting deeper than five levels is elided. A function is written as `[function name]`, a system as `[System t=… s]` and a byte array as `[bytes N]`.
- Anything passed to `console.log`, `console.info` or `log` comes first, under `logs:`, followed by a line `---`; `console.warn` and `console.error` lines are prefixed `warn:` and `error:`.
- A step that returns nothing and logs nothing gives `undefined (use return to see a value)`.
- A step that throws gives `Error:` and the message, followed by `(line N)` with the line of the step's code when the stack names it.
- A step that ran without the loop guard starts with `note: this step ran without the loop guard (WHY), so a synchronous endless loop in it cannot be stopped.`, where WHY is `acorn did not load` or `acorn could not parse it:` and acorn's message.
- A failed step is returned to the model as a tool error.
- The text is cut at 9,000 characters:

▸ workbench/src/agent.js

```
  if (text.length > 9000) text = text.slice(0, 9000) + `\n… cut (${text.length} chars). Return less:
      pick fields, slice arrays.`;
```

The agent panel has three modes. In Auto each step runs as it comes. In Ask each step waits for the person's Run this step or Decline; a declined step shows `The person declined this step.` and returns `Declined by the person. Ask what they want instead, or explain the change without running it.` to the model. Read-only is described in Section B.3. Every step keeps a snapshot of all the files taken before it ran, and a step that changed files offers Revert files to before this step: the files return to the snapshot, the step and every later step of the turn are marked reverted, and the files are applied. The panel also chooses the model (Fast, Balanced, Deep), the thinking effort (Auto, Low, Med, High) and, where the host allows images, Attach screen, which sends a picture of the panel with each message.

B.2 The step's time limit

A step stops after 30 s of real time, a synchronous `for (;;) {}` included. Before the step runs, `instrumentStep` parses it with acorn (as a script, with `return` and `await` allowed at the top level) and puts a call to `__wb_guard()` at the top of the body of every `for`, `for…in`, `for…of`, `while` and `do` loop and of every function, function expression and arrow function. A loop body that is not a block is wrapped in braces, and an arrow's expression body becomes `(__wb_guard(), expression)`. Every insertion goes on the line where the body starts, so the line numbers in errors stay right. The guard reads the clock only every few calls:

▸ workbench/src/agent.js

```
  const fn = () => {
    if (dead) throw new Error(dead);
    if (++n < stride) return;
    n = 0;
    const now = performance.now();
    stride = now - last < 0.5 ? Math.min(stride * 2, 65536) : Math.max(1, stride >> 1);
    last = now;
    if (now > deadline) { dead = msg; throw new Error(msg); }
    if (signal && signal.aborted) { dead = "stopped"; throw new Error("stopped"); }
  };
```

The stride doubles, up to 65,536 calls, while calls come faster than one per half millisecond, and halves when they come slower, so a tight loop pays a counter and a heavy one is checked on every call. In the headless tests a loop that adds 2×10^7 numbers took 78 ms

guarded and 13 ms unguarded, about 3 ns of real time per iteration (`workbench/tests/features.out.txt`). Once the guard has thrown, every later call throws the same error, so a step that catches it cannot carry on. The errors are `eval timed out after 30 s` at the deadline, `stopped` after the Stop button, and `this eval step has already ended` for code the step left running (an un-awaited loop, a timer), which stops at its next loop or call; the page does not report these as uncaught errors. An `async` step is also raced against a 30 s timer, which ends it even while it awaits. `host.evalTimeoutMs` changes the limit (the tests set 2 s). If acorn did not load or cannot parse the step, it runs as written and its result begins with the note of Section B.1.

B.3 Read-only

Read-only makes three guarantees, each checked by `workbench/tests/features.js`.

1. Eight scope functions throw `read-only mode: NAME is off`: `files.edit`, `files.write`, `files.remove`, `apply`, `reboot`, `evals.pin`, `evals.unpin` and `evals.record`.
2. The step's parameters shadow these page globals with `undefined`: `host`, `store`, `window`, `globalThis`, `self`, `document`, `localStorage`, `sessionStorage`, `indexedDB`, `fetch`, `parent`, `top`, `frames`, `opener`, `XMLHttpRequest` and `WebSocket`. `book` stays the scope's own.
3. The shadowing is no fence, since the `Function` constructor reaches the page, so the protected state is copied before the step and compared after it. That state is the files, the shipped files, what is applied to the live system, the live system itself (the object and its `PicoCalc` instance), the code loaded into it since, the pinned baseline, and the copies of the edits and the baseline in the browser's local storage. Whatever changed by any route is put back: a file is restored, a replaced or rebooted live system is swapped back (still running), code loaded around `apply` is loaded again from what is applied and its worlds reloaded, and the baseline and the stored copies are restored. The step then fails with `Error: read-only: this step changed X, by a route around the scope; the change was undone.`, where X lists what changed (`the file js/apps/snake.js`, `the live system (it was rebooted or replaced)`, `the pinned baseline`, ...), followed by the step's own error if it had one.

Changes the person makes in the page while the step runs (typing in Code, Apply, Pin, ↻) are left alone. Read-only does not undo driving the live system (`key`, `type`, `run`, `crash`, `restart`, `battery`), eval runs and their recorded results, fresh systems, diffs, the views,

network requests or anything else outside the protected state: a step that reaches around the scope can do what the page can while it runs, and the protected state is as it was when it ends.

B.4 The scope

These names are in scope in every step: `files`, `manifest`, `apply`, `reboot`, `run`, `key`, `type`, `crash`, `restart`, `battery`, `worlds`, `world`, `handles`, `stats`, `frame`, `text`, `screen`, `uart`, `faults`, `messages`, `test`, `evals`, `diff`, `property`, `book`, `open`, `focus`, `select`, `console`, `log` and `wb`. A path argument is resolved by dropping a leading `./` or `/` and trying it as given, then with `js/` in front, then with `workbench/`; an unknown path throws `no file P (files.list() shows the paths)`.

Table B.1. Files and applying edits. The files are the OS's files under js/ and the workbench's own (workbench/evals.js), with the person's edits, kept in the browser's local storage.

Name	Returns	Notes
`files.list()`	`[{ path, lines, edited?, applied? }]`, sorted by path	`edited: true` when the file `applied: false` when an O
`files.read(path, from?, to?)`	The file's text	With `from`, only lines `from`
`files.grep(pattern, path?)`	Up to 80 strings `file:line text`	`pattern` is a regular express at 160 characters. Without `pa`
`files.edit(path, find, replace)`	`{ path, line, applied }`	Replaces the one occurrence `find` began. Throws `edit: text not found in` or `edit: text occurs N t`
`files.write(path, src)`	`{ path, lines }`	Creates or replaces a file. A ne gets `js/` in front.
`files.remove(path)`	`{ removed }`	Removes the file.
`files.diff(path)`	`no change`, or one hunk	The hunk is `@@ line N @@` f (`+`), against the file the page
`manifest()`	`js/system.json`, parsed	As edited, applied or not; `nul`
`apply(path?)`	The report of Table B.2	Applies one OS file, or every u each world that runs it is start count cleared, mailbox kept). prelude or a file no world runs Then the system runs until ea `js/` need no applying.
`reboot()`	`{ ok, errors }`	A new live system from the fil

`apply` tells the truth about what happened. A reload only starts a world again, and a source that throws crashes a moment later, so after a reload or a reboot `settleApply` runs the system in steps of 2 ms, for at most 200 ms, until each watched world is idle (started) or has a fault frame, is restarting or is stopped (crashed). The watched worlds are those that run a reloaded file; after a reboot caused by the manifest or the prelude, every world.

Table B.2. Apply's report. A key marked ? is present only when it has something to say.

Key	Meaning
`ok`	False when a reload failed, the reboot did not boot, or a watched world crashed.
`reloaded`	The worlds hot-reloaded (not after a reboot).
`rebooted`	`true` when the system was rebooted instead.
`errors`	One line per problem: `w: reason` for a failed reload, up to 12 boot lines for a reboot that did not boot, and `about crashed 0.05 ms after the reload: SyntaxError: …` (or `after the reboot`) for each watched world that crashed.
`watched_ms`	How long the system ran after the apply, in virtual ms.
`started` ?	The watched worlds that started.
`still` ?	The watched worlds still starting after 200 ms.
`crashed` ?	`[{ world, after_ms, state, last_error, faults }]`, where each fault is `{ t, why, file, handling?, crashes, detail? }`, `detail` being the first four lines of the stack.
`nothing` ?	`every file is already applied`, with `ok: true`, when there was nothing to apply.

The Code view shows a failed apply under the editor of the file it concerns, with each `(file.js:line:column)` in the errors as a link to that line.

The names in Table B.3 act on the live system, the one the Device view shows. Each throws `the system is not running` when there is none. When the live system did not boot, `run` throws `the system did not boot:` and the boot lines, and `key` and `type` throw `command failed:` and the command.

Table B.3. The live system.

Name	Returns
`run(ms = 500)`	`{ t_s }`
`key(name, holdMs = 60)`	`{ t_s }`
`type(text)`	`{ t_s }`
`crash(world)`	A sentence, such as `snake killed at 3.214 s`
`restart(world)`	`X restarted`
`battery(pct, charging)`	`{ pct, charging }`
`worlds()`	`[{ name, kind, state, turns, cpu_ms, mem_kb, limit_`
`world(name)`	One world's full entry, plus `manifest` and `file`
`handles()`	`[{ id, type, claim, rights, parent, revoked, minter`

Name	Returns
`stats()`	`{ t_s, lane_busy, turns, published, delivered, crash`
`frame(world)`	`{ title, tree, hints }` or `null`
`text(world?)`	The text the frame shows, or `null`
`screen()`	`{ front, text, hash, lit }`
`uart(last = 20, grep?)`	Lines of the console UART
`faults(last = 20)`	`[{ t, world, why, handling, crashes }]`
`messages({ last = 40, world, type, kind })`	Trace records

Table B.4. Fresh systems, the trace diff, generated evals, evals, the book, the workbench's views and helpers.

Name	Returns
`test({ script?, ms?, files?, manifest?, trace?, then? })`	`{ booted, errors?, t_s, worlds, fault`
`diff({ script?, ms?, before?, after?, context?, max? })`	Table B.5
`property({ seed, runs, length, ms, invariant, … })`	`{ ok: true, runs, keys, vt_s, summary` `{ ok: false, run, seed, step, length,`
`evals.list()`	`[{ name, about }]`
`evals.run(names?, { runs }?)`	`{ pass, total, failing: [{ name, deta`
`evals.last()`	`[{ name, pass, detail?, metrics? }]` o
`evals.pin()`	`{ run_at, pass, total, edited_files,`
`evals.unpin()`	`unpinned`, or `nothing was pinned`
`evals.baseline()`	As `pin`, or `null`
`evals.record({ name?, checks? })`	`{ name, file, line, inputs, until_s,`

Name	Returns
`book.list()`	`[{ ref, id, title, sections }]`
`book.read(ref, from = 0, length = 6000)`	`{ ref, title, from, to, total, text }`
`book.search(q, limit = 12)`	`[{ ref, chapter, section, matches, sn`
`book.open(ref)`	`shown`
`open(path, line)`	`opened`
`focus(view)`	`shown`
`select(world)`	`selected`
`console`, `log`	
`wb`	

A `ref` for the `book` functions is a chapter or appendix number (`"6"`, `"B"`), a chapter id (`"ch:asks"`, `"app:scope"`, or `"asks"`), a section number (`"6.2"`, with or without a leading `§`), a section id, or the id of any figure or table; `book.open` also accepts `"contents"`, `"references"` and `"ref-KEY"` (or `"@KEY"`) for one reference. `book.read` returns the whole chapter when the reference names a subsection, a figure or a table. The book's text is the one `l4book/build.py` rendered when the workbench was built (`l4book/out/wb-book.json`); a workbench built without it has an empty `book.list()`.

Three points about the systems that `test` and the evals create are easy to miss. `test` reports a summary, while an eval decides pass or fail with `s.expect`. Both give `then` or the eval function a system `s` with the members of Table B.6, whose `stats()` is the kernel's full counters, not the scope's compact `stats()`. And a fresh system does not keep its trace unless it is made with `trace: true` or the code sets `s.tracing = true`, so `s.messages()` is otherwise empty; the live system records every message.

B.5 The trace diff

`diff` (in the scope), `t.diff` (in an eval) and the Diff view all call `host.traceDiff`. It boots two fresh systems with tracing on from the boot, *before* and *after*, drives both with the same script, and compares their traces record by record. `before` is `"shipped"` (the default, also `"base"`), `"now"` (also `"current"`), `"pinned"` (the files of the pinned run; without a pin it throws `diff: no pinned baseline; pin a run in Evals first`) or an object `{ path: source }` laid over the shipped files; `after` takes the same values, defaults to `"now"`, and lays an object over the files as now. Anything else throws `diff: before and after are "shipped", "now", "pinned" or { path: source }`. `script` is script commands, one per line, or a function `(s) => …`; then `ms` more are run, and with neither the systems run 3,000 ms. `context` is the number of records shown on each side of the divergence (3), `max` the records each trace keeps (200,000).

Two records have the same shape when they agree in kind, world, message type, sender, receiver, whether it is an ask, segment kind, bus, transfer and its size, changed pixels, text and crash, with source locations such as `(apps/about.js:2:68)` removed. The first record whose shape differs is the first change of behaviour; if one trace is a prefix of the other, it is where the shorter ends. Records before it that differ only in time, duration, steps, native time or message size are counted as timing.

Table B.5. The result of diff() in the agent's scope. t.diff returns the same comparison in the host's form: first.context.before and first.context.after hold rows ({ i, t_ms, text, cost, at }), timing.steps replaces steps_changed, timing.first is an object { index, t_ms, what, before, after }, timing adds sizes, and the result adds script and wall_ms.

Key	Meaning
`changed`	The OS files that differ between before and after.
`before`, `after`	`{ files, booted, errors?, records, t_s, faults, cut? }` for each side: which files (`shipp` `as now`, `pinned`, or one of those `+` the paths laid over it), the records traced, the virtual time, the cra and `cut` when the trace reached `max`.
`same`	True when no record's shape differs.
`screens_same`	True when both panels end with the same pixels.
`first`?	`{ index, t_ms, before, after, context_before, context_after }`: the divergent record's ir times on each side, each side's record as a line (`msg compositor → display flush (ask)`, `seg display start (boot)`, `(the trace ends)`), and the records around it as lines, the divergent marked `▸`.
`timing`	`{ records, shifted, longer, shorter, steps_changed, largest_shift_ms, lane_ms, fi` how many earlier records differ in timing, how many moved in time, lasted longer or shorter or changed step counts, the largest shift, the change in lane time (after – before), and the first such record as a sent

B.6 Evals

`workbench/evals.js` is evaluated as the body of a strict-mode function whose parameters are `define` and `property`, and calls them once per eval:

- `define(name, about, [opts], fn)`, where `name` is a string, `about` one sentence, `opts` optional and `fn` a function. Any other shape throws `define(name, about, [opts], fn)`.
- `fn(s, t)` receives a fresh, booted system `s` and a helper `t`. It may be `async`.
- `s.expect(cond, message)` records `message` as a failure when `cond` is false, without stopping the eval, and returns whether `cond` held.
- `s.metric(name, value, { unit, tol, better })` reports a number and returns it; a value that is not a finite number throws `metric NAME: VALUE is not a number`.
- `t.fresh(opts)` boots a second fresh system with the same options as an eval's; `t.manifest()` returns `js/system.json` as edited, parsed; `t.diff(opts)` is the trace diff of Section B.5; `t.property(opts)` runs a property once, as `property` in the scope.

- `opts.files` adds or replaces files: `{ "apps/rogue.js": "…" }`, a path without `js/` getting it in front.
- `opts.manifest` changes the manifest: a function that receives the parsed manifest and returns it (or changes it and returns nothing), or an object whose keys replace the top-level keys of the manifest.
- `opts.expectBootFailure: true` runs `fn` even when the system does not boot; check `s.booted` and `s.bootErrors()`.
- `opts.trace: true` records the system's trace from the boot.
- `property(name, [about], opts)` defines a generated eval, described below; a missing name or invariant throws `property(name, [about], { seed, runs, length, ms, invariant })`.

The rules for the result are in `runEvals`. An eval passes when `fn` returns or resolves without throwing and no expectation failed. Its detail is the failures joined by `;`, or the exception's message, or on a pass the value `fn` returned (a string as it is, anything else as JSON; with `expectBootFailure`, only a string). An eval without `expectBootFailure` whose system does not boot fails with `did not boot:` and the first three boot lines. Each result records `name`, `about`, `pass`, `detail`, `ms` (real time), `vt` (virtual time at the end), `regressed`, and, when it has them, `metrics` and a property's `replay` script. A run of some evals is merged into the previous results. The workbench keeps the results, the baseline and the pass counts of the last 24 runs in the browser.

A failing eval is REGRESSED when it passed in the pinned run, or, with no pin, in the previous results. Pinning (`evals.pin()`, or Pin the last run in the Evals view) stores each eval's pass or fail and its metrics with the edits the run was made on. Each metric of a later run is then compared with the pinned value: with `better` `"lower"` (the default) it is worse by `value − pinned`, with `"higher"` by `pinned − value`. `tol` is a number in the metric's unit or a string ending in `%`, relative to the pinned value, and 0 by default. A metric worse by more than `tol` is `regressed` and marks the eval REGRESSED even when it passes; one better by more than `tol` is `improved`; otherwise `same`, or `new` when the pinned run lacks it and `unpinned` without a pin. The suite reports three metrics, all on the virtual clock:

▸ workbench/src/evals.js

```
  s.metric("lane_busy", 100 * busy, { unit: "%", tol: "25%" });   // a quarter more idle work is a
      regression of cost
…
  s.metric("first_pixels", ms, { unit: "ms", tol: 4 });   // one 2 ms step of the measurement, twice
…
  s.metric("slowest", slowest, { unit: "ms", tol: 5 });
```

A generated eval, `property(name, [about], { seed, runs, length, ms, invariant, keys, each, boot, settle, files, manifest })`, is one eval made of `runs` (10) seeded random key sequences. Run *r* boots a fresh system with `files` and `manifest` (the first run uses the eval's own), waits `boot` ms (1,500), and presses `length` keys (30) drawn with seed `seed + r` (`seed` 1) from `keys` (by default `up`, `down`, `left`, `right`, `enter`, `esc`, `f1`, `tab`, `bs`, `space`, `m`, `r`, `s`, `f`, `k`, `a`, `p`, `1`, `x`), each followed by a wait drawn from [`ms`/2, 3`ms`/2] (`ms` 200). After `settle` ms (500) it calls `invariant(s, { run, seed, step, end })`, and after every key as well with `each: true`. The invariant holds when it returns nothing or `true`; `false` gives `the invariant does not hold`, a sentence gives that sentence, and a throw gives `the invariant threw:` and the message. A failure's detail is `seed 5 (run 1 of 3), after 12 of 12 keys [f enter down …]: why`, and its `replay` is a script that reproduces it in `test`; a pass's detail is `3 sequences of 30 keys from seed 1: the property held (31.3 virtual s)`. Generated evals in the Evals view (as written, 10 or 50) or `evals.run(names, { runs })` changes `runs` without editing the file.

A recorded session (`evals.record()`, or Record in the Device view) turns the live system's inputs since its boot into an eval. The live system logs every input with its virtual time to the microsecond: keys down and up, presses, kills, reloads and battery changes. The recorded eval replays them at the same virtual times and checks what was chosen: the app in front, one line of its text, the fault count since the boot. It is appended to `workbench/evals.js` under a heading line, named `session-FRONT-N` unless a name is given, as an ordinary edit that Code can revert, and run once at once. A session that hot-reloaded worlds cannot replay exactly, because the eval boots from the files as now; the result then has a `note`, and the Device view warns before recording.

The suite has 36 evals: 35 made with `define` and one property, `random-keys`. This one shows a second system made with `t.fresh`:

▸ workbench/src/evals.js

```
define("deterministic", "two fresh systems given the same keys show the same pixels and count the same
    turns", async (s, t) => {
  const s2 = await t.fresh();
  for (const x of [s, s2]) { x.run(1500); x.key("s"); x.run(200); x.key("p"); x.run(4000); }
  s.expect(s.screenHash() === s2.screenHash(), "the screens differ");
  s.expect(s.stats().turns === s2.stats().turns, `turns differ: ${s.stats().turns} and
      ${s2.stats().turns}`);
});
```

Table B.6. A system s, as test() and evals receive it (class Sys in workbench/src/core.js).

Member	Meaning
`s.run(ms = 500)`	Runs `ms`; returns the virtual time in ms. Throws if the system did not boot.
`s.key(name, holdMs = 60)`, `s.type(text)`	As the scope's `key` and `type`; return the virtual time in ms.
`s.crash(world)`, `s.battery(pct, charging = false)`	As the scope's.
`s.worlds()`, `s.world(name)`, `s.handles()`	The compact world table; one world's full entry (throws `no world called X`
`s.stats()`, `s.raw()`	The kernel's counters `{ t_ms, lane_busy_ms, turns, segments, published, delivered,` (`events` is the length of the kernel's event queue); everything `pc.stats()`
`s.frame(world)`, `s.text(world?)`	As the scope's.
`s.uart(last = 20, grep?)`, `s.faults(last = 20)`, `s.messages({ … })`	As the scope's; `messages` needs tracing.
`s.screen()`, `s.screenHash()`, `s.lit()`	The panel as 320 × 320 RGBA bytes; its hash; the fraction of sampled pixels th
`s.bootErrors()`	The stderr lines, without those the panel model writes (`panel: …`).
`s.tracing`, `s.trace`	Whether the trace is recorded; the raw records.
`s.inputs`, `s.hotReloads`	The input log a recording is made from (`[t_ms, what, …]`); the hot reloads s
`s.expect`, `s.metric`	In an eval only, as above.
`s.now()`, `s.booted`, `s.focus`, `s.pc`	The virtual time in ms; whether it booted; the app in front, taken from the last `PicoCalc` object, for `s.pc.key(name, down)` and `s.pc.reload(world)`

B.7 The views

The workbench has twelve views, arranged in tiles. The layout menu (▦) offers four layouts (Agent beside the device, Code beside the device, Evals and the trace, Everything in one tile); a window narrower than 640 pixels uses one tile.

Table B.7. The views (VIEWS in workbench/src/views.js).

View	What it shows and does
Device	The panel's pixel memory, the keys under it and a type box. The side column shows the app in front, the worlds (× kills one; a click selects it), four battery levels and charging, and the Record block: the number of inputs since the last reboot, a warning if the session hot-reloaded a world, and Record up to now, which takes the end state and lets the person choose the checks (the app in front, one line of its text from a list, the fault count) and the name before Add to evals.js.
Code	The files in groups (System, Drivers, Services, Apps, Other, Workbench), tabs, and an editor. A file marked • is edited and not applied, ∘ edited and applied. The foot says which worlds run the file or that applying reboots, and has Revert file, Apply (an OS file) or Run evals (`workbench/evals.js`). A failed apply is shown under the editor (Section B.4).
Agent	Threads, the mode, model and thinking settings, the steps of each turn with their code, results and Revert, and the composer.
Evals	Run all, Run failing, and one row per eval: its status (`pass`, `FAIL`, `REGRESSED`, `·` not run), what it checks, virtual and wall time, the detail, its metrics, and a failing property's replay script. The side column has the last run's count, the Baseline block (what runs are compared with, a table of each metric now, pinned and its tolerance, Pin the last run and Unpin), Generated evals (as written, 10, 50), the history of the last 24 runs, and how to write an eval.
Diff	Before (Shipped, or Pinned when a run is pinned), after (the files as now), the script, the files that differ, and Run diff. The result shows each side's files, records, time and faults; then either No change of behaviour or the first change, at its record index and times, with the records around it on both sides; then how many records before it differ only in timing, how many shifted, lasted longer or shorter or changed step counts, the largest shift and the lane time after − before.
Worlds	The nucleus's world table, live. A selected world shows its state, counters, Open source, Restart, Kill and Messages; its quotas in use against the manifest's (handles, asks waiting, mailbox messages and KB, framebuffers, turn limit, posts dropped); its manifest entry, `restart` included; and the text of its last frame.
Messages	The trace, filtered to Messages, Segments, Bus, Faults or All and by a world or type, with Tracing and Clear.
UART	The console line: the kernel log, the `#OCT1` fault frames (Fault frames only) and the boot lines of a boot that failed.
Handles	Every live handle: the kernel's id, type (`ep` for an endpoint), claim (the endpoint's name), rights, each holder with its own number for the handle (`compositor #4`), the parent, and the framebuffer's size, the bus speed or `→ server (types) badge holder`; a revoked handle is dimmed.

View	What it shows and does
Graph	The authority the applied manifest grants. Worlds stand in three columns (the nucleus and drivers, services, apps), each with its claims, its state and `passive` or `supervisor`. One edge goes from each holder to each server it may send to (solid) or ask (dashed), labelled with the endpoints' types when one of its worlds is selected; dotted edges go from each channel's provider (the nucleus for `frames` and `worlds.status`) to the worlds that need it, and one from the nucleus to the supervisor, labelled `fault`.
Reference	`workbench/src/reference.md` with the manifest's schema, which is also the agent's reference; its `wb:` links work.
Book	This book: contents, chapters with a section list, previous and next, and search. A code excerpt's label opens the file in Code when it is an OS file, or shows the 12 lines around the excerpt for a nucleus or workbench source; a citation shows its reference.

The header has the run controls (▶ and ❚❚ run and pause the virtual clock, ⏭ steps 100 ms, ↻ reboots from the files as they are now), the virtual time, the speed (0.25× to 4×), Apply N edits when files are unapplied, and three pills: the worlds up (or boot failed, which opens the UART), the evals passing (failures and regressed metrics counted against it), and the lane's busy fraction. The layout menu also copies the edited files as JSON and reverts all edits, which reboots.

B.8 wb: links

A Markdown link whose target starts with `wb:` drives the workbench when clicked in the agent's answers, in the Reference and in the Book. `wbLink` handles seven actions:

▸ workbench/src/agent.js

```
if (a === "open") host.openFile(needFile(q.get("file")), +q.get("line") || undefined);
else if (a === "world") { host.select(q.get("name")); host.focusView("Worlds"); }
else if (a === "eval") { host.focusView("Evals"); host.runEvals([q.get("name")]).catch((e) =>
    host.toast(e.message, true)); }
else if (a === "view") host.focusView(q.get("name"));
else if (a === "book") { if (!book.open(q.get("ref"))) host.toast(`The book has no ${q.get("ref")}`,
    true); }
else if (a === "key" && host.live) { host.live.key(q.get("name")); host.focusView("Device"); }
else if (a === "ask") { host.focusView("Agent"); if (host.ui.setAgentDraft)
    host.ui.setAgentDraft(q.get("q") || ""); else host.ui.agentDraft = q.get("q") || ""; }
```

Table B.8. The wb: links.

Link	Effect
`wb:open?file=js/apps/snake.js&line=40`	Opens the file in Code at the line.
`wb:world?name=snake`	Selects the world and shows the Worlds view.
`wb:eval?name=boot`	Shows the Evals view and runs that eval.
`wb:view?name=Diff`	Shows the view (any of Table B.7).
`wb:book?ref=6.2`	Opens the Book at the reference (as `book.open`), or reports `The book has no 6.2`.
`wb:key?name=f1`	Presses the key on the live system for 60 ms and shows the Device.
`wb:ask?q=…`	Shows the Agent with the text in its composer, not sent.

The agent's instructions list the first six; the Reference lists all seven, since `wb:ask` is a link for pages and answers written for a person rather than one the agent needs to send.

B.9 The WebAssembly API

`wasm/picocalc.js` loads `wasm/picocalc.wasm` (the build of `host/main_wasm.c`) in a page or a Web Worker and supplies the few WASI calls it makes. The second pass changed neither file; what changed is what `stats()` reports (Table B.12). The API touches no DOM, and nothing happens between calls: the virtual clock advances only in `run` and in script commands that take time (`command("wait 500")`, `command("key right")`); `crash` and `reload` run the system only up to the current instant.

`PicoCalc.load(opts)` returns a promise of a `PicoCalc` object. Its options:

Table B.9. Options of PicoCalc.load.

Option	Meaning
`wasmUrl`, `wasmBytes` or `wasmModule`	The binary: a URL (default `picocalc.wasm`), its bytes, or a compiled `WebAssembly.Module`.
`files` or `filesUrl`	The `js/` tree as an object `{ path: text }`, or the URL of such a JSON file (default `files.json`).
`verbose`	0 (default); 1 copies the kernel log to stderr; 2 also every message.

A failure rejects with an `Error` whose `code` says why:

Table B.10. Error codes of load(), boot() and the worker.

Code	Message	When
`wasm_blocked`	`this browser has no WebAssembly`; `this page may not compile WebAssembly: …`; `cannot instantiate picocalc.wasm: …`	No WebAssembly, a Content Security Policy without `'wasm-unsafe-eval'`, or instantiation failed.
`wasm_invalid`	`cannot compile picocalc.wasm: …`	The bytes do not compile for another reason.
`fetch_failed`	`cannot fetch URL: …`, `cannot fetch URL: HTTP 404`	The binary or the files cannot be fetched.
`boot_failed`	`the system did not boot (see the stderr log)`	Thrown by `boot()`; the reasons are on the stderr log (Appendix A).
`terminated`	`the worker was terminated`	A spawned system's pending calls, after `terminate()`.
`bad_op`, `not_loaded`, `error`	`no operation X`, `load first`, or the error's own message	From the worker: an unknown operation, a call before `load`, or an error without a code.

Table B.11. Methods of a PicoCalc object.

Method	Returns	Meaning
`boot(manifest = "js/system.json")`	`this`	Reads the manifest and creates the worlds; they start in the first `run`. Throws `boot_failed`; a second `boot` fails too.
`run(ms)`	`this`	Runs `ms` virtual ms; nothing before a boot.
`now()`	number	Virtual ms since boot.
`key(name, down)`	`true`, or `false` before a boot	A key goes down (`down` true) or up at the current virtual time. The STM32 reports a hold once a key has been down 300 ms; a second down without an up is ignored. Throws `no key called X`.
`keyCode(name)`	number, or -1	The code for a key name (a number is returned as it is).
`command(line)`	0 done, 1 quit, -1 error	One script command, as in `scripts/*.txt`; -1 also before a boot.
`crash(world)`	`true`, or `false` for no such world	Kills the world, as the scope's `crash`.
`reload(world)`	`true`, or `false` with the reason on stderr	Starts the world again from its file as last given to `addFile`: its crash count and history are cleared, it starts at once, its mailbox is kept. A manifest change needs a new `load`.
`frame(world)`	`{ title, tree, hints }` or `null`	The world's last published frame.
`addFile(path, text)`		Adds or replaces a file (`js/apps/about.js`, ...).
`battery(pct, charging = false)`		Sets what the STM32 reports.

Method	Returns	Meaning
`trace(on)`		Starts or stops the JSON-lines trace, delivered to `onTrace`.
`stats()`	`{ stats, worlds, handles }`, or `{}` before a boot	The kernel's counters (as `s.stats()`), every world (Table B.12) and every handle (as the scope's `handles()`).
`screen()`	`Uint8ClampedArray`	What the glass shows, 320 × 320 RGBA, a copy.
`draw(ctx, x = 0, y = 0)`		Puts the screen into a 2D canvas context.
`onLog(fn)`, `onTrace(fn)`, `onCapture(fn)`	`this`	`fn(line, stream)` with `stream` one of `stdout`, `stderr`, `uart` (the kernel log and `#OCT1` frames); `fn(record)`, one parsed trace line (`{ k: "raw", text }` if it does not parse); `fn(name, rgb, w, h)` for a `capture` command, the panel as 8-bit RGB.

The object also has `booted`, and `exited` with the code if the program ever exits; `PicoCalc.WIDTH` and `PicoCalc.HEIGHT` are 320.

Table B.12. A world's entry in stats().worlds. The first group is what sys.worlds() gives a world; the rest go only to the host.

Fields	Meaning
`name`, `kind`, `title`, `state`, `turns`, `msgs`, `cpu_ms`, `mem_kb`, `limit_kb`, `mbox`, `mbox_peak`, `restarts`, `budget_ms`, `last_error`, `claims`	As in *Small Worlds*, Appendix A. A passive world's `cpu_ms` leaves out the time it spent answering asks, which is billed to its askers.
`file`, `priority`, `crashes`	Its source, its priority (0 for a passive world), and its crashes in the last ten seconds.
`provides`, `needs`	Its channels from the manifest.
`passive`, `served_ms`	Whether it is passive; for a passive world, the lane time it spent answering asks.
`endpoints`	`[{ name, server, rights, types }]`: the endpoints the manifest grants it, with rights from `w`, `c`, `g`.
`asks`	The worlds the manifest's endpoints let it ask.
`quota`	`{ handles, ask_slots, mbox, mbox_kb, bufs, turn_ms }` (Appendix A).
`used`	`{ handles, asks, mbox, mbox_bytes, bufs }`: what it uses now.
`dropped`	Channel posts dropped because its mailbox was full.
`turn`, `turn_ms`	While it is in a turn: what it is handling, and for how many ms.

`PicoCalc.spawn(opts)` runs the same system in a Web Worker (`wasm/picocalc-worker.js`), so a page stays responsive while it runs. Its options are `wasmUrl`, `filesUrl`, `files` and `verbose`, as for `load`, plus `workerUrl` (default `picocalc-worker.js`); `wasmBytes` and `wasmModule` are not passed to the worker. URLs are resolved against the page. It returns an object whose methods mirror the ones above but return promises: `boot(manifest)`, `run(ms, { screen })` (resolving to `{ now, screen? }`), `key`, `command`, `crash`, `reload`, `frame`, `addFile`, `battery`, `trace`, `stats` and `screen`, with `onLog`, `onTrace`, `onCapture`, the `worker` itself and `terminate()`. It has no `now`, `keyCode` or `draw`; `run` reports the time.

The C side exports `pc_alloc`, `pc_free`, `pc_add_file`, `pc_boot`, `pc_run`, `pc_now_us`, `pc_key_code`, `pc_key`, `pc_crash`, `pc_battery`, `pc_command`, `pc_trace`, `pc_screen`, `pc_screen_width`, `pc_screen_height`, `pc_reload`, `pc_frame` and `pc_stats`, and imports `env.pc_emit` (bytes of the UART or the trace) and `env.pc_capture` besides the WASI calls; `picocalc.js` is the only intended caller.

B.10 Keyboard shortcuts

Table B.13. The workbench's keyboard shortcuts.

Where	Key	Effect
Code, in the editor	Ctrl+S or Cmd+S	Applies the open file (as `apply(path)`): hot reload, or a reboot for the manifest, the prelude or a file no world runs; then the apply report.
Code, in the editor	Tab	Inserts two spaces.
Agent, in the composer	Enter	Sends the message.
Agent, in the composer	Shift+Enter	A new line.
Diff, in the script	Ctrl+Enter or Cmd+Enter	Runs the diff.
Device, on the screen (click it first)	Arrow keys, Enter, Escape, Backspace, Tab, Space, Delete, Home, End, Page Up, Page Down, F1 to F10, any printable ASCII character	The PicoCalc key of the same name (`up`, `enter`, `esc`, `bs`, `tab`, `space`, `del`, `home`, `end`, `pgup`, `pgdn`, `f1` ...), down while the key is down.
Device, in the type box	Enter	Types the box's text on the device, one character every 100 ms of real time.
Book, in the search box	Escape	Clears the search.

On the Device screen a key goes down on the browser's key-down and up on its key-up, held for at least 60 ms; the browser's auto-repeat is ignored (the STM32 itself reports a hold after 300 ms), keys pressed with Ctrl or Cmd are left to the browser, and leaving the screen releases every key still held. The buttons under the screen press the same keys with the pointer.

B.11 The headless tests

`workbench/tests/` holds Node scripts that open the built page in headless Chrome through Playwright and check it; `workbench/tests/README.md` describes them. They route the CDN scripts (React 18, React DOM, htm, acorn) to local copies installed beside them (`npm install react@18 react-dom@18 htm acorn`), and `lib.js` holds the paths and the helpers. `test.js` runs the eval suite and a scripted agent turn, with a stand-in for Claude that calls the `eval` tool. `features.js` checks the instruments Chapter 12 designed, on a desktop-sized and a phone-sized page, one section each: the truthful apply, Read-only, the step timeout, the structural trace diff, the pinned baseline with tolerances, recording a session, generated evals, and mutants of the OS that each added eval must catch; it writes `features.out.txt`, which ends `all checks passed`. `book.js` exercises the Book view, and `shot-evals-phone.js` takes screenshots of the Evals view on a phone. The page to test is given in `WB_PAGE`:

```
$ WB_PAGE=$PWD/workbench/picocalc-workbench.html node workbench/tests/features.js
```

FURTHER READING

Chapter 9 describes the host, the live and fresh systems and the views; Chapter 10 the eval tool, its scope and its modes; Chapter 11 the suite and how to write an eval; Chapter 12 the design of the apply report, Read-only, the step guard, the trace diff, the pinned baseline and recorded and generated evals.

Appendix A gives the schema that `manifest()`, `opts.manifest` and the Graph view read.

BACK MATTER

References

1. **Accetta et al. 1986.** Mike Accetta, Robert Baron, William Bolosky, David Golub, Richard Rashid, Avadis Tevanian et al.. Mach: a new kernel foundation for UNIX development. *Proceedings of the Summer 1986 USENIX Conference*.
2. **Aizawa 2025.** Ken Aizawa. Writing effective tools for agents — with agents. *Anthropic Engineering, 11 September 2025*. https://www.anthropic.com/engineering/writing-tools-for-agents
3. **Armstrong 2003.** Joe Armstrong. Making reliable distributed systems in the presence of software errors. *PhD thesis, Royal Institute of Technology (KTH), Stockholm*. https://erlang.org/download/armstrong_thesis_2003.pdf
4. **Bershad 1992.** Brian N. Bershad. The increasing irrelevance of IPC performance for microkernel-based operating systems. *Proceedings of the USENIX Workshop on Micro-kernels and Other Kernel Architectures, Seattle, April 1992*, 205–212. https://www.cs.cmu.edu/afs/cs/project/mach/public/www/doc/abstracts/IPCperf.html
5. **cgroup v2 docs 2026.** The Linux kernel developers. Control Group v2. *Linux kernel documentation (Documentation/admin-guide/cgroup-v2.rst), accessed October 2026*. https://www.kernel.org/doc/Documentation/admin-guide/cgroup-v2.rst
6. **Chen and Bershad 1993.** J. Bradley Chen and Brian N. Bershad. The impact of operating system structure on memory system performance. *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP '93)*, 120–133. https://doi.org/10.1145/168619.168629
7. **Chen et al. 2021.** Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan et al.. Evaluating large language models trained on code. *arXiv:2107.03374*. https://arxiv.org/abs/2107.03374
8. **Chen et al. 2025.** Xiangdong Chen, Zhaofeng Li, Jerry Zhang, Vikram Narayanan and Anton Burtsev. Atmosphere: practical verified kernels with Rust and Verus. *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25), Seoul*. https://doi.org/10.1145/3731569.3764821

9. **Chowdhury et al. 2024.** Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace et al.. Introducing SWE-bench Verified. *OpenAI, 13 August 2024 (updated 24 February 2025)*. https://openai.com/index/introducing-swe-bench-verified/

10. **Claessen and Hughes 2000.** Koen Claessen and John Hughes. QuickCheck: a lightweight tool for random testing of Haskell programs. *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, 268–279. https://doi.org/10.1145/351240.351266

11. **Coffman et al. 1971.** E. G. Coffman, M. Elphick and A. Shoshani. System deadlocks. *ACM Computing Surveys 3(2)*, 67–78. https://doi.org/10.1145/356586.356588

12. **DeMillo et al. 1978.** Richard A. DeMillo, Richard J. Lipton and Frederick G. Sayward. Hints on test data selection: help for the practicing programmer. *Computer 11(4)*, 34–41. https://doi.org/10.1109/C-M.1978.218136

13. **Dijkstra 1972.** Edsger W. Dijkstra. The humble programmer. *Communications of the ACM 15(10) (ACM Turing Award lecture)*, 859–866. https://doi.org/10.1145/355604.361591

14. **Elkaduwe et al. 2008.** Dhammika Elkaduwe, Philip Derrin and Kevin Elphinstone. Kernel design for isolation and assurance of physical memory. *Proceedings of the 1st Workshop on Isolation and Integration in Embedded Systems (IIES '08), Glasgow*, 35–40. https://doi.org/10.1145/1435458.1435465

15. **Elphinstone and Heiser 2013.** Kevin Elphinstone and Gernot Heiser. From L3 to seL4: what have we learnt in 20 years of L4 microkernels?. *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP '13), Farmington, Pennsylvania*, 133–150. https://doi.org/10.1145/2517349.2522720

16. **Erlang/OTP n.d..** Ericsson. supervisor: generic supervisor behavior (STDLIB reference manual). *Erlang/OTP documentation*. https://www.erlang.org/doc/apps/stdlib/supervisor.md

17. **Erlang/OTP gen_server 2026.** Ericsson. gen_server: generic server behavior (STDLIB 8.1.1 reference manual, OTP 29.1.2). *Erlang/OTP documentation, accessed November 2026*. https://www.erlang.org/doc/apps/stdlib/gen_server.html

18. **ESP-IDF memory protection n.d..** Espressif Systems. ESP-IDF Programming Guide (ESP32-P4): Migration from 6.1 to 6.2, Security. *Espressif documentation*. https://docs.espressif.com/projects/esp-idf/en/latest/esp32p4/migration-guides/release-6.x/6.2/security.html

19. **Espressif 2026a.** Espressif Systems. ESP32-P4 Chip Revision v1.3 Datasheet (ESP32-P4 Series Datasheet v1.2). *Espressif Systems, online documentation, accessed October 2026*. https://documentation.espressif.com/esp32-p4-chip-revision-v1.3_datasheet_en.html

20. **Espressif 2026b.** Espressif Systems. ESP32-P4 Series Datasheet, version 0.7 (preliminary; chip revision v3.x). *Espressif Systems, online documentation, accessed October 2026*. https://documentation.espressif.com/esp32-p4_datasheet_en.html

21. **Feske 2026.** Norman Feske. Genode Foundations (Genode OS Framework, version 26.05). *Genode Labs; section "Resource trading" of the architecture chapter*. https://genode.org/documentation/genode-foundations/

22. **First pass 2026.** The PicoCalc OS authors. The first pass: authority and quotas from the manifest. *FIRST-PASS.md, in the PicoCalc OS repository*.

23. **Ge et al. 2019.** Qian Ge, Yuval Yarom, Tom Chothia and Gernot Heiser. Time protection: the missing OS abstraction. *Proceedings of the Fourteenth EuroSys Conference (EuroSys '19), Dresden*, 1–17. https://doi.org/10.1145/3302424.3303976

24. **Hansen 1970.** Per Brinch Hansen. The nucleus of a multiprogramming system. *Communications of the ACM 13(4)*, 238–241, 250. https://doi.org/10.1145/362258.362278

25. **Hardy 1988.** Norm Hardy. The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review 22(4)*, 36–38. https://doi.org/10.1145/54289.871709

26. **Heiser 2020.** Gernot Heiser. The seL4 microkernel: an introduction. *White paper, the seL4 Foundation (first published 2020; revision 1.4.1 of 17 July 2026 consulted)*. https://sel4.systems/About/seL4-whitepaper.pdf

27. **Heiser and Elphinstone 2016.** Gernot Heiser and Kevin Elphinstone. L4 microkernels: the lessons from 20 years of research and deployment. *ACM Transactions on Computer Systems 34(1), Article 1*, 1:1–1:29. https://doi.org/10.1145/2893177

28. **Heiser et al. 2025.** Gernot Heiser, Ivan Velickovic, Peter Chubb, Alwin Joshy, Anuraag Ganesh, Bill Nguyen et al.. Fast, secure, adaptable: LionsOS design, implementation and performance. *arXiv:2501.06234 [cs.OS] (v1 8 January 2025; v2 27 May 2025 consulted)*. https://arxiv.org/abs/2501.06234

29. **Heiser et al. 2026.** Gernot Heiser, Peter Chubb, Alex Brown, Tristan Clinton-Muehr, Courtney Darville, Alwin Joshy et al.. sDDF design: design, implementation and evaluation of the seL4 Device Driver Framework, release 0.7. *Work-in-progress report, Trustworthy Systems, UNSW Sydney, 19 August 2026*. https://trustworthy.systems/projects/drivers/sddf-design.pdf

30. **Herder et al. 2006.** Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg and Andrew S. Tanenbaum. MINIX 3: a highly reliable, self-repairing operating system. *ACM SIGOPS Operating Systems Review 40(3)*, 80–89. https://doi.org/10.1145/1151374.1151391

31. **Herder et al. 2007.** Jorrit N. Herder, Herbert Bos, Ben Gras, Philip Homburg and Andrew S. Tanenbaum. Failure resilience for device drivers. *Proceedings of the 37th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN '07)*, 41–50. https://doi.org/10.1109/DSN.2007.46

32. **Härtig et al. 1997.** Hermann Härtig, Michael Hohmuth, Jochen Liedtke, Sebastian Schönberg and Jean Wolter. The performance of µ-kernel-based systems. *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP '97), Saint-Malo*, 66–77. https://doi.org/10.1145/268998.266660

33. **Jia and Harman 2011.** Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering 37(5)*, 649–678. https://doi.org/10.1109/TSE.2010.62

34. **Jimenez et al. 2024.** Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press et al.. SWE-bench: can language models resolve real-world GitHub issues?. *The Twelfth*

International Conference on Learning Representations (ICLR 2024); arXiv:2310.06770. https://arxiv.org/abs/2310.06770

35. **KIT n.d..** Operating Systems Group, Karlsruhe Institute of Technology. In memoriam Jochen Liedtke (1953–2001). *KIT operating systems group, web page (PDF)*. https://os.itec.kit.edu/downloads/in-memoriam-jochen-liedtke_en.pdf

36. **Klein et al. 2009.** Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin et al.. seL4: formal verification of an OS kernel. *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP '09)*, 207–220. https://doi.org/10.1145/1629575.1629596

37. **Klein et al. 2014.** Gerwin Klein, June Andronick, Kevin Elphinstone, Toby Murray, Thomas Sewell, Rafal Kolanski et al.. Comprehensive formal verification of an OS microkernel. *ACM Transactions on Computer Systems 32(1), Article 2*, 2:1–2:70. https://doi.org/10.1145/2560537

38. **Kuz et al. 2007.** Ihor Kuz, Yan Liu, Ian Gorton and Gernot Heiser. CAmkES: a component model for secure microkernel-based embedded systems. *Journal of Systems and Software 80(5)*, 687–699. https://doi.org/10.1016/j.jss.2006.08.039

39. **Kuz et al. 2010.** Ihor Kuz, Gerwin Klein, Corey Lewis and Adam Walker. capDL: a language for describing capability-based systems. *Proceedings of the First ACM Asia-Pacific Workshop on Systems (APSys '10), New Delhi*, 31–36. https://doi.org/10.1145/1851276.1851284

40. **Lampson 1971.** Butler W. Lampson. Protection. *Proceedings of the 5th Princeton Conference on Information Sciences and Systems; reprinted in ACM SIGOPS Operating Systems Review 8(1), 1974*, 18–24 (reprint). https://doi.org/10.1145/775265.775268

41. **Leslie 2020.** Ben Leslie. The seL4 Microkit (previously "seL4 Core Platform"), an seL4 RFC proposed 19 November 2020 and approved by the Technical Steering Committee on 12 September 2023. *The seL4 RFC Book*. https://sel4.github.io/rfcs/implemented/0050-microkit.html

42. **Liedtke 1992.** Jochen Liedtke. Clans & chiefs. *Architektur von Rechensystemen (ed. A. Jammel), Informatik aktuell, Springer*, 294–305. https://doi.org/10.1007/978-3-642-77422-5_25

43. **Liedtke 1993.** Jochen Liedtke. Improving IPC by kernel design. *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP '93)*, 175–188. https://doi.org/10.1145/168619.168633

44. **Liedtke 1995.** Jochen Liedtke. On µ-kernel construction. *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP '95)*, 237–250. https://doi.org/10.1145/224056.224075

45. **Liedtke 1996.** Jochen Liedtke. Toward real microkernels. *Communications of the ACM 39(9)*, 70–77. https://doi.org/10.1145/234215.234473

46. **Liedtke et al. 1997.** Jochen Liedtke, Nayeem Islam and Trent Jaeger. Preventing denial-of-service attacks on a µ-kernel for WebOSes. *Proceedings of the 6th Workshop on Hot Topics in Operating Systems (HotOS-VI), Cape Cod, Massachusetts, IEEE*, 73–79. https://research.ibm.com/publications/preventing-denial-of-service-attacks-on-a-m-kernel-for-weboses

47. **Lyons et al. 2018.** Anna Lyons, Kent McLeod, Hesham Almatary and Gernot Heiser. Scheduling-context capabilities: a principled, light-weight operating-system mechanism for managing time. *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18), Porto, Portugal*, 1–16. https://doi.org/10.1145/3190508.3190539

48. **Matichuk et al. 2015.** Daniel Matichuk, Toby Murray, June Andronick, Ross Jeffery, Gerwin Klein and Mark Staples. Empirical study towards a leading indicator for cost of formal software verification. *Proceedings of the 37th IEEE/ACM International Conference on Software Engineering (ICSE '15), Volume 1, Florence*, 722–732. https://doi.org/10.1109/ICSE.2015.85

49. **McKeeman 1998.** William M. McKeeman. Differential testing for software. *Digital Technical Journal 10(1)*, 100–107. https://ftp.mirrorservice.org/sites/www.bitsavers.org/pdf/dec/dtj/dtj_v10-01_1998.pdf

50. **Microkit manual 2026.** The seL4 Microkit developers. Microkit User Manual (v2.3.0). *seL4 documentation (docs.sel4.systems), accessed November 2026*. https://docs.sel4.systems/projects/microkit/manual/2.3.0/

51. **Microkit sources 2026.** seL4 Foundation. seL4 Microkit source repository, example/passive_server/passive_server.system. *GitHub, accessed November 2026*. https://raw.githubusercontent.com/seL4/microkit/main/example/passive_server/passive_server.system

52. **Microkit tutorial 2026.** seL4 Foundation. Microkit tutorial, Part 3: Wordle server. *seL4 documentation (docs.sel4.systems), accessed November 2026*. https://docs.sel4.systems/projects/microkit/tutorial/part3.html

53. **Miller et al. 2003.** Mark S. Miller, Ka-Ping Yee and Jonathan Shapiro. Capability myths demolished. *Technical report SRL2003-02, Systems Research Laboratory, Johns Hopkins University*. https://papers.agoric.com/papers/capability-myths-demolished/full-text

54. **Murray et al. 2013.** Toby Murray, Daniel Matichuk, Matthew Brassil, Peter Gammie, Timothy Bourke, Sean Seefried et al.. seL4: from general purpose to a proof of information flow enforcement. *2013 IEEE Symposium on Security and Privacy, San Francisco*, 415–429. https://doi.org/10.1109/SP.2013.35

55. **Mytkowicz et al. 2009.** Todd Mytkowicz, Amer Diwan, Matthias Hauswirth and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong!. *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '09)*, 265–276. https://doi.org/10.1145/1508244.1508275

56. **Newcombe et al. 2015.** Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker and Michael Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM 58(4)*, 66–73. https://doi.org/10.1145/2699417

57. **Paturel et al. 2023.** Mathieu Paturel, Isitha Subasinghe and Gernot Heiser. First steps in verifying the seL4 Core Platform. *Proceedings of the 14th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '23), Seoul*, 9–15. https://doi.org/10.1145/3609510.3609821

58. **Proofcraft 2025.** Proofcraft. Proof that seL4 enforces integrity. Now also on AArch64 (news item, 29 April 2025). *proofcraft.systems*. https://proofcraft.systems/news-2025/#2025-04-29

59. **Saltzer and Schroeder 1975.** Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE 63(9)*, 1278–1308. https://doi.org/10.1109/PROC.1975.9939

60. **Schluntz and Zhang 2024.** Erik Schluntz and Barry Zhang. Building effective agents. *Anthropic, 19 December 2024*. https://www.anthropic.com/research/building-effective-agents

61. **seL4 manual 2026.** seL4 Foundation. seL4 Reference Manual, version 17.0.0 (30 October 2026). *The seL4 Foundation*. https://sel4.systems/Info/Docs/seL4-manual-17.0.0.pdf

62. **seL4 news 2024.** seL4 Foundation. Functional correctness proved for AArch64 (news item, 19 April 2024). *sel4.systems*. https://sel4.systems/news/2024

63. **seL4 news 2026a.** seL4 Foundation. Functional correctness proof for MCS seL4 completed on RISC-V (news item, 29 June 2026). *sel4.systems*. https://sel4.systems/news/2026

64. **seL4 news 2026b.** seL4 Foundation. seL4 security proofs now complete on AArch64 (news item, 24 August 2026). *sel4.systems*. https://sel4.systems/news/2026

65. **seL4 proofs 2026.** seL4 Foundation. seL4 proofs: what the proofs say and how strong they are. *sel4.systems, accessed November 2026*. https://sel4.systems/Verification/proofs.html

66. **seL4 roadmap 2026.** seL4 Foundation. Development roadmap. *sel4.systems, accessed November 2026*. https://sel4.systems/roadmap.html

67. **seL4 verified configurations 2026.** seL4 Foundation. Verified configurations. *seL4 documentation (docs.sel4.systems), accessed November 2026*. https://docs.sel4.systems/projects/sel4/verified-configurations.html

68. **Serebryany et al. 2012.** Konstantin Serebryany, Derek Bruening, Alexander Potapenko and Dmitriy Vyukov. AddressSanitizer: a fast address sanity checker. *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC '12), Boston*, 309–318. https://www.usenix.org/conference/atc12/technical-sessions/presentation/serebryany

69. **Sewell et al. 2011.** Thomas Sewell, Simon Winwood, Peter Gammie, Toby Murray, June Andronick and Gerwin Klein. seL4 enforces integrity. *Interactive Theorem Proving (ITP 2011), Lecture Notes in Computer Science, Springer*, 325–340. https://doi.org/10.1007/978-3-642-22863-6_24

70. **Sewell et al. 2013.** Thomas Sewell, Magnus O. Myreen and Gerwin Klein. Translation validation for a verified OS kernel. *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13), Seattle*, 471–482. https://doi.org/10.1145/2491956.2462183

71. **Shapiro et al. 1999.** Jonathan S. Shapiro, Jonathan M. Smith and David J. Farber. EROS: a fast capability system. *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP '99)*, 170–185. https://doi.org/10.1145/319151.319163

72. **Small Worlds 2026.** The PicoCalc OS authors. Small Worlds: Operating Systems, Isolation and Self-Healing Drivers, Built on a JavaScript Handheld. *Textbook, in the PicoCalc OS repository (book/)*. http

s://claude.ai/artifact/1WUiKLVE3poZ9tjcF2XXPu

73. **Steinberg and Kauer 2010.** Udo Steinberg and Bernhard Kauer. NOVA: a microhypervisor-based secure virtualization architecture. *Proceedings of the 5th European Conference on Computer Systems (EuroSys '10), Paris*, 209–222. https://doi.org/10.1145/1755913.1755935

74. **Tarjan 1972.** Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing 1(2)*, 146–160. https://doi.org/10.1137/0201010

75. **Wang et al. 2024.** Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng et al.. Executable Code Actions Elicit Better LLM Agents. *International Conference on Machine Learning (ICML 2024); arXiv:2402.01030*. https://arxiv.org/abs/2402.01030

76. **WASI ABI 2026.** WASI Subgroup, W3C WebAssembly Community Group. WASI application ABI: commands and reactors (legacy/application-abi.md). *WebAssembly/WASI repository, accessed November 2026*. https://github.com/WebAssembly/WASI/blob/main/legacy/application-abi.md

77. **wasi-sdk 2026.** The wasi-sdk contributors. wasi-sdk: WASI-enabled WebAssembly C/C++ toolchain. *GitHub repository WebAssembly/wasi-sdk (Clang/LLVM and wasi-libc as submodules), accessed November 2026*. https://github.com/WebAssembly/wasi-sdk

78. **Waterman et al. 2021.** Andrew Waterman, Krste Asanović and John Hauser. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 20211203. *RISC-V International*. https://github.com/riscv/riscv-isa-manual/releases/download/Priv-v1.12/riscv-privileged-20211203.pdf

79. **WebAssembly Core 2026.** W3C WebAssembly Working Group. WebAssembly Core Specification, version 2.0. *W3C Candidate Recommendation Draft, 6 November 2026 (editor Andreas Rossberg)*. https://www.w3.org/TR/wasm-core-2/

80. **Weibel et al. 2024.** Trudy Weibel, Zoltan A. Kocsis, Mathieu Paturel, Robert Sison, Isitha Subasinghe and Gernot Heiser. Verifying the seL4 Microkit: verified libmicrokit and CapDL mapping. *Technical report, version 1.0, Trustworthy Systems, UNSW Sydney, 21 June 2024*. https://trustworthy.systems/publications/papers/Weibel_KPSSH_24.pdf

81. **West and Sartori 2026.** Mike West and Antonio Sartori. Content Security Policy Level 3. *W3C Working Draft, 27 October 2026*. https://www.w3.org/TR/CSP3/

82. **Yang et al. 2024.** John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan et al.. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems 37 (NeurIPS 2024); arXiv:2405.15793*, 50528–50652. https://papers.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html

83. **Yao et al. 2023.** Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan et al.. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR 2023); arXiv:2210.03629*. https://arxiv.org/abs/2210.03629

84. **Zhou et al. 2021.** Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen et al.. FoundationDB: a distributed unbundled transactional key value store. *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, 2653–2666. https://doi.org/10.1145/3448016.3457559